

Complexidade

Notas de Aula

Ricardo Dorneles

CCET-UCS

9 de janeiro de 2022

Índice

- Introdução à Disciplina
- Introdução a Complexidade de Algoritmos
 - Exercícios de Complexidade
- Análise Assintótica - Notações Θ e Ω
 - Exercícios de Análise Assintótica
- Cálculo da Complexidade
- Resolução de somatórios no Wolfram Alpha
- Análise da Complexidade de Algoritmos Recursivos
 - Resolução de Recorrências no Wolfram Alpha
 - Método da Substituição
 - Método da Iteração (ou Expansão)
 - Teorema Mestre
 - Árvores de recursão
- Análise da Complexidade Média

- Estratégias para Projeto de Algoritmos
 - Força Bruta
 - Backtracking
 - Aplicações de Backtracking
 - O Algoritmo Minimax
 - Algoritmo Guloso (Greedy)
 - Aplicações de Algoritmo Guloso
 - Divisão e Conquista
 - Aplicações de Divisão e Conquista
 - Programação Dinâmica
 - Aplicações de Programação Dinâmica
 - Programação Dinâmica no Geeks for Geeks
 - Branch and Bound
 - Aplicações de Branch-and-Bound

- Algoritmos Aproximativos
- Tratabilidade de Problemas
- Questões do POSCOMP
- Questões do ENADE

Introdução

O estudo durante o semestre será dividido basicamente em três partes

- Cálculo de Complexidade de Algoritmos - A partir de algoritmos/programas, determinaremos sua complexidade, Exemplo: Qual a complexidade de um algoritmo de busca seqüencial em um vetor;
- Estratégias de projeto de Algoritmos - Metodos para fazer algoritmos/programas mais eficientes (com menor complexidade), que executem com um custo computacional menor.
- Tratabilidade de Problemas

O que é complexidade?

- Basicamente temos complexidade de tempo e complexidade de espaço. O nosso foco de estudo durante este semestre sera basicamente complexidade de tempo, ou simplesmente complexidade de um algoritmo.
- A complexidade de tempo de um algoritmo é a quantidade de trabalho necessária para execução deste algoritmo (para encontrar a solução do problema que este algoritmo resolve).
- Quanto maior a complexidade, maior o tempo de execução do algoritmo/programa.

- É bom salientar aqui que quanto temos programa complexo, não significa necessariamente que seu código é difícil de entender, ou que para resolver um problema é necessário um algoritmo muito complicado. Um programa complexo está diretamente relacionado ao seu tempo de execução.
- A complexidade de espaço é a quantidade de espaço(memória) adicional necessária para resolver o problema. Exemplo: vetor, matriz, pilha, variáveis, e outras estruturas, necessárias adicionalmente para resolver o problema. As estruturas com os dados de entrada e de saída não são consideradas neste cálculo.

Para que calcular a complexidade?

- Com a complexidade de um algoritmo, saberemos qual o seu comportamento quando for executado
- Saberemos também qual o tempo necessário para a solução do problema usando este algoritmo, e teremos condições também de avaliar(escolher) entre dois ou mais algoritmos, qual seria mais eficiente.

Como medir a complexidade?

- Métodos empíricos: executar o programa e considerar o tempo de execução. Dependem do compilador, otimizações, processador, etc.
- Métodos analíticos: Cálculo de complexidade a partir da análise do algoritmo/programa. É independente da máquina. Os métodos analíticos são estudados para o cálculo da complexidade e veremos durante este semestre.

Do que depende o tempo de execução de um algoritmo

- Velocidade do equipamento (tempo para executar uma instrução/operação fundamental)
- Tamanho da entrada, normalmente representado pela letra n .
- Algoritmo usado (complexidade do algoritmo que esta resolvendo este problema, representada por uma função de complexidade)

$\text{TempoTotalExecucao} = \text{FuncaoComplexidade}(\text{TamanhoEntrada}) * \text{TempoExecutar1Instrucao}$

Como comparar tempos de execução (como escolher o melhor)?

Executando todos os algoritmos/programas ou Calculando a complexidade destes.

Alguns algoritmos com suas respectivas complexidades:

- Procurar um elemento em um vetor ordenado:
 - Usando pesquisa seqüencial precisamos no pior caso de n comparações, complexidade n .
 - Usando pesquisa binária precisamos no pior caso de $\log n$ comparações, complexidade $\log n$.
- Qual o elemento que mais vezes aparece em um vetor:
Comparando todos com todos teremos n^2 comparações, complexidade n^2 .
- Ordenar um vetor usando bubble sort, teremos na ordem de n^2 comparações.
- Ordenar um vetor usando merge sort ou quicksort, teremos na ordem de $n \cdot \log n$ comparações.

Introdução a Complexidade de Algoritmos

- A complexidade de tempo de um algoritmo, ou simplesmente complexidade, é a quantidade de trabalho executada pelo algoritmo
- É escolhida uma operação chamada de operação fundamental. A complexidade é baseada na contagem do número de operações fundamentais.
- Podem ser escolhidas mais de uma operação fundamental com pesos diferentes. Ex: comparações ou trocas em um algoritmo de classificação, produtos, somas, etc.

- Normalmente a complexidade é função do tamanho da entrada.
- Chamando E_n o conjunto de entradas possíveis de tamanho n , tomando $e \in E_n$ e chamando $c(e)$ o número de execuções da operação fundamental, tem-se, para a entrada e :
 - $CPC(n) = \max_{e \in E_n} c(e)$
- que é a complexidade no pior caso. Outras medidas utilizadas são a complexidade no caso médio e a complexidade no melhor caso.

Representação Assintótica

- Seja t_n o tempo de um programa manipulando uma entrada de tamanho n
- A função **exata** não interessa porque depende do compilador, da máquina, etc.
- Queremos saber como t_n cresce com n para descobrir se é possível rodar o programa para n grande. Para n pequeno a questão da complexidade não é importante.

Notação O

- $T_n = O(f(n))$ se e somente se $\exists$ constantes positivas C e n_0 tais que
- $|t_n| \leq C |f(n)| \quad \forall n \geq n_0$
- ou seja, $f(n)$ é um limite superior para t_n
- Busca-se obter algoritmos com $t_n = O(f(n))$ onde $f(n)$ seja a "menor" função possível (assim, t_n não cresce rapidamente com n).
- A notação $O(\)$ é chamada de Big Oh. Usa-se a letra O porque a classe de complexidade também é chamada de Ordem (Order) de complexidade.

Exemplos:

- $O(1)$ é melhor que
- $O(\log n)$ é melhor que
- $O(n)$ é melhor que
- $O(n \log n)$ é melhor que
- $O(n^2)$ é melhor que
- $O(n^3)$ é melhor que
- $O(2^n)$ é melhor que
- $O(3^n)$ etc.

Os primeiros 6 casos são algoritmos polinomiais. Os dois últimos são algoritmos exponenciais. Normalmente considera-se algoritmos polinomiais como tratáveis em tempo razoável e algoritmos exponenciais como intratáveis.

A tabela a seguir mostra o tempo necessário para executar 6 funções de diferentes complexidades para diferentes tamanhos de entrada

Função de Complexidade	10	20	30	40	50	
n	.00001 seg.	.00002 seg.	.00003 seg.	.00004 seg.	.00005 seg.	.0
n^2	.0001 seg.	.0004 seg.	.0009 seg.	.0016 seg.	.0025 seg.	.1
n^3	.001 seg.	.008 seg.	.027 seg.	.064 seg.	.125 seg.	.2
n^5	.1 seg.	3.2 seg.	24.3 seg.	1.7 min.	5.2 min.	.3
2^n	.001 seg.	1.0 seg.	17.9 min.	12.7 dias.	35.7 anos	3
3^n	.059 seg.	58 min.	6.5 anos	3855 séculos	2×10^8 séculos	1.3

- Sabendo-se a função de complexidade e a relação entre o tamanho de duas entradas, pode-se estimar a relação entre o número de operações necessário para processar as duas entradas (e consequentemente a relação entre os tempos de execução).
- Para isso aplica-se a função de complexidade ao tamanho da segunda entrada (que é função do tamanho da primeira entrada)
- Ex. sabendo-se que um algoritmo tem complexidade quadrática, $O(N^2)$, qual o efeito no número de operações se o tamanho da entrada é multiplicado por 3?
 - Número de operações para a entrada de tamanho $N = N^2$
 - Número de operações para a entrada de tamanho $3N = (3N)^2 = 9N^2$

A tabela a seguir mostra o aumento no tamanho da entrada para um computador 10 x mais rápido, para o mesmo tempo de processamento.

Complexidade	Tamanho máximo em computador lento	Tamanho máximo em computador 10 x mais rápido
$\log_2 n$	x	x^{10}
n	x	10 x
$n \log n$	x	10x (para x grande suficiente)
n^2	x	$3.16x = x\sqrt{10}$
n^3	x	$2.15x = x\sqrt[3]{10}$
2^n	x	$3.3 + x = \log_2 10 + x$
3^n	x	$2.096 + x = \log_3 10 + x$

Algumas relações entre tamanho de entrada, número de operações, complexidade do algoritmo, velocidade (em operações/unidade de tempo) e tempo de execução:

- Tempo de execução = número de operações / velocidade
 - Assim, se a execução de um algoritmo necessita de 10.000.000 de operações, e a velocidade é de 1.000.000 ops/seg, o tempo total é $10.000.000 / 1.000.000 = 10$ segundos
 - Essa expressão deixa claro que, para o mesmo número de operações, a relação entre velocidade e tempo de execução é inversamente linear. Ao dobrar a velocidade, o tempo de execução é reduzido pela metade.
- Não esquecendo que: número de operações = função de complexidade aplicada ao tamanho da entrada ($Nops = f_c(N)$)
 - Onde N = tamanho da entrada e f_c é a função de complexidade

- Pode-se relacionar o número de operações em duas execuções de algoritmos pela expressão a seguir:

- $f_1(x_1) \cdot r = f_2(x_2)$

- onde

- r é a relação entre o número de operações da primeira execução e da segunda execução. Essa relação depende do tempo de execução e da velocidade dos computadores envolvidos e pode ser representada como $R_v \cdot R_t$, onde R_v representa a relação entre a velocidade do computador na segunda execução e a velocidade na primeira execução, e R_t representa a relação entre T_2 e T_1 . Se a velocidade de um computador é o dobro do outro, mas ele roda metade do tempo, os números de operações serão iguais.

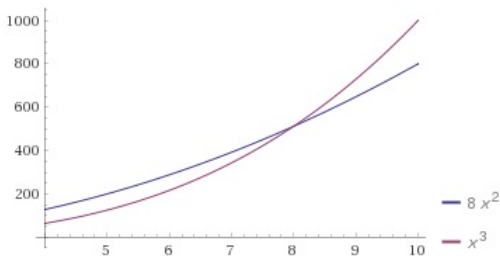
- x_1 e x_2 é o tamanho da instância da primeira e da segunda execução
- f_1 e f_2 são as funções de complexidade da primeira e da segunda execução
- $f_1(x_1)$ e $f_2(x_2)$ é o número de operações da primeira e da segunda execução.
- Da relação anterior, pode-se extrair o tamanho da entrada da segunda execução:
 - $x_2 = f_2^{-1}(rf_1(x_1))$
- Se o coeficiente r é maior do que 1, o número de operações da segunda execução é maior do que da primeira execução.

Exercícios

1) Considere dois algoritmos A e B com complexidade $8n^2$ e n^3 . Qual o valor de n para o qual o algoritmo B é mais eficiente que o algoritmo A ?

Visualização de Funções no Wolfram Alpha

- <https://www.wolframalpha.com/>
- plot $8x^2, x^3, x=4$ to 10



- O gráfico mostra o número de operações necessárias para uma dada entrada N , o que é uma medida da eficiência do algoritmo. Quanto menor o número de operações para uma instância, mais eficiente o algoritmo.
- Constata-se que até um certo valor de N , o algoritmo de complexidade X^3 necessita de menos operações, a partir do qual o custo do algoritmo X^3 é maior. Esse ponto N pode ser obtido igualando-se as duas funções:
 - $x^3 = 8x^2$
obtendo-se $x=8$.
- Assim, para $x < 8$, o algoritmo X^3 é mais eficiente do que o algoritmo $8x^2$.

- 2) Um algoritmo tem complexidade $2n^2$. Num certo computador, num tempo t , o algoritmo resolve um problema de tamanho 25. Imagine agora que você tem disponível um computador 100 vezes mais rápido. Qual o tamanho máximo do problema que o mesmo algoritmo consegue resolver no mesmo tempo t do computador mais rápido?
 - Uma forma de resolver esse problema é calculando o número de operações para a primeira rodada, aplicando a função de complexidade $2n^2$ ao tamanho da entrada (25) resultando em $2 \cdot 25^2 = 1250$ operações.
 - No mesmo tempo, em um computador 100 vezes mais rápido, o número de operações da segunda execução é 100 vezes maior, ou seja, 125.000 operações.
 - Esse mesmo resultado seria obtido se a segunda execução fosse feita no mesmo computador, em um tempo 100 vezes maior.

- 2) (continuação) Um algoritmo tem complexidade $2n^2$. Num certo computador, num tempo t , o algoritmo resolve um problema de tamanho 25. Imagine agora que você tem disponível um computador 100 vezes mais rápido. Qual o tamanho máximo do problema que o mesmo algoritmo consegue resolver no mesmo tempo t do computador mais rápido?
 - Qual o tamanho da entrada para um algoritmo de custo $2n^2$ resultar em 125.000 operações?
 - Pode-se obter o tamanho n da entrada aplicando-se a função inversa da complexidade $2n^2 = 125.000$, o que pode ser feito isolando-se o n .
 - $2n^2 = 125.000 \Rightarrow n^2 = 62500 \Rightarrow n = 250$

- 2) (continuação) Um algoritmo tem complexidade $2n^2$. Num certo computador, num tempo t , o algoritmo resolve um problema de tamanho 25. Imagine agora que você tem disponível um computador 100 vezes mais rápido. Qual o tamanho máximo do problema que o mesmo algoritmo consegue resolver no mesmo tempo t do computador mais rápido?
 - Também pode-se chegar ao mesmo resultado aplicando diretamente a relação $f_1(x_1) = f_2(x_2)$:
 - $2 \cdot 25^2 \cdot 100 = 2 \cdot (n_2)^2$
- 3) E se a complexidade do problema anterior for 2^n ?

- 3) E se a complexidade do problema anterior for 2^n ?
 -
 - $(2^{25}).100 = 2^n \Rightarrow n = \log_2((2^{25}).100)$

4) Suponha que uma empresa utiliza um algoritmo de complexidade n^2 que, em um tempo t , na máquina disponível, resolve um problema de tamanho x . Suponha que o tamanho do problema a ser resolvido aumentou em 20%, mas o tempo de resposta deve ser mantido. Para isso, a empresa pretende trocar a máquina por uma mais rápida. Qual percentual de melhoria no tempo de execução das operações básicas é necessário para atingir sua meta?

5) Suponha que no problema anterior, ainda se queira reduzir em 50% o tempo de resposta. Qual a melhoria esperada para a nova máquina?

6) Um programa tem complexidade n^3 . Este programa precisa de 1 minuto para executar o tamanho de entrada atual. (a) Se tivermos 1 hora, conseguiremos processar uma entrada 5 vezes maior? (b) Cada vez que dobramos o tamanho da entrada, quantas vezes mais rápido precisa ser o equipamento para processar no mesmo tempo?

[Voltar para o índice](#)

7) Um programa de complexidade n^3 precisa de 64 minutos para executar em um computador X com tamanho de entrada 400. (a) Qual o tamanho máximo da entrada para executar em 27 minutos? (b) Se uma alteração no programa reduziu-o para uma complexidade quadrática, qual o tamanho máximo da entrada para executar em 64 minutos?

[Voltar para o índice](#)

8) Um algoritmo leva 1 segundo para uma entrada de tamanho 100.

- (a) Qual é o tamanho da entrada que pode ser resolvida em 1 minuto quando a complexidade do algoritmo é quadrática?
- (b) E se for utilizado um computador com o dobro da velocidade?

[Voltar para o índice](#)

9) Um algoritmo leva 0.5 ms para uma entrada de tamanho 100. Qual é o tamanho da entrada que pode ser resolvida em 1 minuto quando a complexidade do algoritmo é:

- 1 linear
- 2 $O(n \log n)$
- 3 quadrático
- 4 cúbico

[Voltar para o índice](#)

10) Um algoritmo leva 0.5 ms para uma entrada de tamanho 100. Quanto tempo ele levará para terminar uma entrada de tamanho 500 se sua complexidade em tempo é:

- 1 linear
- 2 $O(n \log n)$
- 3 quadrático
- 4 cúbico

[Voltar para o índice](#)

11) Um programa de complexidade n^3 precisa de 4096 segundos para executar em um computador X com um tamanho de entrada 1600.

- 1 Sem alterar o programa e sem trocar o computador, o que acontece com o tempo de execução se o tamanho da entrada for 50% maior?
- 2 Sem alterar o programa, qual o tamanho máximo da entrada que pode ser executado em 4096 segundos considerando um computador Y 8 vezes mais rápido?
- 3 Considerando o computador X , que com o programa n^3 precisa de 4096 s para a entrada de tamanho 1600, qual o tamanho máximo da entrada se um programador conseguir reduzir a complexidade do programa para n^2 ?

12) Um programa com complexidade n^4 processa uma entrada de tamanho 1000 em 625 segundos em um computador X . Usando um computador Y , 4 vezes mais rápido, sem alterar o programa, pergunta-se:

- 1 Em quanto tempo a mesma entrada (1000) é processada?
- 2 Qual o máximo de entrada que esse programa processa em 625 segundos?

[Voltar para o índice](#)

13) Um programa com complexidade n^3 atende as necessidades de uma empresa que processa dados de 1000 funcionários em meia hora.

- 1 Se o número de funcionários reduzir pela metade, qual será o tempo de execução desse programa?
- 2 Se o computador for trocado por um outro 4 vezes mais rápido, qual o número máximo de funcionários que este programa processa em 8 horas?

[Voltar para o índice](#)

14) Um programa com complexidade n^2 processa uma entrada de tamanho 1200 em 1 hora em um computador X.

- 1 Em quanto tempo este programa processa uma entrada de tamanho 20?
- 2 Usando um computador duas vezes mais rápido, qual a maior entrada que esse programa consegue processar em 2 horas?

[Voltar para o índice](#)

15) Um programa com complexidade n^3 calcula o custo de 800 produtos em 2 horas utilizando o computador atual. Sabe-se que a quantidade de produtos será duplicada e que o limite de tempo para executar esse programa é de 4 horas. Diga se o computador atual atende à necessidade de processamento e indique qual a necessidade mínima de desempenho de um novo se for o caso.

[Voltar para o índice](#)

Análise Assintótica - Notações Θ e Ω

- As principais notações utilizadas na análise da complexidade são:
 - $T(n) = O(f(n))$ se existem constantes c e n_0 tais que $T(n) \leq cf(n)$ quando $n \geq n_0$.
 - Essa definição pode ser interpretada como, uma função $T(n)$ é $O(f(n))$ se, a partir de algum valor n_0 , a função $T(n)$ é menor ou igual a $c.f(n)$ para todo valor $n \geq n_0$.
 - Essa condição se realizará somente se o grau do termo de maior grau da função $T(n)$ é menor ou igual ao grau do termo de maior grau da função $f(n)$.
 - Ou seja, $f(n)$ é um limite superior para $T(n)$. Ex: $2n$ é $O(n^2)$ (bem como $O(n)$, $O(n^3)$,...).

■ Outras notações bastante utilizadas são:

- $T(n) = \Omega(g(n))$ se existem constantes c e n_0 tais que $T(n) \geq cg(n)$ quando $n \geq n_0$, ou seja, $g(n)$ é um limite inferior para $T(n)$. Ex: $2n^2 = \Omega(n^2)$ (bem como $\Omega(n)$).
- $T(n)$ é $\Theta(h(n))$ se e somente se $T(n)$ é $O(h(n))$ e $T(n)$ é $\Omega(h(n))$, ou seja, $h(n)$ dá a ordem de complexidade exata de $T(n)$. Ex: $2n^2$ é $\Theta(n^2)$.

Menos utilizadas são as notações:

- $T(n) = o(p(n))$ se $T(n) = O(p(n))$ e $T(n) \neq \Theta(p(n))$.
- $T(n) = \omega(q(n))$ se $T(n) = \Omega(p(n))$ e $T(n) \neq \Theta(q(n))$.

[Voltar para o índice](#)

Exercícios:

1) Um programa tem complexidade $\Theta(n^6)$:

- 1 Podemos afirmar que ele tem complexidade exponencial?
- 2 Poderíamos representar sua complexidade como $O(n^2)$?
- 3 Poderíamos representar sua complexidade como $\Omega(n^4)$?
- 4 Podemos afirmar que ele tem um comportamento mais complexo que $O(n^3)$?

[Voltar para o índice](#)

2) Um programador A analisou um programa e determinou que a sua complexidade é $O(2^n)$. Um programador B também analisou um programa e determinou que sua complexidade é $\Omega(n^4)$. Um programador C, por sua vez, analisou um programa e determinou que sua complexidade é $O(n^4)$.

Supondo que os programadores analisaram corretamente:

(a) Podem os três programadores terem analisado o mesmo programa? Por que?

(b) Se B e C forem o mesmo programa, existe como determinar a função assintótica exata $\Theta(f(n))$ para este programa? Qual seria? Explique.

3) Dois programadores analisaram um mesmo programa, um deles constatou que a complexidade do mesmo é $\Omega(n^2)$ e o outro constatou que a complexidade é $O(2^n)$.

- 1 Os dois programadores podem estar certos? Por que?
- 2 Obrigatoriamente esse programa tem complexidade exponencial? Por que?
- 3 Obrigatoriamente esse programa tem complexidade polinomial? Por que?
- 4 Podemos determinar a função assintótica $\Theta(f(n))$? Qual? Por que?

4) Responda e explique cada resposta:

- 1 Um programa com complexidade $O(n^{10})$ possui necessariamente complexidade polinomial?
- 2 Um programa com complexidade $O(2^n)$ possui necessariamente complexidade exponencial?
- 3 Um programa com complexidade $\Omega(n^3)$ pode ter complexidade exponencial?
- 4 Um programa com complexidade $\Theta(n^8)$ pode ter complexidade exponencial?

5) Responda e explique cada resposta:

- 1 Um programa com complexidade $\Omega(3^n)$ pode ter complexidade polinomial?
- 2 Podemos representar a complexidade de um programa que é $\Theta(n^8)$ como $\Omega(n^2)$ ou $O(n^4)$?
- 3 Um programa com complexidade $\Omega(n^5)$ é necessariamente mais complexo que um $O(n^4)$?
- 4 O algoritmo de ordenação "merge sort" é mais complexo que o algoritmos de ordenação "bubble sort"? Qual a complexidade "exata" de cada um deles?

Quais das opções representam a complexidade de um algoritmo $\Theta(4n^3 - n^2)$?

1 $\Omega(n^4)$

2 $\Theta(n^2)$

3 $\Omega(n^2)$

4 $O(n^4)$

5 $O(n^3)$

[Voltar para o índice](#)

Qual a opção que melhor descreve um algoritmo com comportamento exponencial:

1 $\Theta(n^2)$

2 $\Omega(n^2)$

3 $\Omega(2^n)$

4 $O(n^2)$

5 $\Theta(n.m)$

[Voltar para o índice](#)

- Às vezes pode ser difícil encontrar a relação entre a razão de crescimento de duas funções. Por exemplo, qual a relação entre as funções $\log(n!)$ e $\log(n^n)$? As regras a seguir auxiliam na verificação da relação entre duas funções:

- Obs:

- $\log(n!) = \log(n \cdot (n-1) \cdot (n-2) \dots 1) =$
 $\log n + \log(n-1) + \dots + \log 1$

- $\log(n^n) = \log(n \cdot n \cdot n \cdot n \cdot n \dots) = n \log n,$

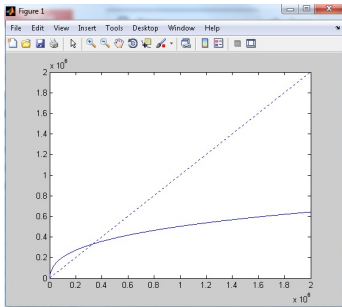
Regra 1: Se $T1(n) = O(f(n))$ e $T2(n) = O(g(n))$, então

- $T1(n) + T2(n) = \max(O(f(n)), O(g(n)))$
- $T1(n) * T2(n) = O(f(n)) * O(g(n))$

Regra 2 : Se $T(x)$ é um polinômio de grau n , então $T(n) = \Theta(x^n)$

Regra 3: $\log^k n = O(n)$ para qualquer constante k , o que mostra que logaritmos crescem devagar.

[Voltar para o índice](#)



[Voltar para o índice](#)

```
clear
for pos=1:2000000
    V(pos)=(log(pos)) ^ 5;
    W(pos)=pos;
end
plot(V,'-')
hold on
plot(W,':')
```

Regra 4: Pode-se determinar a razão de crescimento relativo entre duas funções $f(n)$ e $g(n)$ calculando-se $\lim_{n \rightarrow \infty} f(n)/g(n)$, aplicando-se a regra de l'Hôpital, se necessário (a regra de l'Hôpital diz que se $\lim_{n \rightarrow \infty} f(n) = \infty$ e $\lim_{n \rightarrow \infty} g(n) = \infty$, então $\lim_{n \rightarrow \infty} f(n)/g(n) = \lim_{n \rightarrow \infty} f'(n)/g'(n)$).

Se o limite é 0, $f(n) = o(g(n))$

Se o limite é $c \neq 0$, $f(n) = \Theta(g(n))$

Se o limite é ∞ , $g(n) = o(f(n))$

Normalmente a relação pode ser obtida por álgebra. Por exemplo, se $f(n) = n \log n$ e $g(n) = n^{1.5}$, $f(n)/g(n) = \frac{\log n}{n^{0.5}} = \frac{\log^2 n}{n}$. Como n cresce mais rapidamente que qualquer potência de $\log n$, $g(n)$ cresce mais rapidamente que $f(n)$.

Além disso, é importante manter sempre a relação entre as funções a seguir (de menor crescimento para maior crescimento):

c	constante
$\log n$	logarítmico
$\log^2 n$	logarítmico quadrático
n	linear
$n \log n$	$n \log n$
n^2	quadrático
n^3	cúbico
2^n	exponencial

[Voltar para o índice](#)

Algumas Propriedades da Notação O:

- Se $f(n) = O(g(n))$ e $g(n) = O(h(n))$ então $f(n) = O(h(n))$
- $O(g(n) + h(n)) = O(g(n)) + O(h(n))$
- $O(k.g(n)) = kO(g(n)) = O(g(n))$
- $f(n) = O(f(n))$
- $O(f(n)).O(g(n)) = O(f(n).g(n))$
- $O(f(n)).O(g(n)) = f(n).O(g(n))$

Ex: Classifique as funções de complexidade a seguir por ordem de crescimento assintótico, isto é, encontre a ordem relativa entre as mesmas.

$20n + 3$	2^n	$n.\log n$	$4.n^2$
$\ln n$	e^n	$n.2^n$	1
$4^{\log n}$	$n!$	n^n	

[Voltar para o índice](#)

Ex: Classifique as funções de complexidade a seguir por ordem de crescimento assintótico, isto é, encontre a ordem relativa entre as mesmas.

$20n + 3$	2^n	$n.\log n$	$4.n^2$
$\ln n$	e^n	$n.2^n$	1
$4^{\log n}$	$n!$	n^n	

[Voltar para o índice](#)

$$1 < \ln n < 20n + 3 < n \log n < 4n^2 = 4^{\log n} < 2^n < n.2^n < e^n < n! < n^n$$

Algumas propriedades de logaritmos e potências

- $\log_n 1 = 0$ (log de 1 em qualquer base é 0)
- $\log_a n = \log_b n \cdot \log_a b = \log_b n / \log_b a$ (troca de base)
 - Ex: $\log_4 n = \log_2 n \cdot \log_4 2 = \frac{\log_2 n}{2}$
- $\log_n a \cdot b = \log_n a + \log_n b$ (log de produto)
- $\log_n \frac{a}{b} = \log_n a - \log_n b$ (log do quociente)
- $\log_a n^b = b \cdot \log_a n$ (log da potência)
- $n^{\log_a b} = b^{\log_a n}$
- $\log_a b = \frac{1}{\log_b a}$
- $((a^b)^c) = ((a^c)^b) = a^{bc}$
- $a^b a^c = a^{b+c}$

Progressões Aritméticas

- Uma progressão aritmética (abreviadamente, P. A.) é uma sequência numérica em que cada termo, a partir do segundo, é igual à soma do termo anterior com uma constante r . O número r é chamado de razão ou diferença comum da progressão aritmética.
- Progressões aritméticas ocorrem com frequência na análise de complexidade de algoritmos.
- Alguns exemplos de progressões aritméticas:
 - 1, 4, 7, 10, 13, ..., é uma progressão aritmética em que a razão (a diferença entre os números consecutivos) é igual a 3.
 - -2, -4, -6, -8, -10, ..., é uma P.A. em que $r = -2$.
 - 6, 6, 6, 6, 6, ..., é uma P.A. com $r = 0$.
- Soma dos termos de uma progressão aritmética: $\frac{a_1 + a_n}{2} * n$
- N-ésimo termo de uma P.A.: $a_1 + r(n - 1)$

(POSCOMP 2010 - Questão 20 - Matemática) Qual expressão matemática a seguir gera o n -ésimo termo da sequência $8+13+18+23+28+33+\dots$?

a) $5n^2 + 3n$

b) $3 + 5n$

c) $5\left(\frac{n^2+n}{2}\right) + 3n$

d) $8n + 5$

e) $2,5n^2 + 5,5n^2$

Progressões Geométricas

- Uma progressão geométrica (abreviadamente, P. G.) é uma sequência numérica em que cada termo, a partir do segundo, é igual ao produto do termo anterior por uma constante **q**. O número **q** é chamado de **razão** da progressão Geométrica.
- Progressões geométricas também ocorrem com frequência na análise de complexidade de algoritmos.
- Alguns exemplos de progressões geométricas:
 - 1, 2, 4, 8, 16, ..., é uma progressão geométrica em que a razão (a razão entre termos consecutivos) é igual a 2.
 - 1, 0.5, 0.25, 0.125, ..., é uma progressão geométrica em que a razão é igual a 0.5.
- N-ésimo termo de uma P.G.: $a_1 * q^{n-1}$

Soma dos termos de uma progressão geométrica:

- $S_n = a_1 + a_2 + \dots + a_{n-1} + a_n$
 $q.S_n = q.a_1 + q.a_2 + \dots + q.a_{n-1} + q.a_n = a_2 + a_3 + \dots + a_n + q.a_n$
 $q.S_n - S_n =$
 $a_2 + a_3 + \dots + a_n + q.a_n - (a_1 + a_2 + \dots + a_{n-1} + a_n) = q.a_n - a_1$
 $S_n(q - 1) = q.a_n - a_1 \Rightarrow$

$$S_n = \frac{q.a_n - a_1}{q - 1}$$

p/ P.G. decrescente com número infinito de termos:

- $a_n = 0$
- $S_n = a_1/(1 - q)$

- Em algumas situações é mais fácil identificar o número de termos do que o termo a_n . Nesse caso, substituindo a expressão do termo geral $a_n = a_1 * q^{n-1}$ na expressão de soma da PG:

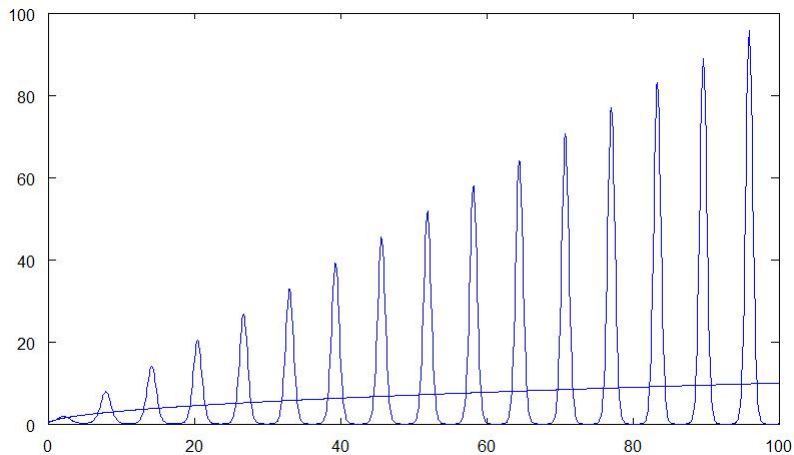
$$S_n = \frac{q * a_1 * q^{n-1} - a_1}{q - 1} =$$

$$S_n = \frac{q^n * a_1 - a_1}{q - 1} =$$

$$S_n = \frac{a_1(q^n - 1)}{q - 1}$$

Visualização de funções no Matlab ou Octave

```
x=linspace(0,100,1000);  
plot(x,x.^sin(x))  
hold on  
plot(x,sqrt(x));
```



Cálculo da complexidade

- A complexidade de um algoritmo é a soma das complexidades de suas partes. A complexidade de uma parte a pode ser absorvida por outra parte b quando
 - $\exists c \exists N \forall n \geq N \text{ complexidade}(a)(n) \leq c \cdot \text{complexidade}(b)(n)$.
 - Ou seja, a função de complexidade de b é "maior" que a de a . Neste caso diz-se que a complexidade de b "absorve" a de a e que portanto
 - $\text{Complexidade}(b) = \text{complexidade}(a) + \text{complexidade}(b)$

Complexidade das estruturas algorítmicas

■ 1. Atribuição : $a \leftarrow b$

- A complexidade depende do tipo de atribuição (matriz, lista, grafo). A complexidade de $a \leftarrow b$ é a complexidade de avaliar b mais a complexidade da atribuição (significativa, por exemplo, se for uma atribuição de matriz)

■ 2. Seqüência : $a; b;$

- A complexidade da seqüência é a soma das complexidades das componentes, isto é, $c(a; b)(n) = c(a)(n) + c(b)(n)$

■ 3. Condicional : **if a then b else c**

- A complexidade do condicional é a complexidade da avaliação de a mais a complexidade do maior entre b e c .

4. Iteração : for $k = i$ to j do a

- A complexidade da iteração tem dois casos a considerar:
- Caso 1: O custo da iteração é sempre o mesmo ao longo das iterações:
 - Nesse caso o custo total pode ser obtido multiplicando-se o número de iterações pelo custo da iteração:
 - No exemplo abaixo, se for considerada como operação fundamental apenas o if, o custo de cada iteração do for é 1 e como ele possui $N-1$ iterações o custo total do for é $(N-1) * 1 = N-1$.

```
for (i=0;i<N-1;i++)  
    if (v[i]>v[i+1]) // $\Theta(1)$   
        {aux=v[i];  
        v[i]=v[i+1];  
        v[i+1]=aux;}
```

■ Caso 1 (continuação):

- O número de iterações, em um laço em que a variável de controle vai de um valor mínimo i até um valor máximo j com incremento de 1, pode ser obtido por: $\text{máximo} - \text{mínimo} + 1$
- Ex: em `for (i=0;i<N;i++)` o maior valor da variável de controle é $N-1$ e o menor valor é 0, logo o número de iterações é $(N-1)-0+1 = N$ iterações
- E o custo da iteração é o custo de todo o bloco dentro da repetição, bloco esse que pode conter outras estruturas de repetição
- $c(\text{for } k = i \text{ to } j \text{ do } a) = (j - i + 1) \cdot c(a)$

■ Caso 1 (continuação):

- Para identificar se o custo de cada iteração é sempre o mesmo ou varia ao longo das iterações basta verificar se a variável de controle do laço está na expressão do custo da iteração. Se a expressão do custo da iteração não contem a variável de controle da iteração, o custo da iteração é sempre o mesmo e basta multiplicar a expressão do custo da iteração pelo número de iterações.
- Se a expressão do custo da iteração depende da variável da iteração, ela se enquadra no segundo caso, visto a seguir:

- Caso 2: O custo da iteração varia ao longo das iterações:
 - Se o custo da iteração varia ao longo das iterações ou, em outras palavras, se o custo da iteração é função da variável da iteração, deve-se calcular o somatório do custo da iteração ao longo das iterações
 - $c(\text{for } k = i \text{ to } j \text{ do } a)(n) = \sum c(a)(t^k(n))$
 - sendo o somatório feito para $i \leq k \leq j$ e t_k é o tamanho da entrada na iteração k .

- Considere o código abaixo, uma versão otimizada do método da bolha:

```
for (j=0;j<N-1;j++)  
    for (i=0;i<N-1-j;i++)  
        if (v[i]>v[i+1]) // $\Theta(1)$   
            {aux=v[i];  
             v[i]=v[i+1];  
             v[i+1]=aux;}
```

- Para se calcular o custo total do for externo (o do j) é necessário saber a expressão do custo do for interno (o do i). Daí se extrai a regra que A ANÁLISE É SEMPRE FEITA DO ESCOPO MAIS INTERNO PARA O MAIS EXTERNO

- Caso 2: (continuação):

```
for (j=0;j<N-1;j++)  
    for (i=0;i<N-1-j;i++)  
        if (v[i]>v[i+1]) //  $\Theta(1)$   
            {aux=v[i];  
              v[i]=v[i+1];  
              v[i+1]=aux;}
```

- No caso acima, para poder calcular o custo do "for j" é necessário ter a expressão do custo total do "for i". Para ter o custo do "for i" é necessário ter o custo do "if", que foi arbitrado como sendo unitário.
- Assim, como a expressão do custo da iteração do for interno ($\Theta(1)$) não depende de i, basta multiplicar o número de iterações (N-1-j) pelo custo da iteração (1) obtendo um custo total de N-1-j

- Caso 2: do custo do for interno calculado acima:

for (j=0;j<N-1;j++)
 $\Theta(N-1-j)$

- Como a expressão do custo da iteração do "for j" é função de j (N-1-j), não se pode simplesmente multiplicar o número de iterações pelo custo da iteração. O custo total de todas as iterações deve ser representado e resolvido como o somatório abaixo:

$$\sum_{j=0}^{N-2} N - 1 - j$$

Algumas propriedades importantes de somatórios são:

- Propriedade Aditiva

$$\sum_{i=m}^n (a_i + b_i) = \sum_{i=m}^n a_i + \sum_{i=m}^n b_i$$

- Propriedade Homogênea

$$\sum_{i=m}^n c(a_i) = c \sum_{i=m}^n a_i$$

- Somatório de constantes. Seja cte uma expressão que independe da variável do somatório:

$$\sum_{i=m}^n cte = cte * (n - m + 1)$$

- O somatório mais comum nesse tipo de cálculo é o somatório abaixo, que se expandido gera a P.A. $m, m+1, m+2, \dots, n-1, n$ cujo somatório é:

$$\sum_{i=m}^n i = (n + m)/2 * (n - m + 1)$$

- O número de termos de um somatório também pode ser calculado a partir dos limites da variável do somatório (último valor - primeiro valor + 1)

■ Somatório dos quadrados

$$\sum_{i=1}^n i^2 = \frac{n^3}{3} + \frac{n^2}{2} + \frac{n}{6} = \frac{2n^3+3n^2+n}{6} = \frac{n(2n^2+3n+1)}{6} = \frac{n(2n+1)(n+1)}{6}$$

■ Somatório dos cubos

$$\sum_{i=1}^n i^3 = \frac{n^4}{4} + \frac{n^3}{2} + \frac{n^2}{4} = \frac{n^4+2n^3+n^2}{4} = \frac{n^2(n^2+2n+1)}{4} = \frac{n^2(n+1)(n+1)}{4}$$

Demonstrações aqui

- Resolva os somatórios a seguir:

$$\sum_{i=1}^n 3i^2 + 2i + n$$

$$\sum_{i=1}^n 3i^2 + 2ni + 3$$

$$\sum_{i=1}^n ni + i + n \text{ (resolvido a seguir)}$$

$$\sum_{i=1}^n ni + i + n$$

- Podemos separar em 3 somatórios utilizando a propriedade aditiva:

$$\sum_{i=1}^n ni + \sum_{i=1}^n i + \sum_{i=1}^n n$$

- em $\sum_{i=1}^n ni$, n é constante em relação a i e pode ser jogado para fora do somatório resultando $n \sum_{i=1}^n i$
- $\sum_{i=1}^n n$ é um somatório de constantes e resulta em n^2
- $\sum_{i=1}^n i$ é uma P.A. e sua soma resulta em $\frac{(1+n)}{2} n$
- E a soma final é $n^2 \frac{(n+1)}{2} + n \frac{(n+1)}{2} + n^2$

Ex: Calcule o número exato de comparações nos dois trechos de um bubblesort a seguir, sendo N a ordem do vetor. Dê a ordem de complexidade de ambas.

```
for (i = 0 ; i < N ; i ++)  
  for (j = 0 ; j < N-1; j ++)  
    if (a[j]>a[j+1])  
      {aux=a[j];  
       a[j]=a[j+1];  
       a[j+1]=aux;}
```

```
for (i = N-1 ; i > 0 ; i -)
    for (j = 0 ; j < i; j ++)
        if (a[j]>a[j+1])
            {aux=a[j];
              a[j]=a[j+1];
              a[j+1]=aux;}
```

- No primeiro caso o número de repetições do laço interno é $N - 1$.
- Como o número de repetições do laço externo é N , o número de comparações é dado por $N * (N - 1)$.
- No segundo caso, o número de repetições do laço interno é i .
- Como i varia no laço externo, a cada execução do laço interno o número de repetições varia.
- O número de comparações é dado pelo somatório, para i variando de 0 a $N - 2$, de i , ou seja, $(N - 1) + (N - 2) + \dots + 1$, que é dada por (soma dos termos de uma progressão aritmética) $((a_1 + a_n)/2) * n$.

Resolução de somatórios no Wolfram Alpha

<http://www.wolframalpha.com/input>



sum $3n^2 + 2n + 6$, $n=1$ to m



Examples Random

Assuming "m" is a variable | Use as a unit instead

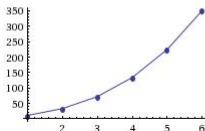
Sum:

$$\sum_{n=1}^m (3n^2 + 2n + 6) = \frac{1}{2} m (2m^2 + 5m + 15)$$

Partial sums:

More terms

Show points



Voltar para o índice

Resolução de somatórios no Wolfram Alpha

Alternate forms:

$$m^3 + \frac{5m^2}{2} + \frac{15m}{2}$$

$$\frac{1}{2} m (m (2m + 5) + 15)$$

$$m \left(m \left(m + \frac{5}{2} \right) + \frac{15}{2} \right)$$

<http://www.wolframalpha.com/input>

Voltar para o índice

Somatórios aninhados no Wolfram Alpha

sum sum sum 1, i=1 to j, j=1 to k, k=1 to N



Extended Keyboard

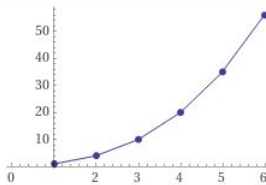


Upload

Sum:

$$\sum_{k=1}^N \left(\sum_{j=1}^k \left(\sum_{i=1}^j 1 \right) \right) = \frac{1}{6} N (N+1) (N+2)$$

Partial sums:



■ 5. Iteração Condicional : **while** a **do** b

- A complexidade do **while** é a complexidade da avaliação da condição *a* mais a complexidade de *b* multiplicado pelo número de execuções (estimativa para o pior caso).

[Voltar para o índice](#)

Exercício : Faça um programa de classificação por inserção e calcule a sua complexidade no melhor e pior caso.

Algoritmo: Ordenação por Inserção.

Entrada: $S[1] \dots S[n]$

Saída: Vetor S ordenado.

```
void Ordena(int * S, int n)
{
    int chave,i,j;
    for(j=1;j<n;++j)
    {
        chave = S[j];
        i = j - 1;
        while(i ≥ 0 && S[i]>chave)
        {
            S[i+1] = S[i];
            --i;
        }
        S[i+1] = chave;
    }
}
```

Complexidade:

$$\sum_{j=1}^{N-1} t(j)$$

onde $t(j)$ é o custo da iteração j do laço externo. No melhor caso (while sempre falha) $t(j) = 1$.

$$\sum_{j=1}^{N-1} 1 = \frac{1+1}{2}(N-1) = N$$

Complexidade do pior caso: (while falha em $i > 0$)

Apenas $t(j)$ muda. $t(j) = j$, para $j := 1, \dots, n - 1$

$$\sum_{j=1}^{n-1} j = \frac{1 + n - 1}{2}(n - 1) = \frac{n(n - 1)}{2}$$

Exercícios

Calcule a complexidade dos algoritmos abaixo:

```
para i de 1 ate n faca
  para j de i+1 ate n faca
    se  $m[i,1] > m[j,1]$  entao
      para k de 1 ate n faca
         $aux := m[i,k] \leftarrow op. fund.$ 
         $m[i,k] := m[j,k] \leftarrow op. fund.$ 
         $m[j,k] := m[i,k] \leftarrow op. fund.$ 
      fimpara
    fimse
  fimpara
fimpara
```

Considerando um vetor qualquer vet, com n elementos.

```
maior ← 0
para i de 1 ate n faca
    cont ← 1
    para j de i+1 ate n faca
        se vet[i]=vet[j] entao cont ← cont + 1 fimse
    fimpara
    se cont>maior entao
        maior ← cont
        posmaior ← i
    fimse
fimpara
escreva(vet[posmaior])
```


Exercícios

1) Calcule a complexidade dos algoritmos a seguir:

Algoritmo 1

```
soma=0;
for (i=1;i<=n;i++)
    for (j=1;j<=n;j++)
        for (k=1;k<=n;k++)
            soma+=k;
```

Algoritmo 2

```
soma=0;
for (i=1;i<=n;i++)
    if (v[i]%2==0)
        for (j=1;j<=i;j++) soma+=j;
    else
        for (j=1;j<=2*i;j++) soma+=j;
```

Algoritmo Qualquer III

```
soma=0;  
for (i=1;i<=n;i++)  
    for (j=1;j<=n;j++)  
        for (k=1;k<=j;k++)  
            soma=soma+k;
```

Algoritmo Qualquer IV

```
soma=0;  
for (i=2;i<=n;i++)  
    soma=soma+i*fun(i);  
/*considere que a função fun(x) é  $O(\log x)$ */
```

Algoritmo Qualquer V

```
soma=0;  
for (i=1;i<=n;i++)  
    for (j=1;j<=i;j++)  
        for (k=1;k<=i;k++)  
            soma=soma+k
```

$$R: \sum_{i=1}^n \sum_{j=1}^i \sum_{k=1}^i 1 = \frac{n^3}{3} + \frac{n^2}{2} + \frac{n}{6}$$

Algoritmo Qualquer VI

```
soma=0;  
for (i=1;i<=n;i++)  
    for (j=1;j<=i;j++)  
        for (k=j;k<=i;k++)  
            soma=soma+k
```

$$R: \sum_{i=1}^n \sum_{j=1}^i \sum_{k=j}^i 1 = \frac{n^3}{6} + \frac{n^2}{2} + \frac{n}{3}$$

- As iterações do laço interno tem um custo constante, então o custo é o número de iterações $(i-j+1)$ multiplicado pelo custo da iteração (1).

$$\sum_{i=1}^n \sum_{j=1}^i (i - j + 1)$$

- o segundo somatório pode ser separado nos termos que dependem da variável do somatório $(i+1)$ e termos que não dependem do somatório (j)

$$\sum_{i=1}^n (\sum_{j=1}^i (i + 1) - \sum_{j=1}^i j)$$

$$\sum_{i=1}^n (i(i + 1) - \frac{i+1}{2} * i)$$

$$\sum_{i=1}^n \frac{i^2+i}{2}$$

$$\frac{1}{2} (\sum_{i=1}^n i^2 + \sum_{i=1}^n i)$$

■ lembrando que $\sum_{i=1}^n i^2 = \frac{2n^3+3n^2+n}{6}$

$$\frac{1}{2} \left(\frac{2n^3+3n^2+n}{6} + \frac{n+1}{2} n \right)$$

$$\frac{1}{2} \left(\frac{2n^3+3n^2+n}{6} + 3 \frac{n^2+n}{6} \right)$$

$$\frac{1}{2} \left(\frac{2n^3+3n^2+n+3n^2+3n}{6} \right)$$

$$\frac{1}{2} \left(\frac{2n^3+6n^2+4n}{6} \right)$$

$$\frac{n^3}{6} + \frac{n^2}{2} + \frac{n}{3}$$

Algoritmo Qualquer VII

```
soma=0;  
for (i=1;i<=n;i++)  
    for (j=1;j<=i;j++)  
        for (k=j;k<=n;k++)  
            soma=soma+k
```

$$R: \sum_{i=1}^n \sum_{j=1}^i \sum_{k=j}^n 1 = \frac{n^3}{3} + \frac{n^2}{2} + \frac{n}{6}$$

- As iterações do laço interno tem um custo constante, então o custo é o número de iterações $(n-j+1)$ multiplicado pelo custo da iteração (1).

$$\sum_{i=1}^n \sum_{j=1}^i (n - j + 1)$$

- o segundo somatório pode ser separado nos termos que dependem da variável do somatório $(n+1)$ e termos que não dependem do somatório (j)

$$\sum_{i=1}^n (\sum_{j=1}^i (n + 1) - \sum_{j=1}^i j)$$

$$\sum_{i=1}^n (i(n + 1) - \frac{i+1}{2} * i)$$

$$\sum_{i=1}^n (ni + i + \frac{i^2+i}{2})$$

$$\frac{1}{2} (\sum_{i=1}^n 2ni + 2i + i^2 + i)$$

$$\frac{1}{2} (\sum_{i=1}^n (2n + 3)i + \sum_{i=1}^n i^2)$$

$$\frac{1}{2} ((2n + 3) \sum_{i=1}^n i + \sum_{i=1}^n i^2)$$

Algoritmo Qualquer VIII

```
i=0;
while (i<n && v[i] != valorprocurado)
    i=i+1;
if (i==n)
    printf(" Não encontrou\n");
else printf(" Encontrou na posicao %d\n",i);
```

R:n (considerando como operação elementar somente o incremento do i dentro do laço)

Ordenação por Inserção

```
for (i=1;i<n;i++)  
    {  
        chave=v[i];  
        j=i-1;  
        while (j>=0 && chave<v[j])  
            {  
                v[j+1]=v[j];  
                j=j-1;  
            }  
        v[j+1]=chave;  
    }
```

```
for (i=1;i<=n;i++)  
    for (j=i+1;j<=n;j++)  
        if (m[i][1] > m[j][1])  
            for (k=1;k<=n;k++)  
                {  
                    aux = m[i][k];  
                    m[i][k] = m[j][k];  
                    m[j][k] = aux;  
                }
```

```
scanf("%d",&n);  
if (n<2) p=0;  
else  
{  
    p=1;  
    i=2;  
    while ((i*i<=n) &&(p==1)) faca  
    {  
        if (n%i==0)  
            p=0;  
        i=i+1;  
    }  
}  
printf("%d",p);
```

```
ini = 1;
fim = N; /* v[1..N] */
repeat
    meio := (ini + fim) div 2;
    if (x > v[meio]) then
        ini = meio + 1
    else
        fim = meio - 1;
until v[meio]=x or ini > fim;
if v[meio]=x then
    writeln("Encontrado posição:", meio)
else
    writeln("Não encontrado")
```

```

{gera todas as combinações das letras do string}
readln(s); /* o primeiro caracter da string fica em s[1] e o último em s[n]*/
n := length(s); /* o tamanho da entrada é o tamanho da string = n */
for i:=1 to n do
    t[i] := 1;
repeat
    for i:=1 to n do
        write(s[t[i]]);
    i := n;
    repeat
        t[i] := t[i] + 1;
        if (t[i]>n) then
            begin
                fim:=0;
                t[i]:=1;
                i:=i-1;
            end
        else
            fim:=1;
        until fim=1 or i=0;
    until i=0;

```


6) Determine a complexidade dos 4 algoritmos abaixo, indicando quais deles resolvem o mesmo problema, escolhendo o mais eficiente entre os que resolvem o mesmo problema, justifique.

Algoritmo 1: encontra1(n:inteiro)

```
1  início
2  |   para  $i$  de 1 ate  $n$  faça
3  |   |   para  $j$  de 1 ate  $i$  faça
4  |   |   |   soma1  $\leftarrow$  0
5  |   |   |   para  $k$  de 1 ate  $j - 1$  faça
6  |   |   |   |   soma1  $\leftarrow$  soma1 +  $k$ 
7  |   |   |   soma2  $\leftarrow$  0
8  |   |   |   para  $k$  de  $j+1$  ate  $i$  faça
9  |   |   |   |   soma2  $\leftarrow$  soma2 +  $k$ 
10 |   |   |   se soma1 = soma2 então
11 |   |   |   |   escreva( $i$ )
12 fim
```

Algoritmo 2: encontra2(n:inteiro)

```
1 início
2   para i de 1 ate n faça
3     ini ← 1
4     fim ← i
5     repita
6       meio ←  $\frac{ini + fim}{2}$ 
7       soma1 ←  $\frac{(1 + meio - 1) * (meio - 1)}{2}$ 
8       soma2 ←  $\frac{(meio + 1 + i) * (i - meio)}{2}$ 
9       se soma1 > soma2 então
10        | fim ← meio
11       senão
12        | ini ← meio
13     até (soma1 = soma2) ou (ini + 1 >= fim) ;
14     se soma1 = soma2 então
15       escreva(i)
16 fim
```

Algoritmo 3: encontra3(n:inteiro)

```
1 início
2   para i de 1 ate n faça
3     para j de 1 ate i faça
4        $soma1 \leftarrow \frac{(1+j-1)*(j-1)}{2}$ 
5        $soma2 \leftarrow \frac{(j+1+i)*(i-j)}{2}$ 
6       se soma1 = soma2 então
7         escreva(i)
8 fim
```

Algoritmo 4: encontra4(n:inteiro)

```
1 início
2   meio ← 1
3   para i de 1 ate n faça
4     repita
5       soma1 ←  $\frac{(1+meio-1)*(meio-1)}{2}$ 
6       soma2 ←  $\frac{(meio+1+i)*(i-meio)}{2}$ 
7       se soma1 < soma2 então
8         meio ← meio + 1
9     até soma1 ≥ soma2 ;
10    se soma1 = soma2 então
11      escreva(i)
12 fim
```

Mergesort

- Outro algoritmo de ordenamento
- Mais eficiente, $O(n \log n)$
- Porém, usa vetor auxiliar (mais memória)

Algoritmo 5: Mergesort(v , $inicio$, fim)

Entrada: Vetor v com elementos a serem ordenados, $inicio$ e fim são índices do início e fim do vetor a considerar

Saída: Vetor v ordenado

```
1 início
2   se ( $fim - inicio + 1$ ) > 1 então
3        $meio \leftarrow \frac{fim + inicio}{2}$  ;                               /* divide  $v$  ao meio */
4       Mergesort( $v$ ,  $inicio$ ,  $meio$ ) ;
5       Mergesort( $v$ ,  $meio + 1$ ,  $fim$ ) ;
6       merge( $v$ ,  $inicio$ ,  $meio + 1$ ,  $fim$ ) ;
7 fim
```

Mergesort

Algoritmo 6: merge($v, inicio, meio, fim$)

Entrada: Vetor v com dois subvetores ordenados, começando em *inicio* e *meio*.

Saída: Subvetor em *v* começando em *inicio* e terminando em *fim*, ordenado.

```

1 início
2   a ← início ;
3   b ← meio ;
4   c ← 1 ;
5   enquanto a < meio ou b ≤ fim faça
6     se a < meio e b ≤ fim então
7       se v[a] ≤ v[b] então
8         aux[c] ← v[a] ;
9         a ← a + 1 ;
10      senão
11        aux[c] ← v[b] ;
12        b ← b + 1 ;
13    senão
14      se a < meio então /* Verifica se sobram elementos na esquerda ou direita */
15        aux[c] ← v[a] ;
16        a ← a + 1 ;
17      senão
18        aux[c] ← v[b] ;
19        b ← b + 1 ;
20  c ← c + 1 ;
21 para c de 1 até tamanho(c) faça /* Copia do auxiliar */
22  v[início + c - 1] ← aux[c];
23 fim

```

- Faça um algoritmo que multiplique duas matrizes quadradas de ordem n . Calcule a complexidade pessimista do mesmo.
- E se no exercício anterior a primeira matriz for $n \times m$ e a segunda $m \times k$?

- Dado dois vetores de tamanhos iguais n , pré-classificados. Faça um algoritmo (merge) que a partir destes dois gere um terceiro vetor também classificado. Calcule sua complexidade.
- E se no exercício anterior os vetores forem de tamanhos diferentes, o primeiro de tamanho n e o segundo m ?
- Faça um algoritmo que procure a primeira ocorrência de uma substring de tamanho m dentro de uma string de tamanho n . Determine sua complexidade.

E quando a variável da iteração não é incrementada de 1?

- Nesse caso o código da função deve ser alterado para que a complexidade da iteração possa ser representada como um somatório.
- Para poder representá-la como um somatório substitui-se a variável de controle por uma função de outra variável que seja incrementada de 1:
 - 1 Se a variável de controle X é incrementada de k a cada iteração, representa-se a variável de controle como $k*Y$ (onde Y é a nova variável de controle)
 - 2 Se a variável de controle X é multiplicada por k a cada iteração, representa-se a variável de controle como k^Y (onde Y é a nova variável de controle)
 - 3 Substitui-se **todas** as ocorrências da variável de controle X (inclusive no incremento e na condição do laço) pela expressão da função

Passo 1 - Representação da variável de controle como função de outra

```
x ← -1
enquanto x ≤ n faça
  para j = 1 até x faça
    soma ← soma + j
  fimpara
  x ← x * 2
fimenquanto
```

$x = 2^y$
 $\Rightarrow$

```
 $2^y \leftarrow 1$      ( $y \leftarrow 0$ )
enquanto  $2^y \leq n$  faça
  para j = 1 até  $2^y$  faça
    soma ← soma + j
  fimpara
   $2^y \leftarrow 2^y * 2$    ( $y \leftarrow y + 1$ )
fimenquanto
```

Passo 2 - Isolamento da variável Y nas atribuições

```
2y ← 1      (y ← 0)
enquanto 2y ≤ n faça
    para j=1 ate 2y faça
        soma ← soma + j
    fimpara
    2y ← 2y * 2  (y ← y+1)
fimenquanto
```

```
y ← 0
enquanto 2y ≤ n faça
    para j=1 ate 2y faça
        soma ← soma + j
    fimpara
    y ← y+1
fimenquanto
```

Passo 3 - Isolamento de Y na condição

```
y ← 0
enquanto 2y ≤ n faça
    para j=1 ate 2y faça
        soma ← soma + j
    fimpara
    y ← y+1
fimenquanto
```

```
y ← 0
enquanto y ≤ log2 n faça
    para j=1 ate 2y faça
        soma ← soma + j
    fimpara
    y ← y+1
fimenquanto
```

$$\text{Complexidade} = \sum_{y=0}^{\log_2 n} 2^y$$

Exercícios

Calcule a complexidade dos algoritmos abaixo:

```
leia n [ para facilitar suponha n potencia de 10 ]  
enquanto n > 0 faca  
    para j=1 to n faca  
        escreva j*n j- considere operacao fundamental  
    fimpara  
    n ← n / 10  
fimenquanto
```

```
leia n —> para facilitar considere n potência de 2
soma <- 0
para i=1 ate n faça
  x<-1
  enquanto x<=n faça
    para j=1 ate x faça
      soma <- soma + j
    fimpara
    x <- x*2
  fimenquanto
fimpara
escreva soma
```

```
leia n
i <- 1
j <- 1
enquanto i<=n faca
  escreva i
  se j=1 entao
    i <- i+1
    j <- i
  senao
    j <- j-1
  fimse
fimenquanto
```



```
leia n
para i=1 ate n faca
  j <- 2
  enquanto j<=2*n faca
    para k=1 ate j faca
      escreva i+j*k <- op. fund.
    fim para
    j <- j + 2
  fim enquanto
fimpara
```

```
leia n <— para facilitar considere n potencia de 2
soma <- 0
para i de 1 ate n faca
  xj-1
  enquanto x<=n faca
    para j de 1 ate x faca
      soma <- soma + j
    fimpara
    x <- x*2
  fimenquanto
fimpara
escreva soma
```

Análise da Complexidade de Algoritmos Recursivos

- A análise da complexidade de programas recursivos é mais complexa que a análise de programas seqüenciais.
- Em algumas situações, particularmente quando há recursão de cauda (situação em que a última operação da função é a chamada recursiva) o programa pode ser convertido em um programa seqüencial e a análise se torna seqüencial.

[Voltar para o índice](#)

- Um exemplo típico é uma rotina recursiva de fatorial que pode ser convertida em um laço para o qual a complexidade é facilmente calculável.
- Quando isso não é possível, sua complexidade pode ser definida em termos de uma função de recorrência, ou seja, uma expressão recursiva de complexidade.
- Por exemplo, no exemplo do fatorial a seguir:

[Voltar para o índice](#)

```
int fat(int n)
{
    if (n<=1)
        return 1;
    else
        return n*fat(n-1);
}
```

escolhido o produto como operação básica, o número de produtos é dado por:

$$T(n) = \begin{cases} 0 & p/n \leq 1 \\ 1 + T(n-1) & p/n > 1 \end{cases}$$

Essa expressão é chamada uma equação de recorrência e, para grande parte dos casos, pode ser resolvida de três formas:

- 1 No **método de substituição**, estima-se uma ordem de complexidade e utiliza-se indução matemática para demonstrar que a estimativa é correta.
- 2 No **método de iteração** converte-se a recorrência em uma soma e busca-se uma expressão para representar a soma.
- 3 O **Teorema Mestre** (Master Theorem) provê soluções para recorrências na forma $T(n) = aT(n/b) + f(n)$, onde $a \geq 1$, $b > 1$ e $f(n)$ é uma função qualquer.

Resolução de recorrências no Wolfram Alpha



`a[1]==0;a[n]==4*a[n/2]+n^2;`



Extended Keyboard



Upload

Input:

$$a(1) = 0 \quad | \quad a(n) = 4a\left(\frac{n}{2}\right) + n^2$$

Recurrence equation solution:

$$a(n) = \frac{n^2 \log(n)}{\log(2)}$$

- Nas expressões resultantes no Wolfram Alpha, os logaritmos são expressos na base 10, mas pela propriedade de mudança de base:

$$\frac{\log_{10} n}{\log_{10} 2} = \log_2 n$$

- e a expressão

$$\frac{n^2 \log(n)}{\log(2)} = n^2 \log_2(n)$$

- Na expressão acima, onde a base não é expressa, é assumida base 10.

Exercícios

Extraia a equação de recorrência dos algoritmos a seguir:

```
procedimento p1(n:inteiro positivo; a:inteiro positivo)
se n>1 entao
  para i=1 ate a faca p1(n/a) fimpara
  para i=1 ate n faca
    escreva i*a <- considere operacao fundamental
  fimpara
fimse
fimprocedimento
```

[Voltar para o índice](#)

Exercícios

Extraia a equação de recorrência dos algoritmos a seguir:

```
procedimento p2(n:inteiro positivo)
se  $n > 1$  entao
  p2( $n/2$ )
  para  $i=1$  ate  $n$  faca
    para  $j=1$  ate  $n$  faca
      escreva  $i*j*n$  <- considere operacao fundamental
    fimpara
  fimpara
fimse
fimprocedimento
```

[Voltar para o índice](#)

```
procedimento proc(n:inteiro positivo)
se n > 1 entao
  proc(n/8)
  para i=1 ate n faca
    escreva i*n <- op. fund.
  fimpara
  proc(n/8)
fimse
fimprocedimento
```

[Voltar para o índice](#)

```
procedimento p2(n:inteiro positivo)
se  $n > 1$  entao
    para  $i=1$  ate 9 faca p2( $n/3$ ) fimpara
    para  $i=1$  ate  $n$  faca
        escreva  $i*a$  <- op. fund.
    fimpara
fimse
fimprocedimento
```

[Voltar para o índice](#)

1. O método da substituição

- O método da substituição baseia-se em tentar identificar a forma da solução a partir da equação de recorrência e soluções conhecidas de recorrências semelhantes, encontrar os coeficientes de cada termo da solução e utilizar indução matemática para provar que a solução é equivalente à recorrência.
- A identificação dos termos da solução é facilitada pelo fato de, para grande parte das recorrências, a solução é da forma n , $n \log n$ ou $\log n$.

Exemplo: Achar a solução da equação da recorrência da função fatorial(n), dada acima.

$$T(n) = \begin{cases} 0 & p/n = 1 \\ 1 + T(n-1) & p/n > 1 \end{cases}$$

Primeiro passo: Estimar a forma da solução:

Da complexidade da versão seqüencial do fatorial, estima-se que a expressão da complexidade seja uma função linear com um termo linear e outro constante no formato

$$T(n) = c.n + d$$

Segundo passo: Utilizando-se a recorrência, calcula-se o valor de $T(n)$ para os primeiros valores de n (uma quantidade de valores de n igual à quantidade de coeficientes a serem calculados)

- $T(1)=0$
- $T(2)=1+T(1)=1$
- Assim, $T(1)=c+d$ e $T(2)=2c+d$.
- Como $T(1) = 0$ (aplicando-se diretamente a recorrência), tem-se que $c + d = 0$.
- Para $n = 2$ (pela recorrência), $T(2) = 1 + T(1) = 1$, tem-se que $2c + d = 1$ resultante o sistema a seguir:

Terceiro passo: Substituir os mesmos valores de n do passo anterior no modelo de solução, obtendo um sistema de equações:

- Assim, $T(1)=c+d$ e $T(2)=2c+d$.
- Como $T(1) = 0$ (aplicando-se diretamente a recorrência), tem-se que $c + d = 0$.
- Para $n = 2$ (pela recorrência), $T(2) = 1 + T(1) = 1$, tem-se que $2c + d = 1$ resultante o sistema a seguir:

$$c + d = 0$$

$$2c + d = 1$$

- que pode ser resolvido subtraindo a primeira equação da segunda que resulta $c=1$, e substituindo c na primeira equação que resulta em $d = -1$
- assim, substituindo c e d em $T(n) = cn + d$ resulta:

$$T(n) = n - 1$$

Solução de sistemas de equações lineares no Matlab

O sistema

$$c + d = 0$$

$$2c + d = 1$$

pode ser representado matricialmente por:

$$\begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} c \\ d \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

[Voltar para o índice](#)

- Chamando de **M** a matriz de coeficientes, **x** o vetor com as variáveis e **b** o vetor de termos independentes, o sistema pode ser representado por $Mx = b$ e pode-se encontrar o vetor **x** da forma a seguir:

$$M = [1 \ 1; 2 \ 1]$$

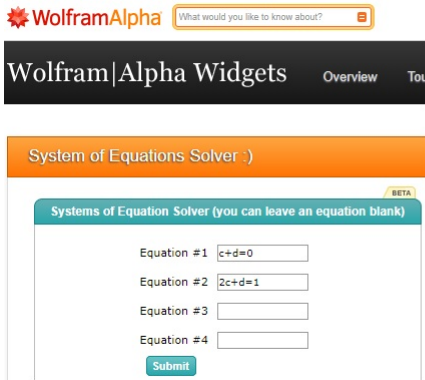
$$b = [0 \ 1]'$$

$$x = M \backslash b$$

[Voltar para o índice](#)

Solução de sistemas de equações lineares no Wolfram

- Procure no Google: Wolfram Systems of Equations Solver



The screenshot shows the Wolfram|Alpha Widgets interface. At the top, there is a search bar with the text "What would you like to know about?". Below this, the "Wolfram|Alpha Widgets" header is visible, with links for "Overview" and "Tou". The main content area features an orange banner titled "System of Equations Solver :)", followed by a teal header for the "Systems of Equation Solver (you can leave an equation blank)" widget, which also includes a "BETA" label. The widget contains four input fields for equations: "Equation #1" with the value $c+d=0$, "Equation #2" with the value $2c+d=1$, "Equation #3" which is empty, and "Equation #4" which is empty. A teal "Submit" button is located at the bottom of the widget.

WolframAlpha What would you like to know about?

Wolfram|Alpha Widgets Overview Tou

System of Equations Solver :)

BETA

Systems of Equation Solver (you can leave an equation blank)

Equation #1

Equation #2

Equation #3

Equation #4

Submit

O que acontece se a candidata a solução não tiver todos os termos da solução correta?

- Considere a recorrência a seguir:

$$T(n) = \begin{cases} 1 & p/n = 1 \\ 2 + T(n/2) & p/n > 1 \end{cases}$$

- E suponha que foi escolhida como solução candidata a função:
 $T(n) = an + b$
- Pela recorrência:
 - $T(1)=1$
 - $T(2)=2+T(1)=2+1=3$
- gerando o sistema
 - Eq.1) $a+b=1$
 - Eq.2) $2a+b=3$

- A solução do sistema pode ser obtida subtraindo a Eq.1 da Eq.2 resultando $a=2$, e substituindo esse valor na Eq.1 resulta $b=-1$, logo a solução é:
 - $T(n)=2n-1$
- que resulta nos valores:
 - $T(1)=2.(1)-1=1$
 - $T(2)=2.(2)-1=3$
- ou seja, para $n=1$ e $n=2$ a expressão obtida é equivalente à recorrência. Entretanto, para $n=4$, a recorrência resulta:
 - $T(4)= 2+T(2) = 2+3 = 5$
- E a expressão obtida resulta em
 - $T(4)=2.(4)-1 = 7$
- ou seja, para $n=4$ a expressão obtida já não é equivalente à recorrência

Prova por indução matemática da correção da solução:

Uma prova por indução matemática baseia-se em mostrar que se a solução é válida para um valor n , então ela também vale para um valor $n + 1$. Isso provado, se houver algum valor para o qual a solução é válida, ela será válida para qualquer valor maior que ele. A prova é formada de três partes:

- 1 **Base da indução:** É o valor para o qual sabe-se que a solução é válida. No caso, sabe-se que $T(1) = 0$.
- 2 **Hipótese da indução:** Assume-se que a solução é válida para todos os valores de n até um valor k .
- 3 **Passo de indução:** Mostra-se que a solução é válida também para $k + 1$.

Ex: Mostre que:

$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

Base da indução: A expressão é válida para $n = 1$.

Como **hipótese de indução**, assume-se que a expressão vale até n .

Passo de indução:

$$\sum_{i=1}^{n+1} i^2 = \sum_{i=1}^n i^2 + (n+1)^2$$

$$\frac{(n \cdot (n+1) \cdot (2n+1))}{6} + (n+1)^2 = (n+1) \left(\frac{n \cdot (2n+1)}{6} + (n+1) \right) =$$

$$(n+1) \frac{(2n^2 + 7n + 6)}{6} = \frac{(n+1)(n+2)(2n+3)}{6}$$

Assim,

$$\sum_{i=1}^{n+1} = \frac{(n+1)[(n+1)+1] \cdot [2(n+1)+1]}{6}$$

que é a expressão esperada.

Exercícios: Mostre, utilizando indução matemática, que os seguintes somatórios estão corretos:

1)

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

$$\sum_{i=1}^{n+1} i = \sum_{i=1}^n i + n + 1 = \frac{n(n+1)}{2} + n + 1 =$$

$$\frac{(n(n+1) + 2n + 2)}{2} = \frac{n^2 + 3n + 2}{2} = \frac{(n+2)(n+1)}{2}$$

que é a forma fechada esperada.

2)

$$\sum_{i=0}^n 2^i = 2^{n+1} - 1$$

$$\sum_{i=0}^{n+1} 2^i = \sum_{i=0}^n 2^i + 2^{n+1} = 2^{n+1} - 1 + 2^{n+1} =$$

$2^{n+2} - 1$ que é a forma fechada esperada.

[Voltar para o índice](#)

3)

$$\sum_{i=0, m \neq 1}^n m^i = \frac{m^{n+1} - 1}{m - 1}$$

$$\sum_{i=0}^{n+1} m^i = \sum_{i=0}^n m^i + m^{n+1} =$$

$$\frac{m^{n+1} - 1}{m - 1} + m^{n+1} = \frac{m^{n+1} - 1 + m^{n+2} - m^{n+1}}{m - 1} =$$

$$\frac{m^{n+2} - 1}{m - 1}$$

que é a forma fechada esperada

Resolva pelo método da substituição as recorrências a seguir, e comprove por indução que a solução encontrada está correta:

$$T(n) = \begin{cases} 1 & p/n = 1 \\ 2 + T(n/2) & p/n > 1 \end{cases}$$

$$T(n) = \begin{cases} 1 & p/n = 1 \\ n + T(n/2) & p/n > 1 \end{cases}$$

[Voltar para o índice](#)

Ex. Mostre por indução matemática que $T(n) = n - 1$ é solução para a recorrência do fatorial do problema anterior.

$$T(n) = \begin{cases} 0 & p/n = 1 \\ 1 + T(n - 1) & p/n > 1 \end{cases}$$

Queremos mostrar que, se $n - 1$ é solução de $T(n)$, então $(n + 1) - 1$ (ou seja, n) é solução de $T(n + 1)$.

Base de Indução: $p/n = 1$ e $n = 2$ a expressão é válida

Hipótese de Indução: vale para qualquer valor até n .

Passo de indução:

$$T(n + 1) = 1 + T((n + 1) - 1) = 1 + T(n) = 1 + n - 1 = n$$

Ex. Faça um algoritmo que encontre o maior e o menor elementos de um vetor de N elementos. Calcule a complexidade do algoritmo considerando como operação elementar a comparação entre dois elementos.

Solução 1:

Seqüencial, com $2N$ comparações.

```
void maxmin ()
{
    MAX = MIN =V[1];
    for ( i=2;i<=N; i++)
    {
        if (V[i]>MAX) MAX=V[i];
        if (V[i]<MIN) MIN=V[i];
    }
}
```

Solução 2:

Divisão e Conquista. Divide em duas metades, chama recursivamente para cada metade e compara os máximos e mínimos de cada metade.

```
Procedure MAXMIN (S) // retorna par (max,min)
If  || S || = 2 then // suponha S={a,b}
{
  if (a>b) return (a,b)
  else return (b,a)
}
else
{
  divide S em S1 e S2;
  (max1,min1) = MAXMIN(S1)
  (max2,min2) = MAXMIN(S2)
  return (MAX(max1,max2), MIN(min1,min2))
}
```

[Voltar para o índice](#)

equação de recorrência:

$$T(n) = \begin{cases} 1 & p/n = 2 \\ 2.T(n/2) + 2 & p/n > 2 \end{cases}$$

Mostrar por indução para n potência de dois, que a solução para a recorrência é $T(n) = cn + b$, e encontrar os coeficientes.

$$p/n = 2 \quad 2c + b = 1 \Rightarrow b = 1 - 2c$$

$$p/n = 4 \quad 4c + b = 4 \Rightarrow 4c + 1 - 2c = 4 \Rightarrow c = \frac{3}{2}, b = -2$$

$$\text{logo, } T(n) = (3/2)n - 2$$

para qualquer potência de 2:

$$\text{Devemos achar que } T(2n) = (3/2)2n - 2 = 3n - 2$$

A partir da equação de recorrência...

$$T(2n) = 2.T(n) + 2 = 2.((3/2)n - 2) + 2 = 3n - 4 + 2 = 3n - 2,$$

Modifique o algoritmo anterior de modo que a recursão vá até $|||S||| = 1$. Qual o número de comparações resultante?

```
Procedure MAXMIN2 (S) // retorna par (max, min)
If    || S || = 1 then // suponha  $S=\{a\}$ 
return (a, a)
else
{
divide S em S1 e S2;
(max1, min1) = MAXMIN2(S1)
(max2, min2) = MAXMIN2(S2)
return (MAX(max1, max2), MIN(min1, min2))
}
```

■ recorrência:

$$T(n) = \begin{cases} 0 & p/n = 1 \\ 2 \cdot T(n/2) + 2 & p/n > 2 \end{cases}$$

$$p/n = 1 \quad T(n) = 0$$

$$p/n = 2 \quad T(n) = 2$$

$$p/n = 4 \quad T(n) = 6$$

$$p/n = 8 \quad T(n) = 14...$$

forma da solução fechada: $a.n + b$

$$p/ \ n = 1 \ a + b = 0 \Rightarrow a = -b$$

$$p/ \ n = 2 \ 2.a + b = 2 \Rightarrow a = 2 \text{ e } b = -2$$

solução : $2n - 2$

Comprovacao por indução:

$$T(2n) = 2.T(n) + 2 = 2.(2n - 2) + 2 = 4n - 4 + 2 = 4n - 2 ,$$

como queremos achar.

Solução 3 :

- Comparar cada elemento de posição ímpar com o elemento de posição par imediatamente após ele (primeiro com segundo, terceiro com o quarto, etc.) gerando dois vetores de tamanho $N/2$ cada um.
- O primeiro contém os maiores elementos resultantes de cada comparação e o segundo contém os menores elementos resultantes de cada comparação (custo : $N/2$ comparações).
- Buscar no primeiro vetor o maior de todos ($N/2$ comparações) e no segundo vetor o menor de todos ($N/2$ comparações).
- Total: $3/2$ de N comparações.

2. O método da iteração (ou expansão)

- A idéia desse método é expandir a recorrência e expressá-la como uma soma de termos dependente apenas de n e das condições iniciais.
- Normalmente a série de termos recai em uma P.A., uma P.G, uma soma de termos constantes ou uma combinação dos três.

Caso 1 - Somatório de termos constantes

$$T(n) = \begin{cases} 0 & p/n = 1 \\ T(n/2) + 1 & p/n \geq 2 \end{cases}$$

Essa recorrência surge, por exemplo, no método de busca binária, onde a busca do elemento intermediário tem custo constante e o subproblema resultante da divisão é da metade do tamanho do problema original.

- Calcula-se o valor de $T(n)$, a partir da recorrência, para os primeiros valores de n , desde o valor de n no caso não recursivo da recorrência (no caso, $n=1$), e para os valores seguintes, obtidos de modo a , ao aplicar a recorrência sobre n , acabe resultando no valor inicial.
- No exemplo, como na recorrência o valor de n é dividido por 2, calcula-se o próximo n multiplicando por 2.
 - $T(1) = 0$
 - $T(2) = T(1) + 1 = 0 + 1$
 - $T(4) = T(2) + 1 = 0 + 1 + 1$
 - $T(8) = T(4) + 1 = 0 + 1 + 1 + 1$

- O resultado da expansão é uma sequência de valores iguais. Para calcular o somatório de uma sequência de valores iguais, deve-se saber a quantidade de valores, e o valor de cada elemento, ambos em função de n .
 - $T(1) = 0$
 - $T(2) = T(1) + 1 = 0 + 1$
 - $T(4) = T(2) + 1 = 0 + 1 + 1$
 - $T(8) = T(4) + 1 = 0 + 1 + 1 + 1$
- No caso, observa-se que a cada vez que n é multiplicado por 2, a quantidade de elementos é aumentada de 1, o que sugere uma relação logarítmica. No caso, a quantidade de elementos é $\log_2 n$.
- E o valor de cada elemento é constante, independente de n , e é igual a 1. Logo, o valor do somatório é
 - $T(N) = \log_2 N * 1$
 - R: $T(N) = \log_2 N$

Exemplo 1: A recorrência do fatorial

$$T(n) = 1 + T(n-1)p/n > 1$$

$$= 1 + 1 + T(n-2)$$

$$= 1 + 1 + 1 + T(n-3)$$

$$= 1 + 1 + 1 + 1 + \dots + 0 = \text{Quantos 1's?}$$

$$T(1) = 0$$

$$T(2) = 1 + 0$$

$$T(3) = 1 + 1 + 0$$

$$T(n) = n - 1$$

Resolver por iteração a recorrência a seguir:

$$T(n) = \begin{cases} 1 & p/n = 1 \\ 3.T(n/4) + n & p/n > 1 \end{cases}$$

$$T(1) = 1$$

$$T(4) = 3T(1) + 4 = 3 + 4$$

$$T(16) = 3T(4) + 16 = 3(3+4) + 16 = 3^2 + 3.4 + 16$$

$$T(64) = 3T(16) + 64 = 3(3^2 + 3.4 + 16) + 64 = \\ 3^3 + 3^2.4 + 3 * 16 + 64$$

que é uma P.G com $q = \frac{4}{3}$, $a_1 = 3^{\log_4 N}$ e $a_n = N$

- $Soma_{PG} = \frac{qa_n - a_1}{q-1}$
- $Soma_{PG} = \frac{\frac{4}{3}N - 3^{\log_4 N}}{\frac{4}{3} - 1}$
- multiplicando por 3 em cima e em baixo...
- $Soma_{PG} = \frac{4N - 3 \cdot 3^{\log_4 N}}{4 - 3} = 4N - 3 \cdot 3^{\log_4 N}$

Exercícios do método da iteração (ou expansão). Soluções mais adiante:

1) Resolva as relações de recorrência a seguir:

$$1 \quad T(n) = \begin{cases} 0 & p/n = 1 \\ T(n/2) + 1 & p/n \geq 2 \end{cases}$$

$$2 \quad T(n) = \begin{cases} 0 & p/n = 1 \\ T(n/2) + n & p/n \geq 2 \end{cases}$$

$$3 \quad T(n) = \begin{cases} 0 & p/n = 1 \\ 2T(n/2) + 1 & p/n \geq 2 \end{cases}$$

$$4 \quad T(n) = \begin{cases} 0 & p/n = 1 \\ 2T(n/2) + n & p/n \geq 2 \end{cases}$$

$$5 \quad T(n) = \begin{cases} 0 & p/n = 1 \\ 2T(n/2) + n^2 & p/n \geq 2 \end{cases}$$

$$6 \quad T(n) = \begin{cases} 0 & p/n = 1 \\ T(n/2) + 1 + n & p/n \geq 2 \end{cases}$$

$$7 \quad T(n) = \begin{cases} 0 & p/n = 1 \\ T(n/2) + bn + c & p/n \geq 2 \end{cases}$$

$$8 \quad T(n) = \begin{cases} 0 & p/n = 1 \\ 4T(n/2) + n & p/n \geq 2 \end{cases}$$

$$9 \quad T(n) = \begin{cases} 0 & p/n = 1 \\ 2T(n/4) + 1 & p/n \geq 2 \end{cases}$$

$$10 \quad T(n) = \begin{cases} 0 & p/n = 1 \\ 4T(n/2) + n^2 & p/n \geq 2 \end{cases}$$

a)

$$T(n) = \begin{cases} 0 & p/n = 1 \\ T(n/2) + 1 & p/n \geq 2 \end{cases}$$

Essa recorrência surge, por exemplo, no método de busca binária, onde a busca do elemento intermediário tem custo constante e o subproblema resultante da divisão é da metade do tamanho do problema original.

$$T(1) = 0$$

$$T(2) = T(1) + 1 = 0 + 1$$

$$T(4) = T(2) + 1 = 0 + 1 + 1$$

$$T(8) = T(4) + 1 = 0 + 1 + 1 + 1$$

$$T(N) = \log_2 N * 1$$

$$R: T(N) = \log_2 N$$

b)

$$T(n) = \begin{cases} 0 & p/n = 1 \\ T(n/2) + n & p/n \geq 2 \end{cases}$$

$$T(1) = 0$$

$$T(2) = T(1) + 2 = 0 + 2$$

$$T(4) = T(2) + 4 = 0 + 2 + 4$$

$$T(8) = T(4) + 8 = 0 + 2 + 4 + 8$$

...

$$\sum_{i=1}^k 2^i =$$

P.G. com $A_1 = 2$, $A_n = N$ e $q = 2$.

$$R: T(n) = 2.n - 2$$

c)

$$T(n) = \begin{cases} 1 & p/n = 1 \\ T(n/2) + n & p/n \geq 2 \end{cases}$$

$$T(1) = 1$$

$$T(2) = T(1) + 2 = 1 + 2$$

$$T(4) = T(2) + 4 = 1 + 2 + 4$$

$$T(8) = T(4) + 8 = 1 + 2 + 4 + 8$$

...

$$\sum_{i=0}^k 2^i =$$

P.G. com $A_1 = 1$, $A_n = N$ e $q = 2$.

R: $T(n) = 2.n - 1$

d)

$$T(n) = \begin{cases} 0 & p/n = 1 \\ 2.T(n/2) + n & p/n \geq 2 \end{cases}$$

$$T(1) = 0$$

$$T(2) = 2.T(1) + 2 = 2$$

$$T(4) = 2.T(2) + 4 = 2.2 + 4 = 4 + 4$$

$$T(8) = 2.T(4) + 8 = 2(4 + 4) + 8 = 8 + 8 + 8$$

$$R: T(N) = N \log N$$

e)

$$T(n) = \begin{cases} 0 & p/n = 1 \\ 2.T(n/4) + n & p/n \geq 2 \end{cases}$$

$$T(1) = 0$$

$$T(4) = 2.T(1) + 4 = 4$$

$$T(16) = 2.T(4) + 16 = 2.4 + 16 = 8 + 16$$

$$T(64) = 2.T(16) + 64 = 2(8 + 16) + 64 = 16 + 32 + 64$$

$$T(256) = 2.T(64) + 256 = 2(16 + 32 + 64) + 256 = \\ 32 + 64 + 128 + 256$$

P.G. com $q = 2$, $A_n = N$ e número de termos é $\log_4 N$

$$\begin{aligned} A_1 &= \frac{A_n}{q^{n-1}} = \frac{N}{2^{n-1}} = \frac{N}{2^{\log_4 N - 1}} = \frac{N}{2^{\log_4 N} 2^{-1}} = \\ &= \frac{2N}{2^{\log_4 N}} = \frac{2N}{N^{\log_4 2}} = \frac{2N}{N^{\frac{1}{2}}} = \frac{2N}{\sqrt{N}} = 2\sqrt{N} \end{aligned}$$

$$S_{P.G.} = \frac{q A_n - A_1}{q - 1} = \frac{2N - 2\sqrt{N}}{2 - 1} = 2N - 2\sqrt{N}$$

Exercícios

Extraia as equações de recorrência dos algoritmos a seguir e resolva-as pelo método da expansão. Demonstre a correção da solução por indução matemática:

```
procedimento proc(n:inteiropositivo)
se n>1 entao
  para i=1 ate 8 faca proc(n/2) fimpara
  para i=1 ate n faca
    para j=1 ate n faca
      escreva i*j*n <— op. fund.
    fimpara
  fimpara
fimse
fimprocedimento
```

```
procedimento p1(n:inteiro positivo)
se  $n > 1$  entao
    p1( $n/4$ )
    p1( $n/4$ )
    para  $i=1$  ate raizquadrada( $n$ ) faca
        escreva  $i*n$  <- op. fund.
    fimpara
fimse
fimprocedimento
```

Resolução do Wolfram Alpha

$RSolve[\{a[n] == 2a[n/4] + n^{(1/2)}, a[1] == 0, a[4] == 2\}, a[n], n]$



$a[1]==0;a[n] == 4*a[n/2]+ n^2;$



Extended Keyboard



Upload

Input:

$$a(1) = 0 \quad | \quad a(n) = 4 a\left(\frac{n}{2}\right) + n^2$$


```
procedimento p2(n:inteiro positivo)
se n>1 entao
  para i=1 ate 27 faca p2(n/3) fimpara
  para i=1 ate n faca
    para j=1 ate n faca
      escreva i*j*a <- op. fund.
    fimpara
  fimpara
fimse
fimprocedimento
```

```
procedimento p1(n:inteiro positivo)
se n>1 entao
    p1(n/2)
    p1(n/2)
    p1(n/2)
    p1(n/2)
    para i=1 ate n*n faca
        soma <- soma + i <- op. fund.
    fimpara
fimse
fimprocedimento
```

```
procedimento p1(n:inteiro positivo)
se n>1 entao
    p1(n/4)
    p1(n/4)
    para i de 1 ate n faca
        para j de 1 ate i faca
            escreva j <- op. fund.
        fimpara
    fimpara
fimse
fimprocedimento
```

```
procedimento p2(n:inteiropositivo)
se n>1 entao
  p2(n/2)
  p2(n/2)
  para i=1 ate n faca
    escreva i*n <- op. fund.
  fimpara
fimse
fimprocedimento
```

```
procedimento proc(n,o,d,a)
se  $n > 0$  entao
    proc(n-1,o,a,d)
    m(o,d) // este procedimento tem custo 1
    proc(n-1,a,d,o)
fimse
fimprocedimento
```

3. O Teorema Mestre (Master Theorem)

O **Teorema Mestre** (Master Theorem) provê limites assintóticos para recorrências na forma $T(n) = aT(n/b) + f(n)$, onde $a \geq 1$, $b > 1$ e $f(n)$ é uma função qualquer. A função de recorrência $T(n)$ pode ser limitada assintoticamente da seguinte forma:

- 1 Se $f(n) = O(n^{\log_b a - \epsilon})$ para alguma constante $\epsilon > 0$, então $T(n) = \Theta(n^{\log_b a})$
- 2 Se $f(n) = \Theta(n^{\log_b a})$ então $T(n) = \Theta(n^{\log_b a} \log n)$
- 3 Se $f(n) = \Omega(n^{\log_b a + \epsilon})$ para alguma constante $\epsilon > 0$, e se $a.f(n/b) \leq c.f(n)$ para alguma constante $c < 1$ e para n suficientemente grande, então $T(n) = \Theta(f(n))$

Observações:

- No caso 1, o ϵ no expoente do logaritmo significa que $f(n)$ deve ser polinomialmente menor que $f(n) = n^{\log_b a}$, ou seja, não basta que seja assintoticamente maior (como $n \log n$ é maior que n , mas não polinomialmente). O mesmo ocorre no caso 3.
- Podem ocorrer funções que não recaem em nenhum dos casos descritos acima. Neste caso, o teorema mestre não pode ser utilizado.

- Ex.1: $T(n) = 9T(n/3) + n$
- Para esta recorrência, temos que, $a = 9$, $b = 3$, $f(n) = n$.
- Desta forma, $n^{\log_b a} = n^{\log_3 9} = \Theta(n^2)$.
- Como $f(n) = O(n^{\log_3 9 - \epsilon})$, onde $\epsilon = 1$, podemos aplicar o caso 1 do teorema e concluir que a solução da recorrência é $T(n) = \Theta(n^2)$

- Ex.2: $T(n) = T(2n/3) + 1$
- Para esta recorrência, temos que, $a = 1, b = 3/2, f(n) = 1$.
- Desta forma, $n^{\log_b a} = n^{\log_{3/2} 1} = n^0 = 1$.
- O caso 2 se aplica já que $f(n) = \Theta(n^{\log_b a}) = \Theta(1)$.
- Temos então que a solução da recorrência é $T(n) = \Theta(\log n)$.

- Ex.3: $3T(n/4) + n \log n$
- Para esta recorrência, temos que,
 $a = 3, b = 4, f(n) = n \log n$.
- Desta forma, $n^{\log_b a} = n^{\log_4 3} = O(n^{0.793})$.
- Como $f(n) = \Omega(n^{\log_4 3 + \epsilon})$, onde $\epsilon \approx 0.2$, o caso 3 se aplica se mostrarmos que a condição de regularidade é verdadeira para $f(n)$.
- Para um valor suficientemente grande de n ,
 $a.f(n/b) = 3.(n/4)\log(n/4) \leq (3/4)n \log n = c.f(n)$ para $c = 3/4$.
- Consequentemente, usando o caso 3, a solução para a recorrência é $T(n) = \Theta(n \log n)$.

- Ex.4: $T(n) = 2.T(n/2) + n \log n$
- Neste caso o teorema não se aplica apesar da recorrência ter a forma apropriada : $a = 2, b = 2, f(n) = n \log n$ e $n^{\log_b a} = n$.
- Aparentemente o caso 3 deveria se aplicar já que $f(n) = n \log n$ é assintoticamente maior que $n^{\log_b a} = n$.
- Mas no entanto, não é polinomialmente maior.
- A fração $f(n)/n^{\log_b a} = (n \log n)/n = \log n$ que é assintoticamente menor que n^ϵ para toda constante positiva ϵ .
- Consequentemente, a recorrência cai na situação entre os casos 2 e 3 onde o teorema não pode ser aplicado.

Exercício:

1. Use o Teorema Mestre para resolver as seguintes recorrências:

1 $T(n) = 4.T(n/2) + n$

2 $T(n) = 4.T(n/2) + n^2$

3 $T(n) = 4.T(n/2) + n^3$

(Cormen) Calcule limites superior e inferior para as recorrências a seguir. Assuma que $T(n)$ é constante para $n \leq 2$. Encontre limites tão estreitos quanto possível, e justifique suas respostas.

1 $T(n) = 2T(n/2) + n^3$

2 $T(n) = T(\frac{9n}{10}) + n$

3 $T(n) = 16T(n/4) + n^2$

4 $T(n) = 7T(n/3) + n^2$

5 $T(n) = 7T(n/2) + n^2$

6 $T(n) = 2T(n/4) + \sqrt{n}$

7 $T(n) = T(n-1) + n$

8 $T(n) = T(\sqrt{n}) + 1$

(Cormen) Calcule limites superior e inferior para as recorrências a seguir. Assuma que $T(n)$ é constante para $n \leq 2$. Encontre limites tão estreitos quanto possível, e justifique suas respostas.

1 $T(n) = 3T(n/2) + n \log n$

2 $T(n) = 3T(n/3 + 5) + \frac{n}{2}$

3 $T(n) = 2T(n/2) + \frac{n}{\log n}$

4 $T(n) = T(n-1) + \frac{1}{n}$

5 $T(n) = T(n-1) + \log n$

6 $T(n) = \sqrt{n}T(\sqrt{n}) + n$

Exercício: As movimentações de discos para resolver o conhecido problema da Torre de Hanoi podem ser gerados pela função a seguir:

```
void hanoi(int n, char origem, char destino, char intermediario)
{
    if (n == 1)
        printf("Move de %c para %c\n", origem, destino);
    else
    {
        hanoi(n-1, origem, intermediario, destino);
        printf("Move de %c para %c\n", origem, destino);
        hanoi(n-1, intermediario, destino, origem);
    }
}
```

Mostre que o número de movimentações de discos no algoritmo recursivo da Torre de Hanoi é dado por $2^n - 1$, onde n é o número de discos.

Res: O algoritmo recursivo para a solução da torre de Hanoi recai na recorrência:

$$T(n) = \begin{cases} 1 & p/n = 1 \\ 2.T(n-1) + 1 & p/n \geq 2 \end{cases}$$

que tem solução $T(n) = 2^{n-1}$

Base da Indução: $T(1) = 1$

$T(n+1)$ (pela recorrência)

$$= 2.T(n) + 1 = 2.(2^{n-1}) + 1 = 2^{n+1} - 1$$

$$T(n+1) \text{ (pela fórmula fechada)} = 2^{n+1} - 1$$

- Algumas recorrências não podem ser resolvidas pelos métodos anteriores. Entre elas a recorrência da série de fibonacci:

$$T(n) = \begin{cases} 0 & p/n = 0 \\ 1 & p/n = 1 \\ T(n-2) + T(n-1) & p/n \geq 1 \end{cases}$$

Árvores de recursão

- Uma árvore de recursão (recursion tree) é uma forma de visualizar o que acontece ao expandir uma recorrência. Ela mostra a árvore de chamadas recursivas e a quantidade de trabalho realizado em cada chamada.
- De modo geral, a mesma informação pode ser visualizada no método da expansão mas a árvore de recursão fornece uma forma talvez mais visível de identificar limites (ou mesmo a equação completa) para a complexidade.

Análise da Complexidade Média

- Para efetuar a análise da complexidade média de algoritmos é necessário possuir informação sobre a probabilidade de ocorrência de cada entrada. Essa informação é difícil de obter e depende do contexto onde o algoritmo é aplicado. Normalmente faz-se suposições simplificadoras sobre a distribuição dos dados de entrada, como dizer que todos tem a mesma probabilidade.

Considerando-se $p(I)$ como a probabilidade que a entrada I ocorra e $t(I)$ o custo computacional de processar a entrada I , o custo médio $A(n)$ (average) de um algoritmo é dado por:

$$A(n) = \sum_{I \in D_n} p(I) t(I)$$

Onde D_n é o conjunto de todas as entradas possíveis. A seguir serão apresentados dois exemplos da análise da complexidade média.

Exemplo 1 - Busca seqüencial

Consideramos que o número buscado tem a probabilidade q (p.ex. 0.5) de estar na lista e, se estiver, tem a mesma probabilidade de estar em qualquer uma das posições (q/n)

$$A(n) = ((1 - q)n + \frac{q}{n} \cdot \sum_{i=1}^n i) =$$

$$(1 - q)n + \frac{q}{n} \cdot \frac{(1 + n) \cdot n}{2} =$$

$$(1 - q)n + \frac{q \cdot (1 + n)}{2} =$$

$$n - q \cdot n + \frac{q}{2} + \frac{q \cdot n}{2} =$$

$$n(1 - \frac{q}{2}) + \frac{q}{2}$$

- O primeiro termo $(1 - q).n$ refere-se ao custo das buscas onde o número não é encontrado (o que é descoberto após percorrer todo o vetor).
- No caso de o número procurado sempre estar no vetor, $q = 1.0$ e a expressão resulta em $\frac{n+1}{2}$.
- No caso do elemento nunca estar no vetor ($q = 0.0$), a expressão se reduz para n (em todas as pesquisas o vetor todo é percorrido).

Exemplo 2 - Quicksort

Para determinar o tempo médio requerido pelo quicksort, assume-se que todas as entradas são igualmente prováveis e $T_{av}(n)$ é o tempo médio requerido pelo quicksort para classificar um vetor de tamanho n .

- A probabilidade de qualquer elemento ser escolhido como pivô é $1/n$.
- A seleção do p -ésimo elemento resultará em dois vetores de tamanho $p - 1$ e $n - p$ respectivamente.
- Uma vez que o número de comparações feitas no particionamento é n , podemos escrever a seguinte relação de recorrência para o tempo médio utilizado pelo quicksort:

$$T_{av}(n) = cn + \frac{1}{n} \sum_{p=1}^n [T_{av}(p-1) + T_{av}(n-p)]$$

Observando que

$$\sum_{p=1}^n T_{av}(p-1) = \sum_{p=1}^n T_{av}(n-p)$$

,

a equação anterior resulta em:

$$T_{av}(n) = cn + 2/n \sum_{p=1}^n T_{av}(p-1) (Eq.1)$$

Utilizando a Eq.1 para $n - 1$, resulta

$$T_{av}(n - 1) = c(n - 1) + 2/(n - 1) \sum_{p=1}^{n-1} T_{av}(p - 1) (Eq.2)$$

Se multiplicarmos ambos os lados da Eq.1 por n e ambos os lados da Eq. 2 por $(n - 1)$ e fizermos Eq.1 - Eq.2, resulta

$$n.T(n) - (n-1)T(n-1) = cn^2 + 2 \sum_{p=1}^n T(p-1) - c(n-1)^2 - 2 \sum_{p=1}^{n-1} T(p-1)$$

eliminando os somatórios...

$$n.T(n) - (n-1)T(n-1) = cn^2 - c(n-1)^2 + 2T(n-1)$$

$$n.T(n) = (n-1)T(n-1) + cn^2 - c(n^2 - 2n + 1) + 2T(n-1)$$

$$n.T(n) = (n-1)T(n-1) + cn^2 - cn^2 + 2cn - c + 2T(n-1)$$

$$n.T(n) = (n-1)T(n-1) + 2cn - c + 2T(n-1)$$

$$n.T(n) = (n+1)T(n-1) + 2cn - c$$

$$n.T(n) = (n+1)T(n-1) + c(2n-1) \text{ CONTINUA...}$$

Uso da indução para demonstração de corretude

A indução matemática também pode ser utilizada para demonstrar a corretude de funções recursivas. Neste caso a base da indução é a condição de não-recursão.

Ex: Mostre que a seguinte função recursiva computa $5^n - 3^n, \forall n \geq 0$.

```
int g(n)
{
if (n <=) 1
return (2n)
else return (8.g(n-1) - 15. g(n-2));
}
```

Base de indução : $g(0) = 0, g(1) = 2$

$$\begin{aligned}g(n+1) &= 8.g(n) - 15.(g(n-1)) \\&= 8.(5^n - 3^n) - 15.(5^{n-1} - 3^{n-1}) \\&= 8.5^n - 8.3^n - 3.5.5^{n-1} + 5.3.3^{n-1} \\&= 8.5^n - 8.3^n - 3.5^n + 5.3^n \\&= (8 - 3).5^n - 8.3^n + 5.3^n \\&= (8 - 3).5^n - (8 - 5).3^n \\&= 5.5^n - 3.3^n = 5^{n+1} - 3^{n+1}\end{aligned}$$

Estratégias para projeto de algoritmos

Utilizadas para resolver algumas classes de problemas:

- 1 Força Bruta
- 2 Algoritmo Guloso
- 3 Backtracking
- 4 Divisão e Conquista
- 5 Programação Dinâmica
- 6 Branch and Bound

[Voltar para o índice](#)

Força Bruta

- **Definição** : Um algoritmo que resolve ineficientemente um problema, frequentemente testando cada uma de todas as soluções possíveis (busca exaustiva) ou de uma ampla gama de soluções.
- Na maioria dos casos/problemas representa a solução mais simples, porém a mais custosa.
- Utilizada como base de comparação com algoritmos desenvolvidos através de alguma técnica mais sofisticada.
- Ex: Pesquisa linear, bubblesort, algoritmo ingênuo de pesquisa de strings.

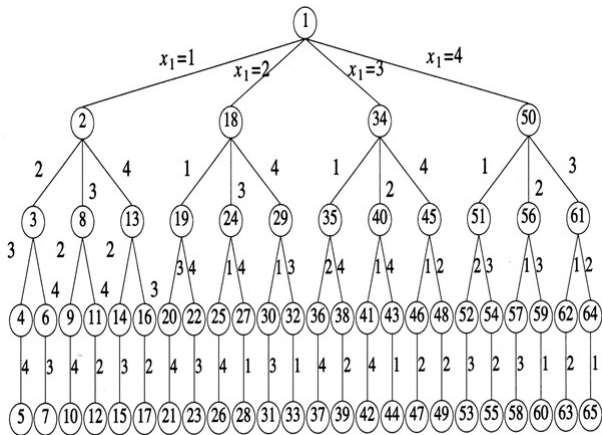
- É caracterizado pela total ausência de sofisticação no desenvolvimento da solução.
- Tipicamente segue o caminho mais óbvio e direto sem tentar minimizar o número de operações necessárias para a solução.
- Em algumas situações é a única solução possível.
- Pode ser utilizada para instâncias pequenas do problema tratado.

- Exemplo: Considere o problema de contar, em um dado texto, o número de substrings que começam com um A e terminam com um B. (Por exemplo, há quatro substrings dessa forma em CABAAXBYA.)
 - 1 Projete um algoritmo força bruta para esse problema e determine sua classe de eficiência.
 - 2 Projete um algoritmo mais eficiente para esse problema.

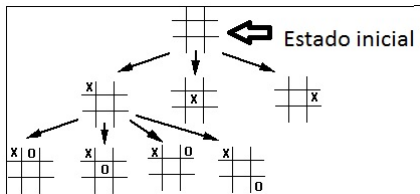
[Voltar para o índice](#)

Backtracking

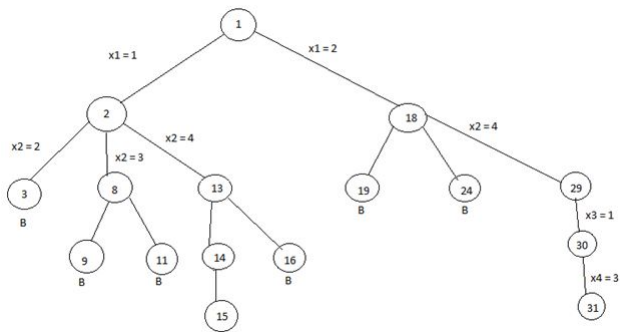
- *Backtracking* (chamada por alguns de "tentativa-e-erro" :- ()) é uma técnica para gerar de uma forma sistemática todas as soluções possíveis de um espaço de soluções.
- Se consideramos o espaço de soluções de um problema como uma árvore onde a cada nodo é feita uma decisão sobre o próximo elemento a compor a solução, o *backtracking* pode ser visto como uma busca em profundidade no mesmo, gerando, em muitos casos, uma implementação otimizada (ou pelo menos mais simples de implementar).
- O *backtracking* pode ser utilizado para:
 - Achar uma solução para um problema
 - Achar todas as soluções para um problema
 - Achar uma solução ótima sujeita a restrições explícitas.



A figura abaixo mostra parte do espaço de estados para o jogo da velha.



- A árvore de soluções é produzida durante o exame das soluções possíveis. O problema sendo resolvido pode ter restrições que fazem com que, durante a geração das soluções, alguns nodos não precisem ser examinados.
- Neste caso, toda a subárvore abaixo do nodo pode ser descartada.
- Para usar backtracking, a solução deve ser expressa como um n-tuplo $(x_1, x_2, \dots, x_n)$, onde x_i assume valores $\in S_i$.



- Normalmente o backtracking é implementado de forma recursiva.
- Alguns elementos que fazem parte de uma implementação de backtracking são:
 - A geração das possíveis escolhas a cada nível da árvore (que corresponde a cada nível da recursão)
 - Uma verificação da validade da escolha naquele momento.
 - A atualização do estado após efetuada a escolha, antes de passar para o próximo nível
 - Ao retornar da recursão, o estado deve ser restaurado ao que tinha antes da escolha.

- De modo geral uma solução com backtracking tem a estrutura a seguir:

```
int back(estado)
    se estado é solução retorne TRUE
    mostre solução
    retorne TRUE
    Para cada movimento válido
        efetue movimento atualizando estado
        se back(estado) == TRUE retorne TRUE
        desfça movimento
    retorne FALSE
```

- Esse modelo encerra a execução ao encontrar a primeira solução. Se busca-se todas a soluções ou a melhor solução, faz-se pequenas modificações nesse modelo.

- Uma outra possibilidade é enviar o movimento e testá-lo ao entrar na função:

```
int back(estado,movimento)
    se movimento é inválido retorne FALSE
    efetue movimento atualizando estado
    se estado é solução retorne TRUE
    Para cada movimento
        se back(estado,movimento) == TRUE retorne TRUE
    desfça movimento
    retorne FALSE
```


■ Alguns problemas que podem ser resolvidos por backtracking:

- O problema das 8 rainhas (8 Queen)
- O problema do percurso do cavalo (Knight Tour)
- Caminho em um labirinto (Labirinto)
- Sudoku
- Puzzle 8
- Problema da Partição
- Soma de Subconjuntos
- Resta 1
- Problema do Caixeiro viajante
- Problema da Mochila

Ex. Problema das 8 rainhas.

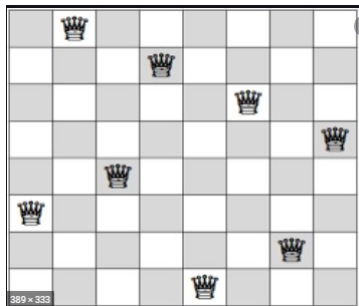
- Colocar 8 rainhas num tabuleiro de xadrez de forma que nenhuma ataque a outra.
- Visto que cada rainha deve estar numa fileira (linha) diferente, pode-se supor que a rainha i esteja na linha i . As soluções ao problema são 8-tuplos $(x_1, x_2, \dots, x_8)$ onde x_i é a coluna da rainha i .

[Voltar para lista de backtracking](#)

Uma solução para o problema da Rainha

			Q				
	Q						
						Q	
		Q					
					Q		
							Q
				Q			
Q							

- O código a seguir, disponível no ava, gera todas as soluções válidas para o problema das 8 rainhas (existem 96 soluções).
- A solução é mantida em um vetor em que cada posição **j** tem o número da linha **i** da rainha da coluna **j**.
- A configuração abaixo seria representada pelo vetor [5,0,4,1,7,2,6,3]



- A função **valida**(int c, int i) verifica se é possível preencher a posição (i,c).
- Se não houver nenhuma outra posição ocupada na mesma linha ou diagonal, a função retorna 1, caso contrário retorna 0.
- Altere o programa para gerar somente uma solução, ou seja, parar ao encontrar a primeira solução.

```
#include <stdio.h>
#include <stdlib.h>
int cont=0;
int tab[8]={0,0,0,0,0,0,0,0};

int valida(int c,int i)
{
    int j;
    for (j=0;j<c;j++)
    {
        if (tab[j]==i) return 0;
        if (tab[j]-j==i-c || tab[j]+j==i+c) return 0;
    }
    return 1;
}
```

```

void back(int c)
{
    int i;
    if (c==8)
    {
        printf("%d - ", ++cont);
        for (i=0; i<8; i++) printf("%d ", tab[i]+1);
        printf("\n\n");
        return;
    }
    for (i=0; i<8; i++)
    {
        if (valida(c,i))
        {
            tab[c]=i;
            back(c+1);
            tab[c]=0;
        }
    }
}

int main()
{
    back(0);
    system("pause");
}

```

Problema do percurso do cavalo

O cavalo no jogo de xadrez move-se em L, ou seja, duas casas em qualquer direção e uma para o lado. A figura a seguir mostra, a partir de uma posição (marcada com C), as casas que um cavalo pode atingir (marcadas com X).

	X		X	
X				X
		C		
X				X
	X		X	

Tabela: Movimentos possíveis para um cavalo no jogo de xadrez

[Voltar para lista de backtracking](#)

- O problema do percurso do cavalo consiste em encontrar, a partir de qualquer casa no tabuleiro, um percurso que passe exatamente uma vez em cada casa.
- Se da casa final pode-se atingir a casa inicial, diz-se que o percurso é *fechado*.
- A imagem da lâmina a seguir mostra um possível percurso fechado:

Solução com percurso fechado

Closed Knight's Tour

3	30	49	34	53	6	47	26
50	33	4	29	48	27	54	7
31	2	35	52	5	56	25	46
36	51	32	1	28	45	8	55
63	16	37	20	9	24	57	44
38	19	64	13	60	41	10	23
15	62	17	40	21	12	43	58
18	39	14	61	42	59	22	11

By Dan Thomasson, August 5, 2001
DTHOMASSON@carolina.rr.com

- O programa a seguir, disponível no ava, encontra as 100 primeiras soluções para o problema do cavalo
- Altere o programa para que:
 - 1 encontrar apenas uma solução
 - 2 encontrar um percurso fechado para o problema do percurso do cavalo
 - 3 não utilize um contador global
 - 4 Teste a validade do movimento antes de efetuar a chamada recursiva. Compare seu tempo de execução com a versão que testa a validade do movimento dentro da chamada recursiva (para a versão que gera as 100 primeiras soluções)

```
#include <stdio.h>
#include <stdlib.h>

int cont=0;
int tab[8][8]={ {0,0,0,0,0,0,0,0},
                {0,0,0,0,0,0,0,0},
                {0,0,0,0,0,0,0,0},
                {0,0,0,0,0,0,0,0},
                {0,0,0,0,0,0,0,0},
                {0,0,0,0,0,0,0,0},
                {0,0,0,0,0,0,0,0},
                {0,0,0,0,0,0,0,0}};

void mostra()
{
    int i,j;
    printf(" Solução %d:\n\n", ++cont);
    for (i=0;i<8;i++){
        for (j=0;j<8;j++){
            printf("%2d ",tab[i][j]);
            printf("\n\n");
        }
    }
    printf("\n\n");
}
```

```

void back(int nivel, int lin, int col)
{
    if (lin<0 || lin>7 || col<0 || col>7) return;
    if (tab[lin][col]!=0) return;
    tab[lin][col]=nivel;
    if (nivel==64)
    {
        mostra();
        tab[lin][col]=0;
        return;
    }
    //movimentos possíveis do cavalo
    int deltalin[8]={-2,-2,-1,-1,1,1,2,2};
    int deltacol[8]=-1,1,-2,2,-2,2,-1,1;
    for (int i=0;i<8;i++)
        back(nivel+1,lin+deltalin[i],col+deltacol[i]);
    tab[lin][col]=0;
}

```

```

int main()
{
    back(1,0,0);
    system(" pause");
}

```

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int main()
{
    time_t t1,t2;
    time(&t1);

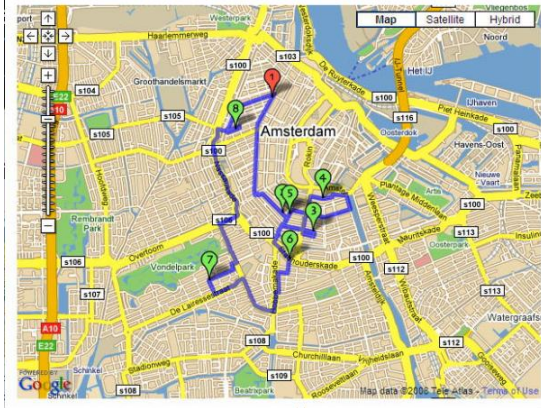
    /* Trecho a ter o tempo medido */

    time(&t2);
    printf("tempo_total:_%d_segundos_\n", (int) t2-t1);
    system(" pause");
}
```

O Problema do Caixeiro Viajante

- O Problema do Caixeiro Viajante (TSP - Traveling Salesman Problem) consiste em, dado um conjunto de cidades, achar o caminho de menor custo total que passa uma vez em cada cidade voltando à cidade inicial
- Para N cidades, o número de caminhos possíveis é da ordem de $(N-1)!$
- Utilizando Programação Dinâmica, é possível encontrar a solução ótima em tempo $N^2 2^N$
- Para instâncias muito pequenas (até 20 cidades) esse problema pode ser resolvido por backtracking
- Em Teoria de Grafos estudam-se algumas heurísticas para encontrar soluções aproximadas da ótima

Google Maps Fastest Roundtrip Solver



Utilize backtracking para resolver uma instância do Sudoku. O objetivo é preencher a matriz com números de 1 a 9 de modo que não haja dois números iguais na mesma linha, coluna ou submatriz 3x3 (em **negrito**). Teste seu programa com o caso abaixo. Teste, também, com outros casos que você encontre na Web. Faça uma versão que encontre somente a primeira solução e outra que encontre (e conte) todas as soluções.

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

- Utilize backtracking para encontrar uma solução para o puzzle "eight". O programa deve imprimir todas as configurações intermediárias da inicial até encontrar a solução. Teste seu programa com o caso abaixo. Teste, também, com outros casos que você queira.
 - Como o programa pode impedir que a busca passe por posições já analisadas?
 - Como o programa pode guardar todas as posições intermediárias?

8	3	
2	1	5
7	6	4

1	2	3
4	5	6
7	8	

Caminho em um labirinto

- Seja um labirinto descrito através de uma matriz de inteiros, onde cada posição com valor igual a 0 corresponde a uma passagem livre e uma posição com valor igual a 1 representa uma parede.
- Escreva um algoritmo que encontre um caminho que leve uma posição inicial qualquer a uma saída do labirinto, caso exista.
- Uma saída é uma posição livre na borda da matriz que define o labirinto. Utilize o valor 2 para marque o caminho válido da entrada até a saída.

[Voltar para lista de backtracking](#)

Problema da Partição

- Dada uma lista de números inteiros positivos (incluindo zero), particione a lista em K sub-listas (ou partições) onde a soma dos números em cada partição é aproximadamente a mesma. A ordem dos elementos na lista de entrada não pode se alterada e a diferença entre as somas deve ser mínima.
- Por exemplo, caso a lista informada seja 3, 1, 1, 2, 2, 1 a solução para $K = 2$ seriam as partições 3, 1, 1 e 2, 2, 1.
- Utilize gerador de números randômicos para montar casos de teste assumindo números inteiros no intervalo que vai de 0 a 10. Teste seu programa com listas de tamanhos diferentes e com diversas quantidades de partições.

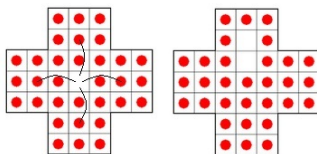
[Voltar para lista de backtracking](#)

Soma de Subconjuntos

- Seja S o conjunto de números inteiros sem repetição.
- Deseja-se saber se S contém um subconjunto S' tal que soma dos seus elementos seja exatamente igual à metade da soma dos elementos de S .
- Ou seja, trata-se de saber se o problema da partição para S tem uma solução e, em caso afirmativo, apresentar um subconjunto S que responde ao problema.
- Você deve resolver este problema usando um algoritmo de backtracking onde:
 - As soluções são representadas por tuplas de tamanho variável.
 - O valor do k -ésimo elemento da tupla é sempre menor do que o valor dos $(k-1)$ elementos que o antecedem na tupla.

Resta-1 (Chinese Checkers)

O objetivo é "comer" os pinos, deixando só um pino no tabuleiro.



[Voltar para lista de backtracking](#)

- Solução por Backtracking: Tentar movimentar os pinos de todas as maneiras.
 - (só funciona bem para versões reduzidas...).



[Voltar para lista de backtracking](#))

Problema da Mochila

- O problema da mochila (em inglês, Knapsack problem) é um problema de otimização combinatória.
- O nome dá-se devido ao modelo de uma situação em que é necessário preencher uma mochila com objetos de diferentes pesos e valores. O objetivo é que se preencha a mochila com o maior valor possível, não ultrapassando o peso máximo.

[Voltar para lista de backtracking](#)

O problema da mochila possui alguns variantes que são:

- O Problema simples da mochila: Na formulação mais geral, há n tipos de itens, de 1 a n . Cada tipo de item i tem um valor v_i e um peso w_i . Normalmente assume-se que todos os valores e pesos são não-negativos. O peso máximo que pode ser transportado na mochila é W . Matematicamente o problema pode ser formulado como:

$$\text{maximizar } \sum_{i=1}^n x_i v_i$$

$$\text{sujeito a } \sum_{i=1}^n x_i w_i < W$$

- O Problema múltiplo da mochila: Há mais de uma mochila ou uma mochila com vários bolsos e vários objetos de cada tipo com o seu valor (ex: URI1970 - Primeiro contato).
- Problema da mochila fracionado: Os objetos podem ser fracionados
- Knapsack 0-1: O número de objetos de cada tipo é 0 ou 1.
- Kapsack ilimitado: Há um número ilimitado de objetos de cada peso e valor.

- Suponha que você tenha em sua máquina uma pasta cheia de arquivos e deseja gravá-los em CD, só que você já sabe de antemão que não vai caber tudo num CD só.
- Podem ser dois, três... dez CD's.
- Como proceder para encaixar o máximo de arquivos em cada CD desperdiçando o mínimo espaço em cada disco?

Outra aplicação do problema da mochila

CHOTCHKIES RESTAURANT	
~ APPETIZERS ~	
MIXED FRUIT	2.15
FRENCH FRIES	2.75
SIDE SALAD	3.35
HOT WINGS	3.55
MOZZARELLA STICKS	4.20
SAMPLER PLATE	5.80
~ SANDWICHES ~	
BARBECUE	6.55



- Para instâncias pequenas o problema da mochila pode ser resolvido utilizando Backtracking
- A eficiência desta estratégia depende da possibilidade de limitar a busca, ou seja, podar a árvore, eliminando os ramos que não levam a solução desejada.
- Para tanto é necessário definir um espaço de solução para o problema.
- Este espaço de solução deve incluir pelo menos uma solução ótima para o problema. Depois é preciso organizar o espaço de solução de forma que seja facilmente pesquisado. A organização típica é uma árvore. Só então se pode realizar a busca em profundidade.

Faça uma função em C que resolva o problema da mochila usando backtracking. Use as estruturas de dados e cabeçalho a seguir:

```
int w[10]={10,15,12,13,21,12,5,8,4,10};  
float v[10]={10.0,30.0,6.0,13.0,60.0,12.0,25.0,40.0,20.0,5.0};  
int pesomax=20;  
  
float mochila(int peso, float valor, int pos)
```

```
#include <stdio.h>
#include <conio.h>

int w[10]={10,15,12,13,21,12,5,8,4,10};
float v[10]={10.0,30.0,6.0,13.0,60.0,12.0,25.0,40.0,20.0,5.0};
int pesomax=300;

float mochila(int peso, float valor, int pos)
{
    float valmax=valor;
    int i;
    for (i=pos;i<=9;i++)
        if (peso+w[i]<pesomax)
        {
            float vaux=mochila(peso+w[i],valor+v[i],i+1);
            if (vaux>valmax)
                valmax=vaux;
        }
    return valmax;
}

int main()
{
    int i;
    printf("%f\n",mochila(0,0,0));
    getch();
}
```

- Exercícios de backtracking do URI Online Judge:
 - 1682 - Código Genético

Árvore de Jogo

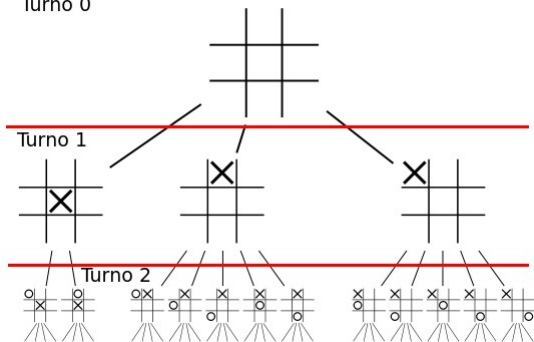
- Em jogos de turno pode-se construir a árvore de jogo, para se avaliar a melhor jogada a fazer a cada momento.
- A árvore de jogo é construída com Backtracking, para se verificar todas as possibilidades.
- Se o número de níveis for baixo, como no jogo da velha, pode-se esgotar a árvore de possibilidades.
- Caso contrário pode-se avaliar a árvore com a técnica **Minimax**.

O algoritmo Minimax

- O algoritmo Minimax é um algoritmo recursivo para escolher o próximo movimento em um jogo, normalmente de 2 jogadores.
- Um valor é associado a cada posição ou estado do jogo. Esse valor é computado a partir de fatores dependendo do jogo envolvido. Em xadrez, alguns fatores levados em conta seriam:
 - Ganho material
 - Desenvolvimento e alcance das peças
 - Situações específicas (xeque, xeque-mate, etc.)
- Esse valor é computado levando em conta que a cada nível da árvore o objetivo é invertido. O jogador **A** objetiva maximizar seu ganho, enquanto o adversário tenta minimizar o ganho de **A**.

- A figura a seguir mostra os 2 primeiro níveis da árvore de jogo para a o Jogo da Velha.
- O jogador X tem 3 possibilidades de jogada inicial (as outras 6 são rotações dessas 3).
- Como o tabuleiro tem apenas 9 casas, o número de níveis da árvore de possibilidade é 9, gerando uma quantidade pequena (765) de posições diferentes.
- Cada ramo da árvore terminará em uma posição de vitória para X (1) ou O (-1), ou empate (0).
- No nível 0, se houver alguma jogada que lhe garanta vitória, essa será a jogada escolhida por X. Se não houver, ele escolherá alguma que garanta ao menos um empate.

Turno 0



- Em jogos em que a árvore de jogadas tem uma profundidade e fator de ramificação maior, não é viável explorá-la completamente.
- O jogo de xadrez, por exemplo, tem um fator de ramificação médio de 40 possibilidades de jogada, e a duração média de uma partida é de 40 lances.
- Nesse caso, explora-se até uma certa profundidade e avalia-se a posição resultante mediante critérios específicos do jogo.

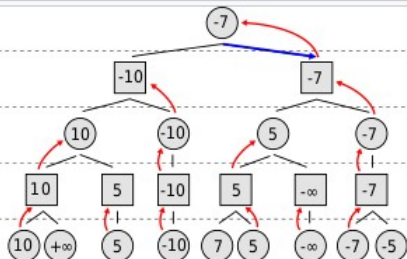
0 (max)

1 (min)

2 (max)

3 (min)

4 (max)



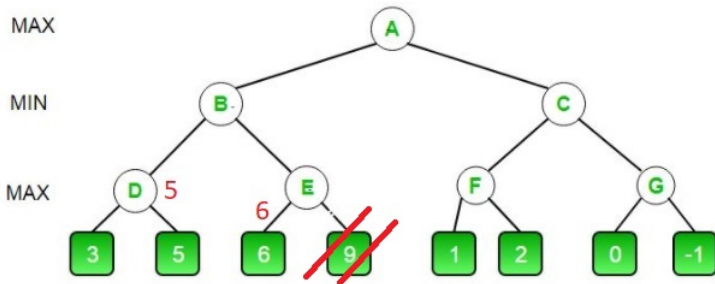
A minimax tree example



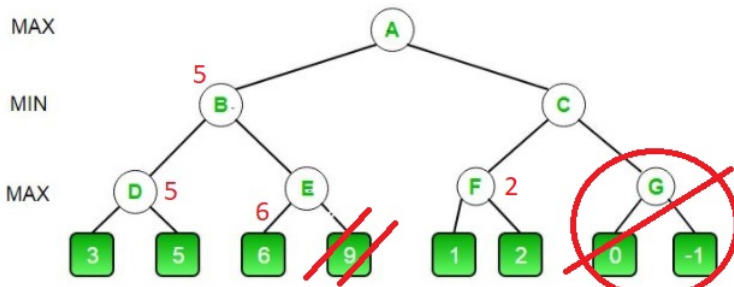
O Corte Alpha-Beta (Alpha-Beta Pruning)

- O corte alpha-beta é uma otimização do algoritmo minimax para reduzir a quantidade de ramos a serem explorados. A ideia é que, ao explorar um ramo, a avaliação parcial do mesmo já dê subsídios para a interrupção de sua avaliação.
- Considere a árvore de avaliação a seguir:

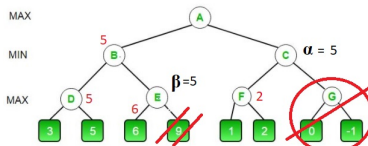
- A avaliação do nodo D resultou em 5. Se a avaliação do nodo E resultar em um valor maior que 5, ele será descartado pelo nodo B, que busca minimizar o valor.
- Ao encontrar o valor 6, a avaliação de E pode ser encerrada, pois qualquer valor menor que 6 não será escolhido por E, e qualquer valor maior que 6 não será escolhido por B.



- O 5 sobe para B, a análise de F resulta em 2 e a análise de G pode ser descartada, porque se G resultar mais de 2 não será escolhido por C, e se resultar menos de 2 não será escolhida por A.



- Essa otimização pode ser implementada passando para cada nodo dois valores:
 - **Alpha** é o melhor valor que o **maximizador** obteve até o momento
 - **Beta** é o melhor valor que o **minimizador** obteve até o momento
- Ao explorar um ramo de maximização, ao encontrar um valor maior que Beta, a avaliação pode ser descontinuada.
- Da mesma forma, ao explorar um ramo de minimização, ao encontrar um valor menor que Alpha, a avaliação pode ser descontinuada.



- O pseudo-código abaixo mostra a seleção do movimento do jogador maximizador

```
function minimax(node, depth, isMaximizingPlayer, alpha, beta):  
  
    if node is a leaf node :  
        return value of the node  
  
    if isMaximizingPlayer :  
        bestVal = -INFINITY  
        for each child node :  
            value = minimax(node, depth+1, false, alpha, beta)  
            bestVal = max( bestVal, value)  
            alpha = max( alpha, bestVal)  
            if beta <= alpha:  
                break  
        return bestVal
```

[Voltar para o índice](#)

- Uma estratégia utilizada também é a avaliação antecipada em maior profundidade de nodos mais promissores, em que se captura peças do adversário, ou a redução do nível em nodos em que se perde para o adversário (iterative deepening).

[Voltar para o índice](#)

Algoritmo guloso (greedy)

- O método guloso (Greedy) é uma abordagem de solução de problemas, normalmente aplicado a problemas em que a solução é um subconjunto de elementos.
- Na abordagem greedy, a cada passo seleciona-se, aplicando algum critério de otimização, o próximo elemento a ser incorporado à solução (melhor solução local).
- Algoritmos gulosos encontram a solução ótima para alguns problemas de otimização, mas podem encontrar soluções menos que ótimas para algumas instâncias de outros problemas.

- Durante a montagem da solução, qualquer subconjunto que satisfaça as restrições do problema é chamado de solução viável.
- Então o que queremos é uma solução viável que maximize ou minimize uma dada função objetivo.
- Essa função viável que satisfaz a função objetivo é chamada de Solução Ótima.

- Podem ser usados para solucionar problemas do tipo:
"Calcular um subconjunto S' de um conjunto de N objetos, onde S' deve satisfazer a alguma(s) propriedade(s) P ."
- Uma definição mais formal para o problema a ser resolvido por um algoritmo guloso é a seguinte:
 - Seja um conjunto S . Deseja-se determinar um subconjunto $S' \subseteq S$ tal que:
 - 1 S' satisfaz uma dada propriedade P e,
 - 2 S' é máximo (ou mínimo) em relação a algum critério dado α

- O algoritmo guloso consiste de um processo iterativo em que S' é construído adicionando-se ao mesmo, elementos de S , um a um.
- Seja S'_{i-1} o subconjunto assim construído após a iteração $i - 1$. Na iteração i determina-se o elemento $s \in S - S_{i-1}$ tal que:
 - 1 $S_{i-1} \cup s$ satisfaz P , e
 - 2 s é tal que $S_{i-1} \cup s$ é maior (menor), ou igual segundo α , do que $S_{i-1} \cup r$, que satisfaz P , p/ todo $r \in S - S_{i-1}$

- Ou seja, a cada passo, determina-se o elemento $s \in S - S_{i-1}$ que, adicionado a S_{i-1} , maximiza (minimiza) α e satisfaz P .
- A aplicação deste algoritmo garante que o subconjunto S' obtido satisfaz P .
- Há aplicações em que não se pode garantir a maximalidade de S_i , obtido segundo o processo acima.
- Assim sendo, para a aplicação deste método, é necessária uma prova de que o conjunto obtido será de fato um máximo (mínimo).
- Obs: nunca se retiram elementos de S' durante o processo de construção do mesmo.

- Duas condições que um problema deve atender para poder ser aplicada a abordagem gulosa são:
 - Escolhas ótimas locais devem levar a uma solução ótima global
 - Os subproblemas tem uma estrutura ótima, ou seja, a solução ótima para os subproblemas leva à solução ótima para o problema.

- Como vantagens do método guloso, pode-se citar
 - A escolha do próximo elemento é normalmente simples e barato (frequentemente $O(n)$).
 - Pode exigir (e normalmente exige) a ordenação dos elementos
- Desvantagens:
 - A demonstração de que leva à solução ótima pode ser complicada. Frequentemente pode ser encontrada por indução.
 - Nem sempre é clara a possibilidade de sua aplicação, e pode facilmente levar a soluções não ótimas.

Problemas clássicos resolvíveis com abordagem Greedy

- 1 Problema da Mochila Fracionada
- 2 Algoritmo de Dijkstra para distância mínima em grafo valorado
- 3 Algoritmos de Prim e Kruskal para árvore geradora mínima
- 4 Codificação de Huffman
- 5 Número mínimo de moedas
- 6 Job sequencing Problem
- 7 Solução do percurso do cavalo
- 8 Seleção de intervalos sem sobreposição
- 9 Algoritmos aproximativos

O Problema da Mochila por Algoritmo Guloso

- Para algumas variantes do problema da mochila pode-se encontrar a solução ótima com o método guloso. Para o problema da mochila fracionada pode-se obter a solução ótima da seguinte forma:
 - 1 Calcula-se para cada objeto i a relação $c_i = \frac{v_i}{w_i}$
 - 2 Ordena-se os objetos em ordem decrescente da relação c_i
 - 3 Vai-se acrescentando os objetos até esgotar a capacidade da mochila. O último objeto a ser colocado é fracionado e coloca-se na mochila a fração exata para completar a capacidade da mochila

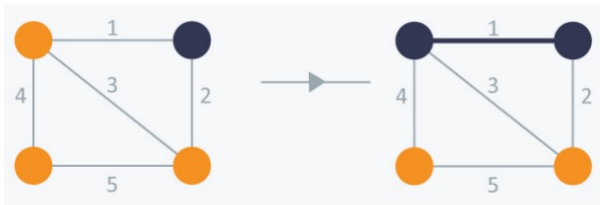
[Voltar para lista de problemas](#)

Algoritmo de Prim

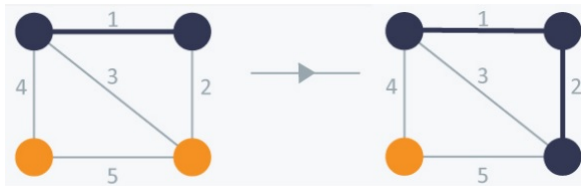
- Um algoritmo bastante usado para cálculo da árvore geradora mínima é o algoritmo de Prim. Ele mantém dois conjuntos de vértices, os que já foram incorporados à árvore e os ainda não selecionados.
- Ele inicia a montagem da árvore a partir de um vértice inicial, e a cada passo acrescenta à árvore geradora a aresta de menor valor que liga um vértice já selecionado a um vértice ainda não selecionado.
- A sequência de figuras a seguir ilustra o processo:

[Voltar para lista de problemas](#)

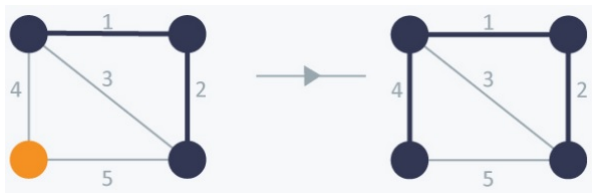
- Iniciando a construção da árvore pelo vértice superior direito da figura (poderia ser qualquer um). Escolhe-se a aresta de menor valor que liga um vértice ainda não selecionado a um vértice já selecionado (no caso, a aresta de valor 1).



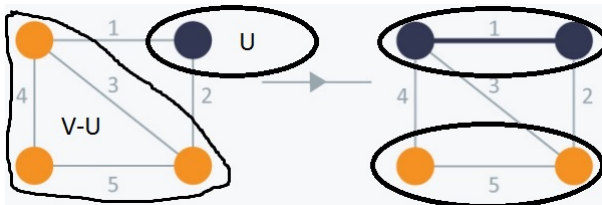
- No passo seguinte seleciona-se a aresta de menor valor que liga um vértice já selecionado (azuis) a um ainda não selecionado e acrescenta-se o vértice e a aresta à árvore.

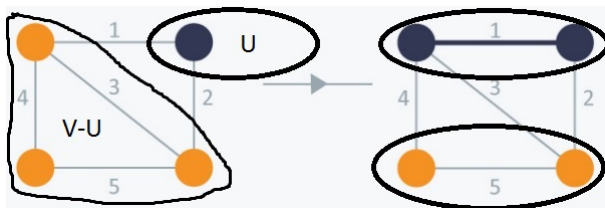


- Isso se repete até que todos os vértices tenham sido incluídos na árvore geradora

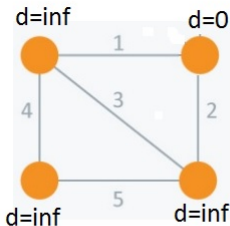


- O algoritmo de Prim pode ser resumido assim.
- Considere V o conjunto de todos os vértices do grafo original e U o conjunto de vértices já selecionados (que já fazem parte da árvore):
- Enquanto houver vértices em $V-U$:
 - Selecione a menor aresta (u,v) em que o vértice u está em U e o vértice v está em $V-U$
 - Adicione o vértice v ao conjunto U e a aresta (u,v) à árvore

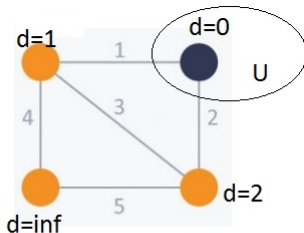
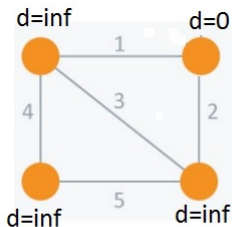


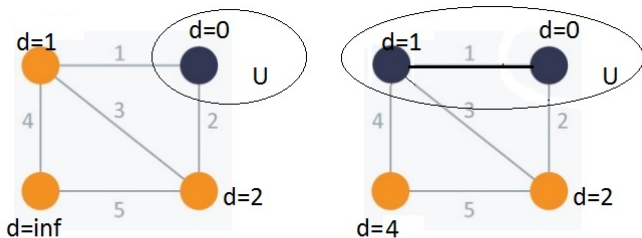


- A identificação de qual vértice deve ser selecionado a cada passo pode ser feita de forma eficiente mantendo, para cada vértice, a menor distância dele à subárvore já montada (chamemos esse vetor de **d**).
- Inicialmente considera-se que o vértice inicial está a uma distância 0 e todos os outros vértices estão a uma distância infinita.

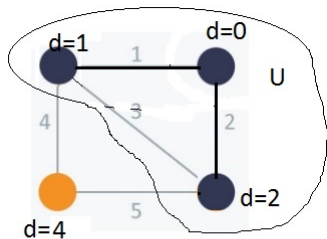
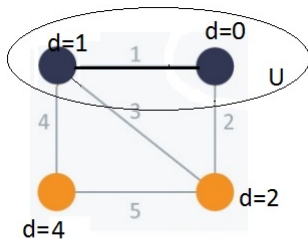


- Para identificar se um dado vértice v já foi selecionado, pode-se usar um vetor de flags (U).
- A cada passo seleciona-se, dentre os vértices ainda em $V-U$, o vértice u de menor valor d .
- Acrescenta-se o vértice u a U , e recalcula-se a distância mínima de cada vértice adjacente a u .





- Seleciona-se, dentre os vértices que ainda não estão em U , o de menor distância (no caso, o vértice com $d=1$)
- Atualiza-se a distância dos vértices adjacentes ao selecionado



- Seleciona-se, dentre os vértices que ainda não estão em U , o de menor distância (no caso, o vértice com $d=2$)
- Atualiza-se a distância dos vértices adjacentes ao selecionado

■ Código Prim1.c. Retorna o custo da árvore mínima.

```
int prim(int G[TAM][TAM]) {
    int i;
    int U[TAM], d[TAM]; // inicialmente nenhum vértice está em U
    for (i=0; i<TAM; i++) {U[i]=0; d[i]=INF;}
    d[0]=0; // iniciará montagem a partir do vértice 0
    for (i=0; i<TAM; i++)
    {
        int menor=INF, j, u; // seleciona vértice u de menor aresta
        for (j=0; j<TAM; j++)
            if (U[j]==0 && d[j]<menor) {menor=d[j]; u=j;}
        U[u]=1; // incorpora vértice u ao conjunto U
        for (j=0; j<TAM; j++) // atualiza distância de todos
            // os vértices adjacentes a u
            if (U[j]==0 && G[u][j]<d[j])
                d[j]=G[u][j];
    }
    int soma=0;
    for (i=0; i<TAM; i++) soma+=d[i];
    return soma;
}
```

- A versão anterior do código retorna a soma total da árvore geradora, mas não fornece o conjunto de arestas que a compõe.
- Uma forma de obter essa informação é manter, para cada vértice, a informação de qual seu vértice "pai", ou seja, a partir de qual vértice ele foi atingido, que resultou na menor distância.
- O código Prim2 mostra as alterações necessárias.

■ Código Prim2.c. Retorna o custo da árvore mínima.

```
int prim(int G[TAM][TAM]) {
    int i;
    int U[TAM], d[TAM]; // inicialmente nenhum vértice está em U
    for (i=0; i<TAM; i++) {U[i]=0; d[i]=INF;}
    d[0]=0; // iniciará montagem a partir do vértice 0
    for (i=0; i<TAM; i++)
    {
        int menor=INF, j, u; // seleciona vértice u de menor aresta
        for (j=0; j<TAM; j++)
            if (U[j]==0 && d[j]<menor) {menor=d[j]; u=j;}
        U[u]=1; // incorpora vértice u ao conjunto U
        for (j=0; j<TAM; j++) // atualiza distância de todos
            // os vértices adjacentes a u
            if (U[j]==0 && G[u][j]<d[j])
                {d[j]=G[u][j]; pai[j]=u;}
    }
    int soma=0;
    for (i=0; i<TAM; i++) soma+=d[i];
    return soma;
}
```

- O passo de maior custo a cada iteração é a seleção da aresta de menor valor, que tem um custo $O(N)$.
- O custo desse passo pode ser reduzido mantendo as distâncias em uma min-heap, um fila de prioridades em que o custo para obter o menor valor tem custo $O(1)$ e o custo para reorganizar a fila é de $O(\log N)$, tornando o custo total do algoritmo da ordem de $O(N \log N)$.

- Implemente o algoritmo de Prim sobre uma matriz de adjacências.
- HackerRank: Prim's (MST) : Special Subtree
- URI Online Judge (todos podem ser resolvidos com matriz de adjacências):
 - 1552 - Resgate em Queda Livre
 - 1774 - Roteadores
 - 2404 - Reduzindo detalhes em um mapa
 - 2522 - Rede do DINF
 - 2550 - Novo Campus
 - 2683 - Espaço de Projeto

- **Algoritmo de Kruskal:** Um algoritmo para calcular uma árvore geradora mínima. Ele mantém um conjunto de árvores geradoras mínimas parciais e repetidamente acrescenta a aresta mais curta no grafo cujos vértices estão em árvores geradoras mínimas diferentes.

[Voltar para lista de problemas](#)

Algoritmo de Kruskal

- O algoritmo de Kruskal é outro algoritmo para cálculo da árvore geradora mínima
 - 1 Iniciar com os n vértices, sem nenhuma aresta
 - 2 A cada etapa do cálculo introduzir, dentre as arestas restantes, a de menor valor que não completa um ciclo.
 - 3 Parar quando o número de arestas introduzidas for $n - 1$

- Como obter, de forma eficiente, a aresta de menor custo entre as restantes?
 - Uma `fila de prioridades` (min-heap).
- Como verificar, de forma eficiente, se a introdução da aresta forma um ciclo?
 - Um `disjoint-set`.

[Voltar para lista de problemas](#)

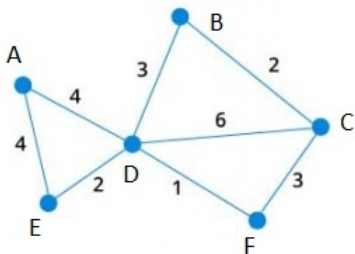
Algoritmo de Dijkstra para dígrafos valorados

- Edsger Dijkstra foi um dos pioneiros da computação.
- Suas principais contribuições foram o uso de semáforos em sistemas operacionais e o algoritmo para caminho mínimo em grafos valorados.
- Seu algoritmo para caminho mínimo foi criado em 1956 e publicado em 1959.

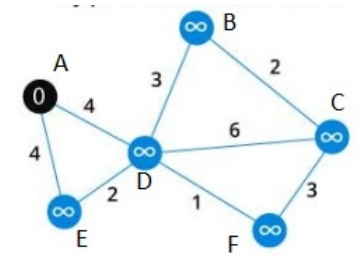


Figura: Edsger Dijkstra (1930-2002)

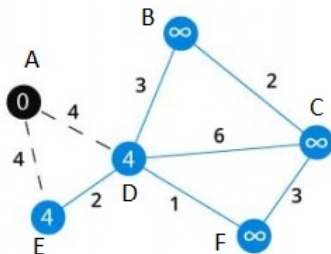
- No algoritmo de Dijkstra, a partir de um vértice inicial, a cada passo é selecionado, dentre os vértices remanescentes (que ainda não foram selecionados), o de menor valor.
- Digamos que se deseja calcular a distância do vértice A aos outros no grafo abaixo:



- Durante todo o processo mantém-se a informação da menor distância já encontrada de cada vértice x ($\text{dist}(x)$) ao vértice inicial.
- Inicialmente o vértice inicial tem distância igual a zero e os outros vértices tem distância infinita (nenhum caminho foi explorado até o momento).



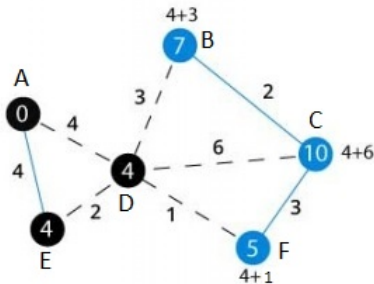
- A cada passo seleciona-se, entre os ainda não selecionados, o vértice **u** de menor distância $\text{dist}(u)$ e atualiza-se a distância de todos seus adjacentes.
- A distância calculada para cada vértice **v** é a distância já calculada para o vértice **u**, mais o custo da aresta (u,v) .
- Se a distância calculada é menor que $\text{dist}(v)$, $\text{dist}(v)$ é atualizada.



-

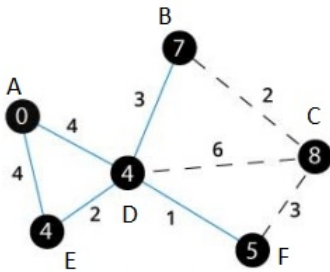
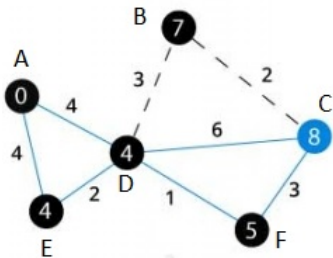


- O próximo vértice selecionado é o vértice D ($\text{dist}(D)=4$).
- Atualiza-se a distância dos vértices adjacentes a D que ainda não foram selecionados.



-
- A graph diagram showing nodes A, B, C, D, E, and F. Node A is black with value 0. Node B is blue with value 7. Node C is blue with value 8. Node D is black with value 4. Node E is black with value 4. Node F is black with value 5. Edges are labeled with weights: A-B (4), A-D (4), A-E (4), B-E (3), B-C (2), C-E (6), C-F (3), D-E (2), and E-F (1). A calculation $5+3 < 10$ is shown near node C.

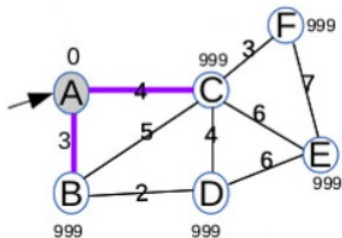
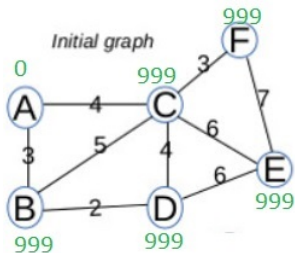
- Isso se repete até que todos os vértices tenham sido selecionados ou até que o vértice destino (caso se procure a distância até um vértice específico) seja selecionado.
- A distância de um vértice no momento em que ele é selecionado é a menor distância possível até ele a partir do vértice inicial.
- Enquanto ele não for selecionado, sua distância pode ser reduzida encontrando-se um caminho mais curto até ele.



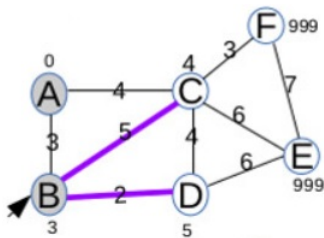
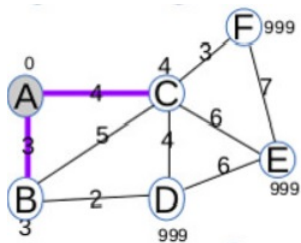
Algoritmo de Dijkstra

Objetivo : Busca encontrar o menor caminho entre s e t . $d[v]$ designa a menor distância de v a s . $l(e)$ designa o peso da aresta e . Q representa o conjunto de vértices para os quais ainda não se calculou a distância.

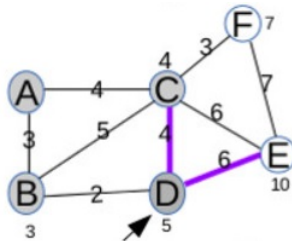
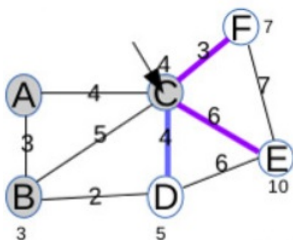
- $d[s] \leftarrow 0$ e para todo $v \neq s, d[v] \leftarrow \infty$
- $Q \leftarrow V$
- Enquanto houver algum vértice não selecionado ($Q \neq \emptyset$)
 - Seja u um vértice em Q para o qual $d[u]$ seja mínimo.
 - Se $u = t$ então termina o algoritmo
 - Para toda aresta $e = (u, v)$,
 - se $v \in Q$ e $d[v] > d[u] + l(e)$
então $d[v] \leftarrow d[u] + l(e)$
 - remova u do conjunto Q



- Inicialmente todos os vértices começam com distância ∞ e o vértice inicial começa com distância 0. O vértice de menor distância (vertex A, com distância 0) é selecionado.

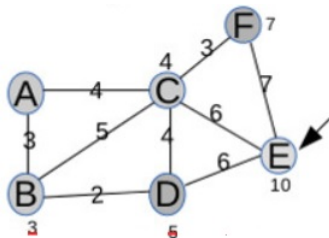
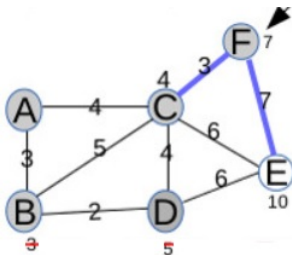


- As distâncias dos vértices adjacentes a A (B e C) são atualizadas. Em seguida B é selecionado e seus adjacentes (C e D) são atualizados. Como $d[C]$ é menor que $d[B] + 5$, $d[C]$ não é alterado.



- O vértice de menor distância entre C, D, E e F (C) é selecionado e seus vértices adjacentes (D, E e F) são atualizados. Como $d[D]$ é menor que $d[C]+4$, $d[D]$ não é alterado.
- Em seguida o vértice de menor distância entre D, E e F (D) é selecionado. Como seu único vértice adjacente (E) tem $d[E]$ menor que $d[D]+6$, $d[E]$ não é alterado.

Exercícios

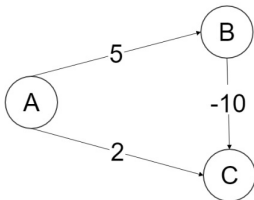


- O vértice de menor distância entre E e F (F) é selecionado. Como seu único vértice adjacente (E) tem $d[E]$ menor que $d[F]+7$, $d[E]$ não é alterado.
- F é selecionado e termina o algoritmo.

Índice

Exercício

- A complexidade do algoritmo depende da estrutura usada para armazenar as distâncias $d[v]$. Se for utilizada uma fila de prioridades, a complexidade do algoritmo é $O(V \log E)$. Se for utilizado um vetor, é $O(V^2)$
- Para grafos não dirigidos, basta substituir cada aresta por dois arcos, um em cada sentido, e aplicar o algoritmo.
- O algoritmo não funciona se as arestas podem ter pesos negativos (figura abaixo).



Codificação de Huffman

- É um esquema de codificação de caracteres de comprimento mínimo baseada na frequência de cada caracter.
- Ele foi desenvolvido em 1952 por David A. Huffman que era, na época, estudante de doutorado no MIT, e foi publicado no artigo "A Method for the Construction of Minimum-Redundancy Codes".
- Cada caracter do texto a ser transmitido/comprimido é representado por uma codificação binária em que o número de bits é inversamente proporcional ao número de ocorrências do caracter no texto.
- A tabela de codificação de cada caracter é montada utilizando uma abordagem gulosa.

- A tabela de codificação montada a partir de uma árvore em que os caracteres são as folhas das árvores, e a codificação de cada caracter é definida pelo caminho da raiz até a folha.
- Inicialmente cada caracter é uma árvore trivial com o caracter como único nodo.
- A frequência do nodo é o número de ocorrências do caracter na frase.
- As duas árvores com menor frequência são juntadas sob um novo nodo raiz cuja frequência é a soma das frequências dos dois nodos.
- Isso se repete até que todos os caracteres estão na árvore.

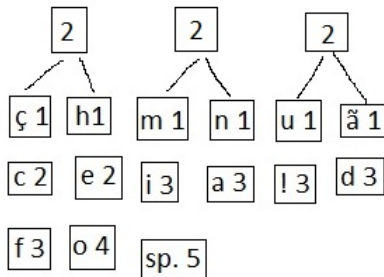
- Considere a frase "Eu odeio a codificação de huffman!!!"
- O número de ocorrências de cada símbolo na frase é:

símbolo	n. de ocorr.	símbolo	n. de ocorr.
a	3	c	2
d	3	e	2
f	3	h	1
i	3	m	1
n	1	o	4
u	1	ã	1
ç	1	!	3
espaço	5		

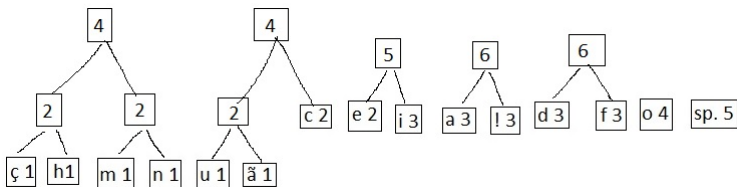
- Inicialmente cada caracter forma uma árvore de um único nodo que contem o caracter e o número de ocorrências dele.

ç 1	h1	m 1	n 1	u 1	ã 1
c 2	e 2	i 3	a 3	l 3	d 3
f 3	o 4	sp. 5			

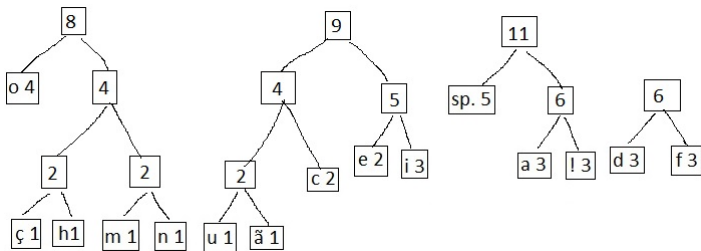
- A cada passo as duas árvores de menor número de ocorrências são selecionadas e unidas em uma única árvore.



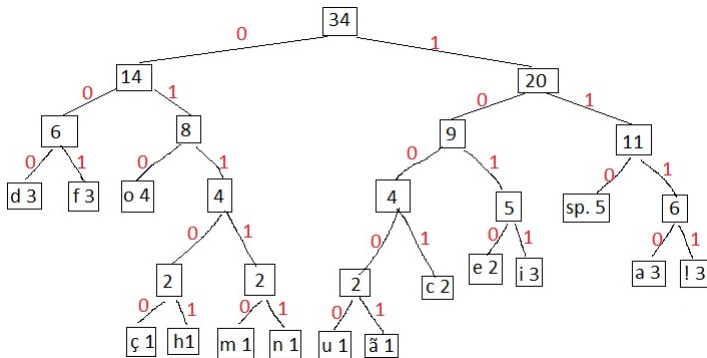
- E continua.
- Se as raízes das árvores forem armazenadas em uma fila de prioridades, a obtenção da raiz de menor valor e a reorganização da fila podem ser feitas em $O(\log N)$, onde N é o número de caracteres diferentes, resultando um custo total de $O(N \log N)$ (praticamente constante, já que o número de caracteres é pequeno (255 em 8 bits))



- A distância de cada caracter à raiz da árvore dará o tamanho de sua codificação
- Dessa forma, os caracteres mais frequentes são os últimos a serem colocados e estarão mais próximos à raiz e terão codificação com menos bits.



- O processo termina quando todos os símbolos foram unidos em símbolos auxiliares, formando uma árvore binária.



- A árvore é então percorrida, atribuindo-se valores binários de 0 quando for tomado o caminho da esquerda e 1 quando for tomado o caminho da direita. Os códigos são gerados a partir desse percurso.
- O caracter 'd', por exemplo, é alcançado pelo caminho '000', e será essa sua representação binária.

símbolo	codificação	símbolo	codificação
a	1110	c	1001
d	000	e	1010
f	001	h	01101
i	1011	m	01110
n	01111	o	010
u	10000	ã	10001
ç	01100	!	1111
espaço	110		

- A codificação de Huffman tem desempenho ótimo quando as probabilidades dos símbolos são potências negativas de dois (2^{-1} , 2^{-2} , ...).
- A codificação gerada tem também a garantia de ser não ambígua, pois nenhum código pode ser o prefixo de outro código uma vez que os caracteres são representados pelas folhas e não por nodos internos.
- Para codificar uma mensagem, basta substituir cada caracter pelo código correspondente

símbolo	codificação	símbolo	codificação
a	1110	c	1001
d	000	e	1010
f	001	h	01101
i	1011	m	01110
n	01111	o	010
u	10000	ã	10001
ç	01100	!	1111
espaço	110		

- A mensagem "Eu odeio a codificação de huffman!!!" seria representada como (espaços colocados para melhorar a legibilidade)

1010 10000 110 010 000 1010 1011 010 110 111 110 1001 010
 000 1011 001 1011 1001 111 01100 10001 010 110 000 1010
 110 01101 10000 001 001 01110 111 01111 1111 1111 1111

- Que ocupa 133 bits, ao invés dos $36 \times 8 = 288$ bits que ocuparia em uma codificação de 8 bits.

Decodificação de Huffman

- 1 Escolha do primeiro bit do código como bit corrente;
- 2 Desça um nível na árvore de Huffman de acordo com o valor do bit corrente (passar ao filho mais velho se o bit for 0 ou passar ao filho mais novo se o bit for 1) e atribuição ao bit corrente do próximo bit do "string" de bits do código a decodificar;
- 3 No caso de ser atingida uma folha da árvore de Huffman deve-se fazer a substituição dos bits correspondentes à trajetória até a folha pelo caractere contido nessa folha;
- 4 Repetição dos passos 2 e 3 até que termine o "string" de bits do código a decodificar.

- Para a decodificação da mensagem é usada a mesma árvore que foi gerada para fazer a codificação. Para isso a tabela de frequências pode ser armazenada junto com o arquivo comprimido e o programa que fará a decodificação montará a árvore a partir da tabela de frequências.

[Voltar para lista de problemas](#)

Número mínimo de moedas

- Dado um valor **N** e um estoque ilimitado de moedas de **m** diferentes valores, busca-se encontrar o número mínimo de moedas para obter o valor.
- Dependendo dos valores das moedas, é possível resolver esse problema por uma abordagem gulosa, tomando a cada vez a moeda de maior valor que seja menor que o valor restante.
- O código a seguir implementa a abordagem, assumindo que o vetor de denominações das moedas tenha sido ordenado previamente em ordem decrescente ($O(m \log m)$).
- Nesse caso, a complexidade é $O(m)$

```
def greedyCoinChanging(M, k):  
    n = len(M)  
    result = []  
    for i in xrange(n - 1, -1, -1):  
        result += [(M[i], k // M[i])]  
        k %= M[i]  
    return result
```

- Em algumas situações, dependendo dos valores das moedas, essa abordagem pode não levar à solução ótima. Ex:
 $M=\{1,6,10\}$ e $N=13$.
- Nesse caso a abordagem por Programação Dinâmica pode ser usada.

Sequenciamento de tarefas (Job Sequencing)

- Dado um conjunto de tarefas onde cada tarefa tem um prazo de término e um ganho associado se terminado dentro do prazo.
- Cada tarefa utiliza **uma unidade de tempo**.
- Cada tarefa pode iniciar a qualquer instante contanto que termine dentro do prazo.
- Busca-se selecionar as tarefas, de modo que haja no máximo uma tarefa ativa a cada momento, de modo a maximizar o ganho.
- Ex: Qual o subconjunto de tarefas que maximiza o lucro para o conjunto de tarefas a seguir?

Tarefa	Prazo	Ganho
a	2	100
b	1	19
c	2	27
d	1	25
e	3	15

- Uma abordagem baseada em força bruta gera todos os subconjuntos viáveis de tarefas e calcula o custo de cada subconjunto.
- Custo exponencial.

- Uma abordagem "greedy" é ordenar as tarefas em ordem decrescente pelo ganho e ir selecionando para cada tarefa o maior slot de tempo disponível dentro de seu prazo.
- Essa abordagem apresenta custo $O(N^2)$

- No exemplo anterior, seria inicialmente selecionada a tarefa de maior ganho, a tarefa **a**, que por terminar no tempo=2, seria colocada no segundo slot de tempo.

	a	
--	---	--

- Em seguida seria selecionada a tarefa **c**, mas como o slot 2 já foi ocupado, a tarefa **c** seria alocada no slot 1.

c	a	
---	---	--

- E por fim seria selecionada a tarefa **e**, que apesar de ter o menor ganho de todas, é a única que pode ocupar o slot 3.

c	a	e
---	---	---

- Na abordagem anterior, o custo $O(N^2)$ se deve ao custo $O(N)$ para encontrar o primeiro slot disponível para um dado deadline.
- Uma solução com custo $O(N \log N)$ pode ser obtida utilizando **Disjoint Sets**.
- Nessa solução é criado um vetor com uma posição para cada possível deadline, mantendo o número do maior slot de tempo compatível com aquele deadline. Se não houver slot disponível compatível com o deadline, a posição conterá 0.

- Nesse vetor, se uma posição P contem um valor Q, diferente de P, significa que o maior slot disponível para o prazo P é o mesmo maior slot disponível para o prazo Q.
- Na figura abaixo, o maior slot disponível para os prazos 3, 4, 7, 8 e 9 é o próprio prazo, enquanto que o maior slot para o prazo 6 é o mesmo do prazo 5 que é o slot 4.

Maior slot :	0	1	3	4	4	5	7	8	9
Prazo :	1	2	3	4	5	6	7	8	9

- E a cada vez que é pesquisado o maior slot para um prazo, a informação é atualizada no vetor usando o mecanismo de path compression abaixo:

```
int find(int x)
{
    if (pai[x] != x)
        pai[x] = find(pai[x]);
    return pai[x];
}
```

- Inicialmente cada posição contém o seu próprio valor, representando que o maior slot compatível com o deadline é o próprio deadline.
- E todos os slots estão disponíveis.

Maior slot	1	2	3
prazo	1	2	3

Tarefa			
Slot	1	2	3

Maior slot	1	2	3
prazo	1	2	3

Tarefa			
Slot	1	2	3

- Aqui também as tarefas são seleccionadas em ordem decrescente de ganho. Inicialmente é seleccionada a tarefa **a**, que tem prazo=2. Para tarefas com prazo=2, o maior slot é 2. A tarefa **a** é alocada para o slot 2 e o maior slot para o deadline 2 passa a ser o da posição à esquerda (1).

Maior slot	1	1	3
prazo	1	2	3

Tarefa		a	
Slot	1	2	3

Maior slot	0	1	3
prazo	1	2	3

Tarefa	c	a	
Slot	1	2	3

- As tarefas seguintes, **b** e **d**, tem prazo=1 o a tabela mostra que não há mais slot disponível e elas não são selecionadas.
- A última tarefa, **e**, tem prazo=3 e a tabela mostra que o maior slot disponível para ela é o 3. Ela é alocada para o slot 3 e o maior slot para o prazo=3 passa a ser o do vizinho à esquerda.

Maior slot	0	1	1
prazo	1	2	3

Tarefa	c	a	e
Slot	1	2	3

Solução do problema do percurso do cavalo utilizando algoritmo guloso

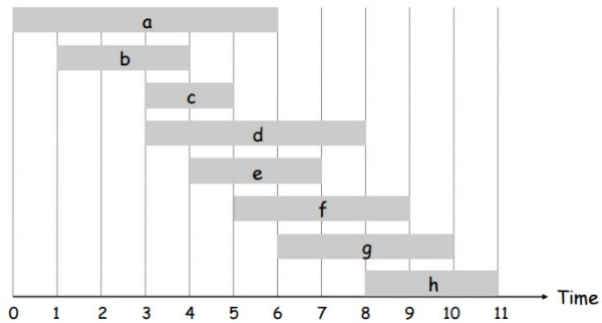
- O problema do percurso do cavalo, visto como exemplo de backtracking, também pode ser resolvido com uma abordagem gulosa. Basta escolher, a cada passo, a casa vizinha que possui menos opções de jogada no próximo passo.

[Voltar para lista de problemas](#)

Seleção de intervalos sem sobreposição

- Nesse problema, dado um conjunto de intervalos (p.ex. intervalos de tempo, cada um com horário de início $s[i]$ e horário de término $f[i]$, para i de 1 a N), busca-se encontrar o subconjunto de intervalos sem sobreposição que maximize a **quantidade** de intervalos do subconjunto.
- Ex:
 - $(1, 4), (3, 5), (0, 6), (5, 7), (3, 8), (5, 9), (6, 10), (8, 11), (8, 12), (2, 13), (12, 14)$

[Voltar para lista de problemas](#)



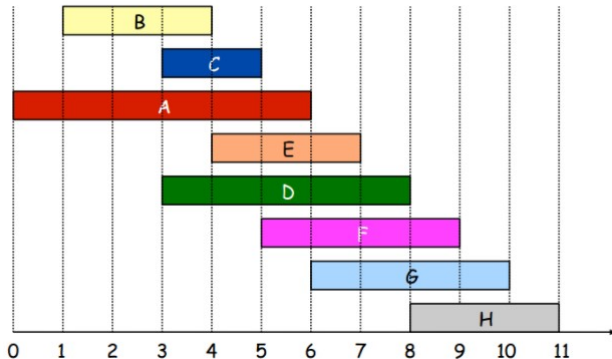
- Ao esboçar uma solução greedy, deve-se especificar o critério a ser utilizado na seleção do próximo elemento. Nesse problema alguns possíveis critérios são: horário de início da tarefa, horário de término da tarefa, duração da tarefa.
- Desses 3 critérios, pode-se mostrar que o único que leva à solução ótima é selecionar a cada passo o intervalo de horário de término mais cedo, e eliminar do conjunto todos os intervalos que tem alguma sobreposição com ele.

[Voltar para lista de problemas](#)

- A figura abaixo mostra situações em que os dois primeiros critérios falham.
- No primeiro, o intervalo **a** é selecionado por ser o de início mais cedo, e impede que quatro outros intervalos sejam selecionados.
- No segundo o intervalo **b** é selecionado por ser o mais curto, e impede que outros 2 intervalos sejam selecionados.



- Utilizando o terceiro critério (selecionar a cada passo o de menor horário de término que não sobreponha qualquer um dos anteriores já selecionados), e ordenando as tarefas em ordem crescente de horário de início:



- Pode-se provar por indução para cada tarefa j que esse critério leva à solução ótima, mas a ideia é que para cada objeto, se ele é compatível com o anterior ele pode ser colocado melhorando a solução, e se ele não for compatível, i.e., colidir com o último já colocado não melhora a solução (pode manter a mesma quantidade de objetos, já que um intervalo já colocado terá que ser removido, ou piorar, se o novo intervalo sobrepôr mais de um dos já colocados).

- Uma abordagem gulosa seria a seguinte:

Ordene os intervalos em ordem crescente de término.

Selecione o intervalo 1

$last = f[1]$

Para i de 2 a N

Se $s[i] \geq last$

Selecione i

$last = f[i]$

- O custo desse algoritmo é **$N \log N$** para a ordenação, e linear para selecionar os intervalos.
- ENADE 2011 - Questão 13 (foi anulada)
- SPOJ - **BUSYMAN** - I AM VERY BUSY

- Qual o subconjunto selecionado para o conjunto de intervalos a seguir?
 - $(1, 4), (3, 5), (0, 6), (5, 7), (3, 8), (5, 9), (6, 10), (8, 11), (8, 12), (2, 13), (12, 14)$
- O primeiro intervalo selecionado é o $(1,4)$. Se o primeiro intervalo selecionado for outro, ele pode ser substituído na solução final pelo $(1,4)$.
- Uma vez selecionado o $(1,4)$, os intervalos $(3,5)$ e $(0,6)$ são descartados, pois iniciam antes do tempo 4.
- Problemas semelhantes são encontrar o conjunto mínimo de intervalos que maximize a área total coberta, com ou sem sobreposição, e a seleção de intervalos sem sobreposição valorados que maximizem o valor total.

Soluções aproximadas por algoritmo guloso

- Algoritmos gulosos também podem ser utilizados para encontrar soluções aproximativas para alguns problemas, principalmente NP-Completo, para os quais não existe algoritmo polinomial que encontre a solução ótima.
- Alguns exemplos são o algoritmo para encontrar o número cromático e para resolver o problema do Caixeiro Viajante a seguir.
- Para mostrar se o algoritmo encontra a solução ótima ou aproximativa, depende do problema e normalmente pode ser demonstrado.

Aplicações do método guloso para soluções aproximadas

- 1 Escalonamento de Processos
- 2 Seleção de bloco de memória
- 3 Obtenção do Número Cromático de um Grafo
- 4 Problema do Caixeiro Viajante
- 5 Cobertura de vértices e/ou arestas

Escalonamento de Processos

- Outra aplicação da abordagem greedy é no escalonamento de processos. Algoritmos de escalonamento de processos em CPUs normalmente utilizam a abordagem greedy. Alguns dos algoritmos usados por CPUs para escalonamento são:
 - First come first serve
 - Shortest job first
 - Escalonamento por prioridade
 - Round-Robin

- Pode-se mostrar que se o escalonador seleciona a cada vez o processo que vai levar menos tempo (baseado no histórico do processo (tempo entre duas operações de I/O, por exemplo), o tempo médio de espera+execução dos processos é o menor possível.

[Voltar para lista de problemas](#)

- Ex: Qual o tempo médio de espera+execução dos processos C_1 , C_2 e C_3 , quando $t(C_1) = 8$, $t(C_2) = 4$ e $t(C_3) = 6$, para todas as ordenações possíveis?
 - Encontre a expressão genérica para o tempo médio de espera para o caso de N processos.
 - Mostre que o tempo médio de espera é mínimo quando o processo selecionado a cada passo é o de menor duração, ou seja, a ordem de entrada dos processos é crescente no tempo de execução estimado.

Ordem de atendimento	Tempo de espera+execução			Tempo médio
	C_1	C_2	C_3	
$C_1 C_2 C_3$	8	8+4	8+4+6	12.7
$C_1 C_3 C_2$	8	8+6+4	8+6	13.3
$C_2 C_1 C_3$	4+8	4	4+8+6	11.3
$C_2 C_3 C_1$	4+6+8	4	4+6	10.7
$C_3 C_1 C_2$	6+8	6+8+4	6	12.7
$C_3 C_2 C_1$	6+4+8	6+4	6	11.3

$$S(n) = t_1 + (t_1 + t_2) + (t_1 + t_2 + t_3) + \dots + (t_1 + t_2 + t_3 + \dots + t_n)$$

$$S(n) = n.t_1 + (n-1).t_2 + (n-2).t_3 + \dots + t_n$$

$$S(n) = n.t_1 + (n-1).t_2 + (n-2).t_3 + \dots + t_n$$

- Supondo que para algum par t_i e t_j , $i < j$, $t_i < t_j$.
 - Se as posições de t_i e t_j forem trocadas entre si, a soma $S(n)$ sofrerá um acréscimo correspondente à diferença entre t_i e t_j , multiplicados pela diferença de posições entre eles, acarretando um aumento no tempo de espera médio de todos os processos.

Seleção de bloco de memória por gerenciador de memória

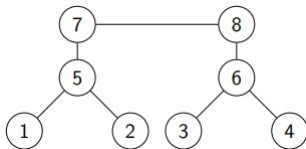
- Ao receber uma solicitação de memória por um processo, o gerenciador de memória pode utilizar várias políticas para escolher o bloco, todas elas usam uma abordagem greedy:
- As principais são:
 - First fit: seleciona o primeiro bloco maior ou igual ao solicitado
 - Best fit: seleciona o menor bloco maior ou igual ao solicitado

[Voltar para lista de problemas](#)

Obtenção do número cromático de um grafo

- O número cromático de um grafo é o número mínimo de cores necessárias para colorir os vértices de um grafo, de modo que dois vértices adjacentes (isto é, ligados por uma aresta) não tenham a mesma cor.
- É um problema NP-difícil (isto é, para o qual não se conhece solução polinomial), mas uma coloração aproximada pode ser obtida utilizando a abordagem a seguir:
 - 1 Ordene os vértices do grafo em ordem decrescente de grau.
 - 2 Atribua a cada vértice a menor cor que não coincida com nenhum dos vértices adjacentes

- Qual a coloração obtida no gráfico abaixo? E qual é o número cromático?



[Voltar para lista de problemas](#)

Solução aproximativa para o problema do Caixeiro Viajante (para grafos completos) :

A cada passo, escolher o arco com menor peso e que:

- 1 Não cause um vértice a ter grau 3 ou mais
- 2 Não forme um ciclo, a não ser quando o número de arcos selecionados for igual ao número de vértices no problema

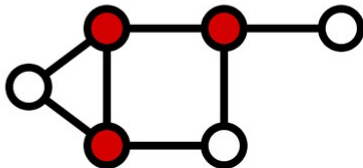
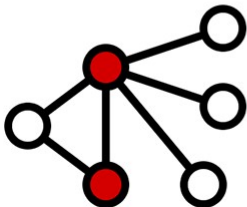
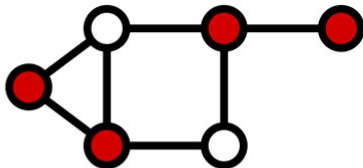
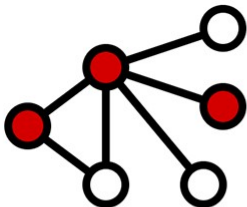
[Voltar para lista de problemas](#)

Cobertura de Vértices

- Uma cobertura de vértices (vertex cover) é um conjunto de vértices que contém ao menos um dos vértices adjacentes a cada aresta do grafo.
- Ao contrário da cobertura mínima de arestas (conjunto de arestas que cobrem todos os vértices) que pode ser encontrado em tempo polinomial expandindo uma associação máxima, o problema de cobertura mínima de vértices é NP-difícil.

[Voltar para lista de problemas](#)

- As 4 figuras abaixo representam coberturas de vértices. As duas de baixo são coberturas mínimas..



- Aplicações de cobertura de vértices incluem aplicações em distribuição de câmeras de vigilância ou postos de distribuição.
- Apesar de ser um problema NP-difícil, há algoritmos polinomiais que podem encontrar uma aproximação que garanta uma solução no máximo igual a 2 vezes o número de vértices da solução ótima.

[Voltar para lista de problemas](#)

Divisão e Conquista (Divide and conquer)

Algoritmos baseados em divisão e conquista resolvem um problema quebrando-o em um ou mais sub-problemas que são similares ao problema original, exceto que são menores em tamanho.

- Cada um dos subproblemas é resolvido independentemente e os resultados dos subproblemas são combinados para produzir a solução do problema original.
- Exemplos clássicos são o Mergesort e o Quicksort.
- Quando o número de subproblemas é igual a 1 (ex. Pesquisa Binária), alguns autores chamam o método de "redução e conquista".

A procedure a seguir é uma representação abstrata da técnica de divisão e conquista (Horowitz):

```
void DeC (p,q) // p e q: limite inferior e superior do vetor
// contendo o problema a ser resolvido
{
//globais n, A(1..n)
int m,p,q;
if (SMALL(p,q)) // verifica o tamanho do problema
return (G(p,q)) // solução não recursiva
else
{
m = DIVIDE (p,q); // m recebe o ponto de divisão do problema
return COMBINE(DeC(p,m),DeC(m+1,q))
}
} //end DeC
```

- Se os tamanhos dos subproblemas são aproximadamente iguais então o tempo de execução de DeC é descrito pela relação de recorrência

$$T(n) = \begin{cases} G(n) & p/n \text{ pequeno} \\ 2.T(n/2) + f(n) & \text{caso contrario} \end{cases}$$

Forma Geral

- O tempo de execução de muitos algoritmos por divisão e conquista pode ser descrito por uma relação de recorrência na forma

$$T(n) = \begin{cases} \Theta(1) & \text{se } n \leq n_0 \\ a \cdot T(n/b) + \Theta(n^k) & p/n > n_0 \end{cases}$$

onde $a \geq 1$, $b > 1$, $k \geq 0$, $n_0 \geq 0$ são constantes e $\Theta(n^k)$ é o tempo necessário para a divisão do problema mais o tempo para a concatenação dos resultados.

Uma forma fechada para a relação, determinada pelos valores de a , b e k , é dada por

$$T(n) = \begin{cases} \Theta(n^k) & \text{se } a \leq b^k \\ \Theta(n^k \log n) & \text{se } a = b^k \\ \Theta(n^{\log_b a}) & \text{se } a > b^k \end{cases}$$

Ex:

a) Quicksort: $a = 2, b = 2, k = 1 \Rightarrow T(n) = \Theta(n \log n)$

b) Mergesort: idem

c) Pesquisa binária:

$a = 1, b = 2, k = 0 \Rightarrow T(n) = \Theta(nk \log n) = \Theta(\log n)$

Ex: Verifique a classe de complexidade das recorrências a seguir pela relação vista:

a) $T(n) = T(n/2) + 1$ $p/n \geq 2$

R: $T(n) = \Theta(\log_2 n)$

b) $T(n) = T(n/2) + n$ $p/n \geq 2$

R: $T(n) = \Theta(n)$

c) $T(n) = 2.T(n/2) + n$ $p/n \geq 2$

R: $T(n) = \Theta(n \log n)$

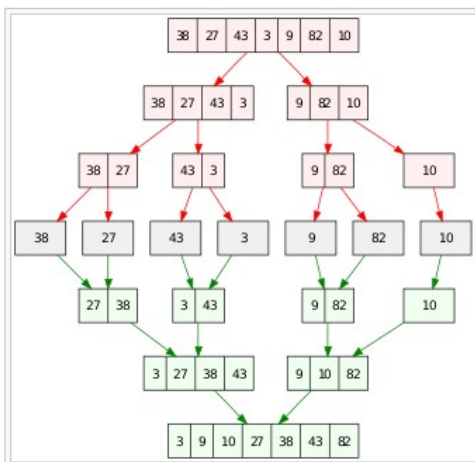
- Um princípio importante na abordagem de Divisão e Conquista é o balanceamento no tamanho dos subproblemas gerados.
- De modo geral o algoritmo é tão mais eficiente quanto mais próximos os tamanhos dos problemas.
- Um bom exemplo disso é a comparação da complexidade do melhor caso e do pior caso do quicksort.

Colocar exemplo do Terada de algoritmo recursivo por divisão e conquista com complexidade linear para encontrar o k-ésimo elemento..

Aplicações clássicas de Divisão e Conquista

- 1 Quicksort
- 2 Merge Sort
- 3 Contagem de inversões em um vetor
- 4 Encontrar par de pontos mais próximos em um plano cartesiano (URI Online Judge 1295)
- 5 Exponenciação
- 6 n-ésimo termo da série de Fibonacci
- 7 Algoritmo de Karatsuba para multiplicação
- 8 Algoritmo de Strassen para Multiplicação de Matrizes

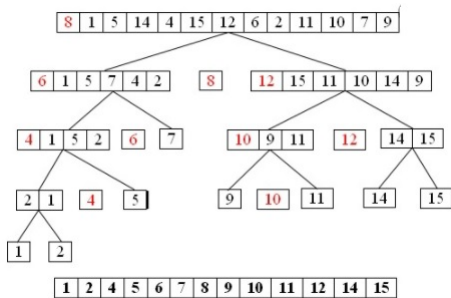
[Voltar para o índice](#)



```
void mergesort(char v[],int ini , int fim)
{
    if ( ini<fim )
    {
        int meio=(fim+ini)/2;
        mergesort(v,ini ,meio);
        mergesort(v ,meio+1,fim );
        merge(v ,ini ,meio ,meio+1,fim );
    }
}
```

```
void merge(char vin[],int ini1, int fim1, int ini2, int fim2)
{
    int i, isai;
    int inicio=ini1;
    for ( isai=ini1; isai<=fim2; isai++)
        if ( ini1>fim1)
            vout[ isai]=vin[ ini2++];
        else if ( ini2>fim2)
            vout[ isai]=vin[ ini1++];
        else if (vin[ ini1]<vin[ ini2])
            vout[ isai]=vin[ ini1++];
        else vout[ isai]=vin[ ini2++];
    for ( i=inicio; i<=fim2; i++) vin[i]=vout[i];
}
```

Contagem de inversões em um vetor por Mergesort modificado



Exponenciação

- Dados inteiros X e y queremos calcular X^y .
- Uma forma de calcular X^y e multiplicar X por ele mesmo $y - 1$ vezes.

Pode-se obter uma solução mais eficiente considerando que:

$$X^y = \begin{cases} X^{\frac{y}{2}} * X^{\frac{y}{2}} & \text{se } n \text{ e par} \\ X^{\frac{y}{2}} * X^{\frac{y}{2}} * X & \text{se } n \text{ e impar} \end{cases}$$

que pode ser implementado recursivamente:

```
int exponent(int x, int n) {  
    if (n == 1)  
        return x;  
    else {  
        y = exponent(x, n / 2);  
        y *= y;  
        if (n & 1) /* n odd */  
            y *= x;  
        return y; }  
}
```

Qual a complexidade da função acima?

A mesma idéia pode ser utilizada para efetuar a potenciação de uma matriz M^x .

[Voltar para lista de Divisão e Conquista](#)

Cálculo do n-ésimo termo da série de fibonacci

O n-ésimo termo da série de fibonacci pode ser calculado pela potenciação a seguir:

$$\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n = \begin{bmatrix} F(n+1) & F(n) \\ F(n) & F(n-1) \end{bmatrix}$$

Qual a complexidade do cálculo acima?

[Voltar para lista de Divisão e Conquista](#)

Algoritmo de Karatsuba para multiplicação de números grandes

- O algoritmo de Karatsuba é um método para multiplicar números grandes eficientemente, descoberto por Anatolii Alexeievitch Karatsuba em 1960 e publicado em 1962. Este algoritmo reduz a multiplicação de dois números de n dígitos a no máximo $O(n^{\log_2 3})$ multiplicações de dígitos simples e a exatamente $n^{\log_2 3}$ quando n é uma potência de 2.
- É mais rápido que o método usual de multiplicação longa, que necessita de n^2 multiplicações de um dígito simples.

- O passo básico do algoritmo de Karatsuba é a fórmula que permite computar o produto de dois números grandes x e y usando 3 multiplicações de números menores, cada um metade dos dígitos de x ou y , mais algumas somas e deslocamentos adicionais.
- Sejam x e y representados como strings de n dígitos em alguma base B . Os dois números podem ser representados como:
 - $x = x_1 B^m + x_0$
 - $y = y_1 B^m + y_0$
- e o produto xy pode ser calculado por
 - $xy = (x_1 B^m + x_0) * (y_1 B^m + y_0)$
 - $xy = z_2 B^{2m} + z_1 B^m + z_0$
- onde
 - $z_2 = x_1 * y_1$
 - $z_1 = (x_1 * y_0) + (x_0 * y_1)$
 - $z_0 = x_0 * y_0$

- Ex: $3456 * 4567 = (34*100+56) * (45*100+67) =$
- $(35*45) * 10000 + (34*67 + 56*45)*100 + 56*67$
- Essa fórmula requer 4 multiplicações. Karatsuba observou que z_1 pode ser calculado com apenas 1 multiplicação, ao custo de algumas adições extras.
- $z_1 = (x_1 + x_0)(y_1 + y_0) - z_2 - z_0$
- Dessa forma, pode-se multiplicar 2 números de n dígitos, com apenas 3 multiplicações de $n/2$ dígitos.
- Equação de recorrência e número de multiplicações?

Algoritmo de Strassen para Multiplicação de matrizes

- Assuma que são dadas duas matrizes $X[n][n]$ e $Y[n][n]$ de ordem grande (p.ex. 50.000).
- O algoritmo ingênuo irá multiplicar uma linha de X por uma coluna de Y obtendo um elemento da matriz resultante Z .
- Tempo total de $O(n^3)$.
- Pode-se usar divisão e conquista. Para simplicidade assuma que n é potência de 2.

Divida cada matriz em 4 de tamanho $\frac{n}{2} \times \frac{n}{2}$

$$X = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \quad Y = \begin{bmatrix} E & F \\ G & H \end{bmatrix} \quad Z = \begin{bmatrix} I & J \\ K & L \end{bmatrix}$$

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix} \begin{bmatrix} E & F \\ G & H \end{bmatrix} = \begin{bmatrix} I & J \\ K & L \end{bmatrix}$$

Assim:

- $I = AE + BG$
- $J = AF + BH$
- $K = CE + DG$
- $L = CF + DH$

- São necessárias 8 multiplicações de subproblemas de tamanho $n/2$
- Também é necessário utilizar tempo $O(n^2)$ para somar os resultados.
- Se $T(n)$ é o tempo para multiplicar duas matrizes de tamanho $n \times n$, então:

$$T(n) = 8T(n/2) + O(n^2)$$

- usando o teorema mestre: $T(n) = \Theta(n^{\log_2 8}) = \Theta(n^3)$.
- Pode-se obter um resultado melhor da seguinte forma:
- Calcule os 7 produtos a seguir (cada um consistindo de dois subproblemas de tamanho $n/2$):
 - $S1 = A(F - H)$
 - $S2 = (A+B)H$
 - $S3 = (C +D)E$
 - $S4 = D(G - E)$
 - $S5 = (A+D)(E +H)$
 - $S6 = (B - D)(G+H)$
 - $S7 = (A - C)(E +F)$

■ Então:

$$I = S5 + S6 + S4 - S2$$

$$= (A+D)(E+H) + (B-D)(G+H) + D(G-E) - (A+B)H$$

$$= AE + DE + AH + DH + BG - DG + BH - DH + DG - DE - AH - BH$$

$$= AE + BG$$

- Da mesma forma pode-se verificar que:

- $J = S1 + S2$

- $K = S3 + S4$

- $L = S1 - S7 - S3 + S5$

- Assim, para calcular I, J, K, e L, precisamos apenas calcular $S_1, \dots, S_7$ que requer a solução de 7 problemas de tamanho $n/2$ mais um número constante (no máximo 16) de somas, cada uma com custo $O(n^2)$.

$$T(n) = 7T(n/2) + O(n^2)$$

- Usando o teorema mestre e como $\log_2 7 = 2.808$:

$$T(n) = O(n^{2.808})$$

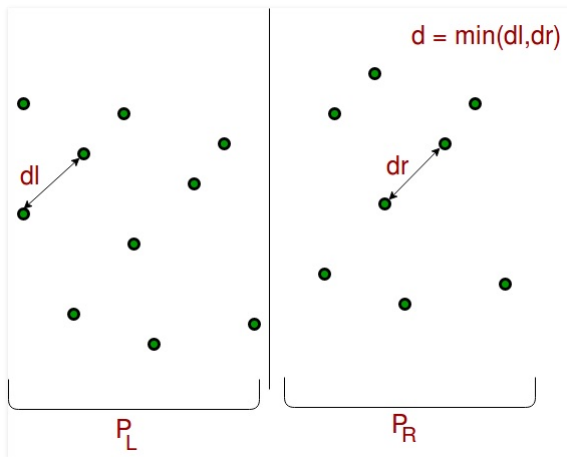
- Para $n = 50,000$: $n^3 = 10^{17}$ e $n^{2.808} = 10^{13}$; este algoritmo é 10.000 vezes mais rápido que o algoritmo de força bruta.

[Voltar para lista de Divisão e Conquista](#)

Encontrar pontos mais próximos entre si no plano cartesiano

- Uma abordagem de força bruta para encontrar o par de pontos mais próximos entre si no plano cartesiano é calcular a distância entre todos os pares de pontos, com custo $O(N^2)$.
- Uma abordagem mais eficiente é dividir o plano em duas metades, e encontrar recursivamente, para cada metade, o par de pontos mais próximos, e o par de pontos mais próximos em que cada ponto está em uma metade.
 - O par de menor distância entre esses 3 pares é o par mais próximo de todo o plano.

[Voltar para lista de problemas](#)



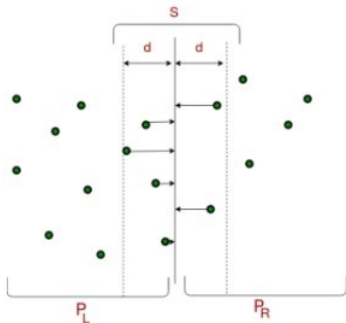
- Resultando a recorrência abaixo...

$$T(N) = \begin{cases} 1 & p/N == 2 \\ 2T(\frac{N}{2}) + (\frac{N}{2})^2 & p/N > 2 \end{cases}$$

- que resolvida resulta em $\Theta(N^2)$, mas...

- Tendo encontrado a menor distância d em cada lado do plano, pode-se usar essa distância para reduzir o número de pontos a serem testados em que cada ponto está em uma das metades.
- Pode-se mostrar que para cada ponto na faixa de um lado, precisa-se testar no máximo 7 pontos na faixa do outro lado.

► Explicação aqui



- Isso reduz a recorrência a

$$T(N) = \begin{cases} 1 & p/N == 2 \\ 2T(\frac{N}{2}) + cte & p/N > 2 \end{cases}$$

- O termo cte na recorrência acima se refere ao cálculo das distância entre pontos de lados distintos, limitado a 7 de cada lado, independente de N.
- A recorrência acima tem complexidade $\Theta(N)$ e o custo total do algoritmo é o custo da ordenação inicial dos pontos, de $\Theta(N \log N)$

Programação Dinâmica

- A abordagem de Divisão e Conquista é útil se o problema pode ser dividido em subproblemas com pouco esforço computacional e a soma dos tamanhos dos subproblemas é mantida pequena.
- Nas situações em que há sobreposição entre os subproblemas, a programação dinâmica normalmente conduz a algoritmos mais eficientes.

[Voltar para o índice](#)

- Soluções baseadas em programação dinâmica normalmente podem ser representadas por uma recorrência...

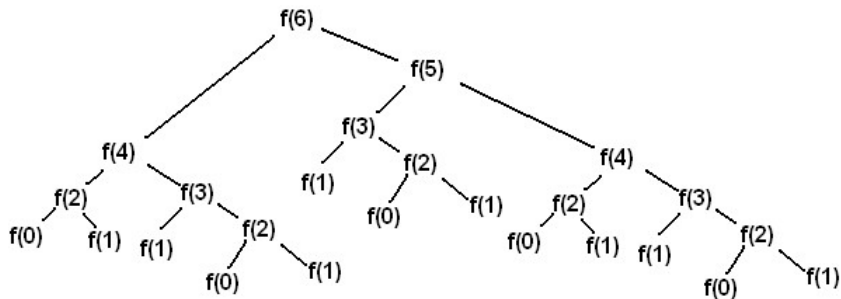
- **Definição:** Uma técnica algorítmica na qual um problema de otimização é resolvido através do armazenamento das soluções dos subproblemas, ao invés do recálculo das mesmas.
 - A obtenção da solução pode ser feita de forma recursiva, em uma abordagem *top-down*, partindo do problema inicial e armazenando os resultados de todos os subproblemas calculados, para que não sejam calculados mais de uma vez.
 - Esse armazenamento das soluções já obtidas na abordagem top-down é chamado *memoization*

- Duas condições que podem ser verificadas para avaliar o uso de Programação Dinâmica são a sobreposição de subproblemas e a subestrutura ótima.
- Subestrutura ótima se refere à propriedade de a solução ótima do problema poder ser obtida a partir da combinação das soluções ótimas dos subproblemas.

Ex.1: Um programa ingênuo para calcular números de Fibonacci:

```
int fib(n) {  
    if (n <= 1 ) return 1;  
    else return fib(n-1) + fib(n-2);  
}
```

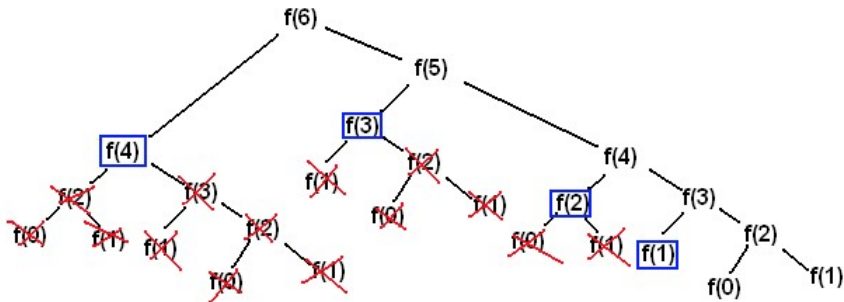
- As chamadas recursivas para o cálculo de fib(6) podem ser representadas pela árvore a seguir:



- Como *fib()* é recalculado diversas vezes sobre o mesmo argumento, o tempo de execução do algoritmo acima é $\Theta(1.6^n)$. Se, ao contrário, o valor de *fib(n)* for salvo a primeira vez que for calculado, o tempo de execução é $\Theta(n)$.
- Alocar e zerar vetor para memo;
- `memo[1] = memo[2] = 1;`

```
int fib(n) {  
    if (memo[n] != 0) return memo[n];  
    else  
        {memo[n] = fib(n-1) + fib(n-2);  
         return memo[n];}  
}
```

E a árvore resultante é:



Obs : Números de fibonacci podem ser calculados em tempo constante.

$$F(n) = \text{round}\left(\frac{s^n}{\sqrt{5}}\right)$$

onde $s = \frac{1+\sqrt{5}}{2}$ (que é o limite da série $\frac{\text{fib}(n)}{\text{fib}(n-1)}$, também chamado de golden number).

- A programação dinâmica também pode ser implementada em uma abordagem *bottom-up*, resolvendo inicialmente os problemas menores e, a partir deles, ir compondo a solução dos problemas maiores, até resolver o problema original.
- Essa abordagem também é chamada de tabulação, porque normalmente monta-se uma tabela para as soluções dos subproblemas.

```
int fib(n) {  
    memo[0]=0;  
    memo[1]=1;  
    for (i=2;i<=n;i++)  
        memo[i]=memo[i-1]+memo[i-2];  
    return memo[n];  
}
```

ou, como para calcular cada elemento bastam os dois anteriores:


```
int fib(n) {  
    m0=0;  
    m1=1;  
    for (i=2;i<=n;i++){  
        mn=m0+m1;  
        m0=m1;  
        m1=mn;}  
    return mn;  
}
```

- Programação dinâmica é utilizada quando os subproblemas não são independentes, ou seja, compartilham alguns subsubproblemas.
- Neste caso a abordagem de divisão e conquista executa mais trabalho que o necessário.
- A programação dinâmica resolve cada subproblema uma única vez e armazena o resultado, evitando recalcular a resposta de um subproblema cada vez que ele for encontrado.

- A estrutura de dados para armazenar os resultados bem como a escolha entre *bottom-up* e *top-down* depende da estrutura dos subproblemas.
- Em problemas em que para a solução são necessários poucos subproblemas, uma abordagem *top-down* pode levar a uma implementação mais eficiente. E o uso de uma tabela hash ou um dicionário pode levar a um uso mais eficiente de espaço.
- Em situações em que todos os subproblemas devem ser resolvidos, uma abordagem *bottom-up* e o uso de uma tabela pode levar a um resultado mais eficiente, por não acarretar o custo das chamadas recursivas

Problemas clássicos (e menos clássicos) usando Programação Dinâmica:

- 1 Problema da Mochila sem restrições
- 2 Problema da Mochila 0-1
- 3 Árvore Binária de Pesquisa Ótima
- 4 Problema do Caixeiro Viajante
- 5 Algoritmos sobre strings e sequências
- 6 Coin Change (Problema do Troco)
- 7 Intervalo de maior soma em vetor (Kadane)
- 8 Soma máxima em matriz

Problemas clássicos etc. (continuação)

- 9 Caixeiro Viajante Unidirecional
- 10 Melhor sequência de Produtos Matriciais
- 11 Maior sequência decrescente não contígua
- 12 Soma de Subconjuntos
- 13 Soma de substrings
- 14 Job Sequencing
- 15 Problemas de *stock trading*

1) Problema da mochila sem restrições

- Se todos os pesos ($w_1, w_2 \dots w_n, W$) são inteiros não-negativos, o problema da mochila pode ser resolvido utilizando programação dinâmica em tempo pseudo-polinomial (proporcional ao valor da entrada, e não ao número de dígitos da mesma) .
- A solução para o problema da mochila sem restrições utilizando programação dinâmica pode ser descrita da seguinte forma:

- Para simplificar as coisas, assuma que todos os pesos são estritamente positivos ($w_i > 0$).
- Queremos maximizar o valor sujeitos à restrição que o peso total é menor ou igual a W .
- Então, para cada valor $w \leq W$, definimos $m[w]$ como o valor máximo que pode ser obtido com um peso total menor que ou igual a W .
- $m[W]$ é então a solução para o problema.

Observe que $m[w]$ tem as seguintes propriedades:

- $m[0] = 0$ (a soma de zero items, i.e, do conjunto vazio)
- $m[w] = \max(m[w - 1], \max_{w_i \leq w} (v_i + m[w - w_i]))$

onde v_i é o valor do i -ésimo item.

- Aqui considera-se como zero o valor máximo para o conjunto vazio.
- A tabulação dos resultados de $m[0]$ até $m[W]$ fornece a solução.
- Uma vez que a solução de cada $m[w]$ envolve o exame de n itens, e que há W valores de $m[w]$ para calcular, o tempo de execução da solução utilizando programação dinâmica é $O(nW)$.
- A divisão de $w_1, w_2, \dots, w_n, W$ pelo seu maior divisor comum é uma forma de melhorar o desempenho do algoritmo.

Uma implementação *top-down*:

```
int peso[2]={5,4};
int valor[2]={10,6};
int W=8;
int m[9]={0,-1,-1,-1,-1,-1,-1,-1,-1};

int knapDP(int w)
{
    if (m[w]!=-1) return m[w];
    int max=0,j;
    for (j=0;j<2;j++)
        if (peso[j]<=w && valor[j]+knapDP(w-peso[j])>max)
            max=valor[j]+m[w-peso[j]];
    m[w]=max;
    return m[w];
}

int main() {
    int valor=knapDP(8);
    return 0;}
```

E uma implementação *bottom-up*:

```
int peso[2]={5,4};
int valor[2]={10,6};
int W=8,w,j;
int m[11];
int main() {
    for (w=0;w<=W;w++)
    {
        int max=0;
        for (j=0;j<2;j++)
        {
            if (peso[j]>w) continue;
            if (valor[j]+m[w-peso[j]]>max)
                max=valor[j]+m[w-peso[j]];
        }
        m[w]=max;
    }
    return 0;
}
```

- Sugestão de exercícios:
 - URI 1487 - Six Flags
 - URI 1798 - Cortando Canos
- A complexidade $O(nW)$ não contradiz o fato que o problema da mochila é NP-completo, já que W , ao contrário de n , não é polinomial no comprimento da entrada do problema.
- O tamanho da entrada do problema é proporcional ao número $\log W$, número de bits em W , e não a W .
- Pode-se reconstituir o conjunto de objetos que compõe a melhor solução, associando a cada posição do vetor, o último objeto que melhorou o valor daquela posição.

2) Problema da mochila 0-1

- Uma solução similar para o problema da mochila 0-1 utilizando programação dinâmica também roda em tempo pseudo-polinomial.
- Como acima, assuma que $w_1, w_2, \dots, w_n, W$ são inteiros estritamente positivos.
- Defina $m[i, w]$ como o máximo valor que pode ser obtido com peso menor ou igual a w usando itens até o i .

- A figura abaixo mostra o preenchimento da tabela para 4 objetos e um limite de peso $W=10$

i	1	2	3	4
v_i	10	40	30	50
w_i	5	4	6	3

$V[i, w]$	0	1	2	3	4	5	6	7	8	9	10
$i = 0$	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	10	10	10	10	10	10
2	0	0	0	0	40	40	40	40	40	50	50
3	0	0	0	0	40	40	40	40	40	50	70
4	0	0	0	50	50	50	50	90	90	90	90

Podemos definir $m[i, w]$ recursivamente como segue:

- $m[0, w] = 0$
- $m[i, 0] = 0$
- $m[i, w] = m[i - 1, w]$; se $w_i > w$ (o novo item pesa mais que o limite da mochila)
- $m[i, w] = \max(m[i - 1, w], m[i - 1, w - w_i] + v_i)$; se $w_i \leq w$.

- A solução pode ser encontrada calculando $m[n, W]$.
- Para fazer isso eficientemente pode-se usar uma tabela para armazenar as computações anteriores.
- Essa solução roda em tempo $O(nW)$ e espaço $O(nW)$.
- Se for buscado apenas o valor da solução, e não os objetos que a compõem, como para cada linha da matriz é utilizada somente a linha anterior, ao invés de manter uma matriz pode-se manter apenas a linha atual e a linha anterior.

- Pode-se usar o mesmo vetor para a situação atual e a anterior (preenchendo-o do fim para o início. Por que?) Isso pode ser feito apenas na solução bottom-up.
- Se o valor e o peso de cada objeto são o mesmo valor, o problema é conhecido como problema da **soma de subconjuntos (subset sum)**

■ Exercícios de Mochila 0-1 do URI Online Judge:

- 1203 - Pontes de São Petersburgo
- 1286 - Motoboy
- 1288 - Canhão de Destruição
- 1624 - Promoção
- 1767 - Saco do Papai Noel
- 2026 - Árvore de Natal
- 2228 - Caça ao Tesouro
- 1970 - Primeiro Contato (múltiplas mochilas)

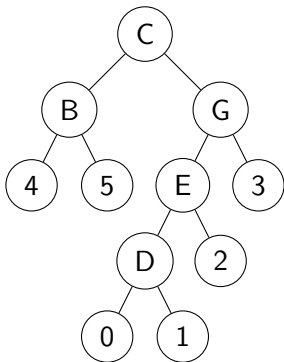
[Voltar para lista de problemas de PD](#)

- Uma variante do problema da mochila é tentar identificar a solução que MINIMIZA algum valor, ao invés de maximizá-lo.
- As abordagens vistas aqui podem ser aplicadas, com poucas alterações.
- Ex: URI 2532 - Demogorgon.
- Exercício do problema múltiplo da mochila:
- URI 1970 - Primeiro Contato.

3) Árvore binária de pesquisa ótima

- Definição da ABP: Uma árvore binária de pesquisa (ABP) T é uma árvore binária; T pode estar vazia ou então cada nodo da árvore contém um identificador, e:
 - 1 Todos os identificadores na sub-árvore à esquerda (direita) de T são "menores" ("maiores") do que o identificador na raiz de T ;
 - 2 As sub-árvores à esquerda e à direita de T são ABP.
- Obs: Todos os identificadores são únicos.

- Representação gráfica de uma ABP (os nodos maiores representam os identificadores, os nodos menores representam os nodos onde termina a busca caso o identificador buscado não esteja em T).



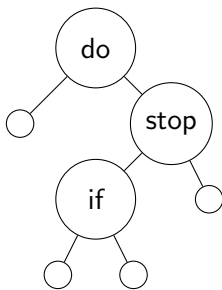
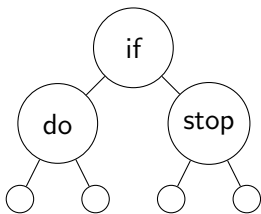
- Há situações em que sabemos com antecedência quais as chaves que serão buscadas e, mais ainda, com que frequência ou probabilidade cada chave será buscada.
- Nesse caso, pode-se construir uma árvore binária de busca ótima que será eficiente para a operação de busca. Inserções e remoções não são usualmente efetuadas na árvore assim construída pois elas podem modificar a sua estrutura.
- Caso inserções e remoções são permitidas, então se deve periodicamente reconstruir a árvore ótima.

- A melhor árvore binária de busca depende das probabilidades de acesso das chaves. Quando a distribuição dessas probabilidades não é uniforme, a melhor árvore binária de busca não é necessariamente uma árvore binária perfeitamente balanceada. Intuitivamente, queremos colocar as chaves mais buscadas mais próximas ao topo da árvore binária.
- Vamos estudar como construir uma árvore binária de busca ótima. Mas, antes, temos que definir o que se entende por árvore binária de busca ótima.

- conjunto de identificadores é $A = a_1, \dots, a_n$ onde $a_1 < a_2 < \dots < a_n$
- P_i é a probabilidade de se pesquisar a_i
- Q_i é a probabilidade de se pesquisar por X onde $a_i < X < a_{i+1}$, ou seja, pesquisar por um identificador não existente entre a_i e a_{i+1} . O nodo fictício (nodo menor) onde estaria este identificador inexistente será representado por E_i .
- Probabilidade de ocorrer um identificador inexistente :
$$\sum_{i=0}^n Q_i$$
- $$\sum Q_i + \sum P_i = 1$$

- Dados A , P_i e Q_i busca-se construir uma ABP ótima (com custo mínimo).
 - Se $X \in T$ e acharmos X no nível k , serão feitas k comparações
 - Se $X \in T$, paramos no nível k após $k - 1$ passos (chegando a um nodo fictício).
 - O custo da ABP (custo médio de pesquisa) é dado por
 - $\sum (P_i * Nivel(a_i)) + \sum Q_i * Nivel(E_i) - 1$
 - O objetivo é minimizar C .
 - Ex:
 - $A = \{do, if, stop\}$
 - $P_i = Q_i = \frac{1}{7}$

Desenhe as ABPs possíveis e calcule o custo de cada uma delas.



A primeira tem custo $13/7$ (ótima) e todas as variantes da segunda tem custo $15/7$.

- Se tivéssemos

- $Q_0 = .15$
- $P_1 = .5$
- $Q_1 = .1$
- $P_2 = .1$
- $Q_2 = .05$
- $P_3 = .05$
- $Q_3 = .05$

- Qual seria a ABP
ótima?

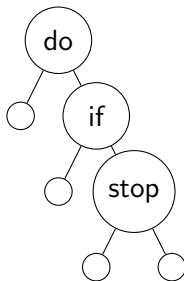
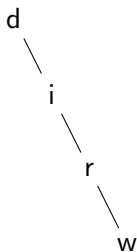


Figura: Custo=1.5

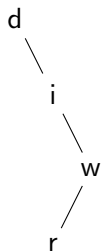
- Como construir a ABP ótima?
- Primeira solução : Gere todas as árvores possíveis e calcule o custo de cada uma.
- Algoritmo para gerar todas as árvores :
- Para cada identificador em A :
 - 1 escolha a_i como raiz
 - 2 recursão com a_1 até $a_i - 1$ à esquerda
 - 3 recursão com a_{i+1} até a_n à direita
- Número de árvores geradas : $O(\frac{4^n}{n^{3/2}})$

- Este número pode ser reduzido através da programação dinâmica.
- Ex: $A = \{\text{do, if, read, while}\}$

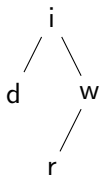
Se soubermos que



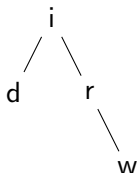
é melhor que



então não adianta testar



porque certamente



será melhor devido a r — w (princípio da otimalidade)

- Solução melhor utilizando programação dinâmica : começa com árvores de 1 elemento, então de 2 elementos, etc. Usa a informação já calculada sobre árvores pequenas para gerar (ou decidir não gerar) árvores maiores.

Lista de Problemas de Programação Dinâmica

4) Problema do Caixeiro Viajante

- Considere um grafo orientado com custos c_{ij} associados a cada aresta.
- O Problema do caixeiro viajante consiste em encontrar o ciclo hamiltoniano de menor custo.
- Utilizando programação dinâmica é possível obter um algoritmo $O(n^2 \cdot 2^n)$, que apesar de exponencial, é mais rápido que o algoritmo de gerar todos os caminhos possíveis, que é da ordem de fatorial de n .

- Considere que o percurso ótimo começa e termina no vértice 1.
- Qualquer ciclo é constituído de uma aresta $(1, k)$, $2 \leq k \leq n$, e um caminho M de k até 1.
- Se o ciclo é de custo mínimo, o caminho M deve ser de custo mínimo (princípio da optimalidade).

- Seja $f(i, C)$ o custo do caminho mínimo do vértice i até 1, e que visita todos os vértices do conjunto C . Do parágrafo anterior tem-se

$$f(1, VG - 1) = \min\{c_{ij} + f(k, VG - \{1, k\})\}(1)$$

- Se conhecemos os valores de $f(k, VG - \{1, k\})$ para todo $k, 2 \leq k \leq n$, pela equação acima podemos resolver o PCV.
- Podemos generalizar e obter

$$f(i, C) = \min_j\{c_{ij} + f(j, C - \{j\})\}(2)$$

- isto é, o caminho mínimo do vértice i até 1 que passa por todos os vértices em C é constituído por (i, j) e um caminho de j até 1 que visita todos os vértices em C , para uma escolha adequada de j .
- Os valores de f necessários em (1) para se resolver o PCV podem ser obtidos de (2) de uma forma ascendente, pelo algoritmo a seguir.
 - 1 $|C| = 0, f(i, \emptyset) = c_{i1}$, para $1 < i \leq n$.
 - 2 Para $|C| = 1, f(i, \{j\}) = c_{ij} + f(j, \emptyset) = c_{ij} + c_{j1}$, para $1 < i, j \leq n$ e $i \neq j$
 - 3 Para $|C| = 2, 3, \dots, n - 2$, determinam-se os valores de $f(i, C)$ através de (2) utilizando-se os valores de f calculados nas iterações anteriores. Deve-se considerar $1 < i \leq n$ e conjuntos C tais que $C \cap \{1, i\} = \emptyset$

Ex: considere o grafo representado pela matriz de adjacências a seguir:

	1	2	3	4
1	0	2	10	7
2	20	0	9	1
3	1	5	0	15
4	7	12	3	0

Passo 1 - ($|C| = 0$):

$$f(2, \emptyset) = c_{21} = 20$$

$$f(3, \emptyset) = c_{31} = 1$$

$$f(4, \emptyset) = c_{41} = 7$$

Passo 2 - ($|C| = 1$):

$$f(2, \{3\}) = c_{23} + f(3, \emptyset) = c_{23} + c_{31} = 9 + 1 = 10$$

$$f(2, \{4\}) = c_{24} + f(4, \emptyset) = c_{24} + c_{41} = 1 + 7 = 8$$

$$f(3, \{2\}) = c_{32} + f(2, \emptyset) = c_{32} + c_{21} = 5 + 20 = 25$$

$$f(3, \{4\}) = c_{34} + f(4, \emptyset) = c_{34} + c_{41} = 15 + 7 = 22$$

$$f(4, \{2\}) = c_{42} + f(2, \emptyset) = c_{42} + c_{21} = 12 + 20 = 32$$

$$f(4, \{3\}) = c_{43} + f(3, \emptyset) = c_{43} + c_{31} = 3 + 1 = 4$$

Passo 3 - ($|C| = 2$):

$$f(2, \{3, 4\}) = \min(c_{23} + f(3, \{4\}), c_{24} + f(4, \{3\})) = \min(9 + 22, 1 + 4) = 5(j = 4)$$

$$f(3, \{2, 4\}) = \min(c_{32} + f(2, \{4\}), c_{34} + f(4, \{2\})) = \min(5 + 8, 15 + 32) = 13(j = 2)$$

$$f(4, \{2, 3\}) = \min(c_{42} + f(2, \{3\}), c_{43} + f(3, \{2\})) = \min(12 + 10, 3 + 25) = 22(j = 3)$$

Passo 4 - ($|C| = 3$)

$$f(1, \{2, 3, 4\}) = \min(c_{12} + f(2, \{3, 4\}), c_{13} + f(3, \{2, 4\}), c_{14} + f(4, \{2, 3\})) = \min(2 + 5, 10 + 13, 7 + 28) = 7(j = 2)$$

- Assim, como $j = 2$ no cálculo de $f(1, \{2, 3, 4\})$ a primeira aresta do caminho mínimo é $(1,2)$.
- Prosseguindo, como $j = 4$ no cálculo de $f(2, \{3, 4\})$, a segunda aresta é $(2,4)$.
- A terceira aresta é $(4,3)$ pois $j = 3$ no cálculo de $f(4, 3)$.
- Portanto, o ciclo de custo mínimo neste exemplo é $(1, 2, 4, 3, 1)$, com custo 7.

Algoritmos sobre strings

- Diversos algoritmos de operações sobre strings ou sequências de valores são baseados em programação dinâmica, havendo uma base comum nas recorrências que os descrevem.
- Longest Common Subsequence
- Shortest Common Supersequence
- Distância de Edição (Levenshtein)
- Maior substring comum a dois strings

5) Encontrar a maior subsequência comum a duas seqüências (lcs - longest common subsequence)

- Dadas duas seqüência de itens, encontrar a maior subsequência presente em ambos.
- Uma subsequência é uma seqüência que aparece na mesma ordem relativa, mas não necessariamente contíguos.
- Por exemplo, na seqüência "abcdefg", "abc", "abg", "bdf", "aeg" são subsequências.

- Um algoritmo exponencial ingênuo é notar que uma cadeia de comprimento n possui $O(2^n)$ subsequências diferentes, tomar a cadeia mais curta, e testar a presença de cada uma das suas subsequências na outra cadeia, em uma abordagem gulosa.

A solução recursiva

- Pode-se resolver o problema em termos de subproblemas menores.
- Dadas duas seqüências X e Y , de comprimento n e m , respectivamente.
- Se a última letra dos dois strings é igual, essa letra faz parte da maior sequência comum (lcs). Por ex:
 $\text{lcs}(\text{"Maria"}, \text{"Joana"})$ é $\text{lcs}(\text{"Mari"}, \text{"Joan"}) + 'a'$.
- Se a última letra dos dois strings é diferente, a última letra de pelo menos um dos strings não faz parte da solução.
 $\text{lcs}(\text{"Mario"}, \text{"Joana"}) = \max(\text{lcs}(\text{"Mario"}, \text{"Joan"}), \text{lcs}(\text{"Mari"}, \text{"Joana"}))$

- Resolve-se o problema de encontrar o maior subseqüência comum de $x = x_1 \dots x_n$ e $y = y_1 \dots y_m$ tomando o melhor dos três casos possíveis:
 - 1 Se x_n é igual a y_m , a maior subseqüência comum dos strings $x_1 \dots x_{n-1}$ e $y_1 \dots y_{m-1}$, seguido pelo último caracter comum.
 - 2 A maior subseqüência comum dos strings $x_1 \dots x_{n-1}$ e $y_1 \dots y_m$.
 - 3 A maior subseqüência comum dos strings $x_1 \dots x_n$ e $y_1 \dots y_{m-1}$.
- Caso base: quando uma das seqüências é vazia, a subseqüência comum é a seqüência vazia de comprimento 0.

- Que pode ser descrito pela recorrência a seguir:

$$LCS[i][j] = \begin{cases} 0 & p/i == 0 \\ 0 & p/j == 0 \\ LCS[i-1][j-1] + 1 & p/X[i] == Y[j] \\ \max(LCS[i-1][j], LCS[i][j-1]) & p/X[i] \neq Y[j] \end{cases}$$

Assim, o tamanho da maior sequência comum aos strings "Sunday" e "Saturday" pode ser obtido pela tabela a seguir:

		S	a	t	u	r	d	a	y
	0	0	0	0	0	0	0	0	0
S	0	1	1	1	1	1	1	1	1
u	0	1	1	1	2	2	2	2	2
n	0	1	1	1	2	2	2	2	2
d	0	1	1	1	2	2	3	3	3
a	0	1	2	2	2	2	3	4	4
y	0	1	2	2	2	2	3	4	5

A partir disso pode-se construir uma solução recursiva (em C):

```
int longest(char x[],char y[])
{
    char x_1[100],y_1[100];
    int tamx=strlen(x);
    int tamy=strlen(y);
    int lcs1,lcs2;
    if (tamx==0 || tamy==0) /* se alguma das strings tem tamanho zero, lcs=0 */
        return 0;
    strncpy(x_1,x,tamx-1);x_1[tamx-1]=0;
    strncpy(y_1,y,tamy-1);y_1[tamy-1]=0; /*calcula as strings sem o último caracter para
    if (x[tamx-1]==y[tamy-1])
        return 1+longest(x_1,y_1);
    lcs1=longest(x,y_1);
    lcs2=longest(x_1,y);
    if (lcs1>lcs2) return lcs1;
    else return lcs2;
}
```


Solução com programação dinâmica

- A solução acima não é muito eficiente, devido à sobreposição de subproblemas.
- Uma solução mais eficiente é armazenar os resultados das comparações dos prefixos de ambos strings (*memoization*).
- Isso pode ser feito utilizando uma abordagem top-down ou uma abordagem bottom-up
- Na abordagem bottom-up, encontramos $\text{lcs}(x_1 \dots x_i, y_1 \dots y_j)$ para cada i e j , a partir das subsequências menores, armazenando os resultados em uma matriz no índice (i, j) à medida que avançamos .
- URI 2824 - Pudim

```

int longest_bottom_up(char x[], char y[])
{
    int tamx=strlen(x);
    int tamy=strlen(y);
    for (int i=0;i<=tamx;i++)
        for (int j=0;j<=tamy;j++)
            if (i==0 || j==0) dist[i][j]=0;
            else if (x[i-1]==y[j-1])
                dist[i][j]=dist[i-1][j-1]+1;
            else if (dist[i-1][j]>dist[i][j-1])
                dist[i][j]=dist[i-1][j];
            else dist[i][j]=dist[i][j-1];
    return dist[tamx][tamy];
}

```

■ E uma abordagem top-down:

```
int lcs_topdown(char x[], char y[], int i, int j)
{
    if (i==0 || j==0) return 0;
    if (tamlcs[i][j]!=-1) return tamlcs[i][j];
    if (x[i]==y[j])
    {
        tamlcs[i][j]=1+lcs_topdown(x,y,i-1,j-1);
        return tamlcs[i][j];
    }
    int tam1=lcs_topdown(x,y,i-1,j);
    int tamj1=lcs_topdown(x,y,i,j-1);
    if (tam1>tamj1) tamlcs[i][j]=tam1;
    else tamlcs[i][j]=tamj1;
    return tamlcs[i][j];
}
```

6) Encontrar a menor seqüência que contenha duas seqüências A e B.

- Dadas duas seqüência de itens, encontrar a menor seqüência que contenha ambas como subsequências.
- Uma subsequência é uma seqüência que aparece na mesma ordem relativa, mas não necessariamente contíguos.
- Por exemplo, na seqüência "abcdefg", "abg", "bdf" e "aeg" são subsequências.

A solução recursiva

- Pode-se definir uma função $\text{scs}(X,Y)$, que retorna o comprimento da menor sequência que contém X e Y .
- Essa função pode ser definida em termos de instâncias menores.
- Dadas duas seqüências X e Y iniciadas na posição 1, de comprimento n e m respectivamente.
- Se o último caracter de X e Y é o mesmo, o comprimento da maior subsequência é $1 + \text{scs}(X_1..X_{n-1}, Y_1..Y_{m-1})$
- Se o último caracter de X e Y é diferente, $\text{scs}(X,Y)$ é $1 + \min(\text{scs}(X_1..X_{n-1}, Y), \text{scs}(X, Y_1..Y_{m-1}))$
- Caso base: quando uma das seqüências é vazia, a seqüência resultante é a outra seqüência.

$$SCS[i][j] = \begin{cases} j & p/i == 0 \\ i & p/j == 0 \\ SCS[i-1][j-1] + 1 & p/X[i-1] == Y[j-1] \\ \min(SCS[i-1][j] + 1, SCS[i][j-1] + 1) & p/X[i-1] \neq Y[j-1] \end{cases}$$

■ URI 2842 - Dabriel e suas Strings

7) Distância de Levenshtein(ou Edit Distance)

- Em Ciência da Computação, a distância de Levenshtein é uma métrica para medir a diferença entre duas seqüências (p.ex. dois strings).
- Ela é definida como o número mínimo de operações para transformar um string no outro, em que as operações permitidas são inserção, deleção ou substituição de um caracter.
- É usada em buscas por proximidade, como em pesquisas no google, pesquisa por chaves aproximadas em bancos de dados (link de implementação em SQL [▶ aqui](#)) e até detecção de plágio (em texto ou código).

- Extensões do algoritmo, como o algoritmo de Smith-Waterman são utilizadas para alinhamento de DNA.
- Algoritmos semelhantes são usados para ajuste entre séries temporais (Dynamic Time Warping).
- Variantes do algoritmo incluem versões em que apenas algumas operações são permitidas, ou pesos diferentes são atribuídos às operações.

- O string "Sunday" pode ser obtido do string "Saturday" a partir das 3 operações a seguir:
 - 1 deleção do primeiro "a" (restando "Sturday")
 - 2 deleção do "t" (restando "Surday")
 - 3 substituição do "r" por "n" (chegando em "Sunday")

- A distância de Levenshtein entre dois strings X [até a posição i] e Y [até a posição j] pode ser definida pela recorrência a seguir:

$$lev[i][j] = \begin{cases} j & i = 0 \\ i & j = 0 \\ lev[i-1][j-1] & X[i-1] == Y[j-1] \\ \min(lev[i-1][j], lev[i][j-1], lev[i-1][j-1]) + 1 & X[i-1] \neq Y[j-1] \end{cases}$$

- Na recorrência acima, a matriz é definida para cada prefixo (substring do início da string) $X[i]$ e $Y[j]$.
- As strings X e Y iniciam na posição 0, que corresponde à coluna e linha 1 da matriz, por isso o $i-1$ e $j-1$ nas referências aos strings.

A distância entre dois strings pode ser obtido por programação dinâmica da forma mostrada no código a seguir:

```
int LevenshteinDistance(char s[1..m], char t[1..n])
{
  // d is a table with m+1 rows and n+1 columns
  declare int d[0..m, 0..n]

  for i from 0 to m
    d[i, 0] := i // deletion
  for j from 0 to n
    d[0, j] := j // insertion
  for j from 1 to n {
    for i from 1 to m {
      if s[i] = t[j] then
        d[i, j] := d[i-1, j-1]
      else
        d[i, j] := minimum (
          d[i-1, j] + 1, // remoção em t
          d[i, j-1] + 1, // inserção em t
          d[i-1, j-1] + 1 // substituição
        )
    }
  }
  return d[m,n]}
```

e a tabela resultante no cálculo da distância entre "Sunday" e "Saturday" é mostrada a seguir:

		S	a	t	u	r	d	a	y
	0	1	2	3	4	5	6	7	8
S	1	0	1	2	3	4	5	6	7
u	2	1	1	2	2	3	4	5	6
n	3	2	2	2	3	3	4	5	6
d	4	3	3	3	3	4	3	4	5
a	5	4	3	4	4	4	4	3	4
y	6	5	4	4	5	5	5	4	3

- A sequência de operações pode ser obtida retroativamente a partir da última posição buscando o caminho das menores diferenças.
- Levenshtein no URI:
 - 1833 - Decoração Natalina
 - 2825 - L de Atreus

Maior substring comum a duas strings

- O maior substring comum a duas strings $S1$ e $S2$ pode ser encontrada guardando, em uma matriz de pd, para cada posição (i,j) , o comprimento do maior sufixo comum até $S1[i]$ e $S2[j]$.
- Isso pode ser expresso pela recorrência a seguir:

$$msc[i][j] = \begin{cases} 0 & p/i == 0 \text{ ou } j == 0 \\ msc[i-1][j-1] + 1 & p/X[i] == Y[j] \\ 0 & p/X[i-1] \neq Y[j-1] \end{cases}$$

- E o tamanho da maior substring é o maior valor da matriz.

- A tabela abaixo mostra o tamanho dos substrings comuns a "abacate" e "facas"

		A	b	a	c	a	t	e
	0	0	0	0	0	0	0	0
f	0	0	0	0	0	0	0	0
a	0	0	0	1	0	1	0	0
c	0	0	0	0	2	0	0	0
a	0	0	1	0	0	3	0	0
s	0	0	0	0	0	0	0	0

- E a complexidade da solução é, para dois strings de tamanhos M e N , $\Theta(M.N)$.

Maior subsequência palíndroma

■ ?????????????

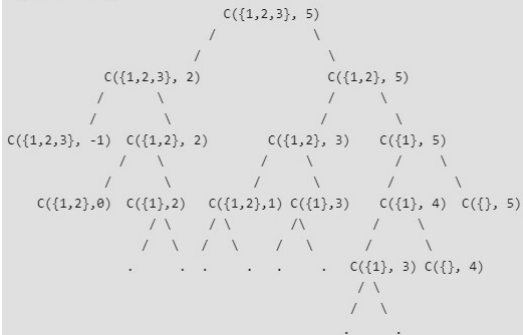
8) Problema do Troco (Coin Change)

- Dado um valor N e uma quantidade infinita de moedas de valores $S = \{S_1, S_2, S_3, \dots, S_n\}$, de quantas maneiras diferentes pode-se obter o valor N com as moedas? Considere que a ordem das moedas não interessa.
- Por exemplo, para $N = 4$ e $S = \{1, 2, 3\}$, há 4 soluções: $\{1, 1, 1, 1\}, \{1, 1, 2\}, \{2, 2\}, \{1, 3\}$. Para $N = 10$ e $S = \{2, 5, 3, 6\}$, há 5 soluções : $\{2, 2, 2, 2, 2\}, \{2, 2, 3, 3\}, \{2, 2, 6\}, \{2, 3, 5\}$ e $\{5, 5\}$.

- Esse problema também pode ser formulado como verificar se há alguma combinação das moedas cuja soma resulte no valor.
 - Nesse caso o problema pode ser resolvido como uma instância do problema da mochila (0-1 ou ilimitada)
- Outra possibilidade é contar o número de soluções diferentes, mas permitindo que cada moeda ocorra apenas uma vez.
- Ou encontrar o número mínimo de moedas para obter um determinado valor (pode ser representado como uma instância do problema da mochila)

- Para contar o total de soluções, pode-se dividir o conjunto de soluções em 2 subconjuntos:
 - 1) Soluções que não contem a m-ésima moeda (S_m).
 - 2) Soluções que contem pelo menos uma m-ésima moeda (S_m)
- Seja $\text{count}(S[], m, n)$ a função que conta o número de soluções que contem moedas até a m-ésima e que totalizam o valor n. Ela pode ser escrita como a soma de $\text{count}(S[], m-1, n)$ e $\text{count}(S[], m, n-S_m)$.
- Logo, o problema tem a propriedade de subestrutura ótima e pode ser resolvido usando as soluções dos subproblemas.
- Obs: diz-se que um problema tem a propriedade de **subestrutura ótima** quando a solução ótima pode ser obtida a partir das soluções ótimas para os subproblemas.

C() --> count()



- O código na lâmina seguinte implementa a seguinte recursão:
 - $C(S,m,N)=C(S,m-1,N)+C(S,m,N-S_m)$
- com os casos-bases:
 - $C(S,m,N)=1, N=0$ (solução única, que é nenhuma moeda)
 - $C(S,m,N)=0, N<0$ (nenhuma solução - quantidade negativa)
 - $C(S,m,N)=0, N \geq 1, m \leq 0$ (nenhuma solução - nenhuma moeda)
- Nessa recursão, há sobreposição de problemas?
- Se busca-se somente verificar a existência de uma solução, ao invés da quantidade delas, a recorrência torna-se:
- $C(S,m,N)=C(S,m-1,N) \parallel C(S,m,N-S_m)$

- Se o problema não permite a repetição da mesma moeda, a possibilidade de usar a moeda m no próximo nível é excluída nas duas chamadas e a recorrência torna-se:
- $C(S, m, N) = C(S, m-1, N) \parallel C(S, m-1, N - S_m)$
- A versão na lâmina seguinte armazena em uma tabela as soluções parciais ainda não calculadas para evitar de calculá-las mais de uma vez.

```

int S[]={2,3,5,6};
int M=sizeof(S)/sizeof(S[0]);

int C(int S[], int m, int N)
{
    if (N==0) return 1;
    if (N<0 || m<0) return 0;
    return C(S,m-1,N)+C(S,m,N-S[m]);
}

int main()
{
    int N=10;
    printf("%d\n",C(S,M-1,N));
    getch();
}

```

```

int S[]={1,2,3,4,5,6,7,8,9,10};
int M=sizeof(S)/sizeof(S[0]);
#define NMax 200
#define MMax 200
int T[MMax][NMax];

int C(int S[], int m, int N)
{
    if (N==0) return 1;
    if (N<0 || m<0) return 0;
    if (T[m-1][N]==-1) T[m-1][N]=C(S,m-1,N);
    if (T[m][N-S[m]]==-1) T[m][N-S[m]]=C(S,m,N-S[m]);
    return T[m-1][N]+T[m][N-S[m]];
}

int main()
{
    int N=120,i,j;
    for (i=0;i<MMax;i++)
        for (j=0;j<NMax;j++)
            T[i][j]=-1;
    printf("%d\n",C(S,M-1,N));
    getch();
}

```


- URI 2446 - Troco
 - bottom-up (mochila) - 0.226 ms
 - top-down - 0.000 ms
- HackerRank - Algorithms → Dynamic Programming → The Coin Change Problem

Intervalo de maior soma em vetor

- Faça um programa que, dado um conjunto de n inteiros (possivelmente negativos) $a_1, a_2, \dots, a_n$, encontre o máximo valor de $\sum_{k=i}^j a_k$.
- Se todos os números são negativos, considere que a maior soma é 0.
- Analise a complexidade do algoritmo.
- Ex. Qual o intervalo de maior soma no vetor abaixo?
 - 3 2 -4 -2 3 4 -3 4 -2 3 -5

Versão 1: n^3

```
int SubsequenciaMaiorSomaN3(int *vet, int n)
{
    int estasoma, maiorsoma, ini, fim, i, j, k;
    maiorsoma=0;
    for (i=0; i<n; i++)
        for (j=i; j<n; j++)
            {
                estasoma=0;
                for (k=i; k<=j; k++)
                    estasoma=estasoma+vet[k];
                if (estasoma>maiorsoma)
                    maiorsoma=estasoma;
            }
    return maiorsoma;
}
```

Escolhendo a soma como operação elementar:

Representando como somatórios:

$$\sum_{i=1}^n \sum_{j=i}^n \sum_{k=i}^j 1$$

Laço interno:

$$\sum_{k=i}^j 1 = 1 \cdot (j - i + 1) = j - i + 1$$

Segundo laço:

$$\sum_{j=i}^n j - i + 1 = \frac{(i - i + 1) + (n - i + 1)}{2} (n - i + 1) =$$

$$\frac{1 + n - i + 1}{2} (n - i + 1) = \frac{(n - i + 2)(n - i + 1)}{2}$$

Laço externo:

$$\sum_{i=1}^n \frac{(n-i+2)(n-i+1)}{2} = \frac{1}{2} \sum_{i=1}^n n^2 + 3n + 2 + i^2 - 2ni =$$

$$\frac{1}{2} [n^3 + 3n^2 + 2n + \sum_{i=1}^n i^2 - \sum_{i=1}^n 2ni + 3i] =$$

$$\frac{1}{2} [n^3 + 3n^2 + 2n + \frac{n(n+1)(2n+1)}{6} - \frac{(2n+3) + (2n^2 + 3n)}{2} n] =$$

$$\frac{1}{2} \cdot \frac{6n^3 + 18n^2 + 12n + (2n^3 + 3n^2 + n) - (6n^3 + 15n^2 + 9n)}{6} =$$

$$\frac{2n^3 + 6n^2 + 4n}{12} = \frac{n^3 + 3n^2 + 2n}{6}$$

Versão 2 : n^2

```
int SubsequenciaMaiorSomaN2(int *vet, int n)
{
    int estasoma, maiorsoma, ini, fim, i, j, k;
    maiorsoma=0;
    for (i=0; i<n; i++)
    {
        estasoma=0;
        for (j=i; j<n; j++){
            estasoma=estasoma+vet[j];
            if (estasoma>maiorsoma)
                maiorsoma=estasoma;
        }
    }
    return maiorsoma;
}
```

Representando como somatórios:

$$\sum_{i=1}^n \sum_{j=i}^n 1$$

Laço interno:

$$\sum_{j=i}^n 1 = 1 \cdot (n - i + 1) = n - i + 1$$

Laço externo:

$$\sum_{i=1}^n n - i + 1 = \frac{(n - 1 + 1) + (n - n + 1)}{2} (n - 1 + 1) = \frac{n + 1}{2} n = \frac{n^2 + n}{2}$$

■ Versão 3 : $n \log n$

- Nesta versão o vetor é dividido em dois, e retorna-se para cada metade o valor da melhor subsequência contida no mesmo, bem como da maior subsequência nos limites do vetor (limite inferior para o primeiro vetor e limite superior para o segundo vetor), em um processo semelhante à identificação dos pontos mais próximos por Divisão e Conquista.

Versão 4 : n (algoritmo de Kadane)

```
int SubsequenciaMaiorSomaN(int vet , int N)
{
    int estasoma , maiorsoma , i , j ;
    maiorsoma = 0;
    estasoma = 0;
    for (j=0;j<N;j++)
    {
        estasoma = estasoma + vet[j];
        if (estasoma > maiorsoma)
            maiorsoma = estasoma;
        else if (estasoma < 0)
            estasoma = 0;
    }
    return maiorsoma;
}
```

- Calculando a complexidade:

$$\sum_{j=0}^{N-1} 1 = \frac{1+1}{2} N = N$$

- O algoritmo de Kadane pode ser utilizado para encontrar o intervalo de soma mínima, feitas poucas alterações.
- Ou outras métricas sobre intervalos de vetor (URI 2952)

- O algoritmo de Kadane é enquadrado como Programação Dinâmica, porque cada intervalo do vetor até a posição anterior é considerado um subproblema para o qual são armazenados o maior intervalo até o momento, e o maior intervalo que contem a última posição até o momento, de forma semelhante ao algoritmo anterior (por divisão e conquista).
- URI 2952 - A Vida Sustentável

Joseph Born **Kadane**

- Nascido em Washington, D.C., em 1941, é professor emérito da Universidade de Carnegie Mellon, mais conhecido pelo seu trabalho em estatística, sendo dos primeiros a propor a estatística Bayesiana.



Problemas da ACM resolvíveis por Programação Dinâmica

(ACM - 108 - Maximum Sum) Faça um algoritmo que leia uma matriz quadrada $\text{Mat}[N][N]$ contendo valores positivos e negativos e encontre a região retangular da matriz que resulta na maior soma, escrevendo a soma calculada. Calcule a complexidade do mesmo. A figura abaixo mostra uma matriz 4×4 e a região retangular de maior soma:

0	-2	-7	0
9	2	-6	2
-4	1	-4	1
-1	8	0	-2

- Uma solução para esse problema utilizando força bruta poderia ser gerar todas as regiões possíveis, utilizando o código abaixo, onde Top, Bottom, Left e Right representam os limites superior, inferior, esquerdo e direito da região retangular considerando uma matriz quadrada de N linhas e colunas:

```
for (Top=0;Top<N;Top++)  
    for (Bottom=Top;Bottom<N;Bottom++)  
        for (Left=0;Left<N;Left++)  
            for (Right=Left;Right<N;Right++)  
                /* calcula a soma da região aqui */
```

- A complexidade da geração das áreas acima é $O(N^4)$. Se o cálculo da soma da região for feita por força bruta:

```
int soma=0,i,j;  
for (i=Top;i<=Bottom;i++)  
    for (j=Left;j<=Right;j++)  
        soma+=M[i][j]
```

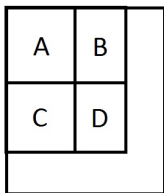
- O cálculo da soma terá complexidade $O(N^2)$ resultando em um custo total $O(N^6)$

- Pode-se calcular a soma de qualquer região em tempo constante ($O(1)$) pré-calculando-se algumas somas parciais e armazenando-as.
- Para cada posição (i,j) armazena-se a soma da região retangular delimitada pelas posições $(0,0)$ e (i,j) .
- Para a matriz do exemplo inicial, mostrada à esquerda, as somas parciais por região seriam (à direita):

0	-2	-7	0	0	-2	-9	-9
9	2	-6	2	9	9	-4	-2
-4	1	-4	1	5	6	-11	-8
-1	8	0	-2	4	13	-4	-3

- Tendo-se precalculado, para cada posição (i,j) a soma da região $(0..i,0..j)$, a soma de qualquer região $(top..bottom, left..right)$ pode ser calculada por
$$\text{soma}(0..bottom, 0..right) - \text{soma}(0..top-1, 0..right) - \text{soma}(0..bottom, 0..left-1) + \text{soma}(0..top-1, 0..left-1)$$

- Por exemplo, para a matriz abaixo, a soma da região D pode ser calculada pela expressão $ABCD - AB - AC + A$, onde $ABCD$ representa a soma das regiões A,B,C e D, AB representa a soma das regiões A e B, e AC representa a soma das regiões A e C.
- Como cada uma das regiões A, AB, AC e ABCD já foi pré-calculada, o cálculo da soma da região D pode ser feito em tempo $O(1)$.
- Qual o custo para pré-calcular a soma de todas as regiões?



- A figura abaixo mostra um exemplo do cálculo de uma das somas parciais. A soma parcial (sp) até a posição (2,2), que contém o valor -4, pode ser calculado pela soma dos dois retângulos vermelhos, menos a soma do quadrado mais o valor da posição (2,2): $-4 + 6 - 9 + (-4) = -11$.

0	-2	-7	0
9	2	-6	2
-4	1	-4	1
-1	8	0	-2

0	-2	-9	-9
9	9	-4	-2
5	6	-11	-8
4	13	-4	-3

- Como o custo para calcular a soma de qualquer região pode ser feita em tempo constante, e o custo de gerar todas as regiões possíveis é $O(N^4)$, o custo para calcular a soma de todas as regiões e encontrar a região que resulta a maior soma pode ser feita em $O(N^4)$.
- Utilizando-se o algoritmo de Kadane pode-se reduzir essa complexidade para $O(N^3)$

- Para isso calcula-se, para cada posição (i,j) a soma dos elementos da coluna, até a linha i , ou seja, a soma dos elementos da coluna acima da posição (i,j) , incluindo a posição (i,j) .
- Para a matriz do exemplo inicial, mostrada à esquerda, as somas parciais das colunas seriam (à direita):

0	-2	-7	0	0	-2	-7	0
9	2	-6	2	9	0	-13	2
-4	1	-4	1	5	1	-17	3
-1	8	0	-2	4	9	-17	1

10) Problema do Caixeiro Viajante Unidirecional

Introdução

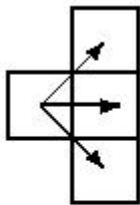
- Problemas que buscam encontrar o caminho mínimo através de algum domínio aparecem em diversas áreas da informática.
- Por exemplo, uma das restrições em problemas de roteamento VLSI é minimizar o comprimento do cabo.
- O Problema do Caixeiro Viajante (TSP) - verificar se é possível visitar todas as cidades na rota de um vendedor exatamente uma vez em um tempo de viagem limitado - é um dos exemplos canônico de um problema NP-completo.

- Soluções para este tipo de problema exigem uma quantidade excessiva de tempo para gerar, mas são fáceis de verificar.
- Esse problema lida com a descoberta de um caminho mínimo através de uma grade de pontos durante a viagem da esquerda para a direita.

O Problema

- Dada uma matriz $m \times n$ de inteiros, você deve escrever um programa que calcula um caminho de peso mínimo.
- Um caminho começa em qualquer lugar na coluna 1 (primeira coluna) e consiste de uma seqüência de etapas que terminam na coluna n (a última coluna).
- Um passo consiste em ir da coluna i até a coluna $i + 1$ em uma linha adjacente (horizontal ou diagonal).

- A primeira e última linhas (linhas 1 e m) de uma matriz são consideradas adjacentes, ou seja, a matriz se "dobra" formando um cilindro horizontal.
- Passos válidos estão ilustrados abaixo:



- O peso de um caminho é a soma dos inteiros em cada uma das n células da matriz que são visitadas.
- Por exemplo, duas matrizes 5x6 ligeiramente diferentes são mostradas abaixo (a única diferença são os números na última linha).

3	4	1	2	8	6
6	1	8	2	7	4
5	9	3	9	9	5
8	4	1	3	2	6
3	7	2	8	6	4

3	4	1	2	8	6
6	1	8	2	7	4
5	9	3	9	9	5
8	4	1	3	2	6
3	7	2	1	2	3

- O caminho mínimo para a primeira matriz é $3 - 1 - 3 - 3 - 2 - 4$ e o caminho mínimo para a segunda matriz é $3 - 1 - 1 - 1 - 2 - 3$.
- Observe que o caminho para a matriz da direita aproveita a propriedade de adjacência da primeira e última linhas.

A Entrada

- A entrada consiste de uma seqüência de especificações de matriz.
- Cada especificação de matriz consiste de uma linha com o número de linhas e colunas (nessa ordem) da matriz, seguida por $m * n$ inteiros onde m é o número de linhas da matriz e n é o número de colunas.
- Os inteiros aparecem por linhas, i.e, os primeiros n inteiros representam os valores da primeira linha, e assim por diante.
- Os inteiros em uma linha serão separados por um ou mais espaços.
- Nota: os inteiros não se restringem a serem positivos.
- Haverá uma ou mais especificações de matriz no arquivo de entrada.
- A entrada é terminada por fim-de-arquivo.

- Para cada especificação o número de linhas será entre 1 e 10, inclusive;
- o número de colunas será entre 1 e 100, inclusive.
- O peso dos caminhos nunca excederá os valores inteiros representáveis com 30 bits.

Saída

- Duas linhas devem ser saída para cada especificação de matriz no arquivo de entrada, a primeira linha representa um caminho de peso mínimo, e, a segunda linha é o custo de um caminho mínimo.
- O caminho é composto por uma seqüência de n inteiros (separados por um ou mais espaços), representando as linhas que constituem o caminho mínimo.
- Se houver mais de um caminho de peso mínimo o caminho que é lexicograficamente menor deve ser a saída.

Exemplo de entrada

5 6

3 4 1 2 8 6

6 1 8 2 7 4

5 9 3 9 9 5

8 4 1 3 2 6

3 7 2 8 6 4

5 6

3 4 1 2 8 6

6 1 8 2 7 4

5 9 3 9 9 5

8 4 1 3 2 6

Exemplo de Saída

1 2 3 4 4 5

16

1 2 1 5 4 5

11

1 1

19

Lista de Problemas de Programação Dinâmica

11) Melhor seqüência de produtos matriciais

- Considere o processo de multiplicação de n matrizes $M = M_1 \times M_2 \times \dots \times M_n$ com dimensões $m_0 \times m_1, m_1 \times m_2, \dots, m_{n-1} \times m_n$, respectivamente. O problema em essência é trivial, pois se multiplicarmos sequencialmente as matrizes obteremos facilmente o resultado esperado.
- A multiplicação de matrizes não é comutativa (em geral, $A \times B \neq B \times A$, mas associativa, o que significa que $A \times (B \times C) = (A \times B) \times C$, o que possibilita diferentes ordens para avaliar os produtos matriciais, cada uma com um número total de operações diferentes.

- Multiplicar uma matriz $m \times n$ por uma matriz $n \times p$ exige $m \times n \times p$ operações de multiplicação. As matrizes são multiplicadas aos pares.
- Considere a sequência de produtos matriciais a seguir:
 - $A(10,100) \times B(100 \times 50) \times C(50,2)$
- Essa sequência de produtos matriciais pode ser resolvida em duas ordens:
 - $(A \times B) \times C$
 - $A \times (B \times C)$
- Qual o número total de operações em cada um dos casos?

- No primeiro caso ($(A \times B) \times C$), o produto $A(10,100) \times B(100,50)$ necessitará de 50.000 operações, resultando em uma matriz $AB(10,50)$.
- O produto da matriz $AB(10,50)$ pela matriz $C(50,2)$ necessitará de 1000 operações, resultando na matriz $ABC(10,2)$.
- O total de operações será 51.000 operações.

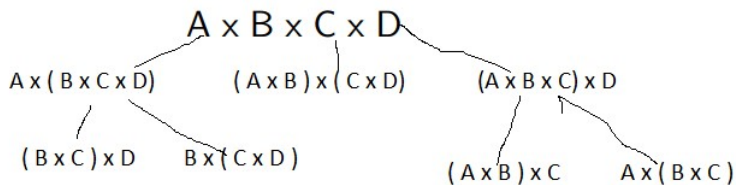
- No segundo caso ($A \times (B \times C)$), o produto $B(100,50) \times C(50,2)$ necessitará de 10.000 operações, resultando em uma matriz $BC(100,2)$.
- O produto da matriz $A(10,100)$ pela matriz $BC(100,2)$ necessitará de 2000 operações, resultando na matriz $ABC(10,2)$.
- O total de operações será 12.000 operações.

- Pode-se considerar como subproblemas os intervalos de matrizes na sequência. No caso acima os subproblemas seriam os intervalos $(A \times B)$ e $(B \times C)$
- No caso da sequência $A \times B \times C \times D$, os subproblemas seriam:
 - $A \times B$, $B \times C$, $C \times D$, $A \times B \times C$, $B \times C \times D$
- Por que $A \times B \times D$ não é um subproblema a ser considerado?

- O custo ótimo dos produtos matriciais pode ser descrito pela recorrência abaixo (Cormen, pg.335):

$$m[i][j] = \begin{cases} 0 & p/i == j \\ \min_{i \leq k \leq j} (m[i][k] + m[k+1][j] + p_{i-1}p_kp_j) & p/i < j \end{cases}$$

- onde $m[i][j]$ é o custo do produto matricial das matrizes no intervalo de i a j , p_{i-1} é o número de linhas da matriz i , p_k é o número de linhas da matriz $k+1$ (igual ao número de colunas da matriz k) e p_j é o número de colunas da matriz j .



	A	B	C	D
4	$A \times B \times C \times D$			
3	$A \times B \times C$	$B \times C \times D$		
2	$A \times B$	$B \times C$	$C \times D$	
1	A	B	C	D

- Para cada intervalo de matrizes, pode-se armazenar o custo do cálculo e as dimensões da matriz resultante.
- O custo e dimensões de cada intervalo pode ser obtido a partir do custo e dimensões das soluções para os intervalos menores.
- Assim, para calcular a solução de $A \times B \times C \times D$ pega-se, dentre as alternativas a seguir, a que resultar no menor custo:
 - $A \times (B \times C \times D)$
 - $(A \times B) \times (C \times D)$
 - $(A \times B \times C) \times D$

Melhor ordem de avaliação de expressões

- Um problema similar ao problema da ordem dos produtos matriciais é o problema da melhor ordem de avaliação de uma expressão.
- Considere uma expressão envolvendo valores numéricos e operadores de soma e multiplicação (ex: $3+2*5+2*4$), em uma linguagem exótica não determinística em que os operadores de soma e multiplicação não possuem precedência ou associatividade.
- Deseja-se saber qual o maior valor possível de se obter a partir da expressão. Ex: $((3+2)*5)+(2*4)=33$
 $(3+(2*5))+(2*4)=38$.
- Descreva como esse problema poderia ser resolvido utilizando programação dinâmica. Descreva o que seriam os subproblemas e descreva, o mais detalhadamente que puder, uma estrutura de dados para armazená-los.

- A função abaixo encontra o maior valor da expressão selecionando um operador de cada vez, e avaliando recursivamente as subexpressões à esquerda e à direita do operador selecionado.

```
int avalia(int inicio , int fim)
{
    //if (res[inicio][fim]!=-1) return res[inicio][fim];
    if (inicio==fim) return exp[inicio]-'0';
    int i , maior=0;
    for (i=inicio+1; i<fim; i+=2)
    {
        int v1=avalia(inicio , i-1);
        int v2=avalia(i+1,fim);
    }
    int res;
    if (exp[i]=='+') res=v1+v2;
    else res=v1*v2;
    if (res>maior) maior=res;
}
res[inicio][fim]=maior;
return maior;
}
```

12) Maior sequência crescente não contígua

- (LIS - Longest increasing sequence) Ou maior sequência decrescente ou maior sequência não-crescente (valores da sequência podem ser iguais ao anterior) ou maior sequência não-decrescente
- Encontrar a maior sequência não-crescente (cada número é menor ou igual ao anterior) de números não necessariamente contíguos em uma sequência **n** de **N** números.
 - Ex: Qual a maior sequência da série de valores a seguir?
10 20 2 4 7 12 6 5 12 17 21 13 19 18 5 3
 - Tentar por backtracking e programação dinâmica.

- Esse problema pode ser descrito pela recorrência abaixo, onde $\text{MaiorSeq}[i]$ é o comprimento da maior sequência que inclui o número $n[i]$.

$$\text{MaiorSeq}[i] = \begin{cases} 1 & p/i == N \\ 1 + \max_{i < j \leq N} (\text{MaiorSeq}[j]) & p/i < N, j \text{ compativel} \\ 1 & \text{se nao ha } j \text{ compativel} \end{cases}$$

- Em uma solução por backtracking, para cada posição $\text{arr}[i]$ busca-se em cada uma das posições seguintes de valor menor ou igual a $\text{arr}[i]$ qual delas inicia a maior sequencia.
- Uma alternativa é armazenar, para cada posição $\text{arr}[i]$, o comprimento da maior sequência que inicia por ele.

- Começando do final do vetor para o início:
 - A maior sequência a partir do 3 tem comprimento 1 (contem somente o próprio 3)
 - A maior sequência a partir do 5 é obtida empareirando com o 3, que tem comprimento 1, resultando a sequencia 5-3, de comprimento 2.
 - A maior sequência a partir do 18 é obtida empareirando com a sequencia a partir do 5 (5-3), que tem comprimento 2, resultando a sequencia 18-5-3, de comprimento 3.

10 20 2 4 7 12 6 5 12 17 21 13 19 18 5 3

- A função abaixo calcula, para cada elemento **arr[i]**, o comprimento da maior sequência crescente que inicia na posição **i** (custo $O(N^2)$).
- o vetor **tam** armazena em cada posição **i** o tamanho da maior sequência que inicia na posição **i**.
- Para encontrar a maior sequência decrescente, ou não-crescente ou não-decrescente basta alterar a comparação **arr[i] > arr[j]** para uma comparação apropriada.

```

int max(int arr, int N)
{
    int tam[N], maior=1;
    tam[N-1]=1;
    for (i=N-2;i>=0;i- -)
    {
        tam[i]=1;
        for (j=i+1;j<N;j++)
            if (arr[i]>arr[j] && tam[j]+1>tam[i])
                tam[i]=tam[j]+1;
        if (tam[i]>maior) maior=tam[i];
    }
    return maior;
}

```

```
int lis( int arr[], int n )
{
    int lis[n];

    lis[0] = 1;
    for (int i = 1; i < n; i++ )
    {
        lis[i] = 1;
        for (int j = 0; j < i; j++ )
            if ( arr[i] > arr[j] && lis[i] < lis[j] + 1)
                lis[i] = lis[j] + 1;
    }
    // Retorna maior valor na lista
    return max_element(lis , lis+n);
}
```

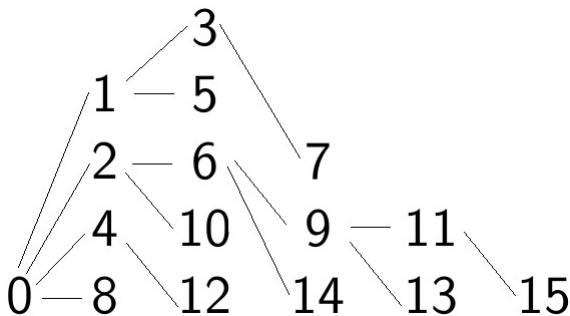
- Um algoritmo de custo $O(N \log N)$ pode ser obtido utilizando-se uma abordagem diferente.
- Aqui será apresentado o algoritmo para obter uma sequência estritamente crescente.
- A ideia é montar, a partir dos dados do array, diversas pilhas decrescentes, da esquerda para a direita.
- A cada valor $arr[i]$ do vetor, coloca-se o valor na pilha mais à esquerda que possui no topo um valor maior ou igual a $arr[i]$.
- Se não houver nenhuma pilha em que o topo seja maior ou igual a $arr[i]$, cria-se uma nova pilha.
- O tamanho da maior subsequência é o número de pilhas.

- Considere a sequência de números:
- 0 8 4 12 2 10 6 14 1 9 5 13 3 11 7 15
- Inicialmente o 0 é colocado na primeira pilha e o 8, por ser maior, é colocado na segunda pilha. E assim vai...

		3			
	1	5			
	2	6	7		
	4	10	9	11	
0	8	12	14	13	15

- E o comprimento da maior subsequência crescente é 6 e uma possível sequência é 0-2-6-9-11-15 (como?)

- Para obter a sequência, basta associar a cada elemento colocado nas pilhas, um apontador para o topo da pilha à sua esquerda:



- Maior sequência decrescente:
 - URI 1517 - Maçãs
 - URI 2024 - Empilhando Presentes

Soma de Subconjuntos

- O problema de soma de subconjuntos pode ser formulado como:
 - "Dado um conjunto (ou **multiconjunto**, que permite repetição de elementos) de valores inteiros (positivos e negativos), existe um subconjunto não-vazio cuja soma resulte zero?"
- Variações desse problema incluem verificar se há um subconjunto cuja soma resulte em um valor dado, ou se há um subconjunto cuja soma resulte exatamente na metade da soma total (problema da partição).
- De modo geral podem ser resolvidos como instâncias do problema da mochila (em que o valor e o peso do objeto são os mesmos) ou de coin change.

Soma de Substrings

- O problema consiste em, dada uma string S representando um número, calcular a soma de todas as substrings desse número.
- P.ex., se o string é "123", os substrings são "1", "2", "3", "12", "23", "123" e a soma resultante é $1+2+3+12+23+123=164$
- Essa soma pode ser calculada armazenando em um vetor $soma[N]$, a soma dos substrings que terminam na posição N .
- Por exemplo, para o string "1234" o vetor conteria o seguinte:
 - $soma[0]=1$
 - $soma[1]=12+2=14$
 - $soma[2]=123+23+3=149$
 - $soma[3]=1234+234+34+4=1506$
- E a soma total de todos os substrings seria $1+14+149+1506=1670$

- As somas anteriores podem ser representadas como:
- $\text{soma}[0]=1$
- $\text{soma}[1]=12+2=1 * 10 + 2 * 2 = 14$
- $\text{soma}[2]=123+23+3= (12+2)+10+3*3 = 14 * 10 + 3 * 3 = 149$
- $\text{soma}[3]=1234+234+34+4= 149*10+4*4=1506$
- Sugerindo a recorrência $\text{soma}[i]=\text{soma}[i-1]*10+\text{dig}[i]*(i+1);$

Sequenciamento de tarefas (Job Sequencing)

- Dado um conjunto de tarefas onde cada tarefa tem um horário de início e um horário de fim e um ganho associado.
- Busca-se selecionar as tarefas, de modo que haja no máximo uma tarefa ativa a cada momento, de modo a maximizar o ganho.
- Ex: Qual o subconjunto de tarefas que maximiza o lucro para o conjunto de tarefas a seguir?

- Entrada: Número de tarefas $n = 4$
- Detalhes de cada tarefa: Horário inicial, horário final, ganho
- Tarefa 1: $\{1, 2, 50\}$
- Tarefa 2: $\{3, 5, 20\}$
- Tarefa 3: $\{6, 19, 100\}$
- Tarefa 4: $\{2, 100, 200\}$
- Saída: Ganho máximo é 250.
- Pode-se alcançar o ganho máximo selecionando as tarefas 1 e 4.
- Há soluções com mais tarefas compatíveis, como as tarefas 1, 2 e 3 mas o ganho obtido é $20+50+100$ que é menor que 250.

- Esse problema pode ser resolvido ordenando-se os processos em ordem crescente de hora de término e encontrando, para cada tarefa i , o maior ganho obtido até a tarefa.
- O maior ganho pode ser obtido através da inclusão da tarefa i ou da não-inclusão da tarefa i .
- A recorrência a seguir representa o ganho máximo até a tarefa N . Nela, **ult** representa a última tarefa compatível com a tarefa n (ou seja, que termina antes do início da tarefa n). $Ult=-1$ significa que não há tarefa anterior compatível com n):

$$GanhoMax(n) = \begin{cases} Ganho[n] & p/ult = -1 \\ \max(GanhoMax(n-1), & p/ult \neq -1 \\ Ganho[n] + GanhoMax(ult) \end{cases}$$

- O custo da recorrência acima depende de como é buscada, para a tarefa **n**, a última tarefa compatível. Se for feita uma busca sequencial da posição **n** para trás, até encontrar a primeira compatível, o custo será $O(n^2)$
- Uma alternativa é aplicar uma busca binária nas tarefas anteriores a **n**, para encontrar a maior que é compatível com **n**. Com isso, o custo total cai para $O(n \log n)$.
- Ou fazer duas ordenações, uma pela hora de término e uma pela hora de início, e associar a cada tarefa **n** a de maior hora de término compatível com o início da tarefa **n** ($O(N \log N)$ para a ordenação e $O(N)$ para associar as tarefas)
- Ex: URI 1838 - A Pedra Filosofal.

Problemas de *stock trading*

- Em problemas de stock trading tem-se um vetor com o valor diário de ações (ou bens ou qualquer coisa) ao longo de um período, e busca-se maximizar o lucro definindo quando comprar e quando vender.
- Por exemplo, se os valores são $\{100, 180, 260, 310, 40, 535, 695\}$, o lucro máximo pode ser obtido comprando no dia 0 e vendendo no dia 3, com lucro de 210, e comprando no dia 4 e vendendo no dia 6, com um lucro de 655.
- No caso mais comum, o problema permite ter somente uma ação, de modo que para comprar uma ação deve-se antes vender a que se tinha.

- Variantes desse problema incluem restrições adicionais, como um número máximo de transações.
- Ou um custo associado a cada transação de compra ou venda.
- O caso mais simples, sem limite de transações, pode ser resolvido pelo algoritmo abaixo:
 - 1 Encontre o mínimo local (ou seja, encontre a primeira posição em que o elemento seguinte é maior que ela) e use como índice inicial. Se não houver, encerre.
 - 2 Encontre o máximo local (ou seja, encontre a primeira posição em que o elemento seguinte é menor que ela) e use como índice final. Se foi atingida a posição final, selecione ela como índice final.
 - 3 Calcule o lucro para esse par início-fim e atualize o lucro total
 - 4 Repita os passos acima até atingir o fim do vetor.

- No caso em que só é permitida uma transação, busca-se o par de valores de maior diferença, em que o menor vem antes do maior.
- Isso pode ser obtido percorrendo uma vez o vetor, mantendo controle do menor valor já encontrado a cada momento, guardando a maior diferença. Algo como:

```
maxDif=0; menor=INF;
```

```
for (i=0;i<N;i++){  
    if (a[i]<menor) menor=a[i];  
    if (a[i]-menor>maxDif) maxDif=a[i]-menor;}
```

- URI 1932 - Bolsa de Valores

Problemas desclassificados de PD do URI e SPOJ

- Algumas recorrências que não se enquadram nos modelos dos problemas anteriores:
- URI 2599 - Contando Radares
- URI 1210 - Produção ótima de ótima vodka
- URI 1689 - Radares
- URI 1810 - Beverly Hills
- URI 1474 - Ônibus
- URI 2475 - Confeção de Presentes
- URI 2197 - Machine Works

Programação Dinâmica no Geeks for Geeks

- Cutting a Rod - DP13 - Dada uma barra de comprimento N , e o valor ganho com a venda de pedaços da barra, qual a melhor forma de cortá-la de modo a maximizar o ganho?
- O vetor $preco[i]$ contem o preço de um pedaço da barra de tamanho i para tamanhos de 1 até M .
- Pode ser expresso pela recorrência $Ganho(n)$, onde n é o comprimento da barra

$$Ganho(n) = \begin{cases} \max_{1 \leq j \leq m, < n} (preco[j] + Ganho[n - j]) & p/n > 1 \\ preco[n] & p/n == 1 \end{cases}$$

- Box Stacking Problem - DP 22: Aplicação direta do LIS
- Maximum Product Cutting - DP 36: Dada uma corda de comprimento N, deve-se cortar a corda em partes de comprimento inteiro de modo a maximizar o produto de todas as partes. Deve ser feito pelo menos um corte.
- Pode ser resolvido pela recorrência:
- $\text{maxProd}(n) = \max(i*(n-i), \text{maxProdRec}(n-i)*i)$ para i em $\{1, 2, 3 \dots n-1\}$

Branch-and-Bound

- Branch and Bound (BB ou B&B) (literalmente **Ramificar e limitar**) é uma estratégia para encontrar soluções ótimas para vários problemas de otimização.
- É considerada a estratégia mais usada para a resolução de problemas de otimização NP-Difíceis.
- Um algoritmo B&B consiste de uma enumeração sistemática de todas as soluções candidatas, onde grandes subconjuntos de soluções são descartadas através do uso de limites inferior/superior (dependendo se é um problema de maximização ou minimização) da quantidade sendo avaliada.

- Para poder aplicar Branch and Bound a um problema, deve ser possível:
 - Calcular um limite inferior/superior para uma instância de um problema
 - Dividir uma região de soluções viáveis em regiões menores
- A estratégia de Branch-and-Bound é muito similar ao backtracking, no sentido que o espaço de estados é usado para resolver o problema
- A principal diferença é que o método branch-and-bound não limita a varredura da árvore a uma única ordem (por exemplo, busca em profundidade)

- Um nodo, na árvore de espaço de estados do BB, representa uma possível solução ou subconjunto de soluções possíveis para o problema.
- Para o problema da mochila, por exemplo, em que a instância do problema é um conjunto de objetos, e uma solução é um subconjunto desses objetos, um nodo representa um estado em que alguns objetos já foram selecionados, alguns já foram removidos e alguns podem ou não fazer parte da solução.

- Um algoritmo Branch-and-bound computa um número (limite) ao nodo para determinar se ele é promissor.
- Esse número é um limite no valor da solução que pode ser obtida expandindo o nodo
- Se esse limite não é melhor que o valor da melhor solução encontrada até agora, o nodo não é "promissor" e pode ser eliminado.
 - De forma semelhante aos cortes feitos no backtracking para problemas de otimização ou à poda alpha-beta do Minimax.

- Esse limite é calculado a partir dos elementos já acrescentados à solução, mais uma estimativa do ganho a partir dos elementos que ainda falta acrescentar.
- Nesse aspecto é semelhante ao algoritmo A-star para calcular distância entre 2 pontos, que ao contrário de Dijkstra, que considera apenas a distância já percorrida, leva em conta também a estimativa de distância até o vértice destino.

- Além de usar o limite para determinar se um nodo é ou não promissor, pode-se comparar os limites de nodos promissores e visitar os filhos daquele com o melhor limite
- Essa abordagem é chamada best-first search com corte branch-and-bound. A implementação dessa abordagem é uma modificação da busca em amplitude com corte branch-and-bound

- O método inicia por:
 - Considere o problema original com a região de soluções viáveis (chamado problema raiz).
 - Os procedimentos de limite inferior (lower-bounding) e limite superior (upper-bounding) são aplicados ao problema raiz.
 - Se os limites coincidem, então uma solução ótima foi encontrada. Caso contrário, a região de soluções viáveis é dividida em duas ou mais regiões, que juntas cobrem a região de soluções viáveis.
 - Estes subproblemas se tornam os nodos filhos do nodo raiz
 - O algoritmo é aplicado recursivamente aos subproblemas, gerando uma árvore de subproblemas

- Se uma solução ótima é encontrada para um subproblema:
 - É uma solução viável para o problema, mas talvez não seja o ótimo global.
 - Uma vez que é viável, pode ser usada para "podar" o resto da árvore.
 - Se o limite inferior de um nodo excede a melhor solução (considerando um problema de minimização), nenhum ótimo global pode existir naquele subespaço de soluções e o nodo pode ser removido.
 - A busca continua até que todos os nodos foram resolvidos ou removidos, ou até que uma solução satisfatória é encontrada.

- Para a aplicação do B&B é necessário tomar cuidado com 3 pontos principais:
 - Como computar um limite inferior/superior
 - Como escolher um nodo para a operação de ramificação (branch)
 - Qual decisão de ramificação tomar
- Em geral não se executa uma busca em profundidade ou amplitude, mas uma busca em que se usa uma fila de prioridade para manter uma lista dos nodos gerados mas ainda não completamente explorados. Em qualquer estágio o próximo nodo a ser explorado é o nodo no início da fila de prioridade.

Aplicações do Branch-and-Bound

- Branch and Bound é bastante utilizado em problemas de otimização, normalmente NP-difíceis. Entre elas pode-se citar:
 - 1 Problemas envolvendo programação linear com coeficientes inteiros (**Integer Programming**).
 - 2 **Bin packing** - Dados n itens de diferentes pesos e recipientes de igual capacidade c , atribuir cada item a um recipiente de modo a minimizar o número de recipientes.
 - É parecido com o problema das múltiplas mochilas, mas nesse as mochilas tem limites diferentes e o objetivo é maximizar o valor.
 - 3 Problema dos k vizinhos mais próximos (knn - k-nearest neighbor)

Aplicações do Branch-and-Bound

4 N-queens problem

5 Puzzle 15

6 Mochila 0-1

7 Job Assignment

8 Problema do Caixeiro Viajante

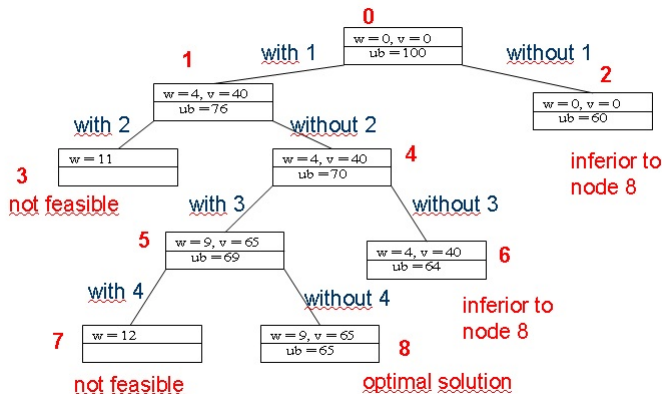
Ex: Problema da Mochila 0-1

- A solução do problema da mochila 0-1 por Programação Dinâmica só pode ser aplicada quando os pesos dos objetos são inteiros. Quando isso não ocorre, pode-se usar **BB**.
- Capacidade da Mochila: 10 kgs

Item	Peso	Valor	Valor/Peso
1	4	\$40	10
2	7	\$42	6
3	5	\$25	5
4	3	\$12	4

- Critério para ramificação: conjuntos que contem ou não cada objeto
- Limite superior: valor atual + peso restante preenchido com objeto de melhor proporção entre valor e peso, que ainda não foi colocado

Árvore do espaço de estados



- No código a seguir, os nodos contem a solução parcial com os objetos selecionados até o momento representados como um vetor de bits.
- A partir desse conjunto as funções $vsol$ e $psol$ calculam o valor e o peso da solução varrendo os bits ligados e somando o peso/valor correspondente.
- Os nodos a serem avaliados são inseridos no final de uma fila, implementada em uma lista encadeada.

```
#include <stdio.h>
#include <stdlib.h>

#define TAM 10

float peso[TAM]={23,31,29,44,53,38,63,85,89,82};
float valor[TAM]={92,57,49,68,60,43,67,84,87,72};
float vprop[TAM];

float capacidade=70;

struct nodo{int solucao; int nivel; float upbound; struct nodo *prox;};
typedef struct nodo tnodo;
tnodo *inic=NULL,*fim=NULL;
```

Funções de avaliação de peso e valor de uma solução

```
float vsol(int solucao)
{
    int i;
    float vaux=0;
    for (i=0;i<30;i++)
    {
        if (solucao%2==1)
            vaux+=valor[i];
        solucao/=2;
    }
    return vaux;
}
```

```
float psol(int solucao)
{
    int i;
    float paux=0;
    for (i=0;i<30;i++)
    {
        if (solucao%2==1)
            paux+=peso[i];
        solucao/=2;
    }
    return paux;
}
```

- Função que devolve qual o valor relativo máximo dos itens restantes, utilizada para estimar o limite superior do nodo.

```
float max(float vprop[TAM], int nivel)
{
    int i;
    float mauz=vprop[nivel+1];
    for (i=nivel+2;i<TAM;i++)
        if (vprop[i]>mauz)
            mauz=vprop[i];
    return mauz;
}
```

```
void insere(int solucao,int nivel)
{
    tnodo *ptaux=(tnodo *)malloc(sizeof(tnodo));
    ptaux->solucao=solucao;
    ptaux->upbound=vsol(solucao)+(capacidade-psol(solucao))*max(vprop,nivel);
    ptaux->nivel=nivel;
    ptaux->prox=NULL;
    if (inic==NULL)
        inic=fim=ptaux;
    else
    {
        fim->prox=ptaux;
        fim=ptaux;
    }
}
```

```
int retira(int *solucao, int *nivel, float *upbound)
{
    if (inic==NULL) return 0;
    *solucao=inic->solucao;
    *nivel=inic->nivel;
    *upbound=inic->upbound;
    tnode *ptaux=inic->prox;
    free(inic);
    inic=ptaux;
    return 1;
}
```



```

int main()
{
    int i;
    for (i=0;i<TAM;i++)
        vprop[i]=valor[i]/peso[i];
    insere(0,-1);
    int solucao, nivel;
    float upbound;
    float valotimo=0;
    while (retira(&solucao, &nivel, &upbound)!=0)
    {
        printf(" Retirei_solucao_%x_nivel_%d_upbound=%f\n", solucao, nivel, upbound);
        if (upbound<valotimo) continue;
        if (vsol(solucao)>valotimo)
        {
            valotimo=vsol(solucao);
            printf(" Novo_valor_otimo=%f\n", valotimo);
        }
        if (nivel==TAM-1) continue;
        int novasolucao=solucao + (1 << (nivel+1));
        if (psol(novasolucao)<=capacidade)
        {
            printf(" Inseri_solucao_%x_nivel_%d\n", novasolucao, nivel+1);
            insere(novasolucao, nivel+1);
        }
        printf(" Inseri_solucao_%x_nivel_%d\n", solucao, nivel+1);
        insere(solucao, nivel+1);
    }
    printf(" Valor_otimo_final=%f\n", valotimo);
    system(" pause");
}

```

- O código anterior varre os nodos na ordem de uma BFS (busca em amplitude), fazendo os cortes, mas não levando em conta para a seleção qual o nodo mais promissor.
- Se os nodos mais promissores forem avaliados primeiro, isso pode levar mais rapidamente a cortes, reduzindo o custo computacional.
- Para isso, ao invés de utilizar uma fila, pode ser utilizada uma fila de prioridades ordenada pelo limite superior do nodo, ou pelo valor já obtido.

- No código anterior retire todos os printf's e rode para instâncias de 5,10,20 e 30 objetos e avalie o tempo. Avalie também o tamanho máximo da fila de prioridades necessário.
- Altere a função retira para, a cada chamada, retornar o nodo de maior valor ao invés do primeiro. Avalie os tempos de execução;
- Altere a função retira para, a cada chamada, retornar o nodo de maior upbound ao invés do primeiro. Avalie os tempos de execução;
- Troque a estrutura usada para a fila por uma fila de prioridades e avalie os tempos.
- Compare os tempos obtidos pelo branch and bound com o tempo utilizado por um backtracking.

Atribuição de Tarefas (Job Assignment)

- Outro problema que pode ser abordado por Branch-and-Bound é o problema de Atribuição de Tarefas (Job Assignment). Dadas N tarefas e N pessoas, com um custo para cada pessoa executar cada tarefa, deseja-se atribuir as tarefas às pessoas de modo a minimizar o custo total.
- Esse problema pode ser resolvido em tempo $O(n^3)$ pelo algoritmo **Húngaro**, mas uma solução possivelmente mais eficiente pode ser obtida por Branch and Bound.

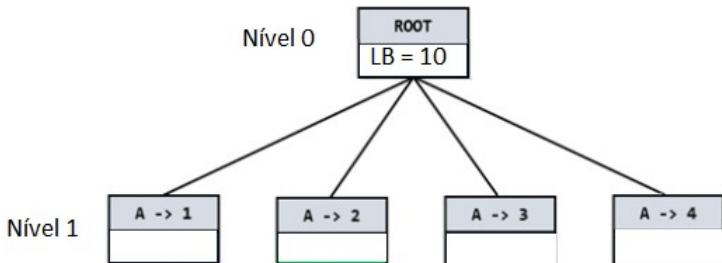
	Job 1	Job 2	Job 3	Job 4
A	9	2	7	8
B	6	4	3	7
C	5	8	1	8
D	7	6	9	4

- A árvore de espaço de estados desse problema pode ser estruturada selecionando a cada nível a tarefa para uma pessoa. No exemplo em questão, a ramificação do primeiro nível seriam as 3 tarefas possíveis para a pessoa A.
- Outra alternativa seria, a cada nível, atribuir uma pessoa à tarefa. Nesse caso a ramificação do primeiro nível seriam cada uma das 3 pessoas atribuídas à tarefa 1.
- Como limite inferior para cada nodo, pode-se utilizar o custo das tarefas já alocadas, mais o menor custo de cada tarefa para as pessoas ainda não alocadas.

- Assim, o limite inferior (LB - Lower Bound) para o nodo inicial seria a soma dos custos das tarefas assinaladas abaixo, totalizando 10:

	Job 1	Job 2	Job 3	Job 4
A	9	2	7	8
B	6	4	3	7
C	5	8	1	8
D	7	6	9	4

- E o primeiro nível da árvore ficaria:



- O limite inferior do nodo $A \rightarrow 1$ é o valor da atribuição $A \rightarrow 1$ mais o menor valor de cada linha, após excluídas a linha A e coluna 1, resultando 17

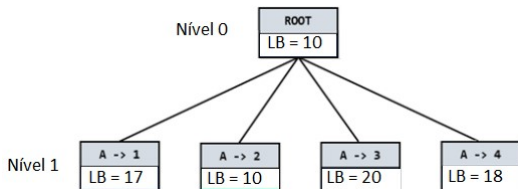
	Job 1	Job 2	Job 3	Job 4
A	9	2	7	8
B	6	4	3	7
C	5	8	1	8
D	7	6	9	4

- Da mesma forma, os limites inferiores das outras atribuições de A seriam:

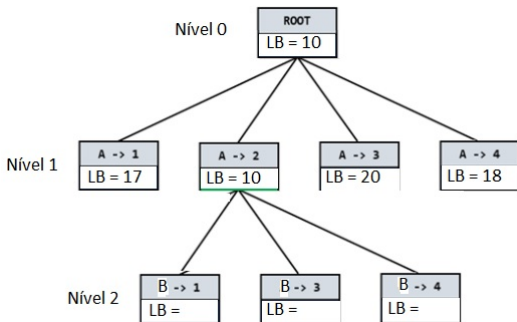
- $A \rightarrow 2 = \mathbf{2} + 3 + 1 + 4 = 10$

- $A \rightarrow 3 = \mathbf{7} + 4 + 5 + 4 = 20$

- $A \rightarrow 4 = \mathbf{8} + 3 + 1 + 6 = 18$



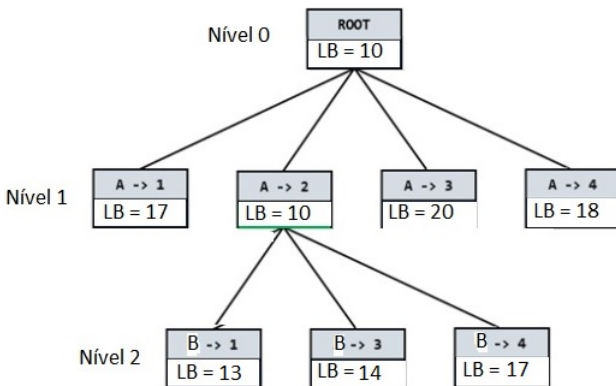
- E assim, a cada passo é selecionado o nó vivo de menor limite inferior, até que o nó selecionado seja um nodo do último nível. No caso, o nó selecionado para expansão é o nó $A \rightarrow 2$.



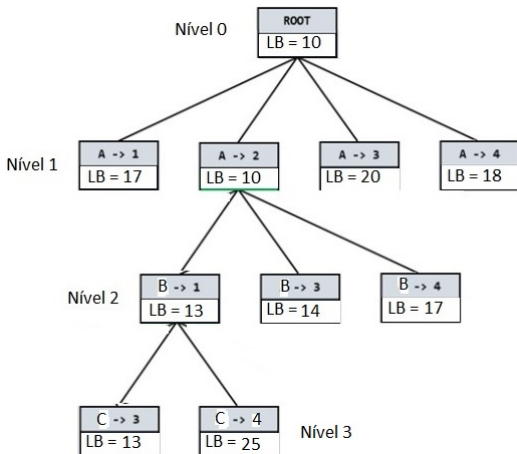
- O limite inferior do nodo $B \rightarrow 1$ é o custo do nodo $A \rightarrow 2$ (2) mais o custo do nodo $B \rightarrow 1$ (6) mais o menor das linhas C e D e colunas 3 e 4 ($1 + 4$) totalizando 13

	Job 1	Job 2	Job 3	Job 4
A	9	2	7	8
B	6	4	3	7
C	5	8	1	8
D	7	6	9	4

- E a árvore da expansão de $A \rightarrow 2$ é:



- Dentre todos os nodos vivos, o de menor LB é o $B \rightarrow 1$ e sua expansão gera:



- E como para cada nodo de C há somente uma escolha para D, não é necessário fazer expansão.
- Nesse exemplo, a escolha a cada passo do nodo de menor LB para efetuar a expansão levou à solução ótima, o que não necessariamente ocorre.

[Voltar para Lista de Aplicações](#)

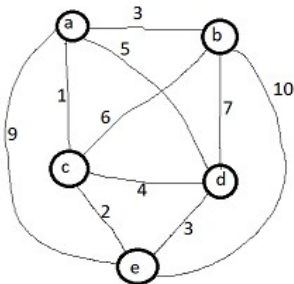
- O algoritmo pode ser descrito nos passos abaixo, mas é o algoritmo-padrão de BB.
- Cada nodo representa a associação de uma tarefa a uma pessoa e contem os seguintes campos:
 - Identificação da tarefa
 - Identificação da pessoa
 - Limite inferior de custo
 - Referência para o nodo "pai"
- E o algoritmo usa duas funções:
 - Add (nodo) - Adiciona o nodo ao "pool" de nodos vivos (de preferência um min-heap)
 - Least () - Retorna o nodo vivo de menor limite inferior e o remove do pool.

- inicializa min-heap com nodo inicial vazio
- while (1)
- /*encontra nodo vivo de menor custo*/
- $E \leftarrow \text{Least}()$
- se E está no último nível:
- retorna E como melhor solução
- /* expande E */
- para cada filho x de E:
- Add (x)

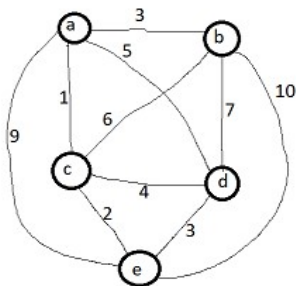
Problema do Caixeiro Viajante

- O Problema do Caixeiro Viajante também pode ser resolvido pelo Branch-and-Bound.
- Cada nível da ramificação pode ser feito a partir das escolhas possíveis para cada cidade.
- E uma possibilidade de cálculo para o limite inferior é a soma das duas arestas de menor valor incidentes a cada vértice, dividido por 2.
- $LB = (\sum_{v \in V} (2 \text{ arestas de menor valor incidentes a } v))/2$

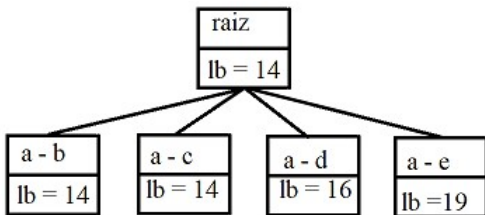
- Para o grafo abaixo o limite inferior seria $((1+3)+(3+6)+(1+2)+(3+4)+(2+3))/2=28/2=14$
- A ramificação seria feita, em cada nível, para uma cidade, e o limite inferior para cada nodo seria o custo das arestas já seleccionadas, mais a média das menores arestas restantes.



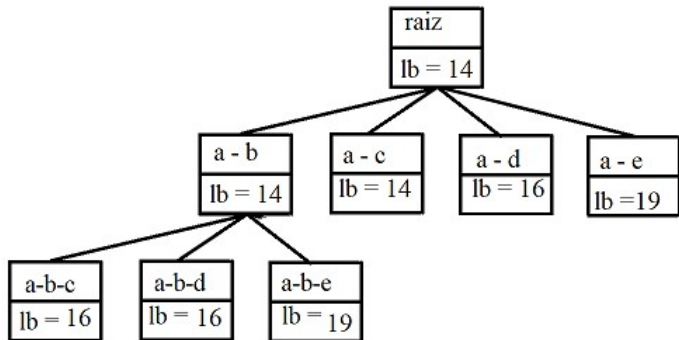
- Assim, o limite inferior do nodo $a \rightarrow b$ seria
 $(3+1)+(3+6)+(1+2)+(3+4)+(2+3))/2=28/2=14$
- E o limite inferior do nodo $a \rightarrow d$ seria
 $(5+1)+(3+6)+(1+2)+(5+3)+(2+3))/2=31/2=15.5$



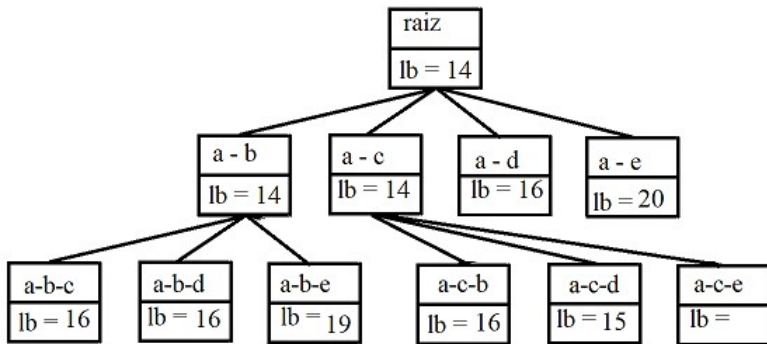
- E o primeiro nível da árvore fica:



- Selecionando o nodo **a** $\rightarrow$ **b** para expansão:



- Expandindo o nodo **a** → **c**:



- E assim vai...

Algoritmos Aproximativos

- Algoritmos aproximativos são algoritmos eficientes (polinomiais) que encontram uma solução cuja distância máxima da solução ótima está dentro de um fator multiplicativo conhecido.
- Diferem-se de algumas abordagens como algoritmo guloso, ou algoritmos genéticos por garantirem a qualidade da solução. Apesar de algoritmos gulosos poderem encontrar a solução ótima em alguns problemas específicos (Prim, Kruskal), são usados em outros problemas para obter soluções "boas", mas no pior caso essas soluções "boas" podem ser arbitrariamente ruins.

- Normalmente são utilizados em problemas de otimização NP-difíceis.
- Não há uma regra geral para a existência de algoritmos aproximativos para problemas. Há alguns problemas NP-difíceis para os quais há diversos algoritmos aproximativos, com diferentes fatores de aproximação (como o Metric TSP), e há problemas para os quais não se conhecem algoritmos aproximativos (como o problema de clique máxima ou o TSP no caso geral).
- A provável inexistência de solução aproximativa para o TSP no caso geral pode ser provada mostrando que uma solução aproximativa para o TSP seria uma solução ótima para o problema do ciclo hamiltoniano, que é NP-Completo.

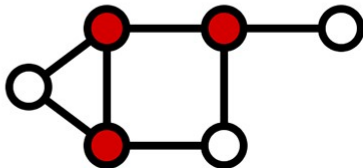
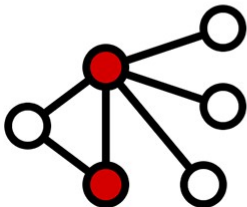
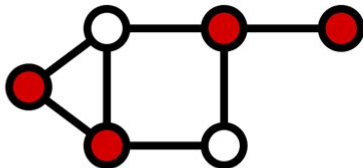
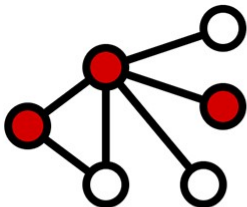
- Para o problema do caixeiro viajante métrico (Metric TSP) sobre grafos completos que obedecem à desigualdade triangular (para quaisquer 3 vértices a, b, c , $l(a, b) < l(a, c) + l(c, b)$), o algoritmo abaixo encontra uma solução dentro de um fator multiplicativo de 2, ou seja, cujo comprimento é no máximo o dobro da solução ótima.
 - 1 Encontre uma árvore geradora mínima T de G .
 - 2 Efetue uma busca em profundidade em T associando um número $L(v)$ a cada vértice v , correspondendo à ordem de visitação.
 - 3 Retorne o ciclo correspondente à ordem de visitação.

Algoritmo de Christofides para o TSP

- O algoritmo de Christofides, mostrado a seguir, obtém uma solução garantidamente menor que 1.5 vezes a solução ótima:
 - 1 Encontre uma árvore geradora mínima T de G .
 - 2 Construa o conjunto V' de vértices de grau ímpar em T e encontre uma associação perfeita M de V' .
 - 3 Construa o grafo Euleriano G' obtido pela adição das arestas de M a T (obs: as arestas de M serão acrescentadas a T mesmo que já existam em T . Nesse caso elas ficarão duplicadas).
 - 4 Encontre o ciclo euleriano C_0 de G' , associando um número $L(v)$ a cada vértice v , correspondendo à ordem de sua primeira visitação.
 - 5 Retorne o ciclo correspondente à ordem de visitação.

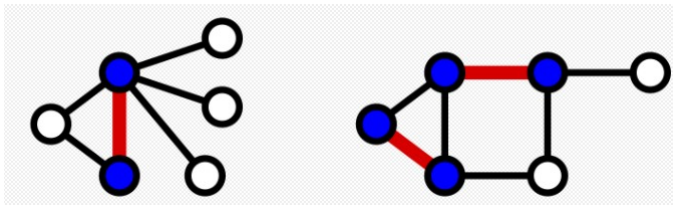
- Outro problema NP-difícil para o qual existe uma aproximação de fator 2 é o problema de cobertura de vértices em um grafo.
- Uma cobertura de vértices (vertex cover) é um conjunto de vértices que contém ao menos um dos vértices adjacentes a cada aresta do grafo.
- Ao contrário da cobertura mínima de arestas (conjunto de arestas que cobrem todos os vértices) que pode ser encontrado em tempo polinomial expandindo uma associação máxima, o problema de cobertura mínima de vértices é NP-difícil.

- As 4 figuras abaixo representam coberturas de vértices. As duas de baixo são coberturas mínimas..

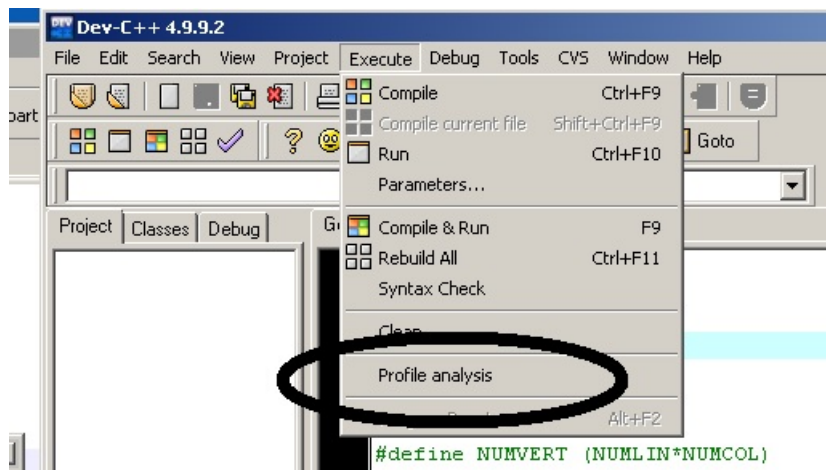


- Aplicações de cobertura de vértices incluem aplicações em distribuição de câmeras de vigilância ou postos de distribuição.
- Apesar de ser um problema NP-difícil, há algoritmos polinomiais que podem encontrar uma aproximação que garanta uma solução no máximo igual a 2 vezes o número de vértices da solução ótima.
- Um desses algoritmos consiste em encontrar uma associação **maximal** no grafo (um conjunto de arestas independentes). Os vértices adjacentes às arestas da associação máxima compõem uma cobertura de vértices.

- A figura abaixo mostra duas associações maximais (arestas em vermelho) e coberturas de vértices correspondentes (em azul).



Perfilamento de Código no Dev C++



Profile analysis

Flat output | Call graph

Function name	% time	Cumul. secs	Self secs	Calls	Self ts/call	Total ts/call
amplitude(int)	53.13	0.17	0.17	1	170.00	170.00
main	46.88	0.32	0.15			
mostra_nivell()	0.00	0.32	0.00	1	0.00	0.00
enchemat()	0.00	0.32	0.00	1	0.00	0.00

% time the percentage of the total running time of the program used by this function.

cumulative seconds a running sum of the number of seconds accounted for by this function and those listed above it.

self seconds the number of seconds accounted for by this function alone. This is the major sort for this listing.

calls the number of times this function was invoked, if this function is profiled, else blank.

self ms/call the average number of milliseconds spent in this function per call, if this function is profiled, else blank.

total the average number of milliseconds spent in this

Close

Opções para perfilador

Tools → Compiler Options → Code Generation/Optimization
→ Generate profiling information for analysis

Tools → Compiler Options → Linker → Generate debugging
information

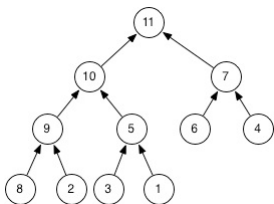
Execute → Rebuild All

Execute → Run

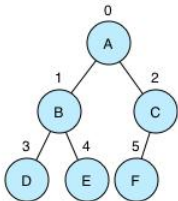
Execute → Profile analysis

- Senão, ao chamar a primeira vez por Profile analysis o dev seta as opções para gerar informações de debug.

- Um heap pode ser implementado sobre um vetor, onde cada elemento $v[i]$ é pai dos elementos nas posições $v[2*i]$ e $v[2*i]+1$ e a raiz da árvore ocupa a primeira posição ($v[1]$).
- Assim, dado o índice de um nodo no vetor, os índices do pai e dos filhos à esquerda e à direita podem ser obtidos por:
 - $PAI(i) : \text{return } i/2$
 - $ESQUERDA(i) : \text{return } i*2$
 - $DIREITA(i) : \text{return } i*2 + 1$
- Assim, o heap abaixo poderia ser representado pelo vetor [11 10 7 9 5 6 4 8 2 3 1]



Heap Properties



heap array, size = 6

Index	0	1	2	3	4	5
value	A	B	C	D	E	F

Index Formulas:

```
left_child_index = parent_index * 2 + 1
right_child_index = parent_index * 2 + 2
                    (or left_child_index + 1)
parent_index = (child_index - 1) / 2
last_parent_index = (size - 2) / 2
```

- As principais operações sobre heaps são:
 - cria-heap: cria um heap vazio
 - heapify: cria um heap a partir de um conjunto de elementos
 - busca-maior (em um heapmax) ou busca-menor(em um heapmin)
 - remove-maior: remove a raiz de um heapmax
 - altera-valor-chave
 - insere: adiciona elemento a um heap

Inserção em Heap

- Pode ser implementada adicionando o novo elemento no final do heap e depois "empurrando-o" para cima enquanto ele for maior que o pai (no caso de um Heap Max) até alcançar sua posição.
- Qual o custo?

```
void corrigeSubindo(int index)
{
    // Se index não é a raiz e o valor do index for maior do que o valor de seu pai,
    // troca seus valores (index e pai(index)) e corrige para o pai
    if (index > 1 && heap[index] > heap[pai(index)])
    {
        troca(heap[index], heap[pai(index)]);
        corrigeSubindo(pai(index));
    }
}
```

Remoção

- A remoção do maior valor da raiz (no caso do Heap Max) pode ser implementada removendo a raiz e substituindo-a pelo último elemento do Heap e "empurrando-o" para baixo na árvore até que o heap esteja corrigido.
- Qual o custo?

```
void corrigeDescendo(int index=1)
{
    // Se index tem filho
    if (filhoE(index) < heap.size())
    {
        // Seleciona o filho com menor valor (esquerda ou direita?)
        int filho = filhoE(index);
        if (filhoD(index) < heap.size() && heap[filhoD(index)] > heap[filhoE(index)])
            filho = filhoD(index);

        // Se o valor do pai é maior do que o valor do maior filho , terminamos
        if (heap[index] > heap[filho])
            return;

        // Caso contrário , trocamos o pai com o filho no heap e corrigimos agora para o filho
        troca(heap[index], heap[filho]);
        corrigeDescendo(filho);
    }
}
```


- Heaps podem ser utilizados para implementar eficientemente filas de prioridades, já que o custo das principais operações (inserção, remoção) é $O(\log n)$ e o custo para obter o maior elemento é $O(1)$.
- Filas de prioridade são utilizadas em diversos algoritmos clássicos tais como:
 - Algoritmo de dijkstra para caminho mínimo
 - Algoritmos de Prim e Kruskal para árvore geradora mínima
 - Algoritmo de Branch-and-bound para buscar nodo de melhores possibilidades

Tratabilidade; as classes P e NP. Problemas NP-completos.

Hierarquias de complexidade

- Um **problema de decisão** é um problema com uma resposta "sim" ou "não". De forma equivalente, uma função cujos valores possíveis são dois, como 0, 1.
- Em um **problema de localização** o objetivo é localizar uma certa estrutura S que satisfaça um conjunto de propriedades.
- Em um **problema de otimização**, o objetivo é identificar, entre as soluções possíveis, a que melhor atenda a um critério de otimização.

- Por exemplo, o problema do caixeiro viajante, no sentido de encontrar a solução de **menor custo**, é um problema de **otimização**, enquanto o problema de **decisão** correspondente pergunta se há um caminho hamiltoniano com um custo menor que um determinado K e o problema de **localização** correspondente busca encontrar uma solução com custo menor do que K .

Algoritmos Pseudo-Polinomiais

- Verificar a primalidade de um número é polinomial?
- Encontrar um fator primo de um número é polinomial?
- A complexidade de um algoritmo é medida em função do **tamanho da entrada**.
- Para algoritmos cuja entrada é um único número, como a identificação de um fator primo, o tamanho da entrada é considerado o número de dígitos (ou de bits) da entrada.
- Algoritmos que são polinomiais no valor da entrada, de modo geral são exponenciais no número de dígitos, e são chamados de **pseudo-polinomiais**.

- Nesse sentido, o algoritmo da Mochila por PD é considerado pseudo-polinomial, uma vez que é função do limite de peso da mochila.
- E o de verificação de primalidade de N pela divisão pelos números menores que $\sqrt{N}$, já que o acréscimo de 1 dígito multiplica o número de operações por 10, o que caracteriza comportamento exponencial.
- Um exemplo de algoritmo "numérico" polinomial, é o de soma de dois números de n dígitos, em que o número de operações elementares é $O(n)$, ou de divisão entre dois números, que é $O(n^2)$.

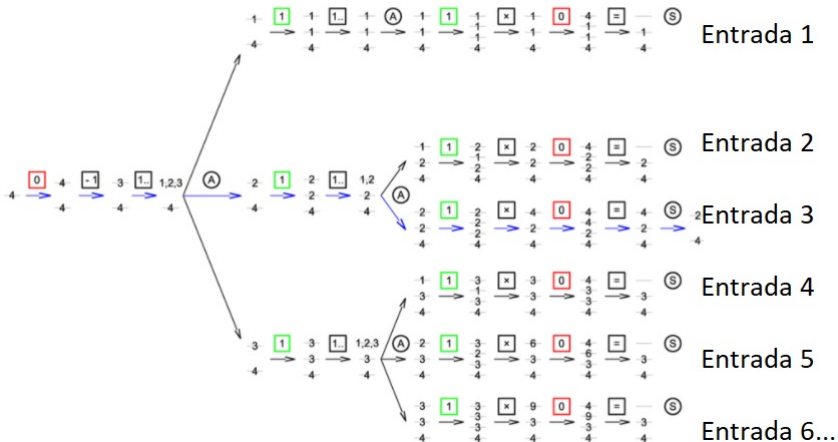
Classe P

- A **classe** P é a classe de problemas resolvíveis em tempo polinomial numa máquina de Turing determinística.
- Uma máquina de Turing determinística é como um autômato determinístico (com uma fita de memória), em que o próximo símbolo e o estado definem o próximo estado.
- Em teoria da computação a Máquina de Turing é o dispositivo padrão para fazer análise de computabilidade. Num sentido muito amplo, um computador é equivalente em computabilidade a uma máquina de Turing.

- Quando um problema NÃO pode ser resolvido em tempo polinomial, isso pode ocorrer por duas razões:
 - 1 A solução tem um tamanho tratável, mas o problema é tão difícil que precisa-se de um tempo exponencial para encontrá-la. P.ex: Problema do caixeiro viajante, número cromático, problemas de otimização de código, sequenciamento de tarefas...
 - 2 A solução tem um tamanho exponencial. P.ex: Encontrar todos os ciclos hamiltonianos de tamanho K ou menor, para um dado grafo. Esses problemas são considerados não realísticos e não serão discutidos.

A Classe NP

- A **classe NP** é a classe de problemas resolvíveis em tempo polinomial em uma máquina de Turing Não-Determinística (NP="Non-Deterministic Polynomial").
- Nesse sentido, a Máquina de Turing não-determinística deve ser visto como uma abstração em que todos os caminhos possíveis são examinados em paralelo e cada ramo examinado chegará a uma solução ou não, independente dos outros ramos.
- Um aspecto importante é que, se algum ramo chegar a uma solução, será possível saber que é uma solução sem consultar os outros ramos.



- Ex: Existe, para um dado grafo, um circuito hamiltoniano com custo menor do que um dado valor k ?
- Solução não determinística:
 - Escolher uma das possíveis trajetórias e calcular sua quilometragem total
 - If "esta quilometragem não excede o valor de k "
 - then considerar encontrada a solução
 - else nada pode ser concluído

- Um exemplo de problema não NP é o problema de encontrar a solução de menor custo para o problema do caixeiro viajante, já que os ramos da máquina de Turing que encontrarem a solução não poderão identificá-la sem comparar com as outras.
- De modo geral, a classificação NP se aplica somente a problemas de decisão.

- A classe **NP** pode ser vista também como os problemas de decisão para o qual a verificação de uma solução pode ser feita em tempo polinomial.
- Sabe-se que $P \subseteq NP$, mas não sabe-se se $P = NP$ ou $P \neq NP$.
- A questão $P=NP$ é um dos 7 problemas do Milênio, definidos pelo Instituto Clay de Matemática, com um prêmio de um milhão de dólares a quem o resolver.

- E na mídia... (Simpsons S07E06 - Treehouse of Horror VI)

The Simpson's: $P = NP$?



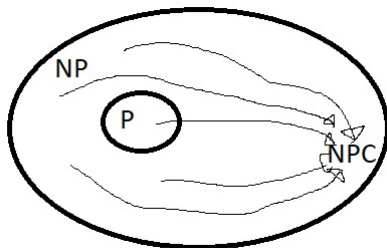
- E Elementary, Futurama e até um filme, Travelling Salesman (2012), sobre 4 matemáticos contratados pelo governo americano para resolver $P=NP$.

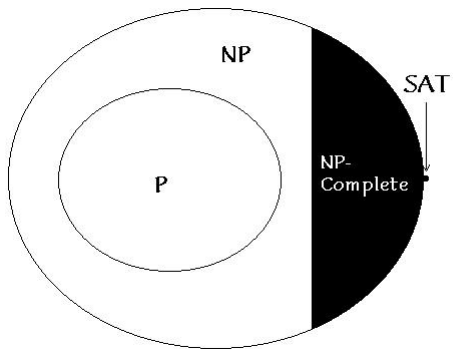
Redução de Problemas

- Diz-se que um problema Q pode ser **reduzido** a um problema R (denota-se por $Q \leq_p R$) se cada instância de Q pode ser transformada em tempo polinomial em uma instância de R que apresenta a mesma resposta Sim/Não (quando se trata de problema de decisão).
- Ou a resposta de R pode ser convertida de volta na resposta de Q .
- Por instância de um problema, considera-se os elementos que o compõem. Para o problema da mochila, uma instância é composta dos objetos, definidos por peso e valor, e o limite de peso da mochila.
- A relação $\leq_p$ é transitiva: se $Q \leq_p R$ e $R \leq_p E$ então $Q \leq_p E$).

- Entre os problemas NP , há uma classe de problemas chamada **NP-Completo**, que tem a propriedade que qualquer problema $\Pi' \in NP$ pode ser mapeado (ou reduzido) para um problema $\Pi \in NP$ — *completo* em tempo polinomial, de forma que a solução de Π também é solução de Π' .
- Um problema Π para ser NP completo, portanto, tem que atender duas propriedades:
 - 1 $\Pi \in NP$
 - 2 todo o problema de decisão Π' satisfaz $\Pi' \Rightarrow \Pi$ (pode ser mapeado para Π)

- Um problema é NP-Completo se todo problema NP pode ser reduzido a ele.





Exemplo simples de redução de problema

- Deseja-se efetuar a expressão de divisão em base 2:
 $(100001110010011)_2 \div (10011)_2$
- Solução possível:
 - 1 Converte-se a expressão para base 10 ($17299 \div 19$)
 - 2 Efetua-se a divisão: $17299 \div 19 = 910.47$
 - 3 Converte-se o resultado de volta para binário:
 $(910.47 \Rightarrow (1110001110)_2$

- Exemplo de redução:
- $\pi_1 =$ contar as cidades de um dado grafo
- $\pi_2 =$ caixeiro-viajante
- A transformação de π_1 em π_2 pode ser feita da seguinte forma:
 - Conjunto de cidades de $\pi_1 =$ Conjunto de cidades de π_2
 - Rota entre todas as cidades de π_2 tem custo 1
 - Resolve-se caixeiro-viajante sobre π_2
 - Número de cidades é o custo do ciclo de π_2

- A importância do estudo dos problemas NP-Completo reside em que a descoberta de uma solução polinomial para um problema NP-Completo implica que todos os problemas NP tem solução polinomial. E que a demonstração que um problema NP é intratável implica que todos os NP-Completo também são.

- O primeiro problema NP-Completo foi descoberto por Stephen Cook em 1972 e é o problema da satisfiability (SAT). A questão é: dada uma expressão lógica com variáveis, constantes, operadores AND, OR e NOT e parênteses, existe uma atribuição de valores às variáveis que torna a expressão verdadeira?

Problema SAT

- **Fórmulas Proposicionais:** Considere um conjunto de variáveis lógicas $x_1, x_2, \dots, x_n$.
 - (A) Um **literal** é uma variável booleana x_i ou sua negação $\neg x_i$.
 - (B) Uma **cláusula** é uma disjunção de literais.

Por exemplo, $x_1 \vee x_2 \vee \neg x_4$ é uma cláusula.
 - (C) Uma fórmula na **Forma Normal Conjuntiva (CNF)** é uma fórmula proposicional que é uma conjunção de cláusulas.
 - (A) $(x_1 \vee x_2 \vee \neg x_4) \wedge (x_2 \vee \neg x_3) \wedge x_5$ é uma fórmula na CNF.
 - (D) Uma fórmula na CNF ϕ está na 3CNF se toda cláusula tem exatamente 3 literais
 - (A) $(x_1 \vee x_2 \vee \neg x_4) \wedge (x_2 \vee \neg x_3 \vee x_1)$ está na 3CNF, mas $(x_1 \vee x_2 \vee \neg x_4) \vee (x_2 \vee \neg x_3) \wedge x_5$ não está.

SAT e 3SAT

- O problema da Satisfatibilidade (SAT) pode ser formulado como: "Dada uma fórmula ϕ na CNF, há alguma atribuição de valores às variáveis que faça com que todas as cláusulas sejam verdadeiras?"
- Ex:
 - $(\neg x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2) \wedge (x_1 \vee x_2)$ é satisfazível.
 - $(x_1 \vee x_2) \wedge (\neg x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2) \wedge (x_1 \vee x_2)$ não é satisfazível.
- Uma instância de SAT pode ser transformada em uma instância de 3SAT. Para isso:
 - Acrescenta-se variáveis às cláusulas que tiverem menos de 3 literais;
 - "Quebra-se" as cláusulas com mais de 3 literais em cláusulas com 3 literais.

- Cook provou que SAT é NP-completo mostrando que qualquer problema de decisão NP pode ser resolvido por uma máquina de Turing não-determinística e que as instâncias de entrada podem ser representadas por cláusulas booleanas para uma máquina de Turing que resolveria o SAT.
- É que ele é NP, uma vez que qualquer atribuição de valores às variáveis pode ser verificada em tempo polinomial.

- Desde Cook e o SAT, milhares de problemas tem sido provados serem NP-Completo.
- Para provar que um dado problema Q é NP-completo, basta mapear algum problema já comprovadamente NP-Completo para Q.
- Essa é a forma usual de demonstrar um problema NP-Completo.
- A figura a seguir mostra alguns problemas clássicos e como foi feita a comprovação da NP-Completeness

Stephen Cook 1971

SAT

Richard Karp 1972

3SAT

Independent Set

Hamiltonian Cycle

Vertex Cover

CLIQUE

Traveling-Salesman Problem(TSP)

Subset-Sum

.....

.....

Por quê é importante conhecer Tratabilidade de Problemas?

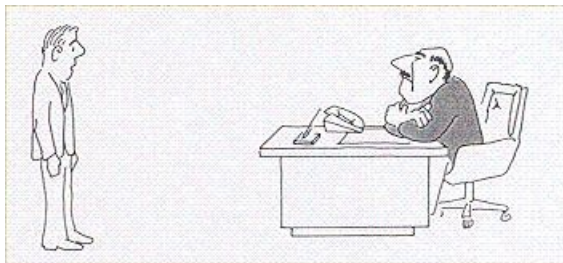
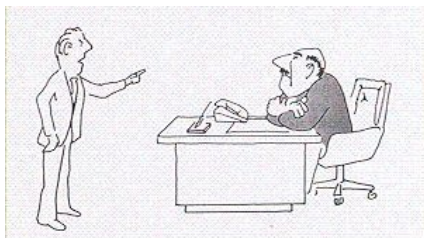


Figura: "Não consegui achar um algoritmo eficiente porque sou incompetente"



Nao consegui achar um algoritmo eficiente porque tal algoritmo
nao existe

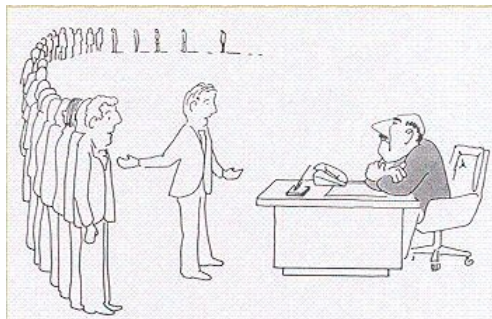


Figura: Não consegui, mas esses cientistas famosos também não conseguiram

Lista de problemas NP-Completo

- Hoje são conhecidos mais de 3000 problemas sabidamente NP-Completo expressos como problemas de decisão.
- A lista a seguir, de áreas de problemas, é tirada do livro Computers and Intractability: A Guide to the Theory of NP-Completeness, de Gary e Johnson, ainda hoje a "Bíblia" de complexidade de algoritmos.
 - Diversos problemas de Grafos: Obtenção do número cromático, cobertura de vértices, conjunto mínimo independente de arestas, homeomorfismo, Particionamento de Grafos
 - Problemas de Projeto de Redes
 - Conjuntos e partições
 - Armazenamento e recuperação

Lista de problemas NP-Completo

- Sequenciamento e escalonamento
- Programação Matemática
- Álgebra e Teoria de Números
- Jogos e quebra-cabeças(p.ex.Solução de Sudoku)
- Lógica
- Autômatos e Linguagens
- Otimização de Programas
- Diversos

■ Algumas abordagens para tratar problemas NP-Completo são:

- Para instâncias pequenas, a solução de custo exponencial pode ser usada. Senão:
- Algoritmos aproximativos, que em alguns casos podem chegar a uma solução próxima da ótima por um fator multiplicativo ou aditivo.
- Randomização: Montecarlo, Algoritmos genéticos
- Restrições no problema: limitando tamanho do problema ou colocando outras restrições (p.ex. se aplicado somente a valores inteiros)
- Heurísticas (como alguns algoritmos gulosos), que não fazem nenhuma garantia da qualidade da solução, mas na maior parte dos casos podem fornecer soluções aceitáveis.

Os problemas no Garey e Johnson normalmente são enunciados da seguinte forma:

- Instância: Conjunto C de m cidades, distância $d(c_i, c_j) \in \mathbb{Z}^+$ para cada par de cidades $c_i, c_j \in C$, inteiro positivo B .
- Questão: Existe um circuito com comprimento menor ou igual a B , i.e., uma permutação $\langle c_{\pi(1)}, c_{\pi(2)}, \dots, c_{\pi(m)} \rangle$ de C tal que

$$\left(\sum_{i=1}^{m-1} d(c_{\pi(i)}, c_{\pi(i+1)}) \right) + d(c_{\pi(m)}, c_{\pi(1)}) \leq B$$

- Referência: Transformação do Problema do Circuito Hamiltoniano
- Comentários: Permanece NP-Completo mesmo se $d(c_i, c_j) \in \{1, 2\}$ para todo $c_i, c_j \in C$. Casos especiais que podem ser discutidos em tempo polinomial podem ser encontrados em [Gilmore and Gomory, 1964]

Problemas NP-Intermediários

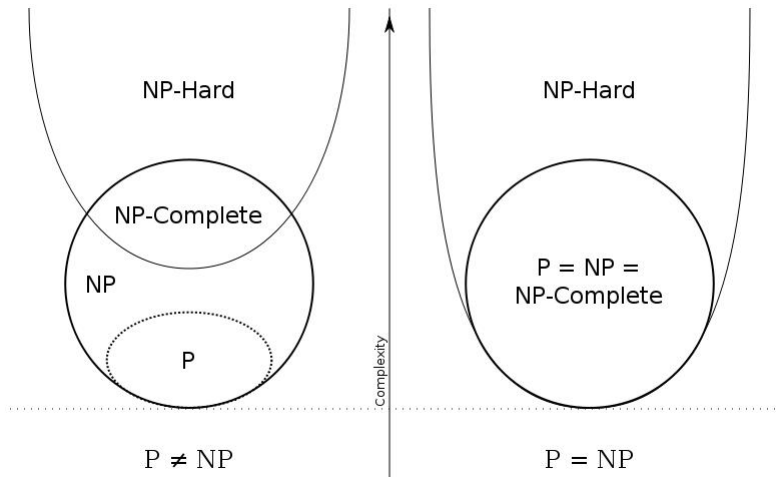
- Problemas que estão na classe NP mas que não estão nem em P e nem em NP-Completo são chamados de NP-Intermediário (NPI)
- É provado que se $P \neq NP$ então NPI não é vazio
- Um problema que acredita-se estar nessa categoria é o problema de isomorfismo em grafos

- Um problema estar em NP e não estar em P não significa necessariamente que não existe solução polinomial para ele.
- Pode ser que o problema não tenha sido pesquisado o suficiente.
- Eventualmente problemas que não estavam em P passam a pertencer a P .
- O primeiro algoritmo polinomial para verificar se um dado número é primo foi encontrado em 2002 por três estudantes indianos (um algoritmo conhecido por AKS, $O(n^{12})$ que foi melhorado e hoje é $O(n^6)$).
- A partir desse momento o problema de testar primalidade passou a estar em P .

Problemas NP-Difíceis

- Um problema **NP-Difícil (NP-hard)** é um problema de otimização associado a um problema de decisão NP-Completo.
- Um problema Π é NP-difícil se todo problema em NP pode ser reduzido a ele.
- Um problema NP-Difícil não precisa necessariamente ser NP.
- Nesse sentido, problemas NP-Difíceis são no mínimo tão difíceis quanto os problemas NP-completos.
- Alguns problemas NP-Difíceis são o problema do caixeiro-viajante (otimização) e o problema da mochila (otimização).

Problemas NP-Difíceis



Exercícios:

1) Qual a classe que não está contida (nem parcialmente) em NP-Difícil:

1 P

2 NP

3 NP-Completo

4 NRA

2) Resolvendo-se de forma determinística em tempo polinomial um problema da classe NP-Difícil podemos afirmar:

1 $P=NP$

2 $P=NP$ -Difícil

3 $NP=NP$ -Difícil

4 NRA

3) Encontrando-se uma solução determinística em tempo polinomial para um problema da classe NP-Completo, podemos afirmar:

1 $P=NP$

2 $P=NP$ -Difícil

3 NP -Completo= NP -Difícil

4 NRA

4) Diga quais das afirmações são verdadeiras e quais são falsas:

- 1 Nem todos os problemas em P são resolvidos em tempo polinomial.
- 2 A classe NP também é conhecida como Classe Não Polinomial.
- 3 A Classe NP representa os problemas que possuem solução não determinística em tempo polinomial.
- 4 Todos os problemas da classe NP -Difícil pertencem também a NP .
- 5 Todos os problemas da classe NP -Completo pertencem à classe NP .

5) Diga quais das afirmações são verdadeiras e quais são falsas :

- 1 Se for solucionado um problema da classe NP-Completo em tempo polinomial podemos afirmar que $P = NP$.
- 2 Se for solucionado um problema da classe NP-Difícil em tempo polinomial podemos afirmar que $P = NP$.
- 3 Se for solucionado um problema da classe NP em tempo polinomial todos os outros NP podem ser resolvidos em tempo polinomial.
- 4 A classe NP-Difícil é uma extensão da Classe NP-Completo, ou seja, a classe NP-Difícil engloba todos os problemas da classe NP-Completo mais outros problemas não pertencentes à classe NP, mas que satisfazem a propriedade da redutibilidade de qualquer problema em NP para problemas em NP-Difícil.
- 5 Os problemas da classe P não pertencem à classe NP.

6) Diga quais das afirmações são verdadeiras e quais são falsas:

- 1 Todos os problemas da Classe P, bem como todos os problemas da classe NP-Completo pertencem à Classe NP.
- 2 Todos os problemas da classe NP-Completo pertencem ao mesmo tempo as classes NP e NP-Difícil.
- 3 Todos os problemas da classe NP-Difícil pertencem ao mesmo tempo as classes NP e NP-Completo.
- 4 Todos os problemas da classe NP pertencem ao mesmo tempo as classes NP-Completo e NP-Difícil.

7) Diga quais das afirmações são verdadeiras e quais são falsas:

- 1 Se for resolvido um problema da classe NP-Difícil em tempo polinomial, todos os NP também terão uma solução em tempo polinomial bem como os NP-Completos.
- 2 Se for resolvido um problema da classe NP-Completo em tempo polinomial, todos os problemas NP-Difíceis também terão solução em tempo polinomial.
- 3 Os problemas P e os problemas NP têm solução em tempo polinomial, a diferença entre eles é que os P possuem solução determinística, e os NP possuem solução não determinística em tempo polinomial.
- 4 A melhor definição para P seria classe de problemas polinomiais e NP seria classe de problemas não polinomiais.

8) Diga quais das afirmações são verdadeiras e quais são falsas:

- 1 Nos problemas NP-Completo encontra-se solução não determinística necessariamente em tempo exponencial.
- 2 Se for resolvido um problema qualquer da classe NP-Difícil em tempo polinomial, todos os problemas NP-Completo também poderão ser resolvidos em tempo polinomial.
- 3 Se for resolvido um problema da classe NP-Completo em tempo polinomial, todos os problemas NP-Difíceis também terão solução em tempo polinomial.
- 4 Ao tentar resolver um problema novo, verificou-se que o mesmo não pode ser resolvido de forma determinística em tempo polinomial. Ao estudá-lo mais percebeu-se que o mesmo pode ser reduzido (mapeado) para um problema NP-Completo. Podemos afirmar seguramente que ele pertence então à classe NP.

BIBLIOGRAFIA

- Aho, A.V., Hopcroft, J.E. and Ullman, J.D., Data Structure and Algorithms, Addison-Wesley, 1983.
- Baase, S., Computer Algorithms - Introduction to Design and Analysis, Second Edition, Addison-Wesley, 1988.
- Gonnet, G.H. and Baeza-Yates, R., Handbook of Algorithms and Data Structures: in Pascal and C Second Edition, 1991.
- Horowitz, Ellis and Sahni, Sartaj., Fundamentals of Data Structures Sixth Printing - Computer Science Press, Inc., 1976.

- Knuth, D., The Art of Computer Programming, Volume 1: Fundamental Algorithms, Addison-Wesley, Second Edition, 1973.
- Knuth, D., The Art of Computer Programming, Volume 3: Sorting and Searching, Addison-Wesley, Second Edition, 1973.
- Sedgewick, R., Algorithms, Second Edition, Addison-Wesley, 1988.
- Wirth, N., Algorithms and Data Structures, Prentice-Hall, 1986.
- Wirth, N., Algoritmos e Estruturas de Dados, Prentice-Hall do Brasil LTDA, 1989.

Questões do ENADE

(2005 - Sistemas de Informação - Questão 13) Julgue os itens a seguir, acerca de algoritmos para ordenação.

I - O algoritmo de ordenação por inserção tem complexidade $O(n \times \log n)$.

II - Um algoritmo de ordenação é dito estável caso ele não altere a posição relativa de elementos de mesmo valor.

III - No algoritmo quicksort, a escolha do elemento pivô influencia o desempenho do algoritmo.

IV - O bubble-sort e o algoritmo de ordenação por inserção fazem, em média, o mesmo número de comparações.

Estão certos apenas os itens

A) I e II.

B) I e III.

C) II e IV.

D) I, III e IV.

E) II, III e IV.

(2005 - Sistemas de Informação - Questão 15) Considere o algoritmo que implementa o seguinte processo: uma coleção desordenada de elementos é dividida em duas metades e cada metade é utilizada como argumento para a reaplicação recursiva do procedimento. Os resultados das duas reaplicações são, então, combinados pela intercalação dos elementos de ambas, resultando em uma coleção ordenada. Qual é a complexidade desse algoritmo?

A - $O(n^2)$

B - $O(n^{2^n})$

C - $O(2^n)$

D - $O(\log n \times \log n)$

E - $O(n \times \log n)$

(2005 - Sistemas de Informação - Questão 21) No modo recursivo de representação, a descrição de um conceito faz referência ao próprio conceito. Julgue os itens abaixo, com relação à recursividade como paradigma de programação.

- I São elementos fundamentais de uma definição recursiva: o caso-base (base da recursão) e a reaplicação da definição.
- II O uso da recursão não é possível em linguagens com estruturas para orientação a objetos.
- III As linguagens de programação funcionais têm, na recursão, seu principal elemento de repetição.
- IV No que diz respeito ao poder computacional, as estruturas iterativas e recursivas são equivalentes.
- V Estruturas iterativas e recursivas não podem ser misturadas em um mesmo programa.

Estão certos apenas os itens

A - I e IV.

B - II e III.

C - I, III e IV.

D - I, III e V.

E - II, IV e V.

(2005 - Ciência da Computação - Questão 43) No processo de pesquisa binária em um vetor ordenado, os números máximos de comparações necessárias para se determinar se um elemento faz parte de vetores com tamanhos 50, 1.000 e 300 são, respectivamente, iguais a:

A - 5, 100 e 30.

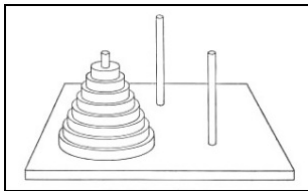
B - 6, 10 e 9.

C - 8, 31 e 18.

D - 10, 100 e 30.

E - 25, 500 e 150.

(2005 - Ciência da Computação - Questão 51) No famoso jogo da Torre de Hanoi, é dada uma torre com discos de raios diferentes, empilhados por tamanho decrescente em um dos três pinos dados, como ilustra a figura abaixo. O objetivo do jogo é transportar-se toda a torre para um dos outros pinos, de acordo com as seguintes regras: apenas um disco pode ser deslocado por vez, e, em todo instante, todos os discos precisam estar em um dos três pinos; além disso, em nenhum momento, um disco pode ser colocado sobre um disco de raio menor que o dele; é claro que o terceiro pino pode ser usado como local temporário para os discos.



Imaginando que se tenha uma situação em que a torre inicial tenha um conjunto de 5 discos, qual o número mínimo de movimentações de discos que deverão ser realizadas para se atingir o objetivo do jogo?

- A) 25
- B) 28
- C) 31
- D) 34
- E) 38

(2005 - Ciência da Computação - Questão 54) Considere que, durante a análise de um problema de programação, tenha sido obtida a seguinte fórmula recursiva que descreve a solução para o problema.

$$T(n) = \begin{cases} 0 & \text{se } i=0 \text{ ou } j=0 \\ 1 + C(i-1, j-1) & \text{se } 0 < i \leq M, 0 < j \leq N \text{ e } i = j \\ \max(C(i, j-1), C(i-1, j)) & \text{se } 0 < i \leq M, 0 < j \leq N \text{ e } i \neq j \end{cases}$$

Qual a complexidade da solução encontrada?

- A) $O(n \times \log n)$
- B) $O(n^2)$
- C) $O(n^2 \times \log n)$
- D) $O(2^n)$
- E) $O(n^3)$

(2005 - Ciência da Computação - Questão 65) A análise de complexidade provê critérios para a classificação de problemas com base na computabilidade de suas soluções, utilizando-se a máquina de Turing como modelo referencial e possibilitando o agrupamento de problemas em classes. Nesse contexto, julgue os itens a seguir.

- I) É possível demonstrar que $P \subseteq NP$ e $NP \subseteq P$.
- II) É possível demonstrar que se $P \neq NP$ então $P \cap NP - \text{Completo} = \emptyset$.
- III) Se um problema Q é NP-difícil e $Q \in NP$, então Q é NP-completo.
- IV) O problema da satisfatibilidade de uma fórmula booleana F (uma fórmula é satisfatível, se é verdadeira em algum modelo) foi provado ser NP-difícil e NP-Completo.
- V) Encontrar o caminho mais curto entre dois vértices dados em um grafo de N vértices e M arestas não é um problema da classe P .

Estão certos apenas os itens

A) I, III e IV.

B) II, III, e IV.

C) III, IV e V.

D) I, II, III, e IV.

E) II, III, IV e V.

(2014 - Ciência da Computação - Questão discursiva 3) O jogo Sudoku consiste em uma matriz 9x9 dividida em 9 sub-matrizes 3x3, como mostrado na figura a seguir.

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Figura: Disponível em <<http://www.en-wikipedia.org>>

A matriz está parcialmente preenchida com números de 1 a 9, e o objetivo do jogo é completar a matriz, de forma que cada linha, coluna e sub-matriz contenham todos os números de 1 a 9.

A partir dessas informações, escreva um algoritmo recursivo baseado em retrocesso (backtracking) para resolver o jogo. A matriz foi transformada em um vetor de 81 posições, contendo zeros nas posições que faltam para serem preenchidas.

Considere que existem duas funções implementadas. A primeira função, `NaoHaViolacao (X, i,V)`, retorna verdadeiro se a inserção do número x na posição i do vetor V não causa violação das restrições do jogo (número repetido em linha, coluna ou sub-matriz). A segunda função, `Imprime (V)`, realiza a impressão do vetor V .

Considere, ainda, que o algoritmo deve imprimir o vetor V com a solução encontrada, se esta existir. (valor: 10,0 pontos)

Observação: Qualquer notação em português estruturado, de forma imperativa ou orientada a objetos pode ser utilizada, assim como em uma linguagem de alto nível, como Pascal, C ou Java.

```

int sudoku(inv v[])
{
    int prox=buscaprox(v);
    if (proxima== -1) {Imprime(v);return 1;}
    int x;
    for (x=1;x<=9;x++)
        if (NaoHaViolacao(x,prox,V)){
            v[prox]=x; if (sudoku(v)) return 1;
        }
    v[prox]=0;
    return 0;}

```

(2014 - Ciência da Computação - Questão discursiva 5) As técnicas de projeto de algoritmos são essenciais para que os desenvolvedores possam implementar software de qualidade. Essas técnicas descrevem os princípios que devem ser adotados para se projetar soluções algorítmicas para um dado problema. Entre as principais técnicas, destacam-se os projetos de algoritmos por tentativa e erro, divisão e conquista, programação dinâmica e algoritmos gulosos.

Nesse contexto, faça o que se pede nos itens a seguir:

- a) Descreva o que caracteriza o projeto de algoritmos por divisão e conquista. (valor: 6,0 pontos)
- b) Apresente uma situação de uso da técnica de projeto de algoritmos por divisão e conquista. (valor: 4,0 pontos)

(2014 - Ciência da Computação - Questão 12) Analise o custo computacional dos algoritmos a seguir, que calculam o valor de um polinômio de grau n , da forma: $P(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$, onde os coeficientes são números de ponto flutuante armazenados no vetor $a[0..n]$, e o valor de n é maior que zero. Todos os coeficientes podem assumir qualquer valor, exceto o coeficiente a_n que é diferente de zero.

]

Algoritmo 1:

soma = a[0]

Repita para i = 1 até n

 Se $a[i] \neq 0.0$ entao

 potencia = x

 Repita para j = 2 até i

 potencia = potencia * x

 Fim repita

 soma = soma + a[i] * potencia

 Fim se

Fim repita

Imprima(soma)

]

Algoritmo 2:

$soma = a[n]$

Repita para $i = n-1$ até 0 passo -1

$soma = soma * x + a[i]$

Fim repita

Imprima($soma$)

Com base nos algoritmos 1 e 2, avalie as asserções a seguir e a relação proposta entre elas.

I. Os algoritmos possuem a mesma complexidade assintótica.

PORQUE

II. Para o melhor caso, ambos os algoritmos possuem complexidade $O(n)$.

A respeito dessas asserções, assinale a opção correta.

- A As asserções I e II são proposições verdadeiras, e a II é uma justificativa correta da I.
- B As asserções I e II são proposições verdadeiras, mas a II não é uma justificativa correta da I.
- C A asserção I é uma proposição verdadeira, e a II é uma proposição falsa.
- D A asserção I é uma proposição falsa, e a II é uma proposição verdadeira.
- E As asserções I e II são proposições falsas.

(2014 - Ciência da Computação - Questão 28) Um cientista afirma ter encontrado uma redução polinomial de um problema NP-Completo para um problema pertencente à classe P. Considerando que esta afirmação tem implicações importantes no que diz respeito a complexidade computacional, avalie as seguintes asserções e a relação proposta entre elas.

I. A descoberta do cientista implica $P = NP$.

PORQUE

II. A descoberta do cientista implica na existência de algoritmos polinomiais para todos os problemas NP-Completos.

A respeito dessas asserções, assinale a opção correta.

- A As asserções I e II são proposições verdadeiras, e a II é uma justificativa correta da I.
- B As asserções I e II são proposições verdadeiras, mas a II não é uma justificativa correta da I.
- C A asserção I é uma proposição verdadeira, e a II é uma proposição falsa.
- D A asserção I é uma proposição falsa, e a II é uma proposição verdadeira.
- E As asserções I e II são proposições falsas.

(2014 - Sistemas de Informação - Questão 10) A área de complexidade de algoritmos abrange a medição da eficiência de um algoritmo frente à quantidade de operações realizadas até que ele encontre seu resultado final. A respeito desse contexto, suponha que um arquivo texto contenha o nome de N cidades de determinado estado, que cada nome de cidade esteja separado do seguinte por um caracter especial de fim de linha e classificado em ordem alfabética crescente. Considere um programa que realize a leitura linha a linha desse arquivo, à procura de nome de cidade.

Com base nessa descrição, verifica-se que a complexidade desse programa é

- A $O(1)$, em caso de busca sequencial.
- B $O(N)$, em caso de busca sequencial.
- C $O(\log N)$, em caso de busca binária.
- D $O(N)$, em caso de transferência dos nomes para uma árvore binária e, então, realizar a busca.
- E $O(\log N)$, em caso de transferência dos nomes para uma árvore binária e, então, realizar a busca

(Ciência da Computação - 2017 - Discursiva 3) Listas lineares armazenam uma coleção de elementos. A seguir, é apresentada a declaração de uma lista simplesmente encadeada.

```
struct ListaEncadeada {  
    int dado;  
    struct ListaEncadeada *proximo;  
};
```

Para imprimir os seus elementos da cauda para a cabeça (do final para o início) de forma eficiente, um algoritmo pode ser escrito da seguinte forma:

```
void mostrar(struct ListaEncadeada *lista) {  
    if (lista != NULL) {  
        mostrar(lista->proximo);  
        printf("%d ", lista->dado);  
    }  
}
```

Com base no algoritmo apresentado, faça o que se pede nos itens a seguir.

- a) Apresente a classe de complexidade do algoritmo, usando a notação Θ . (valor: 3,0 pontos)
- b) Considerando que já existe implementada uma estrutura de dados do tipo pilha de inteiros - com as operações de empilhar, desempilhar e verificar pilha vazia - reescreva o algoritmo de forma não recursiva, mantendo a mesma complexidade. Seu algoritmo pode ser escrito em português estruturado ou em alguma linguagem de programação, como C, Java ou Pascal. (valor: 7,0 pontos)

(Ciência da Computação - 2017 - Questão 22) Um país utiliza moedas de 1, 5, 10, 25 e 50 centavos. Um programador desenvolveu o método a seguir, que implementa a estratégia gulosa para o problema do troco mínimo. Esse método recebe como parâmetro um valor inteiro, em centavos, e retorna um array no qual cada posição indica a quantidade de moedas de cada valor.

```
public static int[] troco(int valor){  
    int[] moedas = new int[5];  
    moedas[4] = valor / 50;  
    valor = valor % 50;  
    moedas[3] = valor / 25;  
    valor = valor % 25;  
    moedas[2] = valor / 10;  
    valor = valor % 10;  
    moedas[1] = valor / 5;  
    valor = valor % 5;  
    moedas[0] = valor;  
    return(moedas);  
}
```

Considerando o método apresentado, avalie as asserções a seguir e a relação proposta entre elas.

I. O método guloso encontra o menor número de moedas para o valor de entrada, considerando as moedas do país.

PORQUE

II. Métodos gulosos sempre encontram a solução global ótima.

A respeito dessas asserções, assinale a opção correta.

- A As asserções I e II são proposições verdadeiras, e a II é uma justificativa correta da I.
- B As asserções I e II são proposições verdadeiras, mas a II não é uma justificativa correta da I.
- C A asserção I é uma proposição verdadeira, e a II é uma proposição falsa.
- D A asserção I é uma proposição falsa, e a II é uma proposição verdadeira.
- E As asserções I e II são proposições falsas.

Questões do POSCOMP

(29 - Fundamentos - 2002) - Qual das seguintes afirmações sobre crescimento assintótico de funções não é verdadeira:

1 $2n^2 + 3n + 1 = O(n^2)$

2 Se $f(n) = O(g(n))$ então $g(n) = O(f(n))$

3 $\log n^2 = O(\log n)$

4 Se $f(n) = O(g(n))$ e $g(n) = O(h(n))$ então $f(n) = O(h(n))$

5 $2^{n+1} = O(2^n)$

(30 - Fundamentos - 2002) - Considere um problema em que são dados 5 objetos com os seguintes pesos e valores:
pesos: $(W_1, W_2, W_3, W_4, W_5) = (6, 10, 9, 5, 12)$

valores: $(P_1, P_2, P_3, P_4, P_5) = (8, 5, 10, 15, 7)$

Além disso, é dada uma mochila que suporta até 30 unidades de peso, para transportar os objetos. O objetivo do problema é preencher a mochila de tal forma que o valor total dos objetos a serem transportados seja o maior possível, mas sem exceder o limite de peso suportado pela mochila. Assuma que é permitido colocar fração de um objeto na mochila. Qual das seguintes alternativas corresponde ao valor máximo obtido no preenchimento da mochila:

1 12.2

2 21.5

3 30.34

4 38.83

5 43.1

(30 - Fundamentos - 2002) - Considere um problema em que são dados 5 objetos com os seguintes pesos e valores:

pesos: $(W_1, W_2, W_3, W_4, W_5) = (6, 10, 9, 5, 12)$

valores: $(P_1, P_2, P_3, P_4, P_5) = (8, 5, 10, 15, 7)$

Além disso, é dada uma mochila que suporta até 30 unidades de peso, para transportar os objetos. O objetivo do problema é preencher a mochila de tal forma que o valor total dos objetos a serem transportados seja o maior possível, mas sem exceder o limite de peso suportado pela mochila. Assuma que é permitido colocar fração de um objeto na mochila. Qual das seguintes alternativas corresponde ao valor máximo obtido no preenchimento da mochila:

1 12.2

2 21.5

3 30.34

4 38.83

5 43.1

(31 - Fundamentos - 2002) - Considere o algoritmo da busca seqüencial de um elemento em um conjunto com n elementos. A expressão que representa o tempo médio de execução desse algoritmo para uma busca bem sucedida é:

1 n^2

2 $n(n+1)/2$

3 $\log_2 n$

4 $(n+1)/2$

5 $n \log n$

(31 - Fundamentos - 2002) - Considere o algoritmo da busca seqüencial de um elemento em um conjunto com n elementos. A expressão que representa o tempo médio de execução desse algoritmo para uma busca bem sucedida é:

1 n^2

2 $n(n+1)/2$

3 $\log_2 n$

4 $(n+1)/2$

5 $n \log n$

(32 - Fundamentos - 2002) - Quais dos algoritmos de ordenação abaixo possuem tempo no pior caso e tempo médio de execução proporcional a $O(n \log n)$.

- 1 Bubble sort e Quicksort
- 2 Quicksort e Merge sort
- 3 Merge sort e Bubble sort
- 4 Heap sort e Selection sort
- 5 Merge sort e Heap sort

(32 - Fundamentos - 2002) - Quais dos algoritmos de ordenação abaixo possuem tempo no pior caso e tempo médio de execução proporcional a $O(n \log n)$.

- 1 Bubble sort e Quicksort
- 2 Quicksort e Merge sort
- 3 Merge sort e Bubble sort
- 4 Heap sort e Selection sort
- 5 Merge sort e Heap sort

(33 - Fundamentos - 2002) - Professor Mac Sperto propôs o seguinte algoritmo de ordenação, chamado de Super Merge, similar ao merge sort: divida o vetor em 4 partes do mesmo tamanho (ao invés de 2, como é feito no merge sort). Ordene recursivamente cada uma das partes e depois intercale-as por um procedimento semelhante ao procedimento de intercalação do merge sort. Qual das alternativas abaixo é verdadeira?

- 1 Super Merge não está correto. Não é possível ordenar quebrando o vetor em 4 partes
- 2 Super Merge está correto, e consome tempo $O(\text{merge sort})$
- 3 Super Merge está correto, mas consome tempo maior que $O(\text{merge sort})$
- 4 Super Merge está correto, e consome tempo menor que $O(\text{merge sort})$
- 5 Nenhuma das afirmações acima está correta

(33 - Fundamentos - 2002) - Professor Mac Sperto propôs o seguinte algoritmo de ordenação, chamado de Super Merge, similar ao merge sort: divida o vetor em 4 partes do mesmo tamanho (ao invés de 2, como é feito no merge sort). Ordene recursivamente cada uma das partes e depois intercale-as por um procedimento semelhante ao procedimento de intercalação do merge sort. Qual das alternativas abaixo é verdadeira?

- 1 Super Merge não está correto. Não é possível ordenar quebrando o vetor em 4 partes
- 2 Super Merge está correto, e consome tempo $O(\text{merge sort})$
- 3 Super Merge está correto, mas consome tempo maior que $O(\text{merge sort})$
- 4 Super Merge está correto, e consome tempo menor que $O(\text{merge sort})$
- 5 Nenhuma das afirmações acima está correta

(39 - Fundamentos - 2003) - Quais das seguintes igualdades são verdadeiras?

I. $n^2 = O(n^3)$

II. $2 * n + 1 = O(n^2)$

III. $n^3 = O(n^2)$

IV. $3 * n + 5 * n \log n = O(n)$

V. $\log n + \sqrt{n} = O(n)$

1 somente I e II

2 somente II, III e IV

3 somente III, IV e V

4 I, II e V

5 I, III e IV

(39 - Fundamentos - 2003) - Quais das seguintes igualdades são verdadeiras?

I. $n^2 = O(n^3)$

II. $2 * n + 1 = O(n^2)$

III. $n^3 = O(n^2)$

IV. $3 * n + 5 * n \log n = O(n)$

V. $\log n + \sqrt{n} = O(n)$

1 somente I e II

2 somente II, III e IV

3 somente III, IV e V

4 I, II e V

5 I, III e IV

(34 - Fundamentos - 2004) - Um algoritmo é executado em 10 segundos para uma entrada de tamanho 50. Se o algoritmo é quadrático, quanto tempo em segundos ele gastará, aproximadamente, no mesmo computador, se a entrada tiver tamanho 100?

1 10

2 20

3 40

4 100

5 500

(34 - Fundamentos - 2004) - Um algoritmo é executado em 10 segundos para uma entrada de tamanho 50. Se o algoritmo é quadrático, quanto tempo em segundos ele gastará, aproximadamente, no mesmo computador, se a entrada tiver tamanho 100?

1 10

2 20

3 40

4 100

5 500

(38 - Fundamentos - 2004) - Para um certo problema foram apresentados dois algoritmos de divisão e conquista, A e B , cujos tempos de execução são descritos, respectivamente, por $T_A(n) = 7T_A(n/2) + n^3$ e $T_B(n) = \alpha T_B(n/4) + n^2$. Qual é o maior valor inteiro para α , tal que o tempo de execução de B seja assintoticamente menor que o de A , isto é, $T_B(n) \in o(T_A(n))$?

1 16

2 49

3 63

4 64

5 65

(38 - Fundamentos - 2004) - Para um certo problema foram apresentados dois algoritmos de divisão e conquista, A e B , cujos tempos de execução são descritos, respectivamente, por $T_A(n) = 7T_A(n/2) + n^3$ e $T_B(n) = \alpha T_B(n/4) + n^2$. Qual é o maior valor inteiro para α , tal que o tempo de execução de B seja assintoticamente menor que o de A , isto é, $T_B(n) \in o(T_A(n))$?

1 16

2 49

3 63

4 64

5 65

(29 - Fundamentos - 2006) - Considere a função Pot que calcula x^n , para x real e n inteiro:

```
Function Pot(x:real;n:integer):real;  
begin  
    if x=0 then  
        Pot := 0  
    else if n=0 then  
        Pot :=1  
    else if n<0 then  
        Pot := 1/Pot(x,abs(n))  
    else if odd(n) then  
        Pot := x * sqr(Pot(x,(n-1) div 2))  
    else  
        Pot := sqr(Pot(x,n div 2))  
end;
```

Seja $T(n)$ o tempo de execução da função Pot para as entradas x e n . A ordem de $T(n)$ é:

1 $T(n) = \Theta(1)$

2 $T(n) = \Theta(\log n)$

3 $T(n) = \Theta(n)$

4 $T(n) = \Theta(n \log n)$

5 $T(n) = \Theta(n^2)$

(31 - Fundamentos - 2006) - Quais algoritmos de ordenação têm complexidade $O(n \log n)$ para o melhor caso, onde n é o número de elementos a ordenar.

- 1 Insertion Sort e Quicksort
- 2 Quicksort e Heapsort
- 3 Bubble Sort e Insertion Sort
- 4 Heapsort e Insertion Sort
- 5 Quicksort e Bubble Sort

(31 - Fundamentos - 2007) - Considere o *problema do caixeiro viajante*, definido como se segue.

Sejam S um conjunto de n cidades, $n \geq 0$, e $d_{ij} > 0$ a distância entre as cidades i e j , $i, j \in S, i \neq j$. Define-se um percurso fechado como sendo um percurso que parte de uma cidade $i \in S$, passa exatamente uma vez por cada cidade de S , e retorna à cidade de origem. A distância de um percurso fechado é definida como sendo a soma das distâncias entre cidades consecutivas no percurso. Deseja-se encontrar um percurso fechado de distância mínima. Suponha um algoritmo guloso que, partindo da cidade 1, move-se para a cidade mais próxima ainda não visitada e que repita esse processo até passar por todas as cidades, retornando à cidade 1.

Considere as seguintes afirmativas.

I. Todo percurso fechado obtido com esse algoritmo tem distância mínima.

II. O problema do caixeiro viajante pode ser resolvido com um algoritmo de complexidade linear no número de cidades.

III. Dado que todo percurso fechado corresponde a uma permutação das cidades, existe um algoritmo de complexidade exponencial no número de cidades para o problema do caixeiro viajante.

Em relação a essas afirmativas, pode-se afirmar que

- 1** I é falsa e III é correta.
- 2** I, II e III são corretas.
- 3** apenas I e II são corretas.
- 4** apenas I e III são falsas.
- 5** I, II e III são falsas.

(33 - Fundamentos - 2007) - Seja $L = \langle r_1, \dots, r_n \rangle$ uma lista qualquer de inteiros não necessariamente distintos.

A esse respeito, assinale a alternativa **INCORRETA**.

- 1 Existe um algoritmo determinístico ótimo de complexidade $O(n)$ para selecionar o maior elemento de L .
- 2 Existe um algoritmo determinístico de complexidade $O(n \lg n)$ para selecionar, para $1 \leq i \leq n$, o i -ésimo menor elemento de L .
- 3 Se existe um algoritmo linear para selecionar o i -ésimo menor elemento de L , então, usando esse algoritmo, é possível projetar um algoritmo linear para ordenar L em ordem não crescente.
- 4 Existe um algoritmo linear para determinar o terceiro maior elemento de L .
- 5 Existe um algoritmo que, percorrendo uma única vez L , pode determinar o menor e o maior elemento de L .

(34 - Fundamentos - 2007) - Seja $V = \langle v_1, \dots, v_n \rangle$ uma lista qualquer de inteiros distintos que se deseja ordenar em ordem *não decrescente*. Analise as seguintes afirmativas.

I. Considere o algoritmo *Quicksort*. Suponha uma execução do algoritmo sobre V tal que a cada sorteio do pivot, a mediana do (sub)problema em questão é escolhida. Então, a complexidade dessa execução é $O(n \lg n)$.

II. Considere o algoritmo *Quicksort*. Suponha uma execução do algoritmo sobre V tal que a cada sorteio do pivot, os dois subproblemas gerados têm tamanho $\frac{1}{10}$ e $\frac{9}{10}$ respectivamente do tamanho do (sub)problema em questão. Então, a complexidade dessa execução é $O(n^2)$.

III. Considere o algoritmo *Mergesort*. A complexidade do pior caso do algoritmo é $O(n \lg n)$ e a complexidade do melhor caso (vetor já ordenado) é $O(n)$.

IV. Considere o algoritmo *Heapsort*. A complexidade do pior caso do algoritmo é $O(n \lg n)$ e a complexidade do melhor caso (vetor já ordenado) é $O(n)$.

V. Se para todo i , v_i é $O(n)$, então a complexidade do algoritmo *Bucketsort* é $O(n)$.

A partir dos dados acima, pode-se concluir que estão **CORRETAS**

- 1 apenas as afirmativas I e II.
- 2 apenas as afirmativas I, II e III.
- 3 apenas as afirmativas I, III e V.
- 4 apenas as afirmativas III, IV e V.
- 5 apenas as afirmativas I e V.

(10 - Matemática - 2010) A relação de recorrência abaixo representa um processo de enumeração por recursão.

$$T(n) = \begin{cases} 0 & p/n = 1 \\ nT(n-1) + n & p/n > 1 \end{cases}$$

Assinale a alternativa que corresponde a um limite superior para o valor da fórmula fechada de tal relação de recorrência.

- a) $T(1)$
- b) 0
- c) n^2
- d) 1024
- e) $n!$

(25 Fundamentos 2010) Considere dois algoritmos A_1 e A_2 , cujas funções de custo são, respectivamente, $T_1(n) = n^2 - n + 1$ e $T_2(n) = 6n \log_2 n + 2n$. Para simplificar a análise, assuma que $n > 0$ é sempre uma potência de 2. Com relação ao enunciado, assinale a alternativa correta.

- a) Como $T_1(n) = \Theta(n^2)$ e $T_2(n) = \Theta(n \log n)$, então A_2 é sempre mais eficiente que A_1 .
- b) O limite superior $T_1(n) = O(n^3)$ é correto e assintoticamente restrito.
- c) O limite inferior $T_2(n) = \Omega(n^3)$ é correto e assintoticamente restrito.
- d) T_1 e T_2 são assintoticamente equivalentes.
- e) A_1 é mais eficiente que A_2 , para n suficientemente pequeno.

(27 Fundamentos 2010) Considere o problema de ordenação onde os vetores a serem ordenados, de tamanho $n > 0$, possuem $\lfloor \frac{n}{2} \rfloor$ valores iguais a um número real x e $\lceil \frac{n}{2} \rceil$ valores iguais a um outro número real y . Considere que os números reais x e y são conhecidos e fixos, porém estão distribuídos aleatoriamente no vetor a ser ordenado. Neste caso, é correto afirmar:

- a) Podemos ordenar estes vetores a um custo $O(n)$.
- b) No caso médio, o Quicksort será o algoritmo mais eficiente para este problema, com um custo $O(n \log n)$.
- c) O algoritmo de ordenação por inserção sempre opera no melhor caso com um custo $O(n)$.
- d) O limite inferior para esta classe de problema é $\Omega(n^2)$.
- e) O limite inferior para esta classe de problema é $\Omega(n \log n)$.

(32 Fundamentos 2010) Considere o algoritmo a seguir.

```
PROC (n)
    se  $n \leq 1$  então
        retorna  $1 + n$ ;
    senão
        retorna  $\text{PROC}(n/2) + \text{PROC}(n/2)$ ;
    fim se
```

Assinale a alternativa que indica corretamente quantas comparações são feitas para uma entrada $n > 0$, onde n é um número natural.

- a) n
- b) $\log n + 1$
- c) $n \log n + 1$
- d) $n^2 + n - 1$
- e) $2n - 1$

(2011 - Fundamentos - 24) Sejam $TA(n)$ e $TB(n)$ os tempos de execução de pior caso de dois algoritmos A e B propostos para um mesmo problema computacional, em função de um certo parâmetro n . Dizemos que o algoritmo A é mais eficiente que o algoritmo B assintoticamente no pior caso quando

- a) $TA(n) = o(TB(n))$.
- b) $TB(n) = o(TA(n))$.
- c) $TA(n) = O(TB(n))$.
- d) $TB(n) = O(TA(n))$.
- e) $TA(n) = \Theta(TB(n))$.

(2011 - Fundamentos - 24) Sejam $TA(n)$ e $TB(n)$ os tempos de execução de pior caso de dois algoritmos A e B propostos para um mesmo problema computacional, em função de um certo parâmetro n . Dizemos que o algoritmo A é mais eficiente que o algoritmo B assintoticamente no pior caso quando

a) $TA(n) = o(TB(n))$.

b) $TB(n) = o(TA(n))$.

c) $TA(n) = O(TB(n))$.

d) $TB(n) = O(TA(n))$.

e) $TA(n) = \Theta(TB(n))$.

(2012 - Fundamentos - 21 e 22) O algoritmo ALGSORT ordena vetores de números inteiros distintos usando apenas comparações. Nesse algoritmo, a função $\text{menor}(V, i, j)$ retorna o índice l , tal que $V[l]$ é o menor número no vetor $V[i..j]$. O custo de tempo de pior caso de $\text{menor}(V, i, j)$ é igual a $j - i$ comparações. De forma similar, a função $\text{maior}(V, i, j)$ retorna um índice g , tal que $V[g]$ é o maior número no vetor $V[i..j]$, também com custo de execução de $j - i$ comparações no pior caso. Para ordenar o vetor $X[1..n]$, $\text{ALGSORT}(V, i, j)$ é chamado com os parâmetros, $V = X$, $i = 1$ e $j = n$.

```
ALGSORT (V,i,j);  
(1) Se  $j-i=0$  então retorne;  
(2) Se  $j-i=1$  então  
    Se  $V[j] < V[i]$  então  
        Troque( $V[j],V[i]$ );  
    Fim;  
    retorne;  
Fim;  
(3)  $l = \text{menor}(V,i,j)$ ;  
(4) Troque( $V[i],V[l]$ );  
(5)  $g = \text{maior}(V,i,j)$ ;  
(6) Troque( $V[j],V[g]$ );  
(7) ALGSORT ( $V,i+1,j-1$ );
```


(2012 - Fundamentos - 21) A função que caracteriza o custo de tempo de pior caso, $T(n)$, para a chamada $ALGSORT(X, 1, n)$ é dada por:

- a) $T(n) = T(n - 1) + 2n - 2$
- b) $T(n) = T(n - 2) + 2n - 2$
- c) $T(n) = T(n - 2) + n - 1$
- d) $T(n) = T(n - 2) + (n - 1)^2$
- e) $T(n) = T(\frac{n}{2}) + 2n$

(2012 - Fundamentos - 22) Com relação ao projeto do algoritmo ALGSORT, assinale a alternativa correta.

- a) O custo de combinação de ALGSORT é $O(n)$ em função do tamanho da entrada para a chamada $\text{ALGSORT}(X, 1, n)$.
- b) Modificando o trecho das linhas de (3) a (6) de ALGSORT, é possível obter um algoritmo assintoticamente menos custoso.
- c) O tempo de execução para a chamada $\text{ALGSORT}(X, 1, n)$ em função de n é $O(n \lg n)$.
- d) O tempo de execução de ALGSORT é $\Theta(n^2)$ em função de n para a chamada $\text{ALGSORT}(X, 1, n)$.
- e) O custo do caso base $n = 1$ para a chamada $\text{ALGSORT}(X, 1, n)$ em função de n é $T(n) = 1$.

(2012 - Fundamentos - 23) Em relação à pesquisa sequencial e binária, assinale a alternativa correta.

- a) A pesquisa binária em média percorre a metade dos elementos do vetor.
- b) A pesquisa binária percorre no pior caso $\log_2 n$ elementos.
- c) A pesquisa binária pode ser feita sobre qualquer distribuição dos elementos.
- d) A pesquisa sequencial exige que os elementos estejam completamente ordenados.
- e) A pesquisa sequencial percorre todos os elementos para encontrar a chave.

(2016 - Fundamentos - 21) Um algoritmo tem complexidade $O(3m^3 + 2mn^2 + n^2 + 10m + m^2)$. Uma maneira simplificada de representar a complexidade desse algoritmo é:

- (A) $O(m^3 + mn^2)$.
- (B) $O(m^3)$.
- (C) $O(m^2)$.
- (D) $O(mn^2)$.
- (E) $O(m^3 + n^2)$.

(2016 - Fundamentos - 22) O tempo de execução $T(n)$ de um algoritmo, em que n é o tamanho da entrada, é dado pela equação de recorrência $T(n) = 8T(n/2) + q \cdot n$ se $n > 1$. Dado que $T(1) = p$, e que p e q são constantes arbitrárias, a complexidade do algoritmo é:

- (A) $O(n)$.
- (B) $O(n \log n)$.
- (C) $O(n^2)$.
- (D) $O(n^3)$.
- (E) $O(n^n)$.

(2016 - Fundamentos - 25) Em relação ao projeto de algoritmos, relacione a Coluna 1 à Coluna 2.

Coluna 1

1. Tentativa e Erro.
2. Divisão e Conquista.
3. Guloso.
4. Aproximado.
5. Heurística.

Coluna 2

- () O algoritmo decompõe o processo em um número finito de subtarefas parciais que devem ser exploradas exaustivamente.
- () O algoritmo divide o problema a ser resolvido em partes menores, encontra soluções para as partes e então combina as soluções obtidas em uma solução global.
- () O algoritmo constrói por etapas uma solução ótima. Em cada passo, após selecionar um elemento da entrada (o melhor), decide se ele é viável (caso em que virá a fazer parte da solução) ou não. Após uma sequência de decisões, uma solução para o problema é alcançada.
- () O algoritmo gera soluções cujo resultado encontra-se dentro de um limite para a razão entre a solução ótima e a produzida pelo algoritmo.
- () O algoritmo pode produzir um bom resultado, ou até mesmo obter uma solução ótima, mas pode também não produzir solução nenhuma ou uma solução distante da solução ótima.

A ordem correta de preenchimento dos parênteses, de cima para baixo, é:

(A) 1 - 2 - 3 - 4 - 5.

(B) 2 - 3 - 4 - 5 - 1.

(C) 3 - 4 - 5 - 1 - 2.

(D) 4 - 5 - 1 - 2 - 3.

(E) 5 - 1 - 2 - 3 - 4

2011/2

1)(2 pontos) Resolva as seguintes equações de recorrência, supondo $T(1)=1$:

1 $T(n) = 2T(n/2) + 1$

2 $T(n) = T(n/2) + b.n + c$

2)(2 pontos) Para o subalgoritmo recursivo abaixo, determine a equação de recorrência que representa a complexidade do mesmo e a equação fechada correspondente. Verifique sua solução por indução matemática.

```
procedimento p2(n: inteiro positivo)
se n > 1 entao
    para i = 1 ate 8 faca p2(n/2) fimpara
    para i = 1 ate n faca
        escreva i * n      <- considere operacao fundamental
    fimpara
fimse
```

3)(2 pontos) Para esse também:

```
void p1(int n, int a)
{
    if (n>1)
    {
        for (i=1;i<=a;i++) p1(n/a,a);
        for (i=1;i<=n;i++)
            printf("\%d\n",i*a); /* op. fundamental */
    }
}
```

4)(2 pontos) Um programa com complexidade n^3 calcula o custo de 800 produtos em 2 horas utilizando o computador atual. Sabe-se que a quantidade de produtos será duplicada e que o limite de tempo para executar esse programa é de 4 horas. Diga se o computador atual atende à necessidade de processamento e indique qual a necessidade mínima de desempenho de um novo se for o caso.

Avaliação da Primeira Área - 02/05/12

1)(2 pontos) Resolva a seguinte equação de recorrência, supondo $T(1)=1$:

$$T(n) = 4.T(n/2) + 3.n$$

2)(2 pontos) Para o subalgoritmo recursivo abaixo, determine a equação de recorrência que representa a complexidade do mesmo e a equação fechada correspondente. Verifique sua solução por indução matemática.

```
void p1(int n)
{
  if (n>1)
  {
    p1(n/2);
    p1(n/2);
    p1(n/2);
    p1(n/2);
    for (i=1;i<=n*n;i++)
      soma = soma + i; <- op. fundamental
  }
}
```

3)(1 ponto)(POSCOMP - Fundamentos - 2004) - Um algoritmo é executado em 10 segundos para uma entrada de tamanho 50. Se o algoritmo é quadrático, quanto tempo em segundos ele gastará, aproximadamente, no mesmo computador, se a entrada tiver tamanho 100?

1 10

2 20

3 40

4 100

5 500

5)(1 ponto)(POSCOMP - Fundamentos - 2003) - Quais das seguintes igualdades são verdadeiras?

I. $n^2 = O(n^3)$

II. $2 * n + 1 = O(n^2)$

III. $n^3 = O(n^2)$

IV. $3 * n + 5 * n \log n = O(n)$

V. $\log n + \sqrt{n} = O(n)$

a) somente I e II

b) somente II, III e IV

c) somente III, IV e V

d) somente I, II e V

e) somente I, III e IV

6)(2 pontos)(POSCOMP - Fundamentos - 2006) - Considere a função Pot que calcula x^n , para x real e n inteiro:

```
Function Pot(x:real;n:integer):real;
begin
  if x=0
    then Pot := 0
    else
      if n=0
        then Pot :=1
        else
          if n<0
            then Pot := 1/Pot(x,abs(n))
            else
              if odd(n)      <- testa se n é ímpar
                then
Pot := x * sqr(Pot(x,(n-1) div 2))  <- sqr=eleva ao quadrado
              else
                Pot := sqr(Pot(x,n div 2))
end;
```


Seja $T(n)$ o tempo de execução da função Pot para as entradas x e n . A ordem de $T(n)$ é:

- a) $T(n) = O(1)$
- b) $T(n) = O(\log n)$
- c) $T(n) = O(n)$
- d) $T(n) = O(n \log n)$
- e) $T(n) = O(n^2)$

Avaliação da Primeira Área - 09/05/12

1) Resolva as seguintes equações de recorrência (2 pontos cada):

$$\text{a) } T(n) = \begin{cases} 1 & p/n = 1 \\ 2 \cdot T(n/2) + n^2 + n & p/n > 1 \end{cases}$$

$$\text{b) } T(n) = \begin{cases} 1 & p/n = 1 \\ T(n/3) + \log_3(n/3) & p/n > 1 \end{cases}$$

2)(2 pontos) Para o subalgoritmo recursivo abaixo, determine a equação de recorrência que representa a complexidade do mesmo e a equação fechada correspondente. Verifique sua solução por indução matemática.

```
procedimento p2(n: inteiro positivo)
se n > 1 então
    p2(n/2)
    para i = 1 até n faça
        para j = 1 até n faça
            escreva i * j * n ← op. fundamental
        fimpara
    fimpara
fimse
```

3)(2 pontos) Um programa de complexidade n^3 precisa de 64 minutos para executar em um computador X com um tamanho de entrada 400.

a) Qual o tamanho máximo da entrada para executar em 27 minutos.

b) Se uma alteração no programa reduziu-o para complexidade quadrática, qual o tamanho máximo da entrada para executar em 64 minutos.

4)(2 pontos) Determine a complexidade do algoritmo abaixo:

```
enquanto n>0 faca  <- para facilitar suponha n potencia de 10
  para j=1 to n faca
    escreva j*n      <- operacao fundamental
  fimpara
  n <- n \ 10        <- lembrando que o operador \ faz a divisao inteira
fimenquanto
```

Avaliação Primeira Área - 07/07/11

1)(2 pontos) Resolva as seguintes equações de recorrência, supondo $T(1)=1$ e verifique a correção de sua solução por indução matemática:

1 $T(n) = 2T(n/2) + 1$

2 $T(n) = T(n/2) + b.n + c$

2)(2 pontos) Para o subalgoritmo recursivo abaixo, determine a equação de recorrência que representa a complexidade do mesmo e a equação fechada correspondente. Verifique sua solução por indução matemática.

```
procedimento p2(n: inteiro positivo)
se n > 1 entao
    para i = 1 ate 8 faca p2(n/2) fimpara
    para i = 1 ate n faca
        escreva i * n      ← op. fundamental
    fimpara
fimse
```

3)(2 pontos) Para esse também:

```
void p1(int n, int a)
{
    if (n>1)
    {
        for (i=1;i<=a;i++) p1(n/a,a);
        for (i=1;i<=n;i++)
            printf("\%d\n",i*a); /* op. fundamental */
    }
}
```


4)(2 pontos) Um programa com complexidade n^3 calcula o custo de 800 produtos em 2 horas utilizando o computador atual. Sabe-se que a quantidade de produtos será duplicada e que o limite de tempo para executar esse programa é de 4 horas. Diga se o computador atual atende à necessidade de processamento e indique qual a necessidade mínima de desempenho de um novo se for o caso.

Avaliação Primeira Área Parte 2 - 08/10/12

1)(3 pontos) Resolva as seguintes equações de recorrência, supondo $T(1)=1$ e verifique a correção de sua solução por indução matemática:

1 $T(n) = 2T(n/2) + 1$

2 $T(n) = T(n/2) + b.n + c$

3 $T(n) = 2.T(n-1) + 1$

2)(2 pontos) Para os dois subalgoritmos recursivos abaixo, determine a equação de recorrência que representa a complexidade do mesmo e a equação fechada correspondente.

```
procedimento p2(n: inteiro positivo)
se n > 1 entao
    para i = 1 ate 8 faca p2(n/2) fimpara
    para i = 1 ate n faca
        escreva i * n      ← op. fundamental
    fimpara
fimse
```

lev

```
void p1(int n, int a)
{
    if (n>1)
    {
        for (i=1;i<=a;i++) p1(n/a,a);
        for (i=1;i<=n;i++)
            printf("\%d\n",i*a); /* op. fundamental */
    }
}
```

Avaliação da Primeira Área - 07/10/13

1)(1.5 pontos) Um programa tem complexidade n^4 . Uma empresa usa este programa e tem no máximo 1 hora e 21 minutos para executá-lo. Atualmente este programa processa um arquivo com 200 itens e precisa 16 minutos de tempo.

- 1 Qual o número máximo de itens que podem ser processados no tempo disponível, que é 1h21m ?
- 2 Se o número de itens dobrar para 400, qual o tempo necessário?
- 3 Se a complexidade deste programa for reduzida para n^2 , qual o número máximo de itens que podemos processar no tempo disponível (1h21m)?

2)(2 pontos) Determine a complexidade do algoritmo abaixo:

```
enquanto n>0 faca  <- para facilitar suponha n potencia de 10 ]  
  para j=1 to n faca  
    escreva j*n      <- considere operacao fundamental  
  fimpara  
  n <- n \ 10        <- lembrando que o operador \ faz a divisao inteira  
fimenquanto
```

Avaliação Primeira Área - Parte 2 - 09/10/13

1)(2 pontos) Resolva as seguintes equações de recorrência, supondo $T(1)=1$ e verifique a correção de sua solução por indução matemática:

1 $T(n) = 2T(n/2) + 1$

2 $T(n) = T(n/2) + b.n$

2)(1.5 pontos) Para o subalgoritmo recursivo abaixo, determine a equação de recorrência que representa a complexidade do mesmo e a equação fechada correspondente. Verifique sua solução por indução matemática.

```
procedimento p1(n: inteiro positivo)
se n > 1 entao
    p1(n/4)
    p1(n/4)
    para i de 1 ate n faca
        para j de 1 ate i faca
            escreva j ← op fundamental
        fimpara
    fimpara
fimse
```


3)(1.5 pontos) Para esse também:

```
void p1(int n, int a)
{
    if (n>1)
    {
        for (i=1;i<=a;i++) p1(n/a,a);
        for (i=1;i<=n;i++)
            printf("\%d\n",i*a); /* op. fundamental */
    }
}
```

Recuperação da Primeira Área - 07/07/11

1)(2 pontos) Para o subalgoritmo recursivo abaixo, determine a equação de recorrência que representa a complexidade do mesmo e a equação fechada correspondente. Verifique sua solução por indução matemática.

```
function p1(int n)
{
  if (n>1)
  {
    p1(n/2);
    for (i=1;i<=n*n;i++)
      soma = soma + i;           // <- op. fundamental
    for (i=1;i<=n;i++)
      soma = soma+i*i;          //<- aqui também
  }
}
```

2) (2.pontos) Determine a complexidade do algoritmo abaixo:

```
leia n
para i=1 ate n faca
    j ← 2
    enquanto j ≤ 2*n faca
        para k=1 ate j faca
            escreva i+j*k      ← op. fundamental
        fimpara
        j ← j + 2
    fimenquanto
fimpara
```

3)(2 pontos) Um programa com complexidade n^3 atende as necessidades de uma empresa que processa dados de 1000 funcionários em meia hora.

- 1 Se o número de funcionários reduzir para a metade, qual será o tempo de execução deste programa?
- 2 Se o computador for trocado por um outro 4 vezes mais rápido, qual o número máximo de funcionários que este programa processa em 8 horas?

4)(2 pontos) Os laços abaixo formam a estrutura para preencher a matriz no problema da ordem dos produtos matriciais. Calcule sua complexidade:

```
for ( i=1; i<N; i++)  
    for ( j=0; j<N-i ; j++)  
        for ( k=i-2; k>=0; k--)  
            c=c+1; // op. fundamental
```

5)(2 pontos) (38 - Fundamentos - 2004) - Para um certo problema foram apresentados dois algoritmos, A e B , cujos tempos de execução são descritos, respectivamente, por $T_A(n) = 7T_A(n/2) + n^3$ e $T_B(n) = \alpha T_B(n/4) + n^2$. Qual é o maior valor inteiro para α , tal que o tempo de execução de B seja assintoticamente MENOR que o de A , isto é, $T_B(n) \in o(T_A(n))$? Sugestão: Use o Teorema Mestre.

Avaliação Primeira Área - Parte 2 - 09/10/13

1)(2 pontos) Resolva as seguintes equações de recorrência, supondo $T(1)=1$ e verifique a correção de sua solução por indução matemática:

1 $T(n) = 2T(n/2) + 1$

2 $T(n) = T(n/2) + b$

2)(1.5 pontos) Para o subalgoritmo recursivo abaixo, determine a equação de recorrência que representa a complexidade do mesmo e a equação fechada correspondente. Verifique sua solução por indução matemática. Considere como operação fundamental somente as chamadas recursivas

```
funcao fibxp1(n:inteiropositivo)
se n<=1
    entao retorna (n)
    senao retorna (fibx(n-1)+fibx(n-1))
fimse
fimfuncao
```


3)(1.5 pontos) Para esse também:

```
funcao fibxp1(n:inteiropositivo)
se n<=1
    entao retorna (n)
    senao retorna (2*fibx(n-1))
fimse
fimfuncao
```

Avaliações da Segunda Área - 02/12/2010

1. (2 pontos) Ao executar-se um programa constata-se que o mesmo leva um tempo excessivo, e procura-se reduzir o tempo de execução do mesmo. Constata-se, também, que a ineficiência do programa resulta da resolução repetida dos subproblemas que resolvem o problema. Que técnica de programação pode ser utilizada para melhorar a eficiência do mesmo, tirando proveito dessa característica do problema (resolução repetida dos subproblemas)?

3. (2 pontos) Descreva como o problema da mochila com itens ilimitados pode ser resolvido usando programação dinâmica. Mostre um exemplo numérico.
4. (2 pontos) Deseja-se eliminar de um vetor $V[1..N]$ todos os números que estão em um vetor $W[1..M]$, compactando assim o vetor V . Os números desses vetores são inteiros no intervalo $[0,99]$. Descreva (ou escreva) um algoritmo que faça isso em tempo linear.

1. (1 ponto) Ao executar-se um programa constata-se que o mesmo leva um tempo excessivo, e procura-se reduzir o tempo de execução do mesmo. Constata-se, também, que a ineficiência do programa resulta da resolução repetida dos subproblemas que resolvem o problema. Que técnica de programação pode ser utilizada para melhorar a eficiência do mesmo, tirando proveito dessa característica do problema (resolução repetida dos subproblemas)?

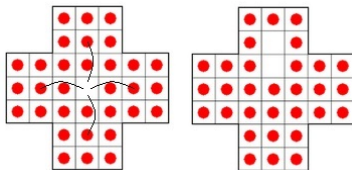
3. (1 ponto) Como se pode demonstrar que um dado problema é NP-Completo?
4. (1 ponto) Deseja-se eliminar de um vetor $V[1..N]$ todos os números que estão em um vetor $W[1..M]$, compactando assim o vetor V . Os números desses vetores são inteiros no intervalo $[0,99]$. Descreva (ou escreva) um algoritmo que faça isso em tempo linear.

6. (1 ponto) O método guloso pode ser utilizado para resolver o problema do Caixeiro Viajante ou alguma variante do mesmo? Se usado, ele encontra a solução ótima? Justifique sua resposta, se necessário usando um exemplo numérico.
7. (1 ponto) Descreva o que é backtracking e mostre um exemplo de código que implemente um backtracking.
8. (1 ponto) Qual a semelhança e qual a principal diferença entre a abordagem de divisão e conquista e a abordagem de programação dinâmica na solução de problemas?

1. (1 ponto) Considere o problema de, dada uma matriz $M[100,100]$ preenchida com 0's 1's, verificar se existe um caminho da posição $[li, ci]$ até a casa $[lf, cf]$. Abaixo é mostrada uma solução que utiliza backtracking.

```
int tem(int li , int ci , int lf , int cf , int M[100][100])
{
    if (li==lf && ci==cf) return 1;
    if (li < 99)
        if (tem(li+1,ci , lf , cf , M)==1) return 1;
    if (li > 0)
        if (tem(li-1,ci , lf , cf , M)==1) return 1;
    if (ci < 99)
        if (tem(li , ci+1,lf , cf , M)==1) return 1;
    if (ci > 0)
        if (tem(li-1, ci , lf , cf , M)==1) return 1;
    return 0;
}
```

4. (2 pontos) No jogo resta um, a partir da posição inicial das peças do tabuleiro, tem-se o objetivo de efetuar jogadas ate que reste apenas uma peça. Em cada movimento uma peça deve saltar sobre outra ocupando a casa vazia imediatamente após, e a peça sobre a qual a primeira saltou é removida do tabuleiro. Veja abaixo a posição inicial e um possível primeiro movimento (a peça na casa [2,4] salta sobre a peça em [3,4] (que é removida) pousando em [4,4]).



- Descreva o mais detalhadamente que conseguir como seria possível resolver o problema utilizando programação dinâmica. Caso não seja possível resolver utilizando programação dinâmica, que técnica poderia ser utilizada?

5. (2 pontos) Cite 3 técnicas das vistas em aula que podem ser usadas para resolver o problema do Caixeiro Viajante, descrevendo como cada uma delas pode ser usada.
6. (1 ponto) O problema da mochila faz parte dos problemas NP-Completo, logo não existe solução polinomial para ele. Mas em aula foram vistas soluções polinomiais para ele. Como se explica isso?
7. (1 ponto) Cite 4 problemas que podem ser resolvidos utilizando a abordagem de programação dinâmica.

1. (1 ponto) Considere o problema de, dada uma matriz $M[100,100]$ preenchida com 0's e 1's, verificar se existe um caminho de 0's da posição $[li, ci]$ até a casa $[lf, cf]$. Abaixo é mostrada uma solução que utiliza backtracking.

```
int tem(int li, int ci, int lf, int cf, int M[100][100])
{
    if (li==lf && ci==cf && M[li][ci]==0) return 1;
    if (li<99)
        if (tem(li+1,ci, lf, cf, M)==1) return 1;
    if (li>0)
        if (tem(li-1,ci, lf, cf, M)==1) return 1;
    if (ci<99)
        if (tem(li,ci+1,lf, cf, M)==1) return 1;
    if (ci>0)
        if (tem(li, ci-1, lf, cf, M)==1) return 1;
    return 0;
}
```

O código acima está completo? Se não, o que está faltando?

- 2. (1 ponto) O método guloso pode ser utilizado para resolver o problema da mochila ou alguma variante do mesmo? Se usado, ele encontra a solução ótima? Justifique sua resposta, se necessário usando um exemplo numérico.
- 5. (1 ponto) Quais são os elementos essenciais para se conseguir um bom desempenho de uma solução ao utilizar a abordagem de Branch and Bound?

- 8. (1 ponto) Qual a semelhança e qual a principal diferença entre a abordagem de divisão e conquista e a abordagem de programação dinâmica na solução de problemas?
- 2. (1 ponto) Na literatura encontra-se às vezes referências ao problema do caixeiro viajante como um problema NP-Completo, às vezes como NP-Difícil. Afinal, ele é NP-Completo, NP-Difícil ou o que? Explique o porquê dessa divergência.

- 4. (1 ponto) Considere uma expressão envolvendo valores numéricos e operadores de soma e multiplicação (ex: $3+2*5+2*4$), em uma linguagem exótica não determinística em que os operadores de soma e multiplicação não possuem precedência ou associatividade. Deseja-se saber qual o maior valor possível de se obter a partir da expressão. Ex: $((3+2)*5)+(2*4)=33$ $(3+(2*5))+(2*4)=38$. Descreva como esse problema poderia ser resolvido utilizando programação dinâmica. Descreva o que seriam os subproblemas e descreva, o mais detalhadamente que puder, uma estrutura de dados para armazená-los.

- 5. (1 ponto) Uma forma de implementar programas para jogar (damas, xadrez, go, jogo da velha) é através de um backtracking em que se simula as jogadas dos dois adversários até um certo número de lances e verifica-se, através de uma função de avaliação, quem atingiu a melhor situação. Nessa abordagem pode ocorrer (e ocorre), de duas sequencias de movimentos diferentes conduzirem a uma mesma posição (e consequentemente a subárvores iguais). Isso é chamado de Transposição de Posição. Discuta o efeito que isso tem no desempenho e como a programação dinâmica pode ser utilizada para melhorar isso.

3. (1 ponto) O cientista John Mc Esperto dedicou anos de sua vida estudando problemas NP e conseguiu apenas mostrar (hipoteticamente falando) que não é possível encontrar um algoritmo polinomial para o problema do caixeiro viajante. Baseado no resultado por ele alcançado, pode-se concluir algo sobre as relações entre P, NP, NP-completo e NP-difíceis? Ou a vida dele foi um fracasso total?

4. (1 ponto) Considere uma expressão envolvendo valores numéricos e operadores de soma e multiplicação (ex: $3+2*5+2*4$), em uma linguagem exótica não determinística em que os operadores de soma e multiplicação não possuem precedência ou associatividade. Deseja-se saber qual o maior valor possível de se obter a partir da expressão. Ex: $((3+2)*5)+(2*4)=33$
 $(3+(2*5))+(2*4)=38$. Descreva como esse problema poderia ser resolvido utilizando programação dinâmica. Descreva o que seriam os subproblemas e descreva, o mais detalhadamente que puder, uma estrutura de dados para armazená-los.

1) (2.0 pontos) Explique a tecnica de projeto de algoritmos denominada programacao dinamica. Cite dois exemplos de uso desta tecnica detalhando o funcionamento. Sem detalhar (1.0 pontos).

2) (2.0 pontos) Considerando um tabuleiro de 8 linhas por 8 colunas. Qual metodo de projeto de algoritmos poderia ser usado para determinar se um cavalo pode ser movimentado por todo este tabuleiro sem repetir nenhuma posicao e sem deixar de visitar posicoes. No tabuleiro de 8x8 entao, o cavalo devera passar uma unica vez em cada uma das 64 posicoes. A movimentacao do cavalo eh sempre em L, de forma que a nova posicao sera calculada "andando" duas posicoes em qualquer direcao e uma para o lado, ou uma posicao em qualquer direcao e duas para o lado. Alem de indicar o metodo, criar um esboco do algoritmo.

3) (2.0 pontos) Fazer um algoritmo que elimine de um vetor todos os numeros pares. Ao eliminar os numeros, deslocar os subsequentes de forma que os que restarem no vetor fiquem dispostos de forma contigua. Colocar 0 nas posicoes finais, conforme mostrado abaixo:

Vetor antes $\Rightarrow$	1	4	3	6	2	5	0	8	3	7	4
Vetor depois $\Rightarrow$	1	3	5	3	7	0	0	0	0	0	0

4) (2.0 pontos) Faça um algoritmo com complexidade máxima $\Theta(n)$ para calcular a soma dos n primeiros elementos da série:

$$1 + \frac{\text{fib}(2)}{2!} + \frac{\text{fib}(3)}{3!} + \frac{\text{fib}(4)}{4!} + \dots + \frac{\text{fib}(n)}{n!}$$

5) (2.0 pontos) Diga quais das afirmacoes sao verdadeiras e quais sao falsas:

- a) ☐ Nem todos os problemas em P sao resolvidos em tempo polinomial.
- b) ☐ A classe NP tambem eh conhecida como Classe Nao Polinomial.
- c) ☐ A Classe NP representa os problemas que possuem solucao nao deterministica em tempo polinomial.
- d) ☐ Todos os problemas da classe NP-Dificil pertencem a classe NP.
- e) ☐ Todos os problemas da classe NP-Completo pertencem a classe NP.
- f) ☐ Se for solucionado um problema da classe NP-Completo de forma deterministica em tempo polinomial podemos afirmar que $P = NP$.
- g) ☐ Se for solucionado um problema da classe NP-Dificil de forma deterministica em tempo polinomial podemos afirmar que $P = NP$.
- h) ☐ Se for solucionado um problema da classe NP de forma deterministica em tempo polinomial, todos os outros NP tambem poderao.
- i) ☐ A classe NP-Dificil eh uma extensao da Classe NP-Completo, ou seja, a classe NP-Dificil engloba todos os problemas da classe NP-Completo mais outros problemas nao pertencentes a classe NP, mas que satisfazem a propriedade da redutibilidade de qualquer problema em NP para problemas NP-Dificil.
- j) ☐ Os problemas da classe P nao pertencem a classe NP.

Recuperação da segunda área, 11/12/13

1) (1.0 ponto) Explique a técnica de projeto de algoritmos denominada branch and bound, destacando as diferenças em relação ao backtracking.

2) (1.0 pontos) 1. (1 ponto) Considere o problema de, dada uma matriz $M[100,100]$ preenchida com 0's 1's, verificar se existe um caminho da posição $[li, ci]$ até a casa $[lf, cf]$. Abaixo é mostrada uma solução que utiliza backtracking.

```
int tem(int li , int ci , int lf , int cf , int M[100][100])
{
    if (li==lf && ci==cf) return 1;
    if (li < 99)
        if (tem(li+1,ci , lf , cf , M)==1) return 1;
    if (li > 0)
        if (tem(li-1,ci , lf , cf , M)==1) return 1;
    if (ci < 99)
        if (tem(li , ci+1,lf , cf , M)==1) return 1;
    if (ci > 0)
        if (tem(li-1, ci , lf , cf , M)==1) return 1;
    return 0;
}
```

O código acima contém os principais elementos em uma solução envolvendo backtracking? Se não, o que está faltando?

- 3) (1 ponto) O que deveria ser alterado na solução acima para que contasse o número de soluções encontradas?
- 3. (2 pontos) Diferencie o método branch-and-bound e o backtracking. Descreva como o problema da mochila pode ser resolvido usando a abordagem de branch-and-bound.

4) (2.0 pontos) Diga quais das afirmacoes sao verdadeiras e quais sao falsas:

- a) ☐ Nem todos os problemas em P sao resolvidos em tempo polinomial.
- b) ☐ A classe NP tambem eh conhecida como Classe Nao Polinomial.
- c) ☐ A Classe NP representa os problemas que possuem solucao nao deterministica em tempo polinomial.
- d) ☐ Todos os problemas da classe NP-Dificil pertencem a classe NP.
- e) ☐ Todos os problemas da classe NP-Completo pertencem a classe NP.
- f) ☐ Se for solucionado um problema da classe NP-Completo de forma deterministica em tempo polinomial podemos afirmar que $P = NP$.
- g) ☐ Se for solucionado um problema da classe NP-Dificil de forma deterministica em tempo polinomial podemos afirmar que $P = NP$.
- h) ☐ Se for solucionado um problema da classe NP de forma deterministica em tempo polinomial, todos os outros NP tambem poderao.
- i) ☐ A classe NP-Dificil eh uma extensao da Classe NP-Completo, ou seja, a classe NP-Dificil engloba todos os problemas da classe NP-Completo mais outros problemas nao pertencentes a classe NP, mas que satisfazem a propriedade da redutibilidade de qualquer problema em NP para problemas NP-Dificil.
- j) ☐ Os problemas da classe P nao pertencem a classe NP.

Recuperação da Segunda Área - parte 1, 09/12/2013

1) Considere que você tem um vetor $V[10000]$ de valores inteiros, e precisa efetuar uma quantidade grande de vezes o cálculo do somatório de intervalos do vetor. Que técnica se poderia utilizar para que essas consultas fossem todas efetuadas com tempo constante, e que o tempo de pré-processamento, antes de iniciar as consultas, fosse o menor possível? Descreva sua solução.

2) Considere o problema de preencher um vetor $P[0..N-1]$ com valores de 1 a N , de modo que cada valor entre 1 e N ocorra exatamente 1 vez, e que a soma de dois números vizinhos seja sempre primo. Faça uma função utilizando um backtracking que receba N (a quantidade de valores) e outros parâmetros que sejam necessários, e escreva, todos os vetores que atendam a essa condição. Ex: Se $N=6$, uma solução seria 1 4 3 2 5 6.

- 1) (1 ponto) Considere o enunciado resumido a seguir do problema da ACM 11450 - Compras de casamento: Dados 6 diferentes modelos de cada peça de vestuário (p.ex. 6 camisas, 6 cintos, 6 sapatos) e o valor de cada modelo, compre um modelo de cada peça de modo que o valor total seja o maior possível, contanto que não ultrapasse o orçamento fornecido. A entrada consiste de dois inteiros $1 < M \leq 200$ e $1 < C < 20$, onde M é o valor máximo a ser gasto e C é o número de peças que deve ser adquirido. A seguir, há a informação do preço de cada um dos 10 modelos das C peças de vestuário.
- Uma possível entrada seria:
 - 200,3 // o orçamento total é 200 unidades monetárias e há 3 peças de vestuário diferentes
 - 6 4 8 7 9 10 // preços dos modelos da peça 1
 - 5 10 7 8 9 7 // preços dos modelos da peça 2
 - 5 3 5 8 6 10 // preços dos modelos da peça 3
- Descreva como esse problema pode ser resolvido utilizando backtracking ou programação dinâmica (no caso de resolver com programação dinâmica essa questão vale 1 ponto a mais de bônus).

- 2) (1 ponto) Considere o problema de identificar a similaridade entre duas sequencias de DNA (uma sequencia de DNA é uma sequencia de letras). Das técnicas e problemas vistos em aula, qual melhor se aplicaria a essa situação?
- 3) (1 ponto) Como se pode demonstrar que um dado problema é NP-Completo?
- 4) (1 ponto) O problema da mochila faz parte dos problemas NP-Completo, logo não existe solução polinomial para ele. Mas em aula foram vistas soluções polinomiais para ele. Como se explica isso?
- 5) (1 ponto) Cite 4 problemas que podem ser resolvidos utilizando a abordagem de programação dinâmica.

Prova 2 - 03/12/14

- 1. (2 pontos) Escreva uma função usando backtracking que receba uma matriz 9x9 com uma configuração de sudoku e escreva todas as soluções para a mesma.

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

- 4. (2 pontos) No puzzle 8 deve-se, a partir de uma posição inicial dos números de 1 a 8 em uma matriz 3x3 chegar-se a uma posição final, movimentando os números através da matriz, trocando-se sempre algum número com a casa vazia vizinha a ele. Dos métodos vistos em aula qual seria uma abordagem adequada para resolvê-lo? Descreva o mais detalhadamente que puder sua solução, tomando cuidado para que a solução não entre em loop, alternando repetidamente entre duas posições.

7	2	4
5		6
8	3	1

	1	2
3	4	5
6	7	8

1)(2 pontos) A complexidade do algoritmo de Strassen para cálculo de produto matricial de duas matrizes $N \times N$ é descrita pela recorrência a seguir:

$$T(n) = \begin{cases} 1 & p/n \leq 1 \\ 7T(n/2) + n^2 & p/n > 1 \end{cases}$$

Resolva a recorrência e verifique a correção de sua solução utilizando indução matemática.

Solução da 1

$$T(1) = 1$$

$$T(2) = 7 T(1) + 2^2 = 7 + 4$$

$$T(4) = 7 T(2) + 4^2 = 7^2 + 7 \cdot 4 + 16$$

$$T(8) = 7 T(4) + 8^2 = 7^3 + 7^2 \cdot 4 + 7 \cdot 16 + 64$$

■ PG:

$$q = \frac{7}{4} \quad a_n = n^2 \quad a_1 = 7^{\log_2 n}$$

$$S_{PG} = \frac{q \cdot a_n - a_1}{q - 1} = \frac{\frac{7}{4} 7^{\log_2 n} - n^2}{\frac{7}{4} - 1}$$

Multiplicando por 4 o numerador e o denominador:

$$S_{PG} = \frac{q \cdot a_n - a_1}{q - 1} = \frac{7 \cdot 7^{\log_2 n} - 4 \cdot n^2}{7 - 4} \Rightarrow T(n) = \frac{7 \cdot 7^{\log_2 n} - 4 \cdot n^2}{3}$$

Prova por indução da 1:

- Base de indução: $T(1)=1$ pela recorrência e pela equação fechada.
- Hipótese de indução: $T(n) = \frac{7 \cdot 7^{\log_2 n} - 4 \cdot n^2}{3}$
- Pela recorrência:

$$\begin{aligned} T(2n) &= 7T\left(\frac{2n}{2}\right) + (2n)^2 = 7\left(\frac{7 \cdot 7^{\log_2 n} - 4 \cdot n^2}{3}\right) + 4 \cdot n^2 = \\ &= 7\left(\frac{7 \cdot 7^{\log_2 n} - 4 \cdot n^2}{3}\right) + 4 \cdot n^2 = \frac{7 \cdot 7 \cdot 7^{\log_2 n} - 7 \cdot 4 \cdot n^2}{3} + \frac{3 \cdot 4 \cdot n^2}{3} = \\ &= \frac{7 \cdot 7 \cdot 7^{\log_2 n} - 28n^2 + 12n^2}{3} = \frac{7^{2+\log_2 n} - 16n^2}{3} \end{aligned}$$

- Pela equação fechada:

$$\begin{aligned} T(2n) &= \frac{7 \cdot 7^{\log_2 2n} - 4 \cdot (2n)^2}{3} = \frac{7 \cdot 7^{\log_2 2 + \log_2 n} - 4 \cdot 4 \cdot n^2}{3} = \\ &= \frac{7 \cdot 7^{1 + \log_2 n} - 16 \cdot n^2}{3} = \frac{7^{2 + \log_2 n} - 16 \cdot n^2}{3} \end{aligned}$$

2)(2 pontos) Para a função recursiva abaixo, determine a equação de recorrência que representa a complexidade da mesma e a equação fechada correspondente. Verifique sua solução por indução matemática.

```
int p(int n)
{
    if (n>1)
    {
        p(n/2);
        for (i=0;i<=N;i++)
            for (j=i;j<N;j++){
                printf("Oi" ); ← considere op. fund.
                printf("Tchau" ); ← essa também
            }
    }
}
```

Solução da 2

- Custo da iteração do "for" mais interno: 2 (constante)
- Custo de todas as iterações do for mais interno:

- Número de iterações * Custo da iteração
- (última iteração - primeira iteração + 1)*2
- $(N-1 - i + 1).2 = (N-i).2$

- Custo de todas as iterações do for externo:

$$\begin{aligned}\sum_{i=0}^N 2(N-i) &= 2 \cdot \sum_{i=0}^N N - 2 \cdot \sum_{i=0}^N i = \\ 2N(N+1) - 2 \sum_{i=0}^N i &= 2N^2 + 2N - 2 \cdot \frac{0+N}{2}(N+1) = \\ 2N^2 + 2N - 2 \cdot \frac{0+N}{2}(N+1) &= 2N^2 + 2N - N^2 - N = N^2 - N\end{aligned}$$

Assim a recorrência fica:

$$T(n) = \begin{cases} 0 & p/n \leq 1 \\ T(n/2) + n^2 - n & p/n > 1 \end{cases}$$

3)(2 pontos) A função abaixo calcula recebe a raiz de uma árvore e retorna a altura da mesma. Determine a equação de recorrência que representa a complexidade da mesma somente em função de N (o parâmetro "raiz" não deve ser considerado para análise da recorrência) e a equação fechada correspondente. Verifique sua solução por indução matemática. Considere como operações fundamentais todas as chamadas recursivas.

```
int altura(struct nodo *raiz , int N)
{
if (N==0) return -1;
if ( altura(raiz->esq ,N-1)>altura(raiz->dir ,N-1))
return altura(raiz->esq ,N-1)+1;
else return altura(raiz->dir ,N-1)+1;
}
```

Solução da 3

Como são feitas duas chamadas recursivas na avaliação da condição e uma no then ou no else, o total de chamadas recursivas é 3 e a recorrência fica:

$$T(n) = \begin{cases} 0 & p/n \leq 1 \\ 3T(n-1) + 3 & p/n > 1 \end{cases}$$

Solução da 3

$$T(0) = 0$$

$$T(1) = 3 T(0) + 3 = 3$$

$$T(2) = 3 T(1) + 3 = 9 + 3$$

$$T(3) = 3 T(2) + 3 = 27 + 9 + 3$$

■ PG:

$$q=3 \quad a_n=3^n \quad a_1=3$$
$$S_{PG} = \frac{q \cdot a_n - a_1}{q-1} = \frac{3 \cdot 3^n - 3}{2}$$

4)(2 pontos) Um programa com complexidade n^3 processa uma entrada de tamanho 1000 em 10 minutos em um computador X. Usando um computador Y, quatro vezes mais rápido, sem alterar o programa, pergunta-se:

a) Em quanto tempo a mesma entrada (1000) é processada?

2.5 minutos

b) Qual o máximo tamanho da entrada que este programa processa em 20 minutos no computador Y? 2000

5)(2 pontos) Calcule a complexidade do trecho de código a seguir:

```
for (i=n-1;i>0;i- -)
    for (j=i-1;j>=0;j- -)
        if (m[i,1]>m[j,1])
            for (k=n-1;k>=0;k- -)
                {
                    aux=m[i,k];
                    m[i,k]=m[j,k];
                    m[j,k]=aux;
                }
```

Resolução da 5

- "for" interno: $(n-1 - 0 + 1) = N$
- "for" do meio: $(i-1 - 0 + 1) \cdot N = Ni$
- "for" externo:

$$\sum_{i=1}^{N-1} Ni = N \cdot \sum_{i=1}^{N-1} i = N \cdot \frac{N-1+1}{2} (N-1) = \frac{N^2(N-1)}{2}$$

1. (2 pontos) Suponha que você emitiu cheques em maio nos valores de $p(1)$, ..., $p(n)$ ao longo do mês de junho. No fim do mês, o banco informa que um total T foi descontado de sua conta. Quais cheques foram descontados? Por exemplo, se $p = \{61, 62, 63, 64\}$ e $T = 125$ então só há duas possibilidades: ou foram descontados os cheques 1 e 4 ou foram descontados os cheques 2 e 3. Esse é o "problema da soma de subconjuntos". Desenvolva uma função utilizando backtracking que receba um vetor com os valores de 10 cheques e o valor total, e escreva uma combinação de cheques que resulte nesse valor.

2. (2 pontos) Seria possível aplicar programação dinâmica ao problema anterior? Esse problema poderia ser associado a algum outro problema estudado em aula? Justifique e detalhe sua resposta.

3. (2 pontos) Diga quais das afirmações são verdadeiras e quais são falsas:

- a) ☐ Nem todos os problemas em P são resolvidos em tempo polinomial.
- b) ☐ A Classe NP representa os problemas que possuem solução não determinística em tempo polinomial.
- c) ☐ Todos os problemas da classe NP-Difícil pertencem também a NP.
- d) ☐ Todos os problemas da classe NP são os problemas que não possuem solução em tempo polinomial.
- e) ☐ Não se conhece solução polinomial para um dado problema, mas pode-se reduzi-lo em tempo polinomial a um problema NP-difícil. Baseado nisso, pode-se afirmar que ele é NP.

4. (2 pontos) O problema de encontrar, em um conjunto de pontos definidos por coordenadas (x,y) pode ser resolvido da seguinte forma. Divide-se o espaço de pontos em duas metades (ordenando-os, por exemplo, pela coordenada y), e encontra-se, recursivamente, em cada uma das duas metades as menores distâncias em cada metade $d1$ e $d2$. Encontra-se também a menor distância $d3$ entre um par de pontos em que cada ponto está em uma das metades. A menor distância de todo o conjunto de pontos é o menor dos 3 valores $d1$, $d2$ e $d3$. Essa descrição se enquadra em qual das estratégias vistas em aula? Identifique, nesse algoritmo, cada um dos elementos que o identifica como usando a estratégia em questão.

5. (2 pontos) "Apagando e Ganhando" é um programa de auditório em que o apresentador escreve em uma lousa um número de N dígitos. O participante deve então apagar D dígitos do número que está na lousa. Os números que restarem equivalem ao prêmio em dinheiro que ele leva para casa!

- Exemplo: Número na lousa: 1 2 3 1 2 3 9
- Dígitos a serem apagados: 3
- Total a ser levado para casa: **1 2 3 1 2 3 9** = 3239!

Descreva uma abordagem para identificar os D dígitos que devem ser removidos de modo a maximizar o valor resultante. Sua abordagem pode ser classificada dentro de alguma das estratégias vistas em aula?

1)(2 pontos) Resolva as seguintes equações de recorrência, supondo $T(1)=1$:

1 $T(n) = 3T(n/2) + 1$

2 $T(n) = 10T(n/4) + n$

2)(2 pontos) A função abaixo escreve todos os movimentos para resolver o problema da Torre de Hanoi. Extraia sua equação de complexidade em função de N e resolva-a.

```
void Hanoi(int N, char DE, char PARA, char INTERMEDIARIO)
{
    if (N==1) printf(" Mover_de_%c_para_%c\n" ,DE,PARA);
    else
    {
        Hanoi(N-1,DE,INTERMEDIARIO,PARA);
        printf(" Mover_de_%c_para_%c\n" ,DE,PARA);
        Hanoi(N-1,INTERMEDIARIO,PARA,DE);
    }
}
```

3)(2 pontos) Para esse também:

```
void p1(int n, int a)
{
    if (n>1)
    {
        for (i=1;i<=4;i++) p1(n/4);
        for (i=1;i<=n*n;i++)
            printf("\%d\n",4*i); /* considere operacao fundamental */
    }
}
```

4)(2 pontos) Um programa com complexidade n^3 calcula o custo de 200 produtos em 2 horas utilizando o computador atual. Se a quantidade de produtos dobrar quanto tempo será necessário para executar o cálculo, utilizando-se uma máquina 40% mais rápida?

- Faça uma função que receba um vetor `int v[20]` contendo uma linha de um nonogram em que as casas que garantidamente são pretas contém 1, as que garantidamente são brancas contem 2 e as que ainda estão indefinidas contem 0. A função deve receber também um vetor `P[20]` com os grupos de pontos a serem posicionados na linha (a lista de grupos termina com 0) e deve gerar todas as possíveis distribuições dos grupos na linha, identificando as casas que são pretas em todas as possíveis soluções e deve marcar essas casas com 1. A função deve identificar também as casas que são brancas em todas as soluções e marcar essas casas com 2.

Soma dos quadrados de 1 a n

- A expressão do termo geral de qualquer série de números geradas por um polinômio pode ser calculada usando diferenças finitas.
- Inicia-se listando os termos da série e a cada nível calcula-se as diferenças entre cada termo e o anterior até que as diferenças sejam constantes.
- O número de níveis até chegar a diferenças constantes é o grau do polinômio gerador

- A terceira linha é a soma dos quadrados de 1 a n .
- Como a primeira linha em que as diferenças são constantes é a dif_3 , o polinômio gerador da série da soma dos quadrados é de grau 3, e pode ser expresso no formato $an^3 + bn^2 + cn + d$

n	1	2	3	4	5	6
n^2	1	4	9	16	25	36
$\sum n^2$	1	5	14	30	55	91
dif_1		4	9	16	25	36
dif_2			5	7	9	11
dif_3				2	2	2

- Dos valores conhecidos do somatório e substituindo na expressão geral do polinômio gerador:

$$\begin{array}{ll}
 \text{Soma}(1)=1 & a+b+c+d=1 \\
 \text{Soma}(2)=5 & 8a+4b+2c+d=5 \\
 \text{Soma}(3)=14 & 27a+9b+3c+d=14 \\
 \text{Soma}(4)=30 & 64a+16b+4c+d=30
 \end{array}$$

- Resolvendo o sistema
(<http://www.calculadoraonline.com.br/sistemas-lineares>)
obtem-se os coeficientes

- $a=\frac{1}{3}$ $b=\frac{1}{2}$ $c=\frac{1}{6}$ $d=0$

- e a solução

- $$\sum_{i=1}^n i^2 = \frac{n^3}{3} + \frac{n^2}{2} + \frac{n}{6} = \frac{2n^3+3n^2+n}{6} = \frac{n(2n^2+3n+1)}{6} =$$

E a soma dos cubos?

- Usando-se o mesmo método pode-se resolver $\sum_{i=1}^n i^3$

n	1	2	3	4	5	6	7
n^3	1	8	27	64	125	216	343
$\sum_{i=1}^n i^3$	1	9	36	100	225	441	784
dif_1		8	27	64	125	216	343
dif_2			19	37	61	91	127
dif_3				18	24	30	36
dif_4					6	6	6

- Como a primeira diferença constante é a quarta diferença,
 $\sum_{i=1}^n i^3 = an^4 + bn^3 + cn^2 + dn + e$

- Dos valores conhecidos do somatório e substituindo na expressão geral do polinômio gerador:

$$\text{Soma}(1)=1 \qquad a+b+c+d+e=1$$

$$\text{Soma}(2)=9 \qquad 16a+8b+4c+2d+e=9$$

$$\text{Soma}(3)=36 \qquad 81a+27b+9c+3d+e=36$$

$$\text{Soma}(4)=100 \qquad 256a+64b+16c+4d+e=100$$

$$\text{Soma}(5)=225 \qquad 625a+125b+25c+5d+e=225$$

- Resolvendo o sistema
(<http://www.calculadoraonline.com.br/sistemas-lineares>)
obtem-se os coeficientes

- $a=\frac{1}{4} \quad b=\frac{1}{2} \quad c=\frac{1}{4} \quad d=0 \quad e=0$

- e a solução

- $$\sum_{i=1}^n i^3 = \frac{n^4}{4} + \frac{n^3}{2} + \frac{n^2}{4} = \frac{n^4+2n^3+n^2}{4} = \frac{n^2(n^2+2n+1)}{4} =$$

1) Um algoritmo leva 0.5 ms para uma entrada de tamanho 100. Quanto tempo ele levará para terminar uma entrada de tamanho 500 se sua complexidade em tempo é:

- 1 linear
- 2 $O(n \log n)$
- 3 quadrático
- 4 cúbico

[Voltar para o índice](#)

■ a) linear:

$$100. R = 500 \Rightarrow R=5 \Rightarrow \text{tempo} = 0.5 \text{ ms} * R = 2.5 \text{ ms}$$

■ b) $n \log n$:

$$100 \log 100. R = 500 \log 500 \Rightarrow R=5*\log(500)/\log(100)=5*1.35 = 6.74 \Rightarrow \text{tempo} = 0.5 \text{ ms} * 6.74 = 3.37 \text{ ms}$$

■ c) quadrático:

$$100^2 R = 500^2 \Rightarrow R=25 \Rightarrow \text{tempo} = 0.5 \text{ ms} * R = 12.5 \text{ ms}$$

■ c) cúbico:

$$100^3 R = 500^3 \Rightarrow R=125 \Rightarrow \text{tempo} = 0.5 \text{ ms} * R = 62.5 \text{ ms}$$

2) Um programa de complexidade n^3 precisa de 4096 segundos para executar em um computador X com um tamanho de entrada 1600.

- 1 Sem alterar o programa e sem trocar o computador, o que acontece com o tempo de execução se o tamanho da entrada for 50% maior?

$$1600^3 * R = (1600 * 1.5)^3 \Rightarrow R = 3.375 \Rightarrow$$

$$\text{tempo} = 4096 * 3.375 = 13.824$$

- 2 Sem alterar o programa, qual o tamanho máximo da entrada que pode ser executado em 4096 segundos considerando um computador Y 8 vezes mais rápido?

$$1600^3 * 8 = X_2^3 \Rightarrow X_2 = 3200$$

2) Um programa de complexidade n^3 precisa de 4096 segundos para executar em um computador X com um tamanho de entrada 1600.

- 3 Considerando o computador X , que com o programa n^3 precisa de 4096 s para a entrada de tamanho 1600, qual o tamanho máximo da entrada se um programador conseguir reduzir a complexidade do programa para n^2 ?

$$1600^3 = X^2 \Rightarrow X = 64000$$

[Voltar para o índice](#)

3) Um programa com complexidade n^4 processa uma entrada de tamanho 1000 em 625 segundos em um computador X . Usando um computador Y , 4 vezes mais rápido, sem alterar o programa, pergunta-se:

- 1 Em quanto tempo a mesma entrada (1000) é processada?

$$1000^4 * R = 1000^4 \Rightarrow R = 1 = R_v * R_t \Rightarrow$$

$$1/4 \text{ do tempo} = 156 \text{ segundos}$$

- 2 Qual o máximo de entrada que esse programa processa em 625 segundos?

$$1000^4 * 4 = X^4 \Rightarrow X = 1000 * \sqrt[4]{4} = 1414$$

4) Um programa com complexidade n^3 atende as necessidades de uma empresa que processa dados de 1000 funcionários em meia hora.

- 1 Se o número de funcionários reduzir pela metade, qual será o tempo de execução desse programa?

$$1000^3 * R = 500^3 \Rightarrow R = 1/8 \Rightarrow \text{tempo} = 30\text{minutos}/8 = 3m45s$$

- 2 Se o computador for trocado por um outro 4 vezes mais rápido, qual o número máximo de funcionários que este programa processa em 8 horas?

$$1000^3 * 4 * 16 = X^3 \Rightarrow X = 4000$$

5) Um programa com complexidade n^2 processa uma entrada de tamanho 1200 em 1 hora em um computador X.

- 1 Em quanto tempo este programa processa uma entrada de tamanho 20?

$$1200^2 * R = 20^2 \Rightarrow R = 1/3600 \Rightarrow$$

$$\text{tempo} = 3600\text{seg} / 3600 = 1 \text{ segundo}$$

- 2 Usando um computador duas vezes mais rápido, qual a maior entrada que esse programa consegue processar em 2 horas?

$$1200^2 * 4 = X^2 \Rightarrow X = 2400$$

6) Um programa com complexidade n^3 calcula o custo de 800 produtos em 2 horas utilizando o computador atual. Sabe-se que a quantidade de produtos será duplicada e que o limite de tempo para executar esse programa é de 4 horas. Diga se o computador atual atende à necessidade de processamento e indique qual a necessidade mínima de desempenho de um novo se for o caso.

$$800^3 * R = 1600^2 \Rightarrow R = 8$$

$$R_v * 2 = 8 \Rightarrow R_v = 4$$

O novo computador deve ser 4 vezes mais rápido.

Calcule a complexidade do trecho de código a seguir:

```
for ( i=0; i<n; i++)  
  for ( j=i+1; j<n; j++)  
    if (m[i ,1]>m[j ,1])  
      for ( k=0; k<n; k++)  
        {  
          aux=m[i , k];  
          m[i , k]=m[j , k];  
          m[j , k]=aux;  
        }
```

- Seleccionando a troca de elementos como operação fundamental (custo unitário, $\Theta(1)$)
- O custo do for interno (for k) é o número de iterações (n) vezes o custo da iteração (1) resultando **n**
- O custo do if é o custo da condição ($O(1)$) mais o custo do **then** (n). Para simplificar, podemos desprezar o custo da condição. Então o if é $\Theta(n)$
- O custo de cada iteração do for intermediário (for j) é n. Como não é função de j, o custo total é o número de iterações $(n-1 - (i+1) + 1 = n-i-1)$ vezes $n = n^2 - ni - n$.

```

for (i=0;i<n;i++)
  for (j=i+1;j<n;j++)
    if (m[i,1]>m[j,1])
      for (k=0;k<n;k++)
        {
          aux=m[i,k];
          m[i,k]=m[j,k];
          m[j,k]=m[i,k];
        }

```

$N * ((N-1) - (i+1) + 1) = N(N-i-1) = N^2 - Ni - N$

1 N N

- Cada iteração do for externo (**for i**) é $n^2 - ni - n$. Como é função de **i** o custo total do **for i** deve ser expresso e resolvido como um somatório.

$$\sum_{i=0}^{n-1} n^2 - ni - n$$

- O somatório pode ser separado em dois somatórios, os dos termos que são função de **i** e os que não são.

$$\sum_{i=0}^{n-1} (n^2 - n) - \sum_{i=0}^{n-1} ni$$

- A expressão do primeiro somatório não é função de **i**, então é tratado como um somatório de constantes (número de termos vezes a expressão)

$$n^3 - n^2 - n \sum_{i=0}^{n-1} i$$

$$n^3 - n^2 - n \sum_{i=0}^{n-1} i$$

- Pela soma da P.A.

$$\sum_{i=0}^{n-1} i = \frac{(0+n-1)}{2} * n$$

- e o custo total do for i é

$$n^3 - n^2 - n \frac{n-1}{2} n$$

- Colocando 2 como denominador comum:

$$\frac{2n^3 - 2n^2 - n^2(n-1)}{2} = \frac{2n^3 - 2n^2 - n^3 + n^2}{2} = \frac{n^3 - n^2}{2}$$

Só para verificar...



sum sum sum 1,k=0 to N-1,j=i+1 to N-1,i=0 to N-1



Extended Keyboard



Upload

Sum:

$$\sum_{i=0}^{N-1} \left(\sum_{j=i+1}^{N-1} \left(\sum_{k=0}^{N-1} 1 \right) \right) = \frac{1}{2} (N-1) N^2$$

Calcule a complexidade do trecho de código a seguir, que triangulariza uma matriz quadrada:

```
for ( i=0; i<n; i++)  
    {  
        for ( k=n-1; k>=i; k--)  
            m[i][k]/=m[i][i];    <- op fund  
        for ( k=i+1; k<n; k++)  
            for ( j=n-1; j>=i; j--)  
                m[k][j]=m[k][j]/  
                m[k][i]-m[i][j]; <- op fund.  
    }
```

- Dentro do **for i** externo há 2 **for k** em sequência (não aninhados). Cada um deles é calculado separadamente e os dois são somados.
- O custo do **for** mais interno (**for j**) é o custo da iteração (1) vezes o número de iterações $((n-1)-i+1 = n-i)$.
- O custo de cada iteração do segundo for k é n-i. Como não é função de k, o custo total das iterações do segundo for k é o custo da iteração (n-i) vezes o número de iterações $((n-1)-(i+1)+1 = n-i-1)$ que é $(n-i)(n-i+1)$
- O custo do primeiro for k é o custo de cada iteração (1) vezes o número de iterações $((n-1)-i+1=n-i)$ que é n-i

```

for (i=0; i<n; i++)
{
    for (k=n-1; k>=i; k--) (n-i)
        m[i][k] /= m[i][i];      ← op fund
    for (k=i+1; k<n; k++) (n-i-1)(n-i)
        for (j=n-1; j>=i; j--)
            m[k][j] = m[k][j] /
            m[k][i] - m[i][j]; ← op fund. 1 n-i
}

```

- Como os dois for k estão em sequência, eles são somados, resultando

$$(n-i) + (n-i-1)(n-i)$$

- Como $(n-i)$ aparece nos dois termos da soma ele pode ser "colocado em evidência" (propriedade distributiva da multiplicação em relação à soma) resultando
- $(n-i)(1+n-i-1)=(n-i)(n-i)=n^2 - 2ni + i^2$
- Cada iteração do for i tem custo $n^2 - 2ni + i^2$. Como é função de i, deve ser expresso e resolvido como somatório:
- $\sum_{i=0}^{n-1} n^2 - 2ni + i^2$

$$\sum_{i=0}^{n-1} n^2 - 2ni + i^2$$

Esse somatório pode ser quebrado em 3 somatórios (um não é função de i, um é função de i e o outro de i^2)

$$\sum_{i=0}^{n-1} n^2 - 2n \sum_{i=0}^{n-1} i + \sum_{i=0}^{n-1} i^2$$

$$n^3 - 2n \frac{0+i-1}{2} n + \sum_{i=0}^{n-1} i^2$$



sum (n-i)^2,i=0 to n-1



Extended Keyboard



Upload

Sum:

$$\sum_{i=0}^{n-1} (n-i)^2 = \frac{1}{6} n (2n^2 + 3n + 1)$$

Calcule a complexidade do trecho de código a seguir. Considere como operação fundamental o comando de escrita:

```
void Moo (int N)
{
    for ( j=1 ; j<=N ; j=j*2 )
        for ( i=j ; i>1 ; i=i/2 )
            printf("%d\n", i);
}
```


- Como a variável do for interno não é incrementada ou decrementada de uma unidade não é possível representá-la como um somatório ou contar o número de iterações.
- Pode-se fazer a simulação da variável i para identificar o comportamento do laço:
- $i = j - j/2 - j/4 - j/8 \dots 2$
- Onde para cada valor de i é executada uma iteração de custo 1. A expansão de i resulta em uma P.G de $a_1=2$, $a_n=j$ e $q=2$. Aplicando o termo geral da P.G.:
- $a_n = a_1 * q^{n-1}$ e o número de termos da P.G. resulta no número de iterações.

- isolando o n (que é o número de termos da P.G. e consequentemente o número de iterações do for i) na expressão do termo geral da P.G. $a_n = a_1 * q^{n-1}$
- $a_n = \frac{a_1 * q^n}{q}$
- $\frac{q * a_n}{a_1} = q^n$
- substituindo $a_1=2$, $a_n=j$ e $q=2$:
- $\frac{2j}{2} = q^n$
- $j = q^n$
- e $n = \log_2 j$
- Assim, o custo total do **for i** é $\log_2 j$

- O tratamento do for externo (for ($j=1; j \leq N; j*=2$) é semelhante. Pode-se expandir o valor de j obtendo:
- $j=1 - 2 - 4 - 8 - 16 \dots N$
- Como a cada valor de j o custo da iteração é $\log_2 j$ os custos das sucessivas iterações serão:
- $\log(j) = 0 + 1 + 2 + 3 + 4 \dots + \log_2 N$
- Cujo somatório é $\frac{1+\log_2 N}{2} * \log_2 N$ que é o custo total do for i.

- Isso pode ser verificado pelo código a seguir, que executa a função Moo e escreve o número de prints:

```
int Moo (int N){
    int cont=0;
    for ( int j=1 ; j<=N ; j=j*2 )
        for ( int i=j ; i>1 ; i=i/2 )
            {cont++;/*printf("%d\n",i);*/}
    return cont;}
int main() {
    for (int i=2;i<=32;i*=2)
        printf("i=%d \t cont=%d\n" , i , Moo(i));
    return 0;
}
```

■ Que resulta em:

i=2 cont=1

i=4 cont=3

i=8 cont=6

i=16 cont=10

i=32 cont=15

- Outra alternativa é substituir no laço interno a variável i por 2^k transformando o laço

```
for ( i=j ; i>1 ; i=i/2 )  
    printf("%d\n",i);
```

- em

```
for (  $2^k=j$  ;  $2^k > 1$  ;  $2^k=2^k/2$  )  
    printf("%d\n",i);
```

- e fazendo as transformações necessárias:

$$2^k = j \Rightarrow k = \log_2 j \quad 2^k > 1 \Rightarrow k > 0$$

for ($k = \log_2 j$; $k > 0$; $k--$)
 $O(1)$

- Que resulta $\Theta(\log_2 j)$ para o for interno
for ($j=1$; $j \leq N$; $j=j*2$)
 $\Theta(\log_2 j)$

- Fazendo as mesmas transformações no for externo:
 $j=2^k$
for ($2^k = 1$; $2^k \leq N$; $2^k = 2^{k+1}$)
 $\Theta(\log_2 2^k)$

for ($2^k = 1$; $2^k \leq N$; $2^k = 2^{k+1}$)

$\Theta(\log_2 2^k)$

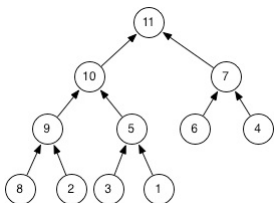
for ($k=0$; $k \leq \log_2 N$; $k++$)

$\Theta(k)$

- resultando, para o for externo:

$$\sum_{k=0}^{\log_2 N} k = \frac{0 + \log_2 N}{2} \log_2 N + 1 = \frac{\log_2 N (\log_2 N + 1)}{2}$$

- Um heap pode ser implementado sobre um vetor, onde cada elemento $v[i]$ é pai dos elementos nas posições $v[2*i+1]$ e $v[2*i+2]$ e a raiz da árvore ocupa a primeira posição ($v[0]$).
- Assim, dado o índice de um nodo no vetor, os índices do pai e dos filhos à esquerda e à direita podem ser obtidos por:
 - $\text{Pai}(i) : (i-1)/2$
 - $\text{Esquerda}(i) : i * 2 + 1$
 - $\text{Direita}(i) : i * 2 + 2$
- Assim, o heap abaixo poderia ser representado pelo vetor [11 10 7 9 5 6 4 8 2 3 1]



- As principais operações sobre heaps são:
 - cria-heap: cria um heap vazio
 - heapify: cria um heap a partir de um conjunto de elementos
 - busca-maior (em um heapmax) ou busca-menor(em um heapmin)
 - remove-maior: remove a raiz de um heapmax
 - altera-valor-chave
 - insere: adiciona elemento a um heap

Inserção em Heap

- Pode ser implementada adicionando o novo elemento no final do heap e depois "empurrando-o" para cima enquanto ele for maior que o pai (no caso de um Heap Max) até alcançar sua posição.
- Qual o custo?

```
void corrigeSubindo(int index)
{
    // Se index não é a raiz e o valor do index for maior do que o valor de seu pai,
    // troca seus valores (index e pai(index)) e corrige para o pai
    if (index > 0 && heap[index] > heap[pai(index)])
    {
        troca(heap[index], heap[pai(index)]);
        corrigeSubindo(pai(index));
    }
}
```

Defines

- No código anterior, uma forma de implementar o cálculo do índice do pai é através de uma função. Uma forma melhor ainda é através de um define.
- Normalmente defines são utilizados para melhorar a legibilidade e manutenção do código. Coisas como o código abaixo, para definir o tamanho de um vetor, de modo que, ao alterar o tamanho do vetor, não seja necessário inspecionar todo o código:
- `#define TAM 10`

- defines podem receber parâmetros, o que possibilita o seu uso no lugar de funções, com a mesma legibilidade de funções mas sem o custo de chamadas e retornos (o código do define é expandido no lugar do uso do define).
- Os parâmetros vão entre parênteses, como o define abaixo, que calcula o maior valor entre duas expressões
- `#define max(A,B) ((A)>(B)?(A):(B))`
- ou os defines abaixo, para o cálculo da posição do pai, filho esquerdo e filho direito no heap.
 - `#define pai(i) ((i-1)/2)`
 - `#define fesq(i) (i*2+1)`
 - `#define fdir(i) (i*2+2)`
- Pode-se passar expressões, mas o que resultaria no exemplo acima ao usar-se "`fesq(1+2)`"?

Remoção

- A remoção do maior valor da raiz (no caso do Heap Max) pode ser implementada removendo a raiz e substituindo-a pelo último elemento do Heap e "empurrando-o" para baixo na árvore até que o heap esteja corrigido.
- Qual o custo?

```

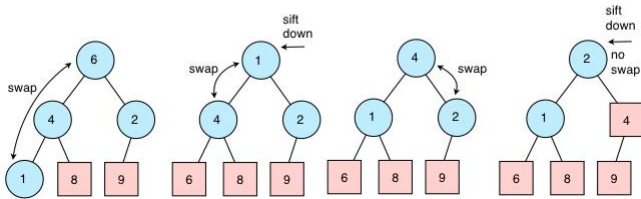
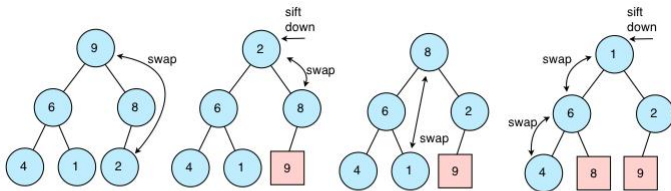
void corrigeDescendo(int index)
{
    if (filhoE(index) < heap.size())    // Se index tem filho
    {
        // Seleciona o filho com menor valor (esquerda ou direita?)
        int filho = filhoE(index);
        if (filhoD(index) < heap.size() && heap[filhoD(index)] > heap[filhoE(index)])
            filho = filhoD(index);
        // Se o valor do pai é maior do que o valor do maior filho , terminamos
        if (heap[index] > heap[filho])
            return;
        // Caso contrário , trocamos o pai com o filho no heap e corrigimos agora para o filho
        troca(heap[index], heap[filho]);
        corrigeDescendo(filho);
    }
}

```


- Heaps podem ser utilizados para implementar eficientemente filas de prioridades, já que o custo das principais operações (inserção, remoção) é $O(\log n)$ e o custo para obter o maior elemento é $O(1)$.
- Filas de prioridade são utilizadas em diversos algoritmos clássicos tais como:
 - Algoritmo de dijkstra para caminho mínimo
 - Algoritmos de Prim e Kruskal para árvore geradora mínima
 - Algoritmo de Branch-and-bound para buscar nodo de melhores possibilidades

- Heaps também são utilizados para implementar o algoritmo de ordenação Heapsort que basicamente é:
 - Coloque os dados a serem ordenados em um heap (custo $O(n \log n)$)
 - A cada iteração remova a raiz do heap (maior valor) e coloque no final do vetor, e reorganize o heap ($O(\log n)$)

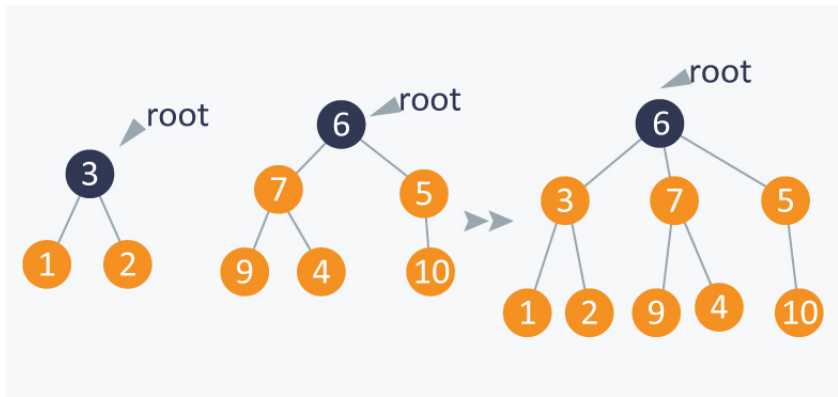
Heapsort



Disjoint-sets (Union-finds)

- Um disjoint-set (também conhecido como union-find ou união-busca) é uma estrutura de dados que mantém o controle de um conjunto de elementos particionados em subconjuntos disjuntos.
- Ele fornece de forma eficiente operações para adicionar novos conjuntos, para mesclar conjuntos existentes e para determinar se os elementos estão no mesmo conjunto.
- Além de muitos outros usos, os disjoint-sets desempenham um papel fundamental no algoritmo de Kruskal de busca de árvore geradora mínima.

- As principais operações sobre um disjoint-set são duas:
 - União: união de dois subconjuntos em um único;
 - Busca: determina em qual subconjunto um elemento em particular está. Esta operação também pode ser utilizada para determinar se dois elementos estão em um mesmo subconjunto.
- Um disjoint-set é representado como uma floresta, onde cada um dos subconjuntos forma uma árvore, na qual cada elemento aponta para seu pai, e a raiz da subárvore contém o elemento representativo do subconjunto.



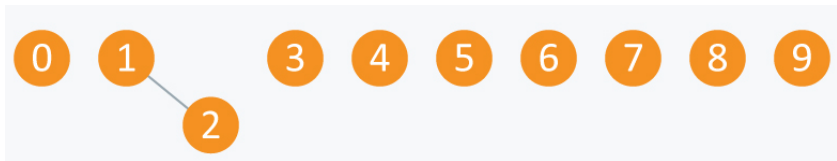
- Essa floresta pode ser representada por um vetor, onde o número da posição representa o vértice, e o conteúdo de cada posição contém o índice do pai. A posição do elemento que está na raiz da árvore contém seu próprio índice, permitindo identificar o elemento como raiz.
- Inicialmente cada elemento representa um subconjunto, sendo raiz da árvore de seu subconjunto, apontando para si mesmo
- Considere o conjunto de valores a seguir:



- Inicialmente cada elemento aponta para si mesmo e o conteúdo do vetor de "pais" é:

Arr	0	1	2	3	4	5	6	7	8	9
	0	1	2	3	4	5	6	7	8	9

- Suponha que se faça a união dos conjuntos 1 e 2. Isso pode ser implementado fazendo com que o pai do conjunto 2 seja o nodo 1.



- E o vetor resultante após essa operação será:

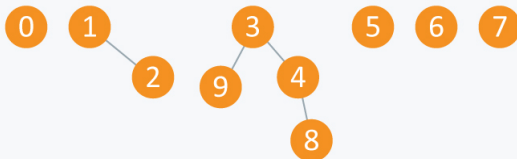
Arr	0	1	1	3	4	5	6	7	8	9
	0	1	2	3	4	5	6	7	8	9

- Supondo que se faça em seguida as operações união(4,3), união(8,4) e união(9,3), os subconjuntos resultantes serão:

Arr

0	1	1	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8	9

- E o vetor resultante será:



- A função abaixo recebe o índice de um elemento e retorna o índice do elemento representativo de seu subconjunto, atualizando, no caminho, a informação do pai de cada nodo no caminho para a raiz (essa atualização do pai durante a busca é chamada de *path compression*):

```
int find(int x)
{
if (pai[x] != x)
    pai[x] = find(pai[x]);
return pai[x];
}
```

- E o código abaixo efetua a união dos subconjuntos dos elementos x e y . Se eles já fazem parte do mesmo subconjunto, a função não faz nada:

```
void union(int x, int y)  
{  
    int xset = find(x);  
    int yset = find(y);  
    if(xset==yset) return;  
    pai[xset]=yset;  
}
```

- A função anterior pode resultar em árvores bem desbalanceadas. Uma forma de melhorar o desempenho é armazenar junto à raiz da subárvore a altura da mesma, e "pendurar" a árvore mais baixa na árvore mais alta. Esse algoritmo é chamado de union-rank ("rank" é a altura da subárvore).
- O código do union-rank é mostrado na lâmina seguinte. Nele, cada elemento contém o índice do pai, e naqueles elementos que são raízes de árvores, o rank contém a altura da árvore:

```

typedef struct {int pai;int rank;} tnode;
tnode ln[100001];
void union_rank(int x, int y)
{
    int xset = find(x);
    int yset = find(y);
    if(xset==yset) return;
    if (ln[xset].rank<ln[yset].rank)
        ln[xset].pai=yset;
    else
        ln[yset].pai=xset;
    if (ln[xset].rank==ln[yset].rank)
        (ln[xset].rank)++;
}

```

A STL (Standard Templates Library) do C++

- A **STL** é uma biblioteca que contém um conjunto de classes úteis para implementar alguns tipos abstratos de dados. Entre essas classes estão **containers**, que são tipos estruturados com métodos para executar diversas operações sobre eles. A STL oferece 5 containers básicos, 3 adaptadores de containers e alguns containers associativos, como dicionários e conjuntos.
- Vantagens do uso dos containers é que simplificam a codificação e algumas operações sobre eles são implementadas de forma muito eficiente.
- Para usar os componentes da stl é necessário colocar no início do código o comando "using namespace std;" para informar ao compilador que serão usadas funções ou classes da biblioteca padrão.
- Ou colocar "std::" em cada uso de elemento da biblioteca.

- Os containers se diferenciam pela estrutura sobre a qual são implementados (vetor, lista encadeada, lista duplamente encadeada) e as operações que podem ser executadas sobre eles, definidas pelos métodos que oferecem.
- Os 5 containers básicos são:
 - vector
 - list
 - deque
 - arrays (a partir do C++11)
 - forward_list (a partir do C++11)

- Além dos tipos básicos, a STL provê também adaptadores de containers (container adaptors), implementados sobre outros containers, oferecendo operações adicionais.
- São eles:
 - Stack (pilha)
 - Queue (fila)
 - Priority Queue (fila de prioridade)

vector

- O vector é o container mais eficiente em relação a acesso a memória.
- Ele é implementado sobre uma área contínua de memória e permite acesso direto aos elementos, como um vetor, e inserção e remoção no final.
- A declaração de um vector é:
 - **vector** <tipo> nome;
- Ex:
 - **vector** <int> var1, var2;
- Para utilizar um vector é necessário colocar `#include <vector>`

- O acesso aos elementos de um vector pode ser feito diretamente como se fosse um vetor (ex: `vet[i]`), e nesse caso não é feita nenhuma verificação se o elemento existe no vector ou não.
- Ou pode ser feito pelo método `at()`, e no caso de ser feito um acesso a um elemento ainda não inserido é gerada uma exceção.

- Alguns dos principais métodos do vector são:
 - `front()` - devolve o valor do primeiro elemento. Se estiver vazio, resulta um comportamento indefinido.
 - `back()` - devolve o valor do último elemento
 - `empty()` - testa se o container está vazio
 - `push_back()` - insere um elemento no final
 - `pop_back()` - remove um elemento do final
 - `assign(int n, int v)` - preenche **n** posições do vetor com o valor **v**

Iteradores

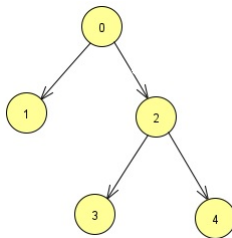
- Os containers possuem também a figura dos **iteradores**, que são objetos utilizados para percorrer sequencialmente os elementos de um container.
- A declaração de um iterador de nome **it** para um vector é feita por:
 - **vector**<int>::iterator it;
- em que o operador :: é usado para referir um elemento de uma classe (no caso, a classe vect<int>).

- e os métodos `begin()` e `end()` retornam iteradores apontando para o primeiro elemento do vector, e para a posição seguinte ao último elemento do vector. Assim, um uso comum para um iterador sobre um vector chamado `vec` seria:

```
for (it = vec.begin(); it!=vec.end(); ++it)
```
- E o acesso ao elemento apontado pelo iterador é feito com o operador `*`, como se fosse um pointer.

- O código abaixo cria uma lista de adjacências para o grafo à esquerda:

```
#include <vector>
// declara vetor de
// listas de adjacências
using namespace std;
vector<int> adj[5];
int main()
{
    adj[0].push_back(1);
    adj[0].push_back(2);
    adj[2].push_back(3);
    adj[2].push_back(4);
}
```



- E o código abaixo implementa uma DFS sobre o grafo:

```
int visit[5]={0};  
vector <int> adj[5];  
  
void dfs(int i)  
{  
    printf("%d\n",i);  
    visit[i]=1;  
    vector <int>::iterator it;  
    for (it=adj[i].begin(); it!=adj[i].end(); it++)  
        if (!visit[*it])  
            dfs(*it);  
}
```


- As principais operações sobre dequeues são:
 - `push_back` - Adiciona elemento no final
 - `push_front` - Adiciona elemento no início
 - `pop_back` - Remove o último elemento
 - `pop_front` - Remove o primeiro elemento
 - `insert` - Insere elementos em qualquer posição
 - `erase` - Remove elementos em qualquer posição
 - `clear` - Remove todos os elementos
- Ex: URI 1594 - Guloso

- O problema de encontrar o valor máximo de cada intervalo de tamanho K em um vetor de tamanho N pode ser resolvido em tempo linear utilizando uma deque e mantendo nela, em ordem decrescente, somente os elementos "úteis", ou seja, os elementos que podem vir a ser o maior de algum intervalo.
- Suponha que o vetor possui os seguintes elementos:
- E o tamanho da janela K é igual a 3.
- Inicialmente o deque "úteis" recebe o primeiro elemento.

3 2 4 7 5 8 1 6 9 6 7 4 5

úteis=[3]

- O próximo elemento do array, 2, por ser menor que o 3, pode vir a ser o maior de uma janela (p.ex, se os elementos após o 2 forem 0 e 1) e é colocado no deque.

3 2 4 7 5 8 1 6 9 6 7 4 5

úteis=[3 2]

- O próximo elemento do array, 4, é maior que todos os elementos do deque, e faz com que nenhum deles possa vir a ser o maior de algum intervalo. Assim, todos os elementos são removidos do deque e o 4 é acrescentado.

3 2 4 7 5 8 1 6 9 6 7 4 5

úteis=[4]

- E isso completa a inicialização do deque, e o maior elemento do primeiro intervalo está no início do deque.

- A partir daí, a janela avança uma posição (na primeira vez, o 3 sai da janela, e entra o 7)
- Se o elemento que é removido da janela (o 3) é igual ao maior elemento do deque (mais à esquerda), este é removido do deque.
- Nesse ponto, todos os valores do deque que são menores que o novo valor da janela (7), são removidos do deque (da direita para a esquerda), pois certamente não serão o maior em alguma janela. E o novo valor da janela é acrescentado à direita no deque.

3 2 4 7 5 8 1 6 9 6 7 4 5

úteis=[4]

3 2 4 7 5 8 1 6 9 6 7 4 5

úteis=[7]

- A cada avanço da janela, o maior elemento da janela é o mais à esquerda no deque.

3 2 4 7 5 8 1 6 9 6 7 4 5
úteis=[7]

3 2 4 7 5 8 1 6 9 6 7 4 5
úteis=[7 5]

3 2 4 7 5 8 1 6 9 6 7 4 5
úteis=[8]

3 2 4 7 5 8 1 6 9 6 7 4 5
úteis=[8 1]

3 2 4 7 5 8 1 6 9 6 7 4 5
úteis=[8 6]

3 2 4 7 5 8 1 6 9 6 7 4 5
úteis=[9]

3 2 4 7 5 8 1 6 9 6 7 4 5
úteis=[9 6]

3 2 4 7 5 8 1 6 9 6 7 4 5
úteis=[9 7]

3 2 4 7 5 8 1 6 9 6 7 4 5

úteis=[9 6]

3 2 4 7 5 8 1 6 9 6 7 4 5

úteis=[9 7]

3 2 4 7 5 8 1 6 9 6 7 4 5

úteis=[7 4]

3 2 4 7 5 8 1 6 9 6 7 4 5

úteis=[7 5]

- Complexidade?
- Estruturas de dados?

- O tipo `list` implementa uma lista duplamente encadeada com as operações de inclusão e remoção nas duas pontas.
- Ao contrário do `vector` e `deque`, ela não permite acesso indexado a todos os elementos.
- A declaração de uma variável do tipo `list` é:
 - **list** <tipo> nome;
- Ex:
 - **list** <int> lista1, lista2;
- Para utilizar listas é necessário colocar `#include <list>`

- As principais operações sobre uma lista são:
 - `front()` - Retorna o valor do primeiro elemento da lista.
 - `back()` - Retorna o valor do último elemento da lista.
 - `push_front(g)` - Adiciona um novo elemento 'g' no início da lista.
 - `push_back(g)` - Adiciona um novo elemento 'g' no final da lista.
 - `pop_front()` - Remove e descarta o primeiro elemento da lista.
 - `pop_back()` - Remove e descarta o último elemento da lista

Queue (fila)

- O container **Queue** implementa uma fila, com as operações padrão de filas, de inserção em uma ponta e remoção na outra.
- A queue é implementada sobre um deque ou uma list.
- A criação de uma fila é feita como qualquer outro container:
 - **queue** <tipo> nome;
- Não esquecer do `#include <queue>`

- Os principais métodos para trabalhar com queues são:
 - `empty()` - retorna true se a fila está vazia
 - `push(valor)` - insere elemento no final da fila
 - `front()` - retorna o primeiro elemento na fila
 - `back()` - retorna o último elemento da fila
 - `pop()` - remove (e descarta) o próximo elemento da fila
- O código a seguir implementa uma BFS sobre o grafo do exemplo anterior, representado por um vetor de listas de adjacências:

```

void bfs(int i)
{
    int visit[5]={0};
    queue<int> fila;
    fila.push(i);
    visit[i]=1;
    while (!fila.empty())
    {
        int v=fila.front();
        printf("%d",v);
        fila.pop();
        vector<int>::iterator it;
        for (it=adj[v].begin(); it!=adj[v].end(); it++)
            if (!visit[*it])
            {
                visit[*it]=1;
                fila.push(*it);
            }
    } // do while
} // da função

```

■ Ex:

- URI 1100 - Movimentos do cavalo
- URI 1704 - Arrumando as tarefas
- URI 1928 - Jogo de Memória (BFS)
- URI 2458 - Setas

Stack (pilha)

- O container **Stack** implementa uma pilha, com as operações padrão de pilhas de inserção e remoção no topo.
- Os principais métodos para manipulação de pilhas são:
 - `empty()` - retorna true se a fila está vazia
 - `push(val)` - empilha um valor
 - `top()` - retorna o valor no topo da pilha
 - `pop()` - remove (e descarta) o valor no topo da pilha
- O código a seguir implementa uma ordenação topológica no grafo, usando o método da DFS

```

#include <stdio.h>
#include <stack>
#include <vector>
using namespace std;
stack<int> pilha;
int visit[5]={0};
vector<int> adj[5];
void dfs(int i)
{
    visit[i]=1;
    vector<int>::iterator it;
    for (it=adj[i].begin(); it!=adj[i].end(); it++)
        if (!visit[*it])
            dfs(*it);
    pilha.push(i);
}
int main(){
    adj[0].push_back(1);
    adj[0].push_back(2);
    adj[2].push_back(3);
    adj[2].push_back(4);
    for (int i=0;i<5;i++)
        if (!visit[i])
            dfs(i);
    while (!pilha.empty()){
        printf("%d\n", pilha.top());
        pilha.pop();}
}

```

- Ex:

- URI 1242 - Ácido Ribonucleico Alienígena
- URI 1903 - Cadeia Alimentar (DFS)

Priority_queue (fila de prioridade)

- O container **priority_queue** implementa uma fila de prioridade. Essa estrutura se caracteriza por permitir identificar o maior (ou menor) elemento em tempo $O(1)$, removê-lo em tempo $O(\log n)$ e inserir novos elementos em tempo $O(\log n)$.
- É útil em algoritmos como o de Prim ou o de Dijkstra, em que um passo importante, e custoso, do algoritmo é a identificação, a cada passo do elemento de menor valor em um conjunto (no caso, a distância de cada vértice ao vértice inicial, no Dijkstra, ou à sub-árvore já formada, no Prim)
- Ele está definido no módulo queue (inclui queue)

- A versão default implementa uma fila de prioridade max-heap, devolvendo a cada vez o maior valor da fila.
 - `priority_queue <int> g;`
- Os principais métodos para manipulação de filas de prioridade são :
 - `empty()` - retorna true se a fila está vazia
 - `push(val)` - empilha um valor
 - `top()` - retorna o valor no topo da pilha
 - `pop()` - remove (e descarta) o valor no topo da pilha
- Para implementar um min-heap, em que o MENOR elemento é mantido no topo, usa-se a sintaxe enigmática abaixo:
 - `priority_queue < int, vector < int >, greater < int >> nome;`

- Containers podem conter tipos escalares, como `int` e `float`, mas também podem conter tipos estruturados, como `structs`.
- Quando o container deve armazenar pares de valores, pode-se utilizar o template `pair` `< tipo1, tipo2 >` que agrupa dois valores como se fosse uma tupla.
- Ex: `vector < pair < int, int >> v;`

- Na implementação do algoritmo de Dijkstra utilizando uma fila de prioridades para recuperar a cada passo o vértice mais próximo do vértice inicial, deve-se armazenar, na fila de prioridades, o número do vértice e a distância associada a ele, ou seja, cada posição da fila de prioridades conterá 2 valores, o número e o peso do vértice.
- Isso pode ser feito definindo um par de valores, com a função `makepair(A,B)`, que retorna um par.
- Para efeito de comparação na fila de prioridades, o primeiro membro da tupla é a primeira chave de comparação, em caso de empate, compara a segunda.
- Ex:
 - URI 1205 - Cerco a Leningrado
 - URI 1633 - SBC
 - URI 2065 - Fila do Supermercado
 - URI 3115 - Estradas

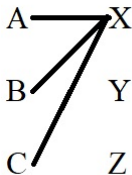
Problemas de Job Assignment pelo Método Húngaro

- São problemas em que se deve associar N tarefas a N pessoas minimizando o custo total. Imagine que se tem a seguinte matriz com o custo de associar a cada pessoa A, B e C, as tarefas X, Y e Z.
- Cada posição (i,j) da matriz representa o custo de atribuir a tarefa j à pessoa i
- Busca-se a atribuição de uma tarefa a cada pessoa que minimize o custo total.

	X	Y	Z
A	10	20	30
B	7	22	17
C	15	21	21

- Problemas de Job Assignment podem ser resolvidos de forma tabular pelo chamado Algoritmo Húngaro, que consiste dos seguintes passos (para uma matriz de ordem N):
 - Para cada linha subtraia de todos os seus valores o valor do menor elemento da linha.
 - Para cada coluna subtraia de todos os seus valores o valor do menor elemento da linha.
 - Encontre o número mínimo L de linhas e colunas para cobrir todos os 0's da matriz
 - Enquanto $L < N$:
 - M = menor número entre as posições não cobertas
 - Subtraia M de todas as posições não cobertas
 - Adicione M a todas as posições que estão em intersecções de linhas e colunas cobertas
 - Recalcule L (o número mínimo de linhas e colunas para cobrir todos os 0's)

- O número mínimo de linhas e colunas para cobrir todos os zeros da matriz pode ser obtido encontrando a **cobertura mínima de vértices** em um grafo bipartido onde os vértices do primeiro grupo correspondem às linhas da matriz, e os vértices do segundo grupo correspondem às colunas.
- Cada 0 na posição **(i,j)** da matriz acrescentará uma aresta no grafo ligando o vértice **i** ao vértice **j**.



	X	Y	Z
A	0	10	20
B	0	15	10
C	0	6	6

- Aplicando no exemplo acima:
- Subtraindo de cada linha o menor elemento da linha:

	X	Y	Z
A	10	20	30
B	7	22	17
C	15	21	21

	X	Y	Z
A	0	10	20
B	0	15	10
C	0	6	6

- Subtraindo de cada coluna o menor elemento da coluna:

	X	Y	Z
A	0	10	20
B	0	15	10
C	0	6	6

	X	Y	Z
A	0	4	14
B	0	9	4
C	0	0	0

- Como o número de linhas/colunas necessárias para cobrir todos os zeros ($L=2$) é menor do que N , subtrai-se de todas as posições não cobertas o menor valor não coberto (4) e soma-se aos elementos nas intersecções das linhas e colunas que cobrem os zeros (no caso, a posição (C,X)).

	X	Y	Z
A	0	4	14
B	0	9	4
C	0	0	0

	X	Y	Z
A	0	0	10
B	0	5	0
C	4	0	0

- Como o número de linhas/colunas necessárias para cobrir todos os zeros ($L=3$) é igual a N , encontrou-se uma atribuição ótima (na verdade, duas), identificadas pelos conjuntos de zeros abaixo:

	X	Y	Z
A	0	0	10
B	0	5	0
C	4	0	0

	X	Y	Z
A	0	0	10
B	0	5	0
C	4	0	0

- Que correspondem às atribuições a seguir, ambas com custo total igual a 48

	X	Y	Z
A	10	20	30
B	7	22	17
C	15	21	21

	X	Y	Z
A	10	20	30
B	7	22	17
C	15	21	21

Voltar para Branch and Bound